PYTHON

Field Notes · from *Introducing Python*, 3rd ed.

KEY IDEA	the concept to remember
TYPE CHECK	what type you have, and what you can do with it
RULE	how to do it right
TRAP CARD	a common mistake
BOSS FIGHT	the hardest idea in the chapter
PRACTICE	questions to check yourself

Contents

1 Introduction **2**
- 1.1 Mysteries . . . 2
- 1.2 Little Python Programs . . . 2
  - A loop . . . 2
  - A list: values by position . . . 3
  - A dictionary: values by name . . . 3
- 1.3 Setup . . . 4
  - Install Python . . . 4
  - Upgrade Python . . . 4
  - Run Python programs . . . 4
- 1.4 Built-In Python Features . . . 5
- 1.5 A Bigger Example . . . 5
- 1.6 Review . . . 8

2 Types and Variables **9**
- 2.1 A Computer . . . 9
- 2.2 Bits and Bytes . . . 9
  - A byte as a number . . . 9
  - A byte as a character . . . 10
- 2.3 Multibyte Types . . . 10
- 2.4 Variables . . . 11
  - Assign a value . . . 11
  - A name is a sticky note . . . 11
  - What an object holds . . . 12
  - Change a value: rebind the name . . . 12
  - Delete a variable . . . 13
  - Two names, one object . . . 13
  - Name variables . . . 13
- 2.5 Python Types . . . 14
  - Type hints do not enforce types . . . 15
- 2.6 Specify Values . . . 16
- 2.7 Objects as Plastic Boxes in Memory . . . 16
- 2.8 Review . . . 16

3 Numbers **18**
- 3.1 Booleans . . . 18
- 3.2 Integers . . . 19
  - Literal integers . . . 19
  - Integer operations . . . 19
  - Integers and variables . . . 21
  - Precedence . . . 21
  - Bases . . . 22
  - Type conversions . . . 22
  - How big is an int? . . . 23
- 3.3 Floats . . . 23
  - Rounding . . . 24

Special float values . . . 24
3.4 Floats Are Not Exact . . . 24
Fractions . . . 25
Decimals . . . 25
3.5 Math Functions . . . 26
3.6 Review . . . 26

4 Strings **27**
4.1 Strings Are Sequences . . . 27
4.2 Create with Quotes . . . 27
4.3 Create with str() . . . 28
4.4 Escape with \ . . . 29
4.5 Combine with + and Duplicate with * . . . 29
4.6 Get a Character by [offset] . . . 30
4.7 Get a Substring with a Slice . . . 31
4.8 Get Length with len() . . . 32
4.9 Methods: Functions That Belong to a String . . . 32
Split with split() . . . 32
Combine with join() . . . 33
Substitute with replace() . . . 33
Prefixes and suffixes . . . 34
Strip with strip() . . . 34
Search and select . . . 35
Change case . . . 35
Set alignment . . . 35
4.10 Apply Formatting . . . 36
Old style: % . . . 36
New style: format() . . . 36
Newest style: f-strings . . . 36
4.11 More String Things . . . 38
4.12 Review . . . 38

5 Bytes and Bytearray **39**
5.1 Bytes . . . 39
Create with quotes . . . 39
Create with `bytes()` . . . 39
Create from a hex string . . . 40
Encode and decode . . . 40
Index, slice, combine, repeat . . . 40
Bytes are immutable . . . 41
5.2 Bytearray . . . 42
Create with `bytearray()` . . . 42
Read: index and slice . . . 42
Change one byte or many . . . 43
`insert()`, `append()`, `extend()` . . . 43
Combine, repeat, freeze . . . 44
5.3 Where bytes show up in AI work . . . 44

6 If and Match **46**
6.1 Comment with # . . . 46
6.2 Continue lines with \ and brackets . . . 46
6.3 Compare with if, elif, and else . . . 47
The shape of an if statement . . . 47
Comparison operators . . . 47

and, or, not . . . 48
Chained comparisons . . . 49
What is true? . . . 49
None is not the same as empty or zero . . . 49
Conditional expressions: `a if cond else b` . . . 50
6.4 Do multiple comparisons with in . . . 51
6.5 The walrus operator . . . 51
6.6 Match . . . 52
Simple matches . . . 52
Structural matches . . . 53
Matching JSON-like data . . . 54

7 For and While **56**
7.1 Repeat with while . . . 56
Cancel with break . . . 56
Skip ahead with continue . . . 57
Check break use with else . . . 57
7.2 Iterate with for and in . . . 58
break, continue, else . . . 58
Counting with enumerate . . . 59
Nested loops . . . 59
7.3 Generate number sequences with range() . . . 60
7.4 Other iterators . . . 61

8 Tuples and Lists **63**
8.1 Tuples . . . 63
Create with commas and () . . . 63
Unpacking and swapping . . . 63
Convert with tuple() . . . 64
What you can do with a tuple . . . 64
8.2 Lists . . . 65
Create with [], list(), and split() . . . 65
Index and slice . . . 65
Add items . . . 66
Change items . . . 66
Remove items . . . 67
Find and count . . . 67
join() is the opposite of split() . . . 68
Sort with sort() or sorted() . . . 68
Assignment makes a second name, not a second list . . . 69
Copy everything with deepcopy() . . . 69
Compare and iterate . . . 70
Iterate several sequences with zip() . . . 70
List comprehensions . . . 71
Lists of lists . . . 71
8.3 Tuples versus lists . . . 72
8.4 There are no tuple comprehensions . . . 72

9 Dictionaries and Sets **74**
9.1 Dictionaries . . . 74
Create with {} and dict() . . . 74
Convert with dict() . . . 74
Add or change an item with [key] . . . 75
Get an item by [key] or with get() . . . 75

Nested dictionaries: reading JSON . . . 76
Iterate with for and in . . . 77
Combine dictionaries . . . 78
Delete items . . . 79
Test for a key with in . . . 79
Assign, copy, deepcopy . . . 79
Compare dictionaries . . . 80
Dictionary comprehensions . . . 80
Counting and grouping patterns . . . 80
9.2 Sets . . . 81
Create with set() or {} . . . 81
Add, remove, length, membership . . . 82
Set operations . . . 82
Set comprehensions and frozenset . . . 83
Which container? . . . 84

10 Functions **85**
10.1 Define and call a function . . . 85
10.2 None, truthiness and falsiness . . . 86
10.3 Positional, keyword and default arguments . . . 87
Positional and keyword arguments . . . 87
Default values . . . 87
10.4 Packing and unpacking: `*args` and `**kwargs` . . . 88
10.5 Keyword-only and position-only parameters . . . 89
10.6 Mutable and immutable arguments . . . 90
10.7 Docstrings . . . 91
10.8 Functions are first-class objects . . . 91
Arguments are not a tuple . . . 92
10.9 Inner functions . . . 93
10.10 Closures . . . 93
10.11 Lambda: anonymous functions . . . 94
10.12 Generators . . . 94
10.13 Decorators . . . 95
Stacking decorators . . . 96
10.14 Namespaces and scope . . . 97
10.15 Dunder names . . . 98
10.16 Recursion . . . 98
10.17 Async functions, briefly . . . 99
10.18 Exceptions . . . 100
try, except, else, finally . . . 100
Exception types form a tree . . . 101
Raise and define your own exceptions . . . 101

11 Objects **104**
11.1 What are objects? . . . 104
11.2 Simple objects . . . 104
Define a class and create objects . . . 104
Assign attributes . . . 105
Methods, `__init__` and `self` . . . 105
11.3 Inheritance . . . 107
Inherit, override, add a method . . . 107
Multiple inheritance and method resolution order . . . 108
Mixins . . . 109
11.4 In self defense . . . 109

11.5 Attribute access . . . 110
Getters, setters and properties . . . 110
Properties for computed values . . . 111
Name mangling . . . 112
11.6 Class attributes and object attributes . . . 112
11.7 Method types . . . 113
11.8 Duck typing . . . 114
11.9 Magic methods . . . 115
Equality, ordering and math . . . 115
__repr__ and __str__ . . . 116
__len__ and __getitem__: subscript syntax . . . 116
__call__: objects you can call . . . 117
11.10 Composition and aggregation . . . 118
11.11 When to use objects or something else . . . 119
11.12 Named tuples . . . 119
11.13 Dataclasses . . . 119
Defaults, options and validation . . . 120
11.14 Nested objects, and attribute versus subscript access . . . 121
From a dict to nested objects . . . 122
11.15 attrs . . . 124

12 Modules and Packages **126**
12.1 Modules and the import statement . . . 126
How an import is resolved . . . 127
The three import forms . . . 127
__name__ and running a module as a script . . . 128
12.2 Packages . . . 128
12.3 The module search path . . . 129
12.4 Relative and absolute imports . . . 130
12.5 Namespace packages . . . 130
12.6 Modules versus objects . . . 131
12.7 Goodies in the standard library . . . 132
Missing keys: setdefault and defaultdict . . . 132
Counting: Counter . . . 133
OrderedDict: mostly history . . . 134
Stack, queue, and deque . . . 134
itertools: loops without the loop code . . . 135
random . . . 135
12.8 More batteries: third-party code . . . 136

13 Development Environment **137**
13.1 Find Python code . . . 137
13.2 Install packages . . . 137
pip . . . 137
Other ways to install . . . 138
13.3 Virtual environments . . . 138
Why you need one . . . 139
Create and use one with venv . . . 139
What "activate" really does . . . 139
Am I inside the venv? . . . 139
The other tools, in a sentence each . . . 140
13.4 Integrated development environments . . . 140
IPython and Jupyter . . . 141
13.5 Source control . . . 141

The three places a change lives . . . 141
A first repository, step by step . . . 141

14 Type Hints and Documentation 144
14.1 Type hints . . . 144
Variable hints . . . 144
Function hints . . . 145
Hints are not enforced . . . 146
Optional values: `str | None` . . . 146
Messages, `Literal` and `TypedDict` . . . 146
14.2 Mypy . . . 147
14.3 Documentation . . . 148
Comments . . . 148
Docstrings . . . 149
Markup files . . . 149

15 Testing 151
15.1 The ladder of checks . . . 151
15.2 Linters: Pylint, Pyflakes and Ruff . . . 151
Ruff (real output) . . . 151
Pylint, Pyflakes, pycodestyle . . . 152
15.3 Unittest . . . 152
Arrange, act, assert . . . 152
15.4 Doctest . . . 154
15.5 Pytest . . . 155
A first test file . . . 156
Parametrization: one test, many cases . . . 157
Fixtures . . . 157
Handy flags . . . 158
Testing code that calls the API . . . 158
15.6 Hypothesis: property-based tests . . . 160
15.7 Nox . . . 160
15.8 Continuous integration . . . 161

16 Debugging 163
16.1 Start here: read the traceback . . . 163
16.2 Assert . . . 164
16.3 Print . . . 165
16.4 F-strings . . . 165
16.5 Pprint . . . 166
16.6 IceCream . . . 166
16.7 Decorators . . . 166
16.8 Logging . . . 167
Levels . . . 167
Configure once, with named loggers . . . 168
Handlers and formatters . . . 169
16.9 Pdb: the debugger . . . 170
Commands you will use . . . 171
Post-mortem: debug a crash where it happened . . . 171
16.10`breakpoint()` . . . 172

17 Text Data 174
17.1 Text Strings: Unicode . . . 174
Python Unicode strings . . . 174

UTF-8: encode and decode . . . 175
HTML entities . . . 177
Normalization . . . 177
17.2 Text Strings: Regular Expressions . . . 178
The five basic functions . . . 178
Patterns . . . 180
Specifying match output . . . 182
17.3 Review and what comes next . . . 182

18 Binary Data **184**
18.1 Convert Binary Data with struct . . . 184
The format string . . . 184
Endianness . . . 185
Reading from the middle of a buffer . . . 185
Typical errors . . . 186
A real-world shape: length, then JSON, then data . . . 186
18.2 Extraction with Binary Data Tools . . . 187
18.3 Convert Bytes/Strings with binascii . . . 187
18.4 Use Bit Operators . . . 188
Using bits as flags and fields . . . 189
18.5 Review and what comes next . . . 190

19 Dates and Times **191**
19.1 Why dates and times are hard . . . 191
19.2 Leap Year . . . 191
19.3 The datetime Module . . . 192
date and timedelta . . . 192
time and datetime . . . 194
Naive versus aware . . . 194
19.4 The time Module . . . 195
19.5 Read and Write Dates and Times . . . 196
ISO 8601: the default choice . . . 196
strftime: object to string . . . 196
strptime: string to object . . . 197
Locale and names . . . 198
19.6 All the Conversions . . . 198
19.7 Alternative Modules . . . 199
19.8 Review and what comes next . . . 199

20 Files **200**
20.1 Files and directories . . . 200
20.2 File Input and Output . . . 200
Create or open with open() . . . 200
with: close files automatically . . . 201
Read a text file . . . 202
Errors you will meet . . . 203
Newlines . . . 204
Binary files and seek() . . . 204
Memory mapping . . . 205
20.3 File Operations . . . 205
20.4 Directory Operations . . . 206
20.5 Pathnames . . . 207
os.path . . . 208
pathlib . . . 208

20.6 BytesIO and StringIO . . . 209
20.7 File Formats: Determination . . . 210
20.8 Review and what comes next . . . 211

21 Data in Time: Concurrency 212
21.1 Programs and Processes . . . 212
Create a process with subprocess . . . 212
Create a process with multiprocessing . . . 213
Kill a process with terminate() . . . 214
21.2 Command Automation . . . 215
21.3 Concurrency . . . 215
Queues . . . 216
Threads . . . 217
concurrent.futures . . . 219
Green threads and gevent . . . 220
Twisted . . . 220
asyncio . . . 220
Asyncio alternatives . . . 223
Async versus 224
Async frameworks and servers . . . 224
Redis . . . 224
Beyond queues . . . 225
21.4 Review and what comes next . . . 225

22 Data in Space: Networks 227
22.1 TCP/IP . . . 227
22.2 Sockets . . . 227
UDP: send one datagram . . . 227
TCP: a connection . . . 228
TCP is a stream, not messages . . . 229
Scapy and Netcat . . . 231
22.3 Networking Patterns . . . 231
22.4 The Request-Reply Pattern . . . 232
ZeroMQ . . . 233
Other messaging tools . . . 233
22.5 The Publish-Subscribe Pattern . . . 234
Redis . . . 234
ZeroMQ . . . 234
22.6 Internet Services . . . 235
Domain Name System (DNS) . . . 235
Python email modules . . . 236
Other protocols . . . 236
22.7 Web Services and APIs . . . 236
22.8 Data Serialization . . . 239
JSON vs pickle . . . 240
Other formats . . . 241
22.9 Remote Procedure Calls . . . 241
XML-RPC . . . 241
Other RPC systems . . . 242
22.10 Remote Management . . . 242
22.11 Big Fat Data . . . 242
22.12 Clouds . . . 243
Docker and Kubernetes . . . 243
22.13 Practice . . . 243

23 Data in a Box: Persistent Storage **245**
23.1 Text Files . . . 245
23.2 Tabular and Delimited Text Files . . . 245
CSV . . . 246
XML . . . 247
JSON . . . 248
YAML . . . 249
TOML . . . 249
Tablib . . . 250
Configuration files . . . 250
23.3 Binary Files . . . 251
Padded binary files and memory mapping . . . 251
Spreadsheets, HDF5 and TileDB . . . 252
23.4 Relational Databases . . . 252
Tables, keys and indexes . . . 252
23.5 SQL . . . 252
A small dataset . . . 253
SELECT, WHERE, ORDER BY, LIMIT . . . 254
Aggregates and GROUP BY . . . 254
JOIN . . . 255
The order SQL really runs . . . 256
UPDATE, DELETE and indexes . . . 256
23.6 DB-API and SQLite . . . 256
Placeholders and what comes back . . . 257
Transactions: commit or lose it . . . 258
23.7 DuckDB, MySQL and PostgreSQL . . . 260
23.8 SQLAlchemy . . . 260
23.9 NoSQL Data Stores . . . 262
The dbm family . . . 262
Memcached . . . 263
Redis . . . 263
Redis alternative: Valkey . . . 265
Other NoSQL families . . . 266
23.10Practice . . . 266

24 The Web **268**
24.1 Web Basics . . . 268
URL anatomy . . . 268
Status codes . . . 269
Cookies in one example . . . 269
24.2 HTTP Testing . . . 269
24.3 Web Clients . . . 270
The standard library . . . 270
Requests . . . 273
Other clients: httpx and aiohttp . . . 273
24.4 Web Servers . . . 274
The simplest Python web server . . . 274
From CGI to WSGI to ASGI . . . 274
Apache, nginx and other servers . . . 275
24.5 Web Server Frameworks . . . 275
Bottle . . . 275
Flask . . . 275
Django, FastAPI, Litestar . . . 276
24.6 Database Frameworks . . . 278

24.7 Web Services and Automation . . . 278
webbrowser and webview . . . 278
Web APIs and REST . . . 278
WebSockets . . . 279
Webhooks . . . 279
24.8 Web Frontends . . . 279
24.9 Crawl and Scrape . . . 280
24.10 Let's Watch a Movie: a client in four small functions . . . 281
24.11 Practice and wrap-up . . . 282

25 Data Science **283**
25.1 Standard Python . . . 283
Sorting with itemgetter and attrgetter . . . 283
Format conversions . . . 283
Math . . . 284
Complex numbers, Decimal and Fraction . . . 285
array, statistics and the @ operator . . . 285
25.2 NumPy . . . 286
Making arrays and reading their shape . . . 287
Reshape and indexing . . . 287
Array math: no loops . . . 288
Shapes and broadcasting . . . 288
Linear algebra, and an embedding example . . . 288
25.3 SciPy . . . 289
25.4 pandas . . . 290
Reading a CSV and looking around . . . 290
Filter, group, summarize . . . 291
Dates . . . 292
25.5 Polars . . . 292
25.6 DuckDB . . . 292
25.7 Data Visualization . . . 292
25.8 Practice and wrap-up . . . 293

26 AI **294**
26.1 Terms that get mixed up . . . 294
26.2 It Turns Out... (a short history) . . . 294
Expert systems . . . 294
Perceptrons . . . 295
The breakthrough: hidden layers and backpropagation . . . 295
Image recognition: ImageNet and AlexNet . . . 297
Large language models . . . 297
The ChatGPT moment . . . 297
Retrieval augmented generation (RAG) . . . 297
Agents, tools and protocols . . . 298
Efficiency . . . 298
26.3 Create Models: Python Frameworks . . . 298
26.4 Current Models . . . 299
26.5 A Million Models: Hugging Face . . . 300
26.6 Working Examples With Ollama . . . 300
What the book's experiments teach . . . 301
References and next steps . . . 302
26.7 Practice and wrap-up . . . 302

27 Performance **304**

27.1 Computer Hardware . . . 304
27.2 Measure Timing . . . 305
A stopwatch with perf_counter . . . 305
timeit . . . 305
A decorator and a context manager . . . 305
27.3 Profile . . . 306
27.4 Algorithms and Data Structures . . . 308
Arrays versus lists and tuples . . . 309
27.5 Cache . . . 310
27.6 Concurrency for waiting . . . 311
27.7 Going faster than CPython . . . 312
Cython . . . 312
NumPy and SciPy . . . 313
C or Rust extensions . . . 313
Taichi, PyPy and Numba . . . 313
The standard-library JIT . . . 313
Mojo . . . 313
27.8 Practice and wrap-up . . . 314

CHAPTER 01

Introduction

This chapter shows what a Python program looks like, how to install Python, and how to run code in two ways: typed line by line, or saved in a file. It ends with a realistic script that reads JSON, the data format almost every AI API speaks. You do not need to understand every line yet. You need to see the shape of the thing.

1.1 Mysteries

A knitting pattern and a cooking recipe both look like nonsense to an outsider. Yet both are *little languages*: short vocabularies, strict rules, steps in order. A program is the same kind of thing, written for a computer. The table maps the parts of a recipe to the parts of a Python program.

Feature	In a recipe	In Python
Vocabulary	"fold", "sift", abbreviations	keywords and names: `for`, `print`
Syntax	what may be written, and where	the grammar rules of the language
Sequence	steps run top to bottom	statements run top to bottom
Loop	"repeat for each ball"	`for` and `while`
Call another recipe	"see the dough recipe"	a **function** call
Assumed context	you know what an oven is	the standard library, the OS
Data	flour, potatoes	values: numbers, text, lists
Tools	bowl, pan	the interpreter, an editor, a terminal
Result	dinner	output, a file, a changed value

> ★ **KEY IDEA: programming is learnable**
>
> Programming is mostly learning a small vocabulary and a few rules. Python keeps both small, which is one reason it dominates data science and AI work.

1.2 Little Python Programs

Python is a general-purpose language created by Guido van Rossum in 1991. It is designed to be easy to read, it runs tiny scripts and huge systems, and it is the main language of modern machine learning. Three small programs show most of its flavor.

A loop

```
# file: loop1.py
for step in 3, 2, 1, "go":
    print(step)
```

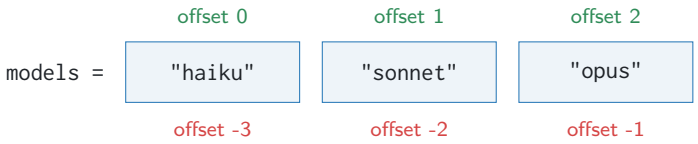
```
3
2
1
go
```

`for` takes each value in turn, puts it in the name `step`, and runs the indented line. Notice that the values are mixed: numbers and text. Python does not mind.

A list: values by position

A **list** keeps values in order inside square brackets. You reach one by its **offset**: the distance from the start. The first item is at offset 0.

```
models = ["haiku", "sonnet", "opus"]
print(models[1])
```



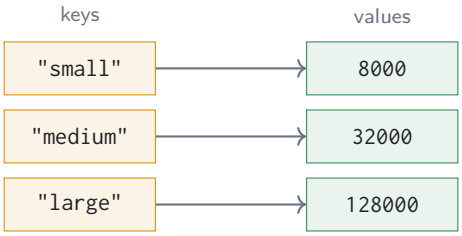
The output of the program above is `sonnet`: offset 1 is the *second* item. Think in offsets, not positions.

A dictionary: values by name

A **dictionary** stores *key*: *value* pairs in curly braces. You look a value up by its key instead of by a number.

```
context_limit = {"small": 8_000, "medium": 32_000, "large": 128_000}
size = "medium"
print(size, "allows", context_limit[size], "tokens")
```

(The numbers are invented for the demo.) Output: `medium allows 32000 tokens`.



TYPE CHECK: square brackets do three different jobs

You write	What it is	You get
`["a", "b"]`	list literal	a `list`
`models[1]`	subscript with an int offset	one item from the list
`context_limit["small"]`	subscript with a key	one value from the dict

Using the wrong thing fails loudly:

```
>>> models = ["haiku", "sonnet", "opus"]
>>> models[3]
Traceback (most recent call last):
  ...
IndexError: list index out of range
>>> {"small": 8_000}["tiny"]
Traceback (most recent call last):
  ...
KeyError: 'tiny'
```

1.3 Setup

To write Python you need two tools: a **terminal** (a window where you type commands) and a **text editor** (plain text, not a word processor). Windows, macOS and Linux all have both. IDEs such as VS Code bundle them; they come in a later chapter.

> ⚠ **TRAP CARD**
> Never write code in Word or similar. They swap straight quotes for curly ones and add formatting. Python then fails with a confusing `SyntaxError`. Use a code editor.

Install Python

Download the latest release from `python.org` (Downloads, then your platform) and follow the installer. On Windows, tick the option that adds Python to `PATH`. The computer may already have an old Python; install a current one anyway.

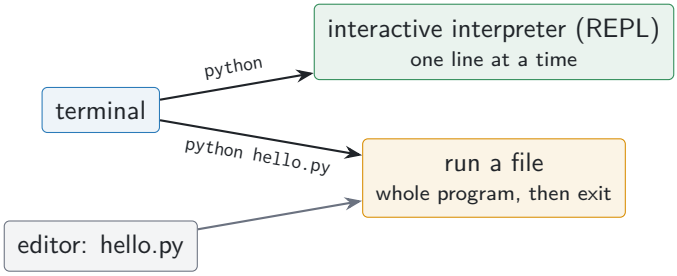
Upgrade Python

Versions are numbered **x.y.z**: a major number, a minor number, a bug-fix number. Python 3 has been the major version for years and no Python 4 is planned. A new minor version (3.12, 3.13, ...) arrives about once a year. Check yours:

```
$ python -V
Python 3.12.10
```

On Windows the launcher may be `py -V`. This series uses Python 3.12. Newer minor versions add features; code written for 3.12 normally runs on later ones.

Run Python programs



Interactive interpreter. Type `python` alone. You get a prompt `>>>`. Type one line of Python, press Enter, see the result at once. This is the fastest way to test an idea.

A file. Save code in a file ending in `.py`, then run `python hello.py`. Use files for anything longer than a few lines.

One difference surprises beginners. The interpreter *echoes* the value of an expression. A file does not. In a file you must call `print()`.

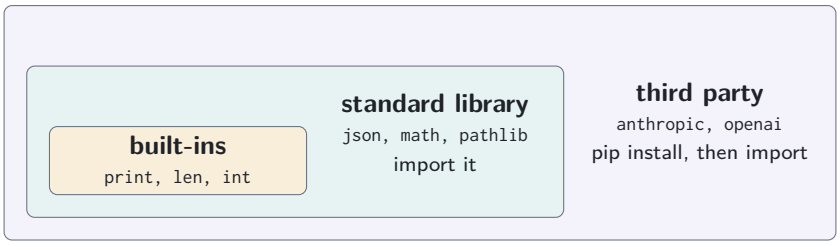
```
>>> 6 * 7
42
>>> print(6 * 7)
42
>>> "hi"
'hi'
>>> print("hi")
hi
```

Note the quotes: the echo shows the value with quotes (a *representation*), while `print()` shows the text for humans.

> ■ **TYPE CHECK: `print()` returns nothing**
>
> `print()` displays text and returns `None`. If you write `x = print("hi")`, then `x` is `None`, not the text. Use `print` to show, not to produce a value.

1.4 Built-In Python Features

Where does code come from? Three layers, each wider than the last.



Layer	How you get it	Example
Built-ins	nothing to do	`len("abc")`
Standard library	`import name`	`import math`
Third-party packages	`pip install name`, then import	`import anthropic`

```
>>> import math
>>> math.sqrt(16)
4.0
>>> from json import dumps
>>> dumps({"ok": True})
'{"ok": true}'
```

Installing third-party packages is done in a terminal, ideally inside a virtual environment (a private folder of packages per project):

```
$ pip install anthropic
```

(`anthropic` is a third-party package; it is not needed to read this chapter.)

> ➤ **RULE**
>
> One virtual environment per project. Never install project packages into the system Python. It keeps versions from clashing.

1.5 A Bigger Example

Here is a short script that does real work: it takes the JSON text an AI API might send back, turns it into Python data, and pulls out two facts. It uses an import, a multi-line string, a function call, nested lookups, an f-string and error handling. (The JSON is canned, so the script runs offline.)

```
# file: reply.py
import json

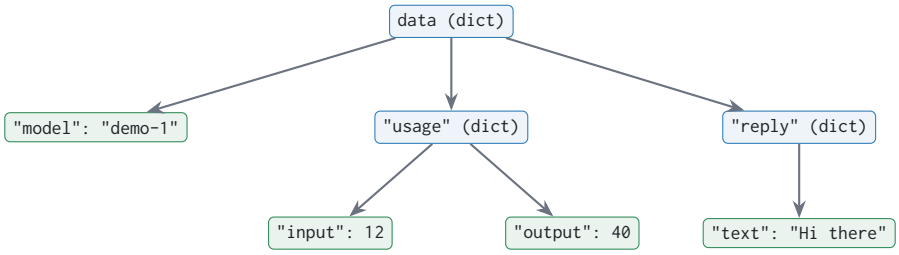
raw = """{
  "model": "demo-1",
  "usage": {"input": 12, "output": 40},
  "reply": {"text": "Hi there"}
}"""

data = json.loads(raw)
try:
    text = data["reply"]["text"]
    total = data["usage"]["input"] + data["usage"]["output"]
    print(f"{data['model']} said {text!r} using {total} tokens")
except KeyError as err:
    print("missing key:", err)
```

```
demo-1 said 'Hi there' using 52 tokens
```

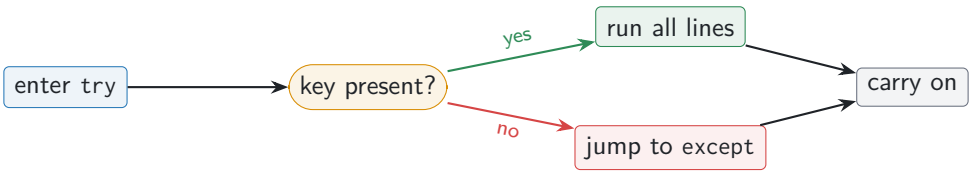
What each part does:

1. `import json`: make the standard-library JSON module available.
2. The triple-quoted string `raw` holds JSON *text*. It is a `str`.
3. `json.loads(raw)` parses text into Python data. Here you get a `dict`.
4. `try:` run the indented block. If a key is missing, jump to `except`.
5. `data["reply"]["text"]` follows two layers of dictionaries.
6. The f-string `f"..."` inserts values inside curly braces.
7. `except KeyError as err` runs only if a key was absent; `err` holds the key.



If the key is missing, the script reports it instead of crashing:

```
>>> import json
>>> data = json.loads('{"model": "demo-1"}')
>>> try:
...     data["reply"]["text"]
... except KeyError as err:
...     print("missing key:", err)
...
missing key: 'reply'
```



★ KEY IDEA: indentation is syntax

Many languages mark blocks with braces. Python uses **indentation**. The lines indented under `try:` belong to it; the unindented line ends it. Use four spaces. Mixing tabs and spaces causes errors.

⚠ TRAP CARD

A bare `except:` (with no error type) hides every bug, even a typo. Name the error you expect: `except KeyError`.

■ TYPE CHECK: what `json.loads` gives you

JSON	Python
object `{...}`	`dict`
array `[...]`	`list`
string	`str`
number	`int` or `float`
`true` / `false`	`True` / `False`
`null`	`None`

A dict uses *subscripts*: `data["model"]`, not `data.model`. Dot access (`obj.name`) is for objects such as SDK responses.

```
>>> d = {"model": "demo-1"}
>>> d["model"]
'demo-1'
>>> d.model
Traceback (most recent call last):
  ...
AttributeError: 'dict' object has no attribute 'model'
```

■ BOSS FIGHT: reading nested data

Chains of subscripts are read left to right; each one peels off one layer. Always know the type of the layer you are in: a list needs an *int*, a dict needs a *key*.

```
>>> resp = {"choices": [{"message": {"text": "ok"}}]}
>>> resp["choices"]
[{'message': {'text': 'ok'}}]
>>> resp["choices"][0]
{'message': {'text': 'ok'}}
>>> resp["choices"][0]["message"]
{'text': 'ok'}
>>> resp["choices"][0]["message"]["text"]
'ok'
```

When one step fails, evaluate the chain one subscript at a time. The error shows which layer was not what you thought.

1.6 Review

You now know the shape of a Python program: names, values, lists, dictionaries, loops, imports, and indentation for blocks. You know two ways to run code and three places code comes from. Next chapter: what a variable really is.

➤ **PRACTICE**

1. Why does `models[1]` on a three-item list give the *second* item?
2. What is the difference between what the interpreter echoes and what a file prints?
3. Name the three sources of Python code and give one example of each.
4. Which error do you get from a list offset that is too large? From a dict key that does not exist?
5. In `r["a"][0]["b"]`, what type must `r`, `r["a"]` and `r["a"][0]` have?
6. What marks the end of a `try` block in Python?

★ **ACHIEVEMENT UNLOCKED: First Run**

- Read a short Python program: loop, list, dictionary, `try`/`except`.
- Check your Python version and start the interpreter.
- Run code interactively and from a `.py` file, and explain the echo difference.
- Tell built-ins, the standard library and third-party packages apart.
- Navigate nested list/dict data with chained subscripts.

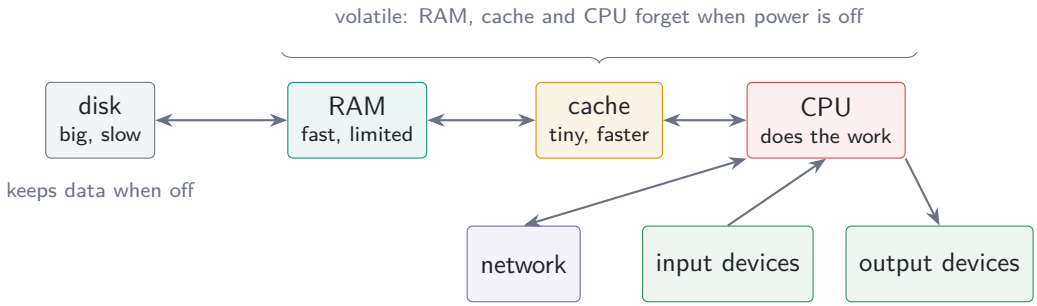
CHAPTER 02

Types and Variables

This chapter explains how a computer stores data, and then how Python handles it in a way that differs from many other languages. The big idea: in Python a variable is just a *name* that points at an *object*, and the object knows its own type. Understanding this now saves you from a whole class of confusing bugs later.

2.1 A Computer

A program is code plus data. The hardware that holds and processes both has a few parts.



Programs and data live on the disk. The CPU can only work on what has been copied into RAM, so data moves `disk` → `RAM` → cache → `CPU` and back. A machine learning model works the same way: its weights sit on disk until the program loads them into memory.

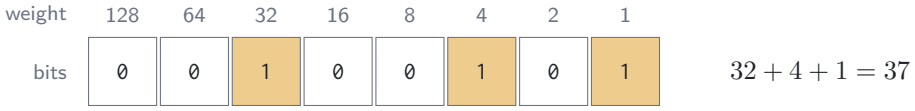
Part	Job	Survives power-off?
CPU	runs instructions (many cores in modern chips)	no
Cache	small, faster copy of what the CPU needs next	no
RAM	working storage for programs and data	no
Disk	long-term storage, much larger, much slower	✓
Input / output	keyboard, mouse, camera, display, printer	n/a
Network	talks to other computers	n/a

2.2 Bits and Bytes

Everything in a computer is made of **bits**. A bit holds one of two states, which you can read as off/on, 0/1 or false/true. Eight bits make a **byte**, which has $2^8 = 256$ possible patterns. A single bit or byte means nothing by itself. Meaning comes from how a program chooses to read the pattern.

A byte as a number

Read a byte in base 2 (binary). Each bit stands for a power of two, like the ones, tens and hundreds places in decimal.



$$32 + 4 + 1 = 37$$

All eight bits set gives $128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$. All clear gives 0. That is the range of an *unsigned* byte: 0 to 255.

```
>>> 2 ** 8
256
>>> 0b11111111
255
>>> bin(37)
'0b100101'
>>> f"{37:08b}"
'00100101'
>>> int("00100101", 2)
37
```

A *signed* byte spends one bit on the sign, so it covers -128 to 127 instead. The same eight bits can therefore mean two different numbers:

```
>>> (255).to_bytes(1, "big")
b'\xff'
>>> int.from_bytes(b"\xff", "big")
255
>>> int.from_bytes(b"\xff", "big", signed=True)
-1
```

A byte as a character

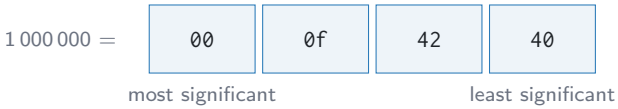
A byte can also stand for a character. ASCII was the old agreement: 128 codes for English letters, digits, punctuation and a few control codes. One byte cannot hold every writing system, so Unicode defines millions of symbols (Chapter 17). Python text uses Unicode.

```
>>> ord("A")
65
>>> chr(97)
'a'
```

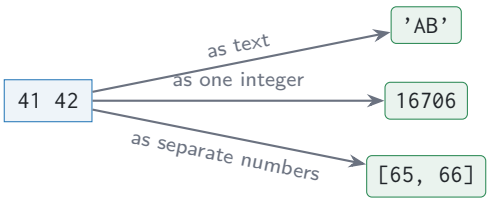
2.3 Multibyte Types

Bytes sit side by side in memory and can be grouped. Two bytes give $2^{16} = 65{,}536$ patterns; four give $2^{32} \approx 4.3$ billion. Memory locations are anonymous, so the program has to remember how many bytes belong together and *what type* they represent.

```
>>> 2 ** 16
65536
>>> 2 ** 32
4294967296
>>> (1_000_000).to_bytes(4, "big").hex()
'000f4240'
```



> ★ **KEY IDEA: the same bytes, different meanings**
>
> Bytes carry no meaning until a type is applied. The two bytes `41 42` are the text `AB`, the integer 16706, or the list of small numbers 65 and 66, depending on the type you read them as.




```
>>> data = b"AB"
>>> data.decode("ascii")
'AB'
>>> int.from_bytes(data, "big")
16706
>>> list(data)
[65, 66]
```

2.4 Variables

A **variable** is a name in your code that refers to a value. Python handles this differently from languages like C or Java, so the picture matters.

Assign a value

```
>>> temperature = 0.7
>>> temperature
0.7
```

The shape is `name = value`. The `=` sign means *assign*, not "equals". The right side is evaluated first; the result is then given the name on the left. Typing a bare name in the interpreter prints its value. A file would not.

A name is a sticky note

In many languages a variable is a *box* with a fixed type, and assignment copies the value into the box. In Python, the value lives in an **object** somewhere in memory. The variable is only a **name** (also called a reference or label) attached to that object. Python is **dynamically typed**: the type belongs to the object, not to the name.

Box model (C, Java)



the name *is* the box

Python



the name points at an object

```
>>> x = 5
>>> type(x)
<class 'int'>
>>> x = "five"
>>> type(x)
<class 'str'>
```

The name `x` was reused for a different type without trouble. Only the object `5` is an `int`; only `"five"` is a `str`.

What an object holds

Every Python object carries some bookkeeping with its value.

Part	Meaning
`id`	a unique number for this object while it lives; `id(obj)`
type	what kind of object it is; `type(obj)`
value	the actual data
reference count	how many names (and other holders) point at it

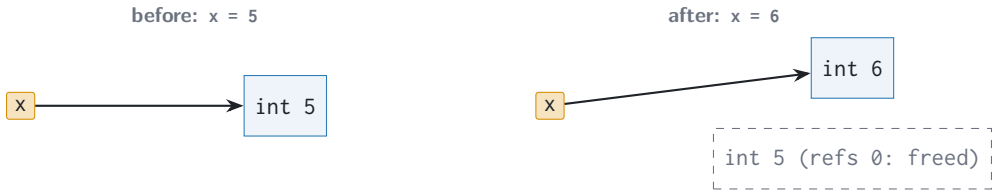
When the reference count drops to zero, nothing can reach the object any more and Python reclaims its memory. This is **garbage collection**. You never call `malloc` or `free` as in C.

```
>>> import sys
>>> a = []
>>> sys.getrefcount(a)
2
>>> b = a
>>> sys.getrefcount(a)
3
```

(`getrefcount` reports one more than you expect, because passing `a` to the function adds a temporary reference.)

Change a value: rebind the name

Assigning again does *not* edit the old object. Python builds (or finds) another object and moves the name to it.




```
>>> x = 1000
>>> first = id(x)
>>> x = 2000
>>> id(x) == first
False
```

⚠ TRAP CARD

CPython keeps ready-made objects for small integers (-5 to 256) and reuses them. So `is` can seem to work on small numbers and fail on larger ones. Compare values with `==`. Use `is` only for `None`.

```
>>> p = int("256"); q = int("256")
>>> p is q
True
>>> p = int("1000"); q = int("1000")
>>> p is q
False
>>> p == q
True
```

Delete a variable

`del name` removes the name (and lowers the object's count). You rarely need it, because Python frees names when their scope ends.

```
>>> tmp = 3
>>> del tmp
>>> tmp
Traceback (most recent call last):
  ...
NameError: name 'tmp' is not defined
```

Two names, one object

Because names are labels, `b = a` does not copy anything. It attaches a second label to the same object. For immutable objects (numbers, text) this is harmless. For mutable ones, it is a classic surprise.

BOSS FIGHT: aliasing a mutable object

Two names, one list. A change made through one name is visible through the other, because there is only one object.



`b = a`: one object `c = a.copy()`: a second object

```
>>> a = [1, 2]
>>> b = a
>>> b.append(3)
>>> a
[1, 2, 3]
>>> a is b
True
>>> c = a.copy()
>>> c.append(4)
>>> a, c
([1, 2, 3], [1, 2, 3, 4])
```

Ask yourself: "how many objects exist here?" If you want an independent copy, make one on purpose (`.copy()`); lists are covered in Chapter 8.

Name variables

Names follow hard rules and soft conventions.

Rule (enforced)	Valid	Invalid
letters, digits, underscore only	`max_tokens2`	`max-tokens`, `name!`
may not start with a digit	`_1st`, `first_1`	`1st_place`
case-sensitive (these are three names)	`Model`, `model`, `MODEL`	—
not a reserved word	`class_name`	`class`, `for`, `None`

```
>>> 1st_place = 1
Traceback (most recent call last):
  ...
SyntaxError: invalid decimal literal
>>> my name = 5
Traceback (most recent call last):
  ...
SyntaxError: invalid syntax
>>> import keyword
>>> len(keyword.kwlist)
35
>>> keyword.iskeyword("lambda")
True
```

Style	Example	Use it for
snake_case	`num_tokens`	variables, functions, modules
PascalCase	`ChatClient`	classes (Chapter 11)
UPPER_SNAKE	`MAX_RETRIES`	"constants": Python does not enforce them
leading underscore	`_cache`	"internal, please do not import"
double underscores both sides	`__init__`	special ("dunder") methods
camelCase	`numTokens`	not Python style (JavaScript, Java)

> ➤ **RULE**
> Name things for what they hold: `prompt_tokens`, not `x1`. Never use a built-in as a name (`list`, `str`, `id`, `type`); it hides the real one.

2.5 Python Types

Bits are meaningless until a type reads them. Python provides built-in types that match common hardware types, plus some software-only ones. The column that matters most for bugs is **mutable**: can the value be changed in place after it is created?

Name	Type	Mutable?	Example	Chapter
Boolean	`bool`	no	`True`, `False`	3
Integer	`int`	no	`47`, `25_000`	3
Float	`float`	no	`3.14`, `2.7e5`	3
Complex	`complex`	no	`3j`, `5 + 9j`	3
Text string	`str`	no	`"alas"`, `'alack'`	4
Bytes	`bytes`	no	`b'ab\xff'`	5
Bytearray	`bytearray`	**yes**	`bytearray(b'ab')`	5
Tuple	`tuple`	no	`(2, 4, 8)`	8
List	`list`	**yes**	`["a", "b"]`	8
Set	`set`	**yes**	`{3, 5, 7}`	9
Frozen set	`frozenset`	no	`frozenset([1, 2])`	9
Dictionary	`dict`	**yes**	`{"a": 1}`	9
Nothing	`NoneType`	no	`None`	here

The last row is not in the table of the book, but you will use it constantly: `None` means "no value here".

Two functions let you ask an object about its type:

```
>>> type(3.5)
<class 'float'>
>>> isinstance(3.5, float)
True
>>> isinstance(True, int)
True
```

(`bool` is a kind of `int` in Python, which Chapter 3 explains.)

TYPE CHECK: mutable or immutable?

Immutable means no operation can change the object in place; you can only make a new one. **Mutable** means it can change, and every name pointing at it sees the change.




```
>>> s = "abc"
>>> s[0] = "X"
Traceback (most recent call last):
  ...
TypeError: 'str' object does not support item assignment
>>> items = ["a", "b", "c"]
>>> before = id(items)
>>> items[0] = "X"
>>> items
['X', 'b', 'c']
>>> id(items) == before
True
```

TYPE CHECK: `None` and truthiness

`None` is the only value of type `NoneType`. It is falsy. Test for it with `is None`, and never call methods on a value that might be `None`.

```
>>> result = None
>>> result is None
True
>>> bool(result)
False
>>> result.upper()
Traceback (most recent call last):
  ...
AttributeError: 'NoneType' object has no attribute 'upper'
```

Type hints do not enforce types

You may annotate a name with a type. Python records the annotation but does not check it while running. Tools such as mypy or your editor check it before you run (Chapter 14).

```
>>> count: int = "five"
>>> count
'five'
```

2.6 Specify Values

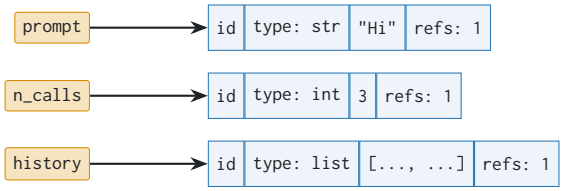
A **literal** is a value written directly in code: `5`, `"hi"`, `[1, 2]`. A **variable** is a name that refers to a value. Each type has its own literal syntax, and punctuation decides the type. Two cases catch beginners:

You write	You get	Why
`(4)`	`int`	parentheses only group; they do not make a tuple
`(4,)`	`tuple`	the comma makes the tuple
`{}`	`dict`	empty braces are a dict; use `set()` for an empty set
`{4}`	`set`	braces with items but no colons
`{"a": 4}`	`dict`	braces with `key: value` pairs

```
>>> type((4))
<class 'int'>
>>> type((4,))
<class 'tuple'>
>>> type({})
<class 'dict'>
>>> type({4})
<class 'set'>
```

2.7 Objects as Plastic Boxes in Memory

A good mental image: each object is a clear plastic box on a memory shelf. Some contents have a fixed size (id, type, count); the value part can be small or large. A list box can sit beside an int box. Your code lives on another shelf, and each variable is a sticky note stuck to a box.



2.8 Review

Variables are names; objects carry an id, a type, a value and a reference count. Assigning moves a name to another object. Mutable objects can change in place, so shared names can surprise you. Next: numbers, the simplest objects.

➤ **PRACTICE**

1. Why can the same name hold an `int` and, a moment later, a `str`?
2. After `a = [1]` and `b = a`, then `b.append(2)`, what is `a`? Why?
3. Which of these is not a valid variable name, and why: `top_p`, `2nd`, `Top_P`, `for`?
4. What is the difference between a mutable and an immutable type? Name two of each.
5. Why is `x is 1000` unreliable, and what should you write instead?
6. What type is `(7)`, and what must you add to make it a tuple?

★ **ACHIEVEMENT UNLOCKED: Name Tamer**

- Explain a variable as a name pointing at an object, with a type, id and reference count.
- Predict what happens to other names when a mutable object is changed.
- Choose valid, idiomatic names and spot illegal ones.

- Read the type table and say which types are mutable.
- Check types with `type()` and `isinstance()`, and test for `None` with `is`.

CHAPTER 03

Numbers

This chapter covers Python's simplest types: `bool`, `int` and `float`, plus two exact alternatives, `Fraction` and `Decimal`. You will learn what each operator returns, how Python converts between number types, and why floats can surprise you. Token counts, prices, batch sizes and probabilities are all numbers, so this is daily material.



When two different number types meet in one expression, Python converts the one on the left of the chain to the one on the right. The result type is the "wider" one.

3.1 Booleans

A `bool` has exactly two values: `True` and `False`. Both start with a capital letter. Write `true` and Python raises a `NameError`.

The function `bool(x)` converts *any* value to a boolean. This is the rule behind **truthiness**: in an `if`, Python asks "is this value truthy?". (Chapter 6 uses it.)

Falsy (counts as False)	Truthy (counts as True)
`False`, `None`	`True`
`0`, `0.0`, `0j`	any other number, including `-1` and `0.001`
`""` (empty string)	any non-empty string, even `"0"` and `"False"`
`[]`, `()`, `{}`, `set()` (empty containers)	any container with at least one item, even `[0]`

```
>>> bool(1), bool(-45), bool(0), bool(0.0)
(True, True, False, False)
>>> bool(""), bool("0"), bool("False")
(False, True, True)
>>> bool([]), bool([0]), bool(None)
(False, True, False)
```

> ⚠ **TRAP CARD**
> `bool("False")` is `True`, because the string is not empty. If you read a setting such as `DEBUG=False` from an environment variable, it arrives as text. Compare the text instead: `value.lower() == "true"`.

> ■ **TYPE CHECK: a `bool` is an `int`**
> `bool` is a subtype of `int`. `True` acts as 1 and `False` as 0 in arithmetic.

```
>>> True + True
2
>>> int(True), int(False)
(1, 0)
>>> isinstance(True, int)
True
>>> type(True + 1)
<class 'int'>
```

3.2 Integers

An `int` is a whole number with an optional sign. In Python 3 it has no fixed size limit.

Literal integers

```
>>> tokens = 1_500_000
>>> tokens
1500000
>>> -42, +42
(-42, 42)
>>> 1,500
(1, 500)
```

- Underscores are allowed as digit separators for readability. They are ignored.
- Do not write commas: `1,500` is a *tuple* of two ints, not fifteen hundred.
- A leading zero followed by digits (`007`) is a `SyntaxError`. Prefixes `0b`, `0o`, `0x` mark other bases (see below).

Integer operations

Op	Name	Example	Result	Result type
`+`	add	`5 + 8`	`13`	`int`
`-`	subtract	`90 - 10`	`80`	`int`
`*`	multiply	`4 * 7`	`28`	`int`
`/`	true division	`7 / 2`	`3.5`	**always `float`**
`//`	floor division	`7 // 2`	`3`	`int` (for int operands)
`%`	remainder	`7 % 3`	`1`	`int`
`**`	power	`3 ** 4`	`81`	`int` (`float` if the exponent is negative)

```
>>> 7 / 2
3.5
>>> 8 / 2
4.0
>>> 7 // 2
3
>>> 7 % 2
1
>>> 7 ** 2
49
>>> 2 ** -1
0.5
```

Quotient and remainder. `//` and `%` are a pair: `a == (a // b) * b + (a % b)`. `divmod(a, b)` returns both in a tuple. This is how you split work into batches.




```
>>> n_tokens, chunk_size = 10_500, 4_000
>>> divmod(n_tokens, chunk_size)
(2, 2500)
>>> -(-n_tokens // chunk_size)
3
```

The last line is a trick for *ceiling* division: how many chunks you need in total, counting the partial one. It works because `//` rounds down, as the next trap explains.

⚠ TRAP CARD

`//` rounds toward minus infinity (it *floors*), not toward zero. They differ for negatives. `int()` truncates toward zero.




```
>>> -7 // 2
-4
>>> -7 % 2
1
>>> divmod(-7, 2)
(-4, 1)
>>> int(-7 / 2)
-3
```

Division by zero raises `ZeroDivisionError` for both `/` and `//`.

```
>>> 5 / 0
Traceback (most recent call last):
  ...
ZeroDivisionError: division by zero
>>> 7 // 0
Traceback (most recent call last):
  ...
ZeroDivisionError: integer division or modulo by zero
```

Integers and variables

An operator on a variable does not change the variable unless you assign the result. The right side of = is computed first.

```
>>> budget = 1000
>>> used = 250
>>> budget - used
750
>>> budget
1000
>>> budget = budget - used
>>> budget
750
```

Shorthand operators combine the operation and the assignment:

Shorthand	Same as	
`x += 5`	`x = x + 5`	
`x -= 5`	`x = x - 5`	
`x *= 2`	`x = x * 2`	
`x /= 4`	`x = x / 4`	(x becomes a `float`)
`x //= 4`	`x = x // 4`	
`x %= 4`	`x = x % 4`	
`x **= 2`	`x = x ** 2`	

```
>>> budget = 750
>>> budget //= 4
>>> budget
187
>>> budget /= 2
>>> budget
93.5
```

Precedence

Python evaluates operators in a fixed order, tightest binding first. When in doubt, add parentheses.

Level	Operators	Note
1 (first)	`(...)`	parentheses always win
2	`**`	right to left: `2 ** 3 ** 2` is `2 ** 9`
3	unary `+x`, `-x`	`-3 ** 2` is `-(3 ** 2)`
4	`*`, `/`, `//`, `%`	left to right
5 (last)	`+`, `-`	left to right

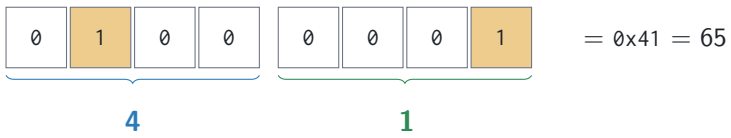
```
>>> 2 + 3 * 4
14
>>> (2 + 3) * 4
20
>>> -3 ** 2
-9
>>> (-3) ** 2
9
>>> 2 ** 3 ** 2
512
```

Bases

Integers are decimal by default. Prefixes write them in binary, octal or hexadecimal. The interpreter always prints the value in decimal. You may never need other bases in daily work, but you will see hex in colors, memory addresses and byte data.

Base	Prefix	Digits	Value of 10 in that base
2 (binary)	`0b`	0, 1	2
8 (octal)	`0o`	0–7	8
10 (decimal)	none	0–9	10
16 (hex)	`0x`	0–9, a–f	16

Hex is handy because one hex digit equals exactly four bits, so a byte is two hex digits.




```
>>> 0b101, 0o17, 0xff
(5, 15, 255)
>>> bin(10), oct(10), hex(255)
('0b1010', '0o12', '0xff')
>>> int("ff", 16), int("101", 2)
(255, 5)
>>> chr(72), ord("H")
('H', 72)
```

Note that `bin()`, `oct()` and `hex()` return *strings*, not ints. `chr()` turns a code number into a one-character string; `ord()` turns it back.

Type conversions

`int(x)` takes one argument and returns an `int`. What it accepts depends on the input type.

Input	Result	Rule
`int(True)`	`1`	bools become 1 or 0
`int(98.6)`	`98`	float: drops the fraction (toward zero)
`int("42")`	`42`	string of digits; sign, _ and surrounding spaces are fine
`int("10", 2)`	`2`	second argument gives the base of the text
`int("3.5")`	**ValueError**	a string with a decimal point is not accepted
`int("")`	**ValueError**	empty text
`int("1e3")`	**ValueError**	exponent notation is not accepted

```
>>> int(3.99), int(-3.99)
(3, -3)
>>> int("42"), int(" 42 "), int("1_000"), int("+7")
(42, 42, 1000, 7)
>>> int("3.5")
Traceback (most recent call last):
  ...
ValueError: invalid literal for int() with base 10: '3.5'
>>> int(float("3.5"))
3
```

> ➤ **RULE**
> User input and JSON values often arrive as strings. Convert at the edge (`int(text)`, `float(text)`) and catch `ValueError`. Do not pass text into arithmetic.

When types are mixed in arithmetic, Python promotes automatically:

Expression	Result type
`int + int`, `int * int`	`int`
`int + float`	`float`
`int / int`	`float`
`int // int`	`int`
`float // int`	`float` (value is still floored: `7.0 // 2` is `3.0`)
`bool + int`	`int`
`bool + float`	`float`

```
>>> 4 + 7.0
11.0
>>> True + 2.5
3.5
>>> 7.0 // 2
3.0
```

How big is an int?

Older languages limit integers to 32 or 64 bits and overflow past that. Python 3 integers grow as needed; only your memory limits them.

```
>>> 2 ** 100
1267650600228229401496703205376
>>> len(str(10 ** 50))
51
```

3.3 Floats

A `float` is a number with a fractional part, stored in binary.

```
>>> 5., 5e1, 2.5e-3, 1_000.5
(5.0, 50.0, 0.0025, 1000.5)
>>> float("3.14"), float(7), float(True)
(3.14, 7.0, 1.0)
>>> float("1e3")
1000.0
>>> float("abc")
Traceback (most recent call last):
  ...
ValueError: could not convert string to float: 'abc'
```

- Exponent form: `2.5e-3` means 2.5×10^{-3}.
- All the integer operators work on floats; `divmod` works too.
- `float()` accepts strings with a decimal point or exponent; `int()` does not.

Rounding

`round(x)` returns an `int`. `round(x, n)` keeps `n` decimal places and returns a `float`. Exact halves round to the *nearest even* number, so `round(2.5)` is 2, not 3.




```
>>> round(3.14159, 2)
3.14
>>> round(2.5), round(3.5), round(-2.5)
(2, 4, -2)
>>> type(round(3.7)), type(round(3.7, 1))
(<class 'int'>, <class 'float'>)
```

Special float values

Floats also include `inf` (overflow) and `nan` ("not a number"). `nan` is not equal to anything, not even itself, so test it with `math.isnan`. NaNs can sneak into data pipelines and silently poison averages.

```
>>> 1e308 * 10
inf
>>> import math
>>> nan = float("nan")
>>> nan == nan
False
>>> math.isnan(nan)
True
```

3.4 Floats Are Not Exact

Binary storage cannot hold most decimal fractions exactly, just as decimal cannot hold $\frac{1}{3}$. The tiny error is small but real.

```
>>> 1 / 3
0.3333333333333333
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
>>> f"{0.1:.20f}"
'0.10000000000000000555'
```

> ➤ **RULE**
>
> Never compare floats with `==`. Use `math.isclose(a, b)`, which allows a small tolerance. Python's standard library offers exact alternatives for when it matters.
>
> ```
> >>> import math
> >>> math.isclose(0.1 + 0.2, 0.3)
> True
> ```

Fractions

`fractions.Fraction` stores a numerator and a denominator, so it is exact for ratios.

```
>>> from fractions import Fraction
>>> Fraction(1, 3) + Fraction(1, 6)
Fraction(1, 2)
>>> Fraction("0.1") + Fraction("0.2") == Fraction("0.3")
True
>>> float(Fraction(1, 8))
0.125
```

Decimals

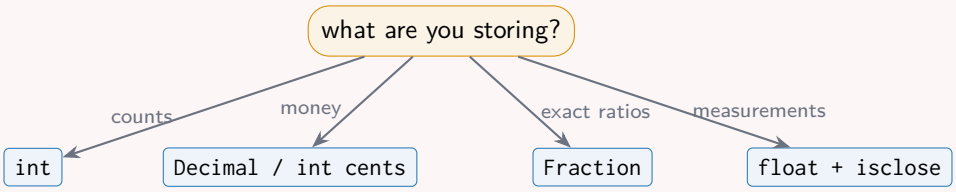
`decimal.Decimal` stores decimal digits exactly. Use it for money. Always build it from a *string*; building it from a float copies the float's error.

```
>>> from decimal import Decimal
>>> Decimal("0.10") + Decimal("0.20")
Decimal('0.30')
>>> Decimal(0.1) == Decimal("0.1")
False
```

Another common approach is to count in the smallest unit as an `int` (cents, or millionths of a dollar) and convert only for display.

```
>>> price_cents, qty = 1999, 3
>>> total_cents = price_cents * qty
>>> f"${total_cents / 100:.2f}"
'$59.97'
```

> ■ **BOSS FIGHT: choose the right number type**
>
> Floats are right for measurements and model scores; they are wrong for money and for exact equality. Ask what you are counting before you pick a type.



A worked example. The prices are invented for the demo.

```
>>> from decimal import Decimal
>>> tokens_in, tokens_out = 12_000, 800
>>> rate_in, rate_out = Decimal("3.00"), Decimal("15.00")
>>> cost = (tokens_in * rate_in + tokens_out * rate_out) / 1_000_000
>>> cost
Decimal('0.048')
```

Rates are dollars per million tokens. The `int` times `Decimal` multiplication is allowed; mixing `Decimal` with `float` is not.

3.5 Math Functions

A few functions are always available (`abs`, `min`, `max`, `round`, `divmod`); the rest live in the `math` module (Chapter 25 covers more). Complex numbers also exist, written with a `j`.

```
>>> import math
>>> abs(-3), min(2, 9), max(2, 9)
(3, 2, 9)
>>> math.floor(2.7), math.ceil(2.1), math.sqrt(2)
(2, 3, 1.4142135623730951)
>>> abs(3 + 4j)
5.0
```

3.6 Review

Booleans are tiny integers with a truthiness rule behind them. Integers have unlimited size. `/` always gives a float, `//` floors. Floats are approximate: compare with a tolerance and use `Decimal` or integer cents for money. Next: text.

➤ **PRACTICE**

1. What type does `9 / 3` return, and what does `9 // 3` return?
2. Predict `-9 // 4`, `-9 % 4` and `int(-9 / 4)`, then check them.
3. Why is `bool("0")` true while `bool(0)` is false?
4. You have 1 234 documents and send them 100 at a time. Write one expression for the number of batches.
5. Why does `0.1 + 0.2 == 0.3` fail, and what do you write instead?
6. Which conversions raise `ValueError`: `int("12")`, `int("12.0")`, `float("12.0")`, `int(" 12 ")`?

★ **ACHIEVEMENT UNLOCKED: Number Cruncher**

- Predict the type and value of any expression with `+ - * / // % **`.
- Split totals into batches with `divmod`, and know that `//` floors.
- Convert between `int`, `float`, `bool` and `str` and handle `ValueError`.
- Explain float inexactness and choose `isclose`, `Decimal`, `Fraction` or integer cents.
- Use truthiness rules without guessing.

CHAPTER 04

Strings

Most programs spend more time on text than on math, and AI work is text work: prompts, model replies, file names, JSON. This chapter covers how to create strings, pick them apart with indexes and slices, search and clean them, and build new ones with f-strings. Everything here applies to prompts you will format and replies you will clean.

4.1 Strings Are Sequences

A `str` is an ordered **sequence** of characters. A character is the smallest unit of writing: a letter, digit, symbol, space or newline. Python has no separate character type. A single character is just a string of length 1, and a string may be empty.

Strings are **immutable**: no operation edits a string in place. Every operation that seems to change one returns a *new* string. This is the most important fact in the chapter.

Every sequence (strings, and later lists and tuples) supports the same core operations:

Operation	Example	Returns	Meaning
membership	`"fun" in s`	`bool`	is it present?
position of a value	`s.index("x")`	`int`	offset of first match
one element	`s[0]`	`str` (length 1)	item at an offset
slice	`s[2:5]`	`str`	a range of items
length	`len(s)`	`int`	number of items
smallest, largest	`min(s)`, `max(s)`	`str` (length 1)	by character code

```
>>> prompt = "Summarize this text"
>>> "text" in prompt
True
>>> prompt.index("this")
10
>>> prompt[0]
'S'
>>> prompt[10:14]
'this'
>>> len(prompt)
19
>>> min("model"), max("model")
('d', 'o')
```

4.2 Create with Quotes

Wrap characters in matching single or double quotes. Python treats both the same. The interpreter echoes strings with single quotes.

```
>>> 'single' == "double"
False
>>> 'tokens' == "tokens"
True
>>> "It's fine"
"It's fine"
>>> 'She said "hi"'
'She said "hi"'
```

Having two quote styles lets you put one kind of quote inside the other. Triple quotes (''' or """) create *multi-line* strings and keep the line breaks. That is the usual way to write a long prompt.

```
>>> text = """line one
... line two"""
>>> text
'line one\nline two'
>>> print(text)
line one
line two
```

Echo versus print. The interpreter echo shows the string's representation, with quotes and escape codes like `\n`. `print()` shows the text itself, separates its arguments with spaces and ends with a newline.

```
>>> print("a", "b", 3)
a b 3
>>> len(""), bool("")
(0, False)
```

Prefixes. A letter before the opening quote changes how the string is read.

Prefix	Kind	Use
none	plain text (`str`)	most strings
`f`	f-string	insert values: `f"hi {name}"`
`r`	raw string	keep backslashes literally (paths, regular expressions)
`fr`	raw f-string	both
`b`	`bytes`, not `str`	binary data (Chapter 5)
`u`	same as plain	historical; you can ignore it

4.3 Create with str()

`str(x)` converts any object to its text form. `print()` and f-strings do this for you. `+` does *not*: mixing `str` and `int` with `+` is a `TypeError`.

```
>>> str(42), str(3.5), str(True), str(None)
('42', '3.5', 'True', 'None')
>>> str([1, 2])
'[1, 2]'
>>> "tokens: " + 42
Traceback (most recent call last):
  ...
TypeError: can only concatenate str (not "int") to str
>>> "tokens: " + str(42)
'tokens: 42'
```

4.4 Escape with \

A backslash gives the next character a special meaning (an **escape sequence**), or removes its special meaning.

Write	You get
`\n`	newline
`\t`	tab
`\\`	one backslash
`\'` and `\"`	a quote character

```
>>> "a\tb"
'a\tb'
>>> print("line1\nline2")
line1
line2
>>> print('C:\\logs\\new')
C:\logs\new
```

A **raw string** (`r"..."`) turns escapes off. It does not affect real line breaks inside triple quotes.

```
>>> print(r"C:\logs\new")
C:\logs\new
>>> len(r"\n"), len("\n")
(2, 1)
```

> ⚠ **TRAP CARD**
> On Windows a path like `"C:\new"` contains a newline, because `\n` is an escape. Use a raw string, double the backslashes, or use forward slashes.
>
> ```
> >>> print("C:\new")
> C:
> ew
> ```

4.5 Combine with + and Duplicate with *

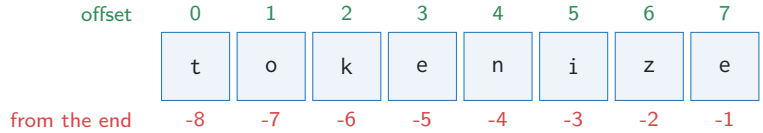
`+` joins two strings; `*` repeats a string by an `int`. Python adds no spaces. Two string *literals* next to each other are joined automatically, which is handy for splitting a long string over several lines inside parentheses.

```
>>> "ab" + "cd"
'abcd'
>>> "ab" "cd"
'abcd'
>>> "-" * 10
'----------'
>>> "ha" * 3 + "!"
'hahaha!'
>>> "5" + "5", "5" * 3
('55', '555')
>>> prompt = ("You are a helpful assistant. "
...           "Answer in one sentence.")
>>> prompt
'You are a helpful assistant. Answer in one sentence.'
```

`*` binds tighter than `+`, so `"ha" * 3 + "!"` repeats first.

4.6 Get a Character by [offset]

Offsets start at 0 on the left. Negative offsets count from the right, so `-1` is the last character.




```
>>> word = "tokenize"
>>> word[0], word[3], word[-1], word[-2]
('t', 'e', 'e', 'z')
>>> word[8]
Traceback (most recent call last):
  ...
IndexError: string index out of range
```

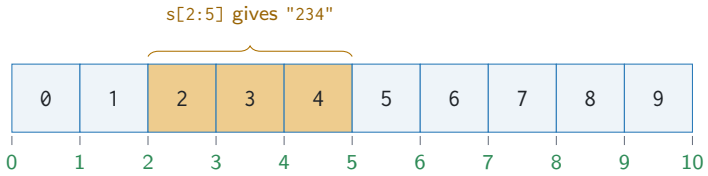
An offset past the end raises `IndexError`. You cannot assign to an offset, because strings are immutable. Build a new string instead:

```
>>> word = "tokenize"
>>> word[0] = "T"
Traceback (most recent call last):
  ...
TypeError: 'str' object does not support item assignment
>>> "T" + word[1:]
'Tokenize'
>>> word.replace("t", "T", 1)
'Tokenize'
>>> word
'tokenize'
```

> ■ **TYPE CHECK: what indexing returns**
>
> `s[i]` returns a `str` of length 1, never a separate char type. `s[i]` needs an `int`; a `str` offset such as `s["a"]` is a `TypeError`. Compare with a dict, where the subscript is a key, not a position.

4.7 Get a Substring with a Slice

`s[start:stop:step]` returns a new string. Think of the offsets as numbers on the *lines between characters*. `start` is included; `stop` is excluded. Omitted parts default to the beginning, the end, and step 1.




```
>>> d = "0123456789"
>>> d[2:5], d[:3], d[7:], d[:]
('234', '012', '789', '0123456789')
>>> d[-3:], d[:-3], d[-5:-2]
('789', '0123456', '567')
>>> d[::2], d[1::2], d[2:9:3]
('02468', '13579', '258')
>>> d[::-1], d[8:2:-2]
('9876543210', '864')
```

Slice	Result for `d`	Reading it
`d[2:5]`	`'234'`	from cut 2 up to (not including) cut 5
`d[:3]`	`'012'`	first three characters
`d[7:]`	`'789'`	from cut 7 to the end
`d[-3:]`	`'789'`	last three characters
`d[:-3]`	`'0123456'`	everything except the last three
`d[::2]`	`'02468'`	every second character
`d[::-1]`	`'9876543210'`	reversed

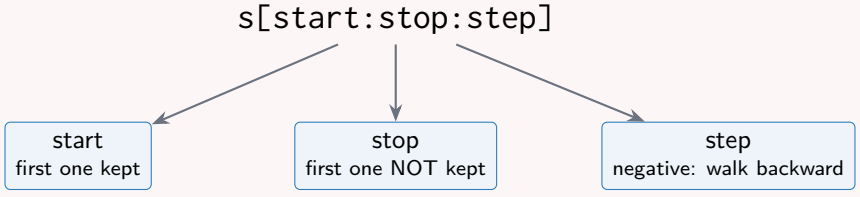
Slices are forgiving. Out-of-range offsets are clamped instead of raising an error. A slice that selects nothing gives the empty string. Indexing (`d[10]`) is strict; slicing is not.

```
>>> d = "0123456789"
>>> d[-50:3], d[7:99], d[20:30], d[5:2]
('012', '789', '', '')
>>> d[10]
Traceback (most recent call last):
  ...
IndexError: string index out of range
```

■ BOSS FIGHT: slices and cut lines

Most slice mistakes come from counting characters instead of cut lines. Rules that always hold:

- `s[a:b]` has length `b - a` when both are in range.
- `s[:n] + s[n:] == s` for any `n`. The two halves meet at one cut.
- With a negative step, start and stop swap roles: you walk backward from `start` down to, but not including, `stop`.
- Prefer simple forms: `s[:n]`, `s[-n:]`, `s[::-1]`.



```
>>> d = "0123456789"
>>> d[:4] + d[4:] == d
True
>>> len(d[3:8])
5
>>> d[:3] + "!" + d[-2:]
'012!89'
```

4.8 Get Length with len()

`len(s)` returns an `int`: the number of characters, including spaces and newlines. The last valid offset is `len(s) - 1`. (Token counts are not character counts; models read tokens, which are chunks of characters.)

```
>>> len("hello world"), len("a\nb"), len("")
(11, 3, 0)
```

4.9 Methods: Functions That Belong to a String

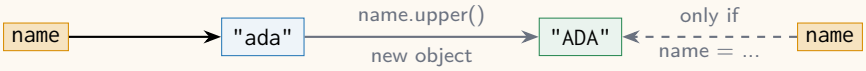
Many string operations are **methods**, called with a dot: `s.method(args)`. The dot (attribute access) is different from the brackets used for indexing. A method *never* changes `s`; it returns a result.

■ **TYPE CHECK: what the common methods return**

Methods	Returns	Examples
transforming	new `str`	`upper lower strip replace title center`
splitting	`list[str]`	`split splitlines`
joining	new `str`	`sep.join(list)`
counting, finding	`int`	`find index count`
testing	`bool`	`startswith endswith isdigit isalpha isalnum`

⚠ **TRAP CARD**

Forgetting to keep the result is the most common beginner bug with strings. Calling `name.upper()` alone computes a new string and throws it away. `name` is untouched.




```
>>> name = "ada"
>>> name.upper()
'ADA'
>>> name
'ada'
>>> name = name.upper()
>>> name
'ADA'
```

Split with split()

`s.split(sep)` cuts a string at each separator and returns a `list` of strings. With no argument it splits on any run of whitespace and drops empty pieces. A second argument limits the number of cuts.

```
>>> line = "role:user,content:hello,tokens:5"
>>> line.split(",")
['role:user', 'content:hello', 'tokens:5']
>>> line.split(",", 1)
['role:user', 'content:hello,tokens:5']
>>> "a  b\tc\nd".split()
['a', 'b', 'c', 'd']
>>> "a,,b".split(",")
['a', '', 'b']
>>> "".split(","), "".split()
([''], [])
>>> "a\nb\nc".splitlines()
['a', 'b', 'c']
```

Combine with join()

`join` is the reverse of `split`. It looks backward: the *separator* is the object, and the list is the argument: `separator.join(list)`. All items must already be strings.




```
>>> ", ".join(["one", "two", "three"])
'one, two, three'
>>> "\n".join(["l1", "l2"])
'l1\nl2'
>>> ",".join([str(1), str(2)])
'1,2'
>>> ",".join([1, 2])
Traceback (most recent call last):
  ...
TypeError: sequence item 0: expected str instance, int found
```

> ➤ **RULE**
> To build one string from many pieces, collect the pieces in a list and call `"".join(pieces)` once. Do not grow a string with `+` inside a loop.

Substitute with replace()

`s.replace(old, new)` returns a copy with every occurrence replaced. A third argument limits the count. It matches raw text, so it can hit the middle of words.

```
>>> s = "the cat sat on the mat"
>>> s.replace("the", "a")
'a cat sat on a mat'
>>> s.replace("the", "a", 1)
'a cat sat on the mat'
>>> s.replace("at", "og")
'the cog sog on the mog'
>>> s
'the cat sat on the mat'
```

For whole-word or pattern replacement you need regular expressions (Chapter 17).

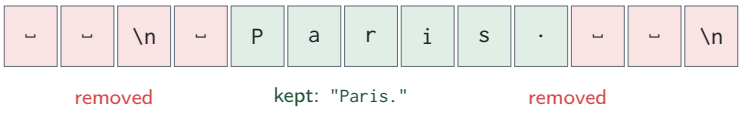
Prefixes and suffixes

```
>>> f = "report_final.json"
>>> f.startswith("report"), f.endswith(".json")
(True, True)
>>> f.startswith(("a", "r"))
True
>>> f.removesuffix(".json")
'report_final'
>>> f.removeprefix("report_")
'final.json'
>>> f.removeprefix("zzz")
'report_final.json'
```

`startswith` and `endswith` accept a tuple of alternatives. `removeprefix` and `removesuffix` (Python 3.9+) do nothing if the text is absent. To add a prefix or suffix, use `+`.

Strip with strip()

`strip()` removes characters from the *ends*. With no argument it removes whitespace (spaces, tabs, newlines). `lstrip()` works on the left end only, `rstrip()` on the right. Given an argument, it treats it as a *set of characters*, not a substring.




```
>>> raw = "  \n  Paris.  \n"
>>> raw.strip()
'Paris.'
>>> raw.lstrip()
'Paris.  \n'
>>> raw.rstrip()
'  \n  Paris.'
>>> "***hi***".strip("*")
'hi'
>>> "..ok?!".strip(".?!")
'ok'
>>> "xxabcxx".strip("xa")
'bc'
```

A common job is cleaning a model reply that wraps JSON in a code fence:

```
>>> reply = "```json\n{\"ok\": true}\n```"
>>> reply.removeprefix("```json").removesuffix("```").strip()
'{"ok": true}'
```

Chaining methods works because each one returns a new string for the next to use.

Search and select

Call	Returns	If not found
`s.find(x)`	offset (`int`) of the first match	`-1`
`s.rfind(x)`	offset of the last match	`-1`
`s.index(x)`, `s.rindex(x)`	same offsets	raises `ValueError`
`s.count(x)`	number of non-overlapping matches	`0`
`x in s`	`True` or `False`	`False`

```
>>> t = "to be or not to be"
>>> t.find("be"), t.rfind("be"), t.find("zz")
(3, 16, -1)
>>> t.index("or")
6
>>> t.index("zz")
Traceback (most recent call last):
  ...
ValueError: substring not found
>>> t.count("to"), t.count("o")
(2, 4)
>>> t.isalnum(), "abc123".isalnum(), "12".isdigit(), "abc".isalpha()
(False, True, True, True)
```

> ⚠ **TRAP CARD**
>
> `find` returns `-1` (truthy) when the text is missing and `0` (falsy) when it is at the start. So `if s.find("x"):` is wrong. To ask "is it there?", use `"x" in s`.
>
> ```
> >>> "abc".find("a"), bool("abc".find("a"))
> (0, False)
> >>> "abc".find("z"), bool("abc".find("z"))
> (-1, True)
> >>> "a" in "abc", "z" in "abc"
> (True, False)
> ```

Change case

```
>>> m = "hello, World"
>>> m.upper(), m.lower()
('HELLO, WORLD', 'hello, world')
>>> m.capitalize(), m.title(), m.swapcase()
('Hello, world', 'Hello, World', 'HELLO, wORLD')
>>> "Ab".lower() == "aB".lower()
True
```

To compare text ignoring case, lowercase both sides first.

Set alignment

`center`, `ljust`, `rjust` pad a string to a total width; an optional second argument sets the pad character. `zfill` pads with zeros on the left.

```
>>> "ok".center(10, "*")
'****ok****'
>>> "ok".ljust(6, ".") + "|"
'ok....|'
>>> "ok".rjust(6) + "|"
'    ok|'
>>> "7".zfill(3)
'007'
```

4.10 Apply Formatting

Building text from values is called **interpolation**. Python has three styles. Know all three so you can read old code, but write f-strings.

Style	Looks like	Notes
Old (`%`)	`"%s: %d" % (name, n)`	C-style; values go in a tuple after `%`
New (`.format`)	`"{}: {}".format(name, n)`	placeholders `{}`, `{0}`, `{name}`
f-string	`f"{name}: {n}"`	Python 3.6+; any expression inside braces

Old style: %

The format is `template % values`. A single value can go alone; several must be in a tuple. Each `%` code has a type: `%s` text (works for anything), `%d` integer, `%f` float, `%x` hex, `%e` exponent, `%%` a literal percent sign.

```
>>> "%s has %d tokens" % ("prompt", 42)
'prompt has 42 tokens'
>>> "%.2f" % 3.14159
'3.14'
>>> "%5d|%-5d|" % (7, 7)
'    7|7    |'
>>> "%d%%" % 95
'95%'
>>> "%x" % 255
'ff'
```

New style: format()

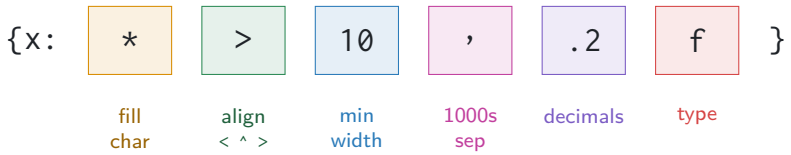
```
>>> "{} has {} tokens".format("prompt", 42)
'prompt has 42 tokens'
>>> "{1} before {0}".format("a", "b")
'b before a'
>>> "{name} costs {cost:.2f}".format(name="draft", cost=1.5)
'draft costs 1.50'
>>> "{0[model]}".format({"model": "demo-1"})
'demo-1'
```

Newest style: f-strings

Put `f` before the quote and write values or whole expressions inside braces.

```
>>> model, n = "demo-1", 1234567
>>> f"{model} saw {n} tokens"
'demo-1 saw 1234567 tokens'
>>> f"{model.upper()} has {len(model)} characters"
'DEMO-1 has 6 characters'
```

After a colon comes a **format spec**, a tiny language shared by `format()` and f-strings. Its parts, in order, are all optional:




```
>>> model, n = "demo-1", 1234567
>>> f"{1234.5:*>10,.2f}"
'**1,234.50'
>>> f"{n:,}"
'1,234,567'
>>> f"{0.4567:.1%}"
'45.7%'
>>> f"{model:>10}|{model:<10}|{model:^10}|"
'    demo-1|demo-1    |  demo-1  |'
>>> f"{7:03d}"
'007'
>>> f"{255:#x} {5:08b}"
'0xff 00000101'
```

Spec	Meaning	f"{x:SPEC}" for x = 3.14159
.2f	2 decimals	3.14
8.2f	width 8, 2 decimals	' 3.14'
<8.2f	left-aligned	'3.14 '
08.2f	zero padded	00003.14
.2e	scientific	3.14e+00
.0%	percent	314%

Useful extras: `=` for debugging, `!r` for the `repr`, a nested field for a dynamic width, and doubled braces for literal braces.

```
>>> model, n = "demo-1", 1234567
>>> f"{model=}"
"model='demo-1'"
>>> f"{n * 2 = }"
'n * 2 = 2469134'
>>> f"{model!r}"
"'demo-1'"
>>> width = 8
>>> f"{model:>{width}}"
'  demo-1'
>>> f'{{"model": "{model}"}}'
'{"model": "demo-1"}'
```

⚠ TRAP CARD

Braces are special in f-strings. To write a literal `{` or `}`, double it. This matters when you build JSON text or a prompt that contains braces. A single `{` that you meant literally makes Python try to evaluate what follows it. For real JSON, prefer `json.dumps(data)`.

```
>>> topic = "tokenizers"
>>> prompt = f"""Explain {topic}.
... Use two sentences."""
>>> prompt
'Explain tokenizers.\nUse two sentences.'
```

➤ RULE

Use f-strings for new code. Never build prompts by gluing user text into a template with `+` and no check: the text may contain braces, quotes or instructions. Keep the template fixed and insert values deliberately.

4.11 More String Things

Python has many more string methods (`partition`, `casefold`, `encode`, `isupper`, `...`). The standard documentation lists them all. Chapter 17 adds Unicode and regular expressions.

4.12 Review

A string is an immutable sequence of characters. Index it with `[i]`, cut it with `[start:stop:step]`, and remember that methods return new strings. Use `split` and `join` to move between text and lists, `in` to ask whether something is present, and f-strings to build text.

```
>>> s1 = "abcdef"
>>> s1[:2] + "!" + s1[-2:]
'ab!ef'
```

➤ PRACTICE

1. What do `"python"[1:4]`, `"python"[-3:]` and `"python"[::-1]` return? Check by hand first.
2. Why does `word[0] = "T"` fail, and what are two ways to get `"Tokenize"` from `"tokenize"`?
3. A reply contains `" Yes.\n"`. Which method returns just `"Yes."`? What does the original value look like afterwards?
4. Why is `if s.find("key"):` a bug? What is the fix?
5. Turn `["a", "b", "c"]` into `"a-b-c"`, and back into a list.
6. Write an f-string that prints the float `0.8765` as a percentage with one decimal, right-aligned in 8 characters.

★ ACHIEVEMENT UNLOCKED: Text Wrangler

- Create strings with quotes, triple quotes, raw strings and f-strings, and explain echo versus print.
- Index and slice any string and predict the result from the cut-line picture.
- Explain why strings are immutable and keep the result of every method.
- Clean and search text with `strip`, `split`, `join`, `replace`, `in` and `find`.
- Format numbers and layouts with the format-spec mini-language.

CHAPTER 05

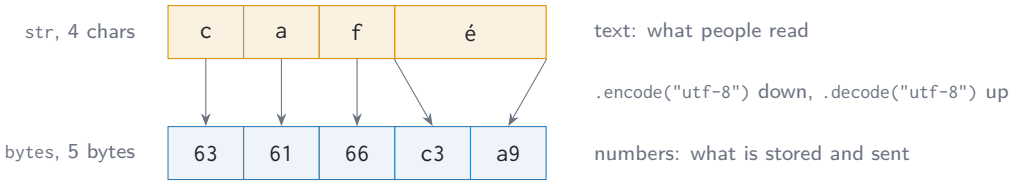
Bytes and Bytearray

What this chapter gives you: the two types Python uses for raw binary data. `bytes` is read-only and `bytearray` can be edited in place. You will meet them every time you send an image to an API, read a streamed response, hash a prompt, or look at what a tokenizer really sees.

5.1 Bytes

A `bytes` object is a fixed row of small integers, each from 0 to 255. Think of beads on a wire: all the same size, no meaning attached. A `str` is different: it is a row of characters. Python never mixes the two on its own. You convert on purpose.

> ★ **KEY IDEA: a byte is just a number**
>
> A byte holds one integer in the range 0–255. Python *prints* some bytes as letters (`b"A"`) because that is easier to read. The data is still the number 65.



One character can take more than one byte. That is why `len(text)` and `len(data)` can differ.

Create with quotes

Put a `b` in front of the quote. Only plain ASCII characters are allowed inside. For any other value, write two hex digits after a backslash-x.

```
>>> raw = b"GET\x00\x01\xff"
>>> raw
b'GET\x00\x01\xff'
>>> type(raw)
<class 'bytes'>
>>> len(raw)
6
>>> b"\x41\x42\x43"
b'ABC'
```

Create with `bytes()`

Give `bytes()` an iterable of integers (a list, a tuple, a `range`), or a single integer.

```
>>> bytes([72, 105, 33])
b'Hi!'
>>> bytes(3)
b'\x00\x00\x00'
>>> len(bytes(range(256)))
256
>>> bytes([256])
Traceback (most recent call last):
  ...
ValueError: bytes must be in range(0, 256)
```

> ⚠ **TRAP CARD: `bytes(3)` is not `b"3"`**
>
> A single integer means *length*, not value. `bytes(3)` gives three zero bytes. To get one byte whose value is 3, pass a list: `bytes([3])`.

Create from a hex string

`bytes.fromhex()` reads pairs of hex digits and ignores spaces between the pairs. A lone digit is an error. It is a *class method*: you call it on `bytes` itself, not on a value (Chapter 11). `.hex()` goes the other way.

```
>>> bytes.fromhex("48 69 21")
b'Hi!'
>>> bytes.fromhex("4 8")
Traceback (most recent call last):
  ...
ValueError: non-hexadecimal number found in fromhex() arg at position 1
>>> b"Hi!".hex()
'486921'
```

Encode and decode

Text becomes bytes with `.encode()`. Bytes become text with `.decode()`. You must name the encoding; use `"utf-8"` unless you have a reason not to. Chapter 17 covers the details.

```
>>> text = "caf\xe9"
>>> data = text.encode("utf-8")
>>> data
b'caf\xc3\xa9'
>>> len(text), len(data)
(4, 5)
>>> data.decode("utf-8") == text
True
>>> data.decode("ascii")
Traceback (most recent call last):
  ...
UnicodeDecodeError: 'ascii' codec can't decode byte 0xc3 in position 3: ...
```

> ➤ **RULE**
>
> Always pass the encoding by name and decode with the same one you encoded with. Text on the wire (JSON, HTTP bodies, files) is almost always UTF-8.

Index, slice, combine, repeat

Bytes are a sequence, so `[ ]`, slices, `+`, `*`, `in`, and `for` all work. The catch is what you get back.

■ **TYPE CHECK: indexing vs slicing**

Expression	Result	Type	Note
`b[2]`	`97`	`int`	one position gives a *number*
`b[2:3]`	`b"a"`	`bytes`	a slice always gives `bytes`
`b + b`	joined	`bytes`	new object, both sides must be `bytes`
`b * 2`	repeated	`bytes`	new object
`for x in b`	`x`	`int`	the loop walks numbers
`97 in b`	`True`	`bool`	an int tests for one byte
`b"a" in b`	`True`	`bool`	bytes test for a run of bytes

```
>>> b = b"\xff\x01abcdef"
>>> b[2]
97
>>> chr(b[2])
'a'
>>> b[2:3]
b'a'
>>> b[-3:]
b'def'
>>> b[0:2] + b"!!"
b'\xff\x01!!'
>>> b"ab" * 2
b'abab'
>>> list(b"Hi")
[72, 105]
>>> b"abc" == "abc"
False
>>> b"abc" + "def"
Traceback (most recent call last):
  ...
TypeError: can't concat str to bytes
```

A useful real-world check is the first few bytes of a file, its “magic number”:

```
>>> data = b"%PDF-1.7 ...rest of file..."
>>> data[:4] == b"%PDF"
True
>>> data.startswith(b"%PDF")
True
```

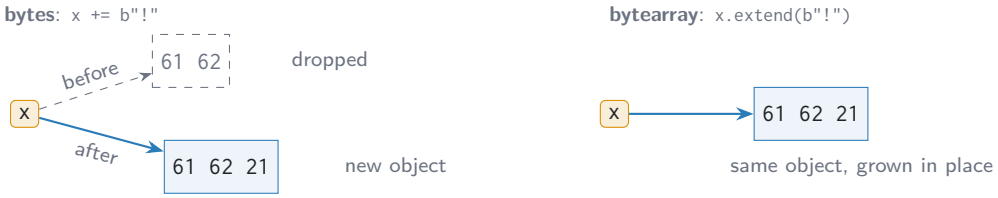
Bytes are immutable

You cannot assign to a position. `b1 += b2` looks like an edit, but Python builds a *new* bytes object and moves the name `b1` onto it. The old one is dropped.

```
>>> data = bytes([1, 2, 3])
>>> data[0] = 9
Traceback (most recent call last):
  ...
TypeError: 'bytes' object does not support item assignment
>>> before = data
>>> data += bytes([4])
>>> data
b'\x01\x02\x03\x04'
>>> before
b'\x01\x02\x03'
>>> data is before
False
```

5.2 Bytearray

bytearray is the editable twin of bytes. Every read operation works the same. On top of that you can change values, change the length, insert, and delete. Use it when you build binary data piece by piece.



Create with bytearray()

```
>>> bytearray()
bytearray(b'')
>>> bytearray([1, 2, 3, 255])
bytearray(b'\x01\x02\x03\xff')
>>> bytearray(4)
bytearray(b'\x00\x00\x00\x00')
>>> bytearray("abc", "ascii")
bytearray(b'abc')
>>> bytearray(b"from bytes")
bytearray(b'from bytes')
```

With a string you must give the encoding as the second argument. bytearray("abc") alone is a TypeError.

Read: index and slice

Same rules as bytes, with one difference. A slice of a bytearray is a bytearray.

```
>>> ba = bytearray(b"\xff\x01abcdef")
>>> ba[2]
97
>>> ba[4:6]
bytearray(b'cd')
```

Change one byte or many

```
>>> ba = bytearray([1, 2, 3, 255])
>>> ba[1] = 127
>>> ba
bytearray(b'\x01\x7f\x03\xff')
>>> ba = bytearray(b"\xff\x01abcdef")
>>> ba[2:5] = b"oops"
>>> ba
bytearray(b'\xff\x01oopsdef')
>>> ba[1] = 300
Traceback (most recent call last):
  ...
ValueError: byte must be in range(0, 256)
```

The slice assignment above replaced three bytes with four, so the array grew. Strings cannot do this.

`insert()`, `append()`, `extend()`

```
>>> ba = bytearray(b"abc")
>>> ba.insert(0, 2)
>>> ba.append(4)
>>> ba.extend(b"!!!")
>>> ba
bytearray(b'\x02abc\x04!!!')
>>> ba.append(b"\x04")
Traceback (most recent call last):
  ...
TypeError: 'bytes' object cannot be interpreted as an integer
```

■ **TYPE CHECK: what each method wants**

Method	Argument	Returns	Effect
`ba.append(n)`	one `int` 0–255	`None`	adds one byte at the end
`ba.extend(bs)`	bytes-like or ints	`None`	adds many bytes at the end
`ba.insert(i, n)`	index, `int`	`None`	adds one byte before position `i`
`ba.pop()`	optional index	`int`	removes and returns one byte
`ba.replace(a, b)`	two bytes-like	`bytearray`	a *new* array; `ba` is unchanged

⚠ **TRAP CARD: `replace()` does not edit in place**

On a `bytearray`, `.replace()` returns a changed *copy*, like `str.replace()`. If you ignore the return value, nothing happens.

```
>>> ba = bytearray(b"\xff\x01abcdef")
>>> new = ba.replace(b"\x01ab", b"XYZ")
>>> new
bytearray(b'\xffXYZcdef')
>>> ba
bytearray(b'\xff\x01abcdef')
```

Combine, repeat, freeze

```
>>> bytearray(b"\x01\x02") + bytearray(b"\x03\x04")
bytearray(b'\x01\x02\x03\x04')
>>> bytearray(b"\x01\x02") * 2
bytearray(b'\x01\x02\x01\x02')
>>> frozen = bytes(bytearray(b"hi"))
>>> type(frozen)
<class 'bytes'>
>>> hash(bytearray(b"hi"))
Traceback (most recent call last):
  ...
TypeError: unhashable type: 'bytearray'
```

`bytes(some_bytearray)` makes an immutable copy. Because a `bytearray` can change, it is unhashable, so it cannot be a dictionary key or a set member (Chapter 9). A `bytes` can.

■ **TYPE CHECK: bytes vs bytearray vs str**

	mutable	hashable	`x[0]` gives	slice gives
bytes	✗	✓	int	bytes
bytearray	✓	✗	int	bytearray
str	✗	✓	str	str

5.3 Where bytes show up in AI work

You will not write much byte-level code, but the types appear at every boundary where data leaves your program.

Situation	What you hold
Reading an image or PDF to send to a vision model	`bytes` from `open(path, "rb")`
Streaming reply arrives in pieces	`bytes` chunks, collected in a `bytearray`
Sending an image as JSON	`base64` turns `bytes` into `bytes` of ASCII, then `.decode()` to `str`
Cache key for a prompt	`hashlib` takes `bytes`, so `.encode()` first
Byte-level tokenizers	the model sees UTF-8 bytes, so one emoji can be several tokens

Two short examples, both from the standard library:

```
>>> import base64, hashlib
>>> base64.b64encode(b"Hi!")
b'SGkh'
>>> base64.b64encode(b"Hi!").decode("ascii")
'SGkh'
>>> hashlib.sha256(b"prompt").hexdigest()[:8]
'cf07194e'
```

Streamed replies can split a multi-byte character across two chunks. Collect the raw bytes first and decode once at the end:

```
chunks = [b'{"name": "caf\xc3', b'\xa9", "tokens": 4}']
buf = bytearray()
for chunk in chunks:
    buf.extend(chunk)
print(len(buf), "bytes")
print(buf.decode("utf-8") == '{"name": "caf\xe9", "tokens": 4}')
```

```
30 bytes
True
```

Decoding each chunk alone would fail, because the two halves of `c3 a9` are in different chunks.

■ **BOSS FIGHT: bytes and str look alike but are not**

The same-looking value behaves differently. `b"abc"[0]` is the number 97. `"abc"[0]` is the string `"a"`. `b"abc" == "abc"` is `False`, and `+` between them is a `TypeError`.

Strategy: ask "is this data text or raw bytes?" at every boundary. Decode as soon as bytes come in and encode as late as possible before they leave. Inside your program, work with `str`.

➤ **PRACTICE**

1. Build a `bytes` value holding the numbers 1 to 5. What does `len()` return?
2. What is the difference between `bytes(5)` and `bytes([5])`?
3. Given `b = b"hey"`, what are the type and value of `b[0]` and of `b[0:1]`?
4. Make a `bytearray` from the numbers 1 to 5, then append 6 and set the first byte to 0.
5. Why does `ba.replace(b"a", b"b")` seem to do nothing when you ignore its result?
6. A friend encodes a string as UTF-8 and decodes it as ASCII and gets an error. Why?

★ **ACHIEVEMENT UNLOCKED: Byte Wrangler**

You can now:

- create `bytes` and `bytearray` from literals, lists, hex strings, and text;
- predict whether an index or slice returns an `int` or a sequence;
- convert between `str` and `bytes` with `.encode()` and `.decode()`;
- pick `bytearray` when data must grow or change, and freeze it with `bytes()`;
- recognise where bytes appear in API, file, and tokenizer work.

CHAPTER 06

If and Match

What this chapter gives you: the first tools that let a program *decide*. You will write comments, split long lines, branch with `if`/`elif`/`else`, test membership with `in`, and pick between patterns with `match`. Truthiness, `None`, and the one-line `a if c else b` get extra attention, because they are the usual sources of silent bugs.

6.1 Comment with

A comment starts at # and runs to the end of the line. Python ignores it. Use comments to say *why*, not to repeat what the code already says. There is no block comment: start every line with #. Inside a string, # is just a character.

```
>>> max_context = 200_000   # tokens the model can read at once
>>> note = "use # for comments"
>>> note
'use # for comments'
```

A triple-quoted string that stands alone is not a comment. It is a string value that Python builds and throws away. Its proper use is as documentation (Chapter 14).

6.2 Continue lines with \ and brackets

Keep lines under about 80 characters. If an expression is too long, you have two choices.

Method	How	Verdict
Backslash	put `\` as the very last character of the line	works, but fragile
Brackets	break the line anywhere inside `( )`, `[ ]`, or `{ }`	preferred

```
>>> total = 1200 + \
...          350 + \
...          40
>>> total
1590
>>> total = (1200 +
...           350 +
...           40)
>>> total
1590
```

> ⚠ **TRAP CARD: a space after the backslash**
>
> A backslash followed by even one trailing space is a `SyntaxError`. You cannot see the space, which makes it painful. Prefer brackets and you never meet this bug.

Brackets also help with long prompts. Two string literals side by side are joined into one:

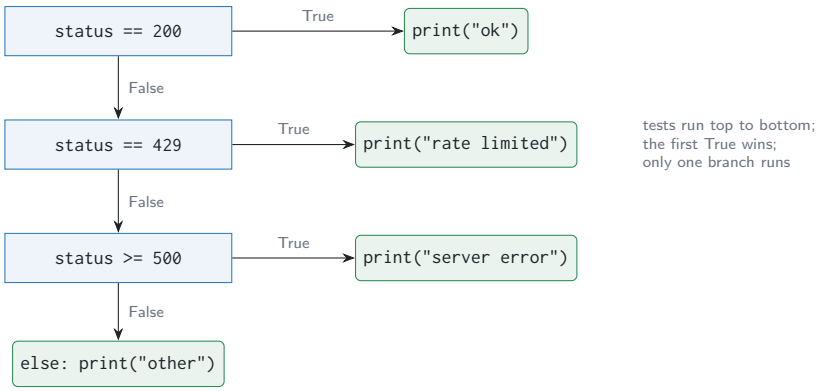
```
>>> prompt = ("You are a helpful assistant. "
...           "Answer in one sentence.")
>>> prompt
'You are a helpful assistant. Answer in one sentence.'
```

> ⚠ **TRAP CARD: a missing comma joins strings**
>
> In a list, `["a" "b"]` is a one-item list containing `"ab"`. A forgotten comma between two string items silently merges them. You get no error, only a shorter list.

6.3 Compare with if, elif, and else

The shape of an if statement

An `if` line ends with a colon. The lines that belong to it are indented by the same amount, and the standard is four spaces. Never mix tabs and spaces. Indentation *is* the structure: when the indent goes back, the block ends.




```
status = 429
if status == 200:
    print("ok")
elif status == 429:
    print("rate limited: wait and retry")
elif status >= 500:
    print("server error")
else:
    print("other:", status)
```

```
rate limited: wait and retry
```

`elif` and `else` are optional. Tests inside tests are allowed, one more indent level each, but flat `elif` chains read better than deep nesting.

Comparison operators

Every comparison produces a `bool`.

Operator	Meaning	Watch for
`==`	equal in value	`5 == 5.0` is `True`; `"5" == 5` is `False`
`!=`	not equal	the opposite of `==`
`< <= > >=`	ordering	mixing `int` and `str` raises `TypeError`
`is` / `is not`	same object	use it for `None`, not for numbers or strings
`=`	*assignment*	`if x = 5:` is a `SyntaxError`

```
>>> 5 == 5.0, "5" == 5
(True, False)
>>> 3 < "4"
Traceback (most recent call last):
  ...
TypeError: '<' not supported between instances of 'int' and 'str'
```

and, or, not

Use `and`, `or`, and `not` to combine tests. Comparisons are worked out first, so no parentheses are needed, but add them when a reader might hesitate. Order of strength: `not`, then `and`, then `or`.

```
>>> tokens, has_key = 800, True
>>> tokens < 1000 and has_key
True
>>> tokens > 1000 or not has_key
False
```

`and` and `or` are *short-circuit* operators. They stop as soon as the answer is known, and they return one of their two *operands*, not necessarily a `bool`.

TYPE CHECK: what and / or / not return

Expression	Rule	Type back
`a and b`	if `a` is falsy, return `a`; otherwise return `b`	type of an operand
`a or b`	if `a` is truthy, return `a`; otherwise return `b`	type of an operand
`not a`	`True` if `a` is falsy, else `False`	always `bool`

```
>>> "" or "default"
'default'
>>> "hello" or "default"
'hello'
>>> "hi" and 42
42
>>> 0 and 1 / 0
0
>>> not []
True
```

The `0 and 1 / 0` line never divides, because the left side already decided the result.

TRAP CARD: `x == "a" or "b"` is always truthy

`model == "claude" or "gemini"` means `(model == "claude") or "gemini"`. The right side is a non-empty string, so the whole expression is never falsy. Each side of `or` must be a full test, or use `in`.

```
>>> model = "gpt"
>>> model == "claude" or "gemini"
'gemini'
>>> model == "claude" or model == "gemini"
False
>>> model in ("claude", "gemini")
False
```

Chained comparisons

Python lets you write a range test the way you would on paper. `lo < x < hi` means `lo < x and x < hi`, and `x` is evaluated once.




```
>>> temperature = 0.7
>>> 0 <= temperature <= 2
True
>>> 5 < 7 < 10 < 999
True
```

What is true?

An `if` does not need a real `bool`. Python asks every object "are you empty or zero?". Empty and zero things are *falsy*; everything else is *truthy*. `bool(x)` shows the verdict.

Falsy (counts as False)	Truthy (counts as True)
`False`, `None`	`True`
`0`, `0.0`	any other number, including `-1` and `0.1`
`""` (empty string)	any non-empty string, even `" "`, `"0"`, `"False"`
`[]`, `()`, `{}`, `set()`, `b""`	any container with at least one item, even `[0]` or `[None]`

```
>>> [bool(v) for v in (0, 0.0, "", [], (), {}, set(), None)]
[False, False, False, False, False, False, False, False]
>>> [bool(v) for v in ("0", "False", " ", [0], [None], 0.1)]
[True, True, True, True, True, True]
```

> ➤ **RULE**
> Test emptiness the Python way: `if messages:` rather than `if len(messages) > 0:`. It also works for strings, dicts, and sets.

None is not the same as empty or zero

`None` means "no value was given". It is falsy, but so are `0` and `""`, and those are real, valid values. To ask "was anything given?" test identity: `x is None` or `x is not None`.

value	`if value:`	`if value is not None:`
`None`	✗ False	✗ False
`0`	✗ False	✓ True (valid temperature)
`""`	✗ False	✓ True (valid empty string)
`0.7`	✓ True	✓ True

■ **BOSS FIGHT: a falsy value that is perfectly valid**

A temperature of `0` is a real setting, yet it is falsy, so `if temperature:` treats it like "not provided". The same bug hides in `temperature or 1.0`, which swaps a valid `0` for `1.0`.

Strategy: when `None` means "missing", test `is None` / `is not None`. Use plain truthiness only when "empty or zero" really should be treated as "nothing".

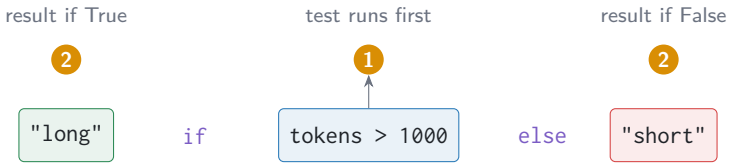
```
for temperature in (0, 0.7, None):
    wrong = temperature or 1.0
    right = 1.0 if temperature is None else temperature
    print(temperature, "wrong:", wrong, "right:", right)
```

```
0 wrong: 1.0 right: 0
0.7 wrong: 0.7 right: 0.7
None wrong: 1.0 right: 1.0
```

(The `for` loop just repeats the block once per value; Chapter 7 explains it.)

Conditional expressions: `a if cond else b`

When you only need to *choose a value*, a one-line form avoids a four-line `if`. The order is unusual: the value for true comes first, the test is in the middle, and the value for false is last. Python runs the test first.




```
>>> tokens = 1500
>>> label = "long" if tokens > 1000 else "short"
>>> label
'long'
>>> print("over limit" if tokens > 4096 else "fits")
fits
```

■ **TYPE CHECK: what a conditional expression gives back**

It is an *expression*: it produces one value, so you can assign it, pass it to a function, or put it inside an f-string. The two branches may be different types; the result has the type of whichever branch ran. It has `else` always, because it must produce a value.

⚠ **TRAP CARD: written backwards, it runs and is wrong**

Putting the test first looks natural but is valid Python with a different meaning. The line below does not raise an error. It treats `"long"` as the test, which is always truthy, so you get `tokens > 1000` (a `bool`) instead of a label.

```
>>> tokens = 1500
>>> tokens > 1000 if "long" else "short"
True
>>> label = "long" if tokens > 1000 else "short"
```

> ⚠ **TRAP CARD: it binds loosely**
>
> The whole thing is chosen *after* the surrounding + is evaluated. Add parentheses when you mix them.

```
>>> flag = False
>>> "Result: " + "yes" if flag else "no"
'no'
>>> "Result: " + ("yes" if flag else "no")
'Result: no'
```

> ➤ **RULE**
>
> Use `a if c else b` to pick between two values. Use a normal `if` block when you want to *do* something (call functions, change variables). Do not chain several in one line.

6.4 Do multiple comparisons with in

Many or-ed equality tests collapse into one `in`. `x in container` gives a `bool`. What "in" means depends on the container.

Container	`in` checks	Speed
`str`	is it a *substring*? (`"at" in "cat"` is True)	scans
`list`, `tuple`	is it equal to an item? (`"at" in ["cat"]` is False)	scans
`set`	is it an item?	instant
`dict`	is it a *key*? (values are ignored)	instant

```
>>> model = "claude-haiku-5-5"
>>> "haiku" in model
True
>>> allowed = {"claude-haiku-5-5", "claude-opus-5-5"}
>>> model in allowed
True
>>> limits = {"input": 100, "output": 20}
>>> "input" in limits, 100 in limits
(True, False)
>>> "at" in "cat", "at" in ["cat"]
(True, False)
>>> "gpt" not in allowed
True
```

> ➤ **RULE**
>
> If you test membership many times against a fixed collection, store it in a `set`. Sets check instantly; lists check one item at a time (Chapter 9).

6.5 The walrus operator

`name := expression` (Python 3.8+) assigns *and* gives back the value, so you can test and keep a result in one step. It looks like a walrus lying on its side.

```
>>> reply = "x" * 120
>>> limit = 100
>>> if (over := len(reply) - limit) > 0:
...     print("trim", over, "characters")
trim 20 characters
>>> over
20
```

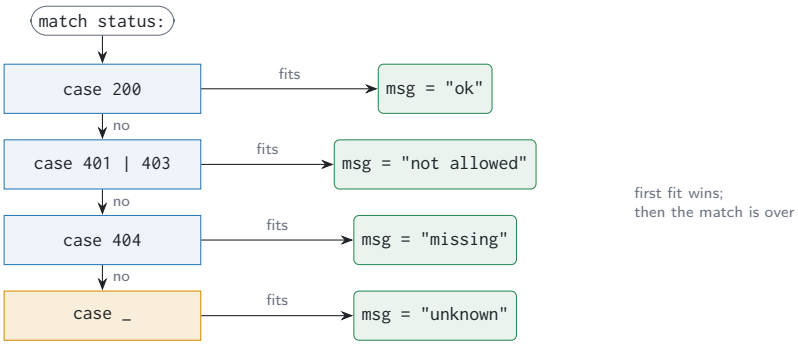
The variable stays alive after the `if`. It also works in `while` loops (Chapter 7).

> ⚠ **TRAP CARD: parentheses decide what is assigned**
>
> Without parentheses around the assignment, the walrus grabs the whole comparison, so you store a `bool`, not the number.

```
>>> reply, limit = "x" * 120, 100
>>> if over := len(reply) - limit > 0:
...     print(over)
True
```

6.6 Match

`match` (Python 3.10+) compares one *subject* against a list of *patterns*, top to bottom. The first pattern that fits runs its block and the whole `match` ends. There is no fall-through and no `break`. `case _` is the default.



Simple matches

```
status = 404
match status:
    case 200:
        msg = "ok"
    case 401 | 403:
        msg = "not allowed"
    case 404:
        msg = "missing"
    case _:
        msg = "unknown"
print(msg)
```

```
missing
```

`401 | 403` is an *or-pattern*: either value fits. The same job can be done with an `if`/`elif` chain or, for pure

lookup, a dictionary. Pick by readability.

Tool	Best when	Default case
`if/elif/else`	conditions are ranges or complex tests	`else`
`match/case`	you compare against shapes or exact values	`case _`
`dict.get(key, default)`	you only map a value to another value	the second argument

```
>>> messages = {200: "ok", 404: "missing"}
>>> messages.get(500, "unknown")
'unknown'
```

Structural matches

"Structural" means the pattern describes the *shape* of the subject and can pull values out of it. A sequence pattern matches a list or tuple of the same length. A name in a pattern *captures* that part into a variable. `*rest` captures the remaining items as a list. A `case` may end with `if condition`, called a *guard*.

```
for line in ("quit", "add gpt-x", "add claude-a 3", "list all now"):
    parts = line.split()
    match parts:
        case ["quit"]:
            print("bye")
        case ["add", name]:
            print("add one:", name)
        case ["add", name, *rest]:
            print("add", name, "with extras", rest)
        case [cmd, *_] if cmd == "list":
            print("listing")
        case _:
            print("unknown")
```

```
bye
add one: gpt-x
add claude-a with extras ['3']
listing
```

■ **TYPE CHECK: pattern cheat sheet**

Pattern	Fits when the subject is	Binds
`200`, `"ok"`, `None`	equal to that literal (`None` by identity)	nothing
`401 | 403`	any one of the alternatives	nothing
`x`	*anything*	`x`
`_`	anything	nothing
`[a, b]` or `a, b`	a list or tuple of exactly 2 (not a `str`)	`a`, `b`
`[a, *rest]`	a sequence of 1 or more	`a`, `rest` (list)
`{"k": v}`	a dict with key `"k"`; extra keys are fine	`v`
`int()`, `str()`	an instance of that type	nothing
`case p if cond`	`p` fits and `cond` is true	as `p`

Matching JSON-like data

Model replies and tool calls are dicts. A mapping pattern checks keys and values and pulls out what you need. Unlike sequences, a dict may have extra keys.

```
events = [
    {"type": "text", "text": "Hello"},
    {"type": "tool_use", "name": "search", "input": {"q": "tokens"}},
    {"type": "ping"},
    42,
]
for event in events:
    match event:
        case {"type": "text", "text": body}:
            print("text:", body)
        case {"type": "tool_use", "name": name}:
            print("tool call:", name)
        case {"type": other}:
            print("ignored:", other)
        case int() | float():
            print("a number")
```

```
text: Hello
tool call: search
ignored: ping
a number
```

If no case fits and there is no `case _`, the `match` simply does nothing. It is not an error.

> ⚠ **TRAP CARD: a bare name captures, it does not compare**
>
> In a pattern, a plain name such as `LIMIT` is a *capture*: it always fits and overwrites the variable. To compare against a constant, use a dotted name (`Config.LIMIT`), a literal, or a guard.

```
>>> LIMIT = 100
>>> match 5:
...     case LIMIT:
...         print("matched; LIMIT is now", LIMIT)
matched; LIMIT is now 5
>>> match 5:
...     case n if n == 100:
...         print("equal to 100")
...     case _:
...         print("not 100")
not 100
```

> ⚠ **TRAP CARD: names are bound even when the guard fails**
>
> A pattern may capture a name and then lose to its guard. The capture stays behind.

```
>>> match [3, 5]:
...     case x, y if y > 6:
...         print("big")
...     case _:
...         print("no match")
no match
>>> x, y
(3, 5)
```

➤ PRACTICE

1. Write an `if/elif/else` that prints `"too low"`, `"too high"`, or `"just right"` when `guess` is compared with `secret`.
2. What does `"yes" and 0` evaluate to? What does `"" or None` evaluate to?
3. Rewrite `label = "big" if n > 10 else "small"` as a normal four-line `if/else`. Then explain what `n > 10 if "big" else "small"` would return.
4. Why is `if temperature:` wrong when 0 is a legal setting? Write the correct test.
5. Write a `match` that handles the strings `"start"` and `"stop"`, plus a default.
6. What is the difference between `"at" in "cat"` and `"at" in ["cat"]`?

★ ACHIEVEMENT UNLOCKED: Fork in the Road

You can now:

- add comments and split long lines with brackets;
- build `if/elif/else` ladders and chained comparisons;
- predict truthiness, and test `None` with `is None`;
- write `a if cond else b` in the right order, and spot it written backwards;
- use `in`, the walrus, and `match` with sequence and mapping patterns.

CHAPTER 07

For and While

What this chapter gives you: the two ways Python repeats work. `while` repeats as long as a test stays true. `for` walks through the items of anything iterable. You will also use `range()` to count, `break` and `continue` to steer, and the odd-looking `else` on a loop. Almost every AI script is a loop over documents, messages, batches, or retries.

7.1 Repeat with while

`while condition:` runs its indented block, then checks the condition again, over and over. When the condition is false, Python skips to the line after the block. Something inside the block must eventually make the condition false, or the loop never ends.

```
budget = 1000
used = 0
while used + 300 <= budget:
    used += 300
    print("sent a batch, used", used)
print("stopped at", used)
```

```
sent a batch, used 300
sent a batch, used 600
sent a batch, used 900
stopped at 900
```

Tracing the loop by hand, one row per check, is the best way to see what happens:

check #	used	`used + 300 <= 1000`	action
1	0	`300 <= 1000` ✓	run body: `used` becomes 300
2	300	`600 <= 1000` ✓	run body: `used` becomes 600
3	600	`900 <= 1000` ✓	run body: `used` becomes 900
4	900	`1200 <= 1000` ✗	leave the loop

> ⚠ **TRAP CARD: forgetting to change the thing you test**
>
> If the body never changes `used`, the test is true forever and the program hangs. Stop it with `Ctrl+C`. When you write a `while`, point at the line that moves it toward the end.

Cancel with break

`break` leaves the loop immediately. The pattern `while True:` plus a `break` inside is the standard way to loop "until something happens". `input()` reads one line from the keyboard and *always returns a string*.

```
while True:
    line = input("prompt> ")
    if line == "q":
        break
    print(line.upper())
```

```
prompt> hi
HI
prompt> q
```

Skip ahead with continue

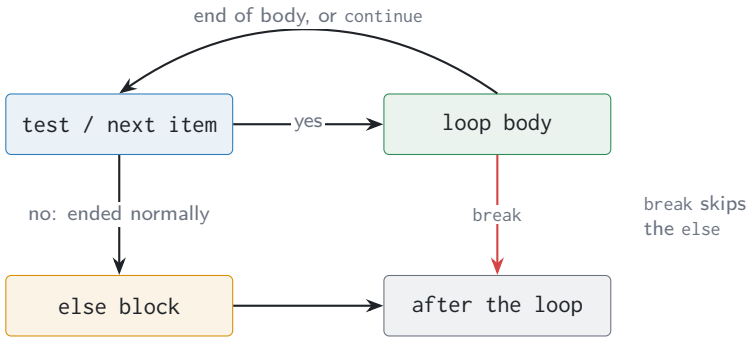
`continue` abandons the rest of this pass and jumps back to the top for the next pass. It is handy for skipping items you do not want to process.

```
lines = ["hello", "", "   ", "world"]
for line in lines:
    if not line.strip():
        continue
    print("got:", line)
```

```
got: hello
got: world
```

Check break use with else

A loop may have an `else` block. It runs only if the loop finished *without* hitting `break`. Read it as "no break happened": the loop was searching, and `else` says it found nothing. It is indented to match the `while`, not the `if`.




```
batch = [3, 5, 7]
pos = 0
while pos < len(batch):
    if batch[pos] % 2 == 0:
        print("found an even number at", pos)
        break
    pos += 1
else:
    print("no even number")
```

```
no even number
```

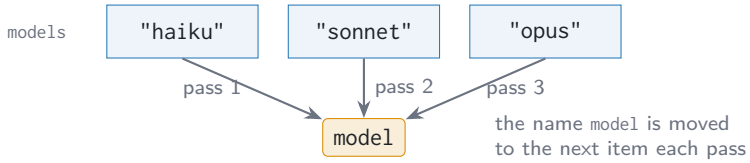
A realistic use is a retry loop: stop at the first success, and the `else` reports that every attempt failed.

```
statuses = [429, 429, 200]     # pretend replies from an API
attempts = 0
while attempts < 3:
    status = statuses[attempts]
    attempts += 1
    if status == 200:
        print("ok after", attempts, "tries")
        break
else:
    print("gave up")
```

```
ok after 3 tries
```

7.2 Iterate with for and in

`for name in iterable:` takes the items of the iterable one at a time, binds each to `name`, and runs the block. You do not manage a counter or know the length in advance. An *iterable* is anything you can step through: strings, lists, tuples, dicts, sets, ranges, files, and more.




```
>>> for letter in "thud":
...     print(letter)
t
h
u
d
>>> for model in ["haiku", "sonnet"]:
...     print(model.upper())
HAIKU
SONNET
```

TYPE CHECK: what `x` is in `for x in ...`

Iterable	x is a ...	Example
`str`	`str` (one character)	`"ab"` gives `"a"`, `"b"`
`list`, `tuple`	the item itself	any type, even another list
`dict`	the *key*	use `.values()` or `.items()` for more (Chapter 9)
`range(n)`	`int`	`0` up to `n - 1`
`bytes`	`int`	one byte as a number (Chapter 5)
`enumerate(seq)`	a `(index, item)` tuple	unpack it: `for i, item in ...`

break, continue, else

All three work exactly as in a `while`. The `else` runs when the loop was not stopped by `break`, and also when the iterable was empty.

```
for letter in "thud":
    if letter == "x":
        print("found an x")
        break
else:
    print("no x in there")
```

```
no x in there
```

A plain flag variable does the same job and some people find it clearer. Choose whichever you can read at a glance six months later.

> ■ **BOSS FIGHT: the `else` on a loop**
>
> `else` on a `for` or `while` does *not* mean "otherwise, if the loop condition was false". It means "the loop ran to its natural end without a `break`". That includes the case where a `for` had nothing to iterate over.
>
> **Strategy:** read it as `nobreak`. Use it only for search loops: `break` means "found it", `else` means "not found". If that reading feels forced, use a flag instead.

```
for item in []:
    print("never printed")
    break
else:
    print("else ran: the list was empty")
```

```
else ran: the list was empty
```

Counting with enumerate

If you need both the position and the item, do not count by hand and do not write `range(len(items))`. `enumerate()` hands you pairs.

```
models = ["haiku", "sonnet", "opus"]
for i, name in enumerate(models, start=1):
    print(i, name)
```

```
1 haiku
2 sonnet
3 opus
```

Nested loops

A loop inside a loop runs its whole inner cycle for every outer pass. `break` and `continue` only affect the *innermost* loop they sit in.

```
for i in range(3):
    for j in range(3):
        if j == 1:
            break          # leaves only the inner loop
        print(i, j)
```

```
0 0
1 0
2 0
```

> ⚠ **TRAP CARD: changing a list while looping over it**
>
> Removing items from the list you are iterating makes Python skip items, because the positions shift. Loop over a copy (`for n in nums[:]`) or build a new list instead (Chapter 8). The loop below should remove all the even numbers, but it leaves a 4 behind.

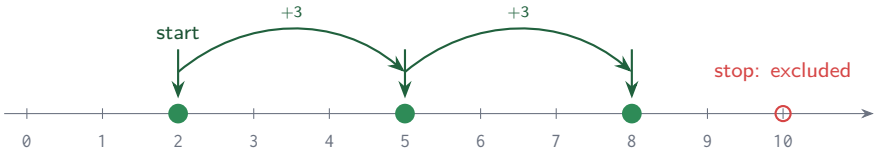
```
nums = [2, 4, 6, 7]
for n in nums:
    if n % 2 == 0:
        nums.remove(n)
print(nums)
```

```
[4, 7]
```

> ⚠ **TRAP CARD: the loop variable lives on**
>
> After a `for`, the name keeps the last item. If the iterable was empty, the name was never set and using it raises `NameError`.

7.3 Generate number sequences with range()

`range(start, stop, step)` produces integers on demand without building a list. Only `stop` is required. `start` defaults to 0 and `step` defaults to 1. The stop value is *never* included, the same rule as slices.




```
>>> list(range(3))
[0, 1, 2]
>>> list(range(2, 6))
[2, 3, 4, 5]
>>> list(range(0, 11, 2))
[0, 2, 4, 6, 8, 10]
>>> list(range(2, -1, -1))
[2, 1, 0]
>>> list(range(5, 2))
[]
```

> ■ **TYPE CHECK: range is its own type**
>
> `range(...)` returns a `range` object: an immutable, lazy sequence of ints. It is not a list, so it prints as `range(2, 10, 3)`; wrap it in `list()` to see the numbers. It supports `len()`, indexing, and `in`, and a huge range uses almost no memory.

```
>>> r = range(0, 10, 3)
>>> r
range(0, 10, 3)
>>> len(r), r[2], 9 in r, 5 in r
(4, 6, True, False)
>>> len(range(10**12))
1000000000000
```

> ⚠ **TRAP CARD: the stop value is not included**
>
> `range(1, 5)` gives 1, 2, 3, 4. To get 1 to 5 you need `range(1, 6)`. A going-down range needs a negative step, or it is silently empty: `range(5, 2)` is `[]`, but `range(5, 2, -1)` is `[5, 4, 3]`.

7.4 Other iterators

The thing a `for` loop really uses is an *iterator*: an object that gives the next item each time it is asked and raises `StopIteration` when it runs out. `iter()` makes one from any iterable and `next()` asks for the next item. You rarely call them yourself, but knowing they exist explains two behaviours.

```
>>> it = iter(["a", "b"])
>>> next(it), next(it)
('a', 'b')
>>> next(it)
Traceback (most recent call last):
  ...
StopIteration
>>> next(it, "done")
'done'
```

> ⚠ **TRAP CARD: an iterator is used up after one pass**
>
> A list can be looped over many times. An iterator (and things like streaming API replies, which behave like iterators) can be consumed only once. A second loop sees nothing.

```
>>> stream = iter([1, 2])
>>> list(stream), list(stream)
([1, 2], [])
```

The walrus operator (Chapter 6) pairs well with `while`: read a piece, test it, and use it in one line. This loop stops at the first empty piece.

```
chunks = iter(["ab", "cd", ""])
while (chunk := next(chunks)) != "":
    print("chunk:", chunk)
```

```
chunk: ab
chunk: cd
```

Later chapters show iteration over files (Chapter 20), how to make your own iterable objects (Chapter 11), and the `itertools` module (Chapter 12).

➤ **RULE**

Prefer `for` when you know what you are looping over, and `while` when you loop until a condition changes. If you catch yourself writing `i = 0 ... i += 1` to walk a list, switch to `for` or `enumerate`.

➤ **PRACTICE**

1. Print the numbers 3, 2, 1, 0 using a `for` loop over a list, then using `range()`.
2. Write a `while` loop with `number = 1` and `guess_me = 7` that prints `"too low"` while `number` is smaller, prints `"found it!"` and stops when equal, and increments `number` each pass.
3. What does `list(range(10, 0, -3))` give? Predict it first.
4. Rewrite `for i in range(len(items)): print(i, items[i])` using `enumerate`.
5. Explain in one sentence when the `else` block of a `for` loop runs.
6. Why does the loop that removes even numbers from `[2, 4, 6, 7]` leave a 4 in the list?

★ **ACHIEVEMENT UNLOCKED: Loop Master**

You can now:

- write `while` loops that end, and stop runaway ones;
- steer loops with `break`, `continue`, and loop `else`;
- choose the right form of `range()` and predict its numbers;
- iterate with `for`, `enumerate`, and nested loops without skipping items;
- explain why an iterator can be read only once.

CHAPTER 08

Tuples and Lists

What this chapter gives you: the two ordered containers you will use all day. A `tuple` is a fixed row of items. A `list` is a row you can grow, shrink, and reorder. You will learn to slice them, sort them, copy them without surprises, and build new ones with comprehensions. A prompt history, a batch of documents, and a list of embeddings are all lists.

★ **KEY IDEA: sequences, mutable and immutable**

A *sequence* holds items in order and is read by position (index). Strings and bytes are sequences of one kind of item. Tuples and lists can hold *any* mix of types, even other lists.

Type	mutable?	hashable?	Literal
`str`, `bytes`	✗	✓	`"ab"`, `b"ab"`
`tuple`	✗	✓(if every item is)	`(1, 2)`
`list`	✓	✗	`[1, 2]`
`bytearray`	✓	✗	`bytearray(b"ab")`

8.1 Tuples

Create with commas and ()

The comma makes the tuple, not the parentheses. Parentheses only make it easier to see. The one place where the comma really matters is a tuple with one item.

```
>>> empty = ()
>>> pair = 1024, "tokens"
>>> pair
(1024, 'tokens')
>>> one = ("haiku",)
>>> one, type(one)
(('haiku',), <class 'tuple'>)
>>> not_a_tuple = ("haiku")
>>> type(not_a_tuple)
<class 'str'>
```

⚠ **TRAP CARD: the one-item tuple**

`("haiku")` is just a string in brackets. You need the trailing comma: `("haiku",)`. This also bites when you pass a one-item tuple to a function: `f(("a",))`, because `f("a",)` is just the string.

Unpacking and swapping

Assigning a tuple to several names takes it apart. The count must match, unless you use `*name` to soak up the extras (it always becomes a list).

```
>>> usage = (812, 143)
>>> tokens_in, tokens_out = usage
>>> tokens_in, tokens_out
(812, 143)
>>> a, b, c = usage
Traceback (most recent call last):
  ...
ValueError: not enough values to unpack (expected 3, got 2)
>>> first, *rest = [10, 20, 30, 40]
>>> first, rest
(10, [20, 30, 40])
>>> x, y = 1, 2
>>> x, y = y, x
>>> x, y
(2, 1)
```

The swap works because the right side is built into a tuple first, then unpacked. No temporary variable is needed.

Convert with tuple()

`tuple(iterable)` builds a tuple from anything you can loop over.

```
>>> tuple(["a", "b"]), tuple("ab"), tuple(b"\x01\x02")
(('a', 'b'), ('a', 'b'), (1, 2))
```

What you can do with a tuple

Read it, slice it, combine it, repeat it, compare it, loop over it. You cannot change it.

```
>>> t = (1, 2, 3)
>>> t[0], t[-1], t[1:]
(1, 3, (2, 3))
>>> t[0] = 9
Traceback (most recent call last):
  ...
TypeError: 'tuple' object does not support item assignment
>>> t + (4,), t * 2
((1, 2, 3, 4), (1, 2, 3, 1, 2, 3))
>>> (7, 2) < (7, 2, 9), (7, 2) == (7, 2, 9)
(True, False)
>>> for word in ("fresh", "out"):
...     print(word)
fresh
out
```

“Modify” with += only *looks* like a change. Python builds a new tuple and moves the name onto it, exactly as for `bytes`.

```
>>> t1 = ("Fee", "Fie")
>>> before = t1
>>> t1 += ("Foe",)
>>> t1, before, t1 is before
(('Fee', 'Fie', 'Foe'), ('Fee', 'Fie'), False)
```

> ⚠ **TRAP CARD: immutable does not mean "deeply frozen"**
>
> A tuple cannot re-point its slots, but if a slot holds a *list*, that list can still change. The tuple is then not hashable.

```
>>> record = ("log", [1, 2])
>>> record[1].append(3)
>>> record
('log', [1, 2, 3])
>>> hash(record)
Traceback (most recent call last):
  ...
TypeError: unhashable type: 'list'
```

A *named tuple* lets you read items by name as well as by position. It is covered in Chapter 11.

8.2 Lists

A list keeps items in order, allows duplicates, and can be changed in place. Use one whenever the contents or the length can change.

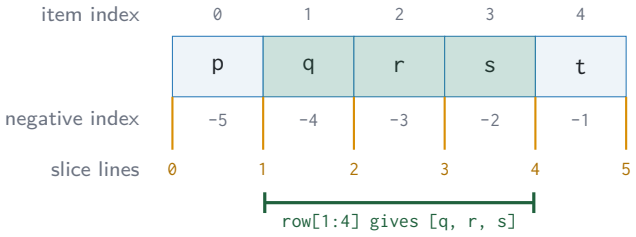
Create with [], list(), and split()

```
>>> empty = []
>>> mixed = [1, "two", [3, 4], None]
>>> list("cat"), list((1, 2)), list(range(3))
(['c', 'a', 't'], [1, 2], [0, 1, 2])
>>> "9/19/2024".split("/")
['9', '19', '2024']
>>> "a/b//c".split("/")
['a', 'b', '', 'c']
>>> "a/b//c".split("//")
['a/b', 'c']
>>> "one  two   three".split()
['one', 'two', 'three']
```

Two repeated separators give an empty string between them. `split()` with no argument splits on any run of whitespace and drops the empties.

Index and slice

An index counts from 0 at the left, or from -1 at the right. A slice `seq[start:stop:step]` returns a *new list* of the same type. The easiest way to read a slice is to imagine the numbers sitting on the *lines between* items: the slice runs from the start line to the stop line, so the stop item is not included.



```
>>> row = ["p", "q", "r", "s", "t"]
>>> row[0], row[-1]
('p', 't')
>>> row[1:4], row[:2], row[-2:]
(['q', 'r', 's'], ['p', 'q'], ['s', 't'])
>>> row[::2], row[::-1]
(['p', 'r', 't'], ['t', 's', 'r', 'q', 'p'])
>>> row[10:], row[-9:2]
([], ['p', 'q'])
>>> row[5]
Traceback (most recent call last):
  ...
IndexError: list index out of range
```

■ TYPE CHECK: index vs slice

`row[2]` returns *one item* (any type; here a `str`) and raises `IndexError` when out of range. `row[2:3]` returns a *list* (here with one item) and never raises: out-of-range ends quietly snap to the edge. A slice is a copy, so assigning to it does not change `row`.

Add items

```
>>> q = ["a", "b", "c"]
>>> q.append("d")
>>> q.insert(0, "z")
>>> q.extend(["x", "y"])
>>> q
['z', 'a', 'b', 'c', 'd', 'x', 'y']
>>> q.append(["m", "n"])
>>> q[-1]
['m', 'n']
>>> q += ["end"]
>>> print(q.append("e"))
None
```

⚠ TRAP CARD: append vs extend

`append(x)` adds *one item*, even if `x` is a list. `extend(it)` (and `+=`) adds each item of `it`. Using `append` with a list gives a list inside the list.

⚠ TRAP CARD: methods that change a list return None

`append`, `extend`, `insert`, `sort`, `reverse`, `remove`, and `clear` change the list in place and return `None`. Writing `names = names.sort()` replaces your list with `None`.

Change items

Assigning to an index replaces one item. Assigning to a slice replaces a whole stretch, and the new stretch can be any iterable of any length.

```
>>> n = [1, 2, 3, 4]
>>> n[0] = "one"
>>> n[1:3] = [7, 8, 9]
>>> n
['one', 7, 8, 9, 4]
>>> n[1:3] = []
>>> n
['one', 9, 4]
>>> n[10] = 1
Traceback (most recent call last):
  ...
IndexError: list assignment index out of range
```

Remove items

Want to	Use	Returns	If missing
delete by position	`del m[i]`	(statement)	`IndexError`
delete by value	`m.remove(v)`	None	`ValueError`; removes the first match only
take out by position	`m.pop(i)`	the item	`IndexError`; default `i` is `-1`
empty the list	`m.clear()`	None	–

```
>>> m = ["a", "b", "c", "b"]
>>> del m[0]
>>> m.remove("b")
>>> m
['c', 'b']
>>> m.pop(), m.pop(0)
('b', 'c')
>>> m.pop()
Traceback (most recent call last):
  ...
IndexError: pop from empty list
>>> ["a"].remove("z")
Traceback (most recent call last):
  ...
ValueError: list.remove(x): x not in list
```

`append()` with `pop()` gives a stack (last in, first out). `append()` with `pop(0)` gives a queue (first in, first out), but `pop(0)` is slow on big lists. For queues, use `collections.deque` (Chapter 12).

Find and count

```
>>> w = ["a", "b", "a"]
>>> w.index("a"), w.count("a"), "b" in w, len(w)
(0, 2, True, 3)
>>> w.index("z")
Traceback (most recent call last):
  ...
ValueError: 'z' is not in list
>>> nums = [3, 9, 4]
>>> min(nums), max(nums), sum(nums)
(3, 9, 16)
```

`index()` returns the *first* position and raises `ValueError` if the value is absent, so check with `in` first when it may be missing. If you test membership a lot, a set is faster (Chapter 9).

join() is the opposite of split()

`join()` is a *string* method. The separator is the string you call it on, and the list is the argument. It returns a `str` and needs every item to already be a string.

```
>>> parts = ["Harry", "Hermione", "Ron"]
>>> ", ".join(parts)
'Harry, Hermione, Ron'
>>> ", ".join(parts).split(", ") == parts
True
>>> ", ".join([1, 2])
Traceback (most recent call last):
  ...
TypeError: sequence item 0: expected str instance, int found
>>> ", ".join([str(n) for n in [1, 2]])
'1, 2'
```

Sort with sort() or sorted()

	Changes the list?	Returns	Works on
`m.sort()`	yes, in place	`None`	lists only
`sorted(x)`	no	a new `list`	any iterable
`m.reverse()`	yes, in place	`None`	lists only
`reversed(x)`	no	an iterator	any sequence

```
>>> scores = [0.9, 0.2, 0.5]
>>> sorted(scores)
[0.2, 0.5, 0.9]
>>> scores
[0.9, 0.2, 0.5]
>>> result = scores.sort(reverse=True)
>>> result is None, scores
(True, [0.9, 0.5, 0.2])
>>> sorted([1, "a"])
Traceback (most recent call last):
  ...
TypeError: '<' not supported between instances of 'str' and 'int'
```

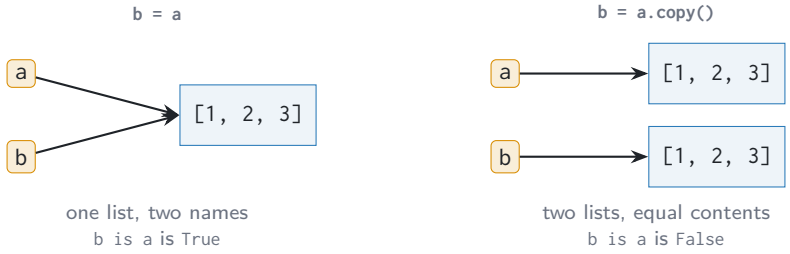
The `key=` argument names a function that turns each item into the value to compare. The items themselves are not changed.

```
>>> words = ["Banana", "apple", "cherry"]
>>> sorted(words)
['Banana', 'apple', 'cherry']
>>> sorted(words, key=str.lower)
['apple', 'Banana', 'cherry']
>>> runs = [("haiku", 0.4), ("opus", 3.1), ("sonnet", 1.2)]
>>> sorted(runs, key=lambda r: r[1], reverse=True)
[('opus', 3.1), ('sonnet', 1.2), ('haiku', 0.4)]
```

(`lambda r: r[1]` is a tiny unnamed function, which Chapter 10 explains. It means "given `r`, give back `r[1]`".) Plain uppercase letters sort before lowercase, and a sort that is *stable* keeps the original order of ties.

Assignment makes a second name, not a second list

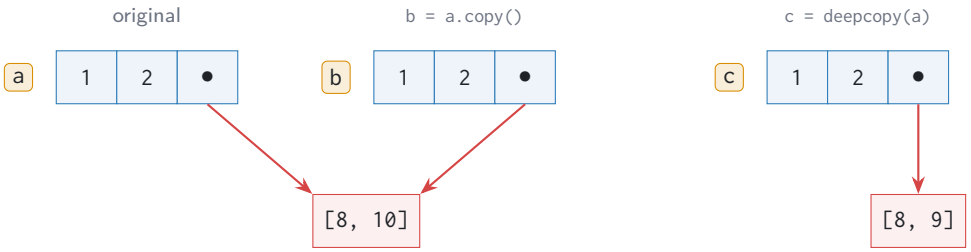
A variable is a label. `b = a` sticks a second label on the *same* list, so a change through one name shows through the other. To get an independent list, copy it with `a.copy()`, `list(a)`, or `a[:]`.




```
>>> a = [1, 2, 3]
>>> b = a
>>> a[0] = "changed"
>>> b
['changed', 2, 3]
>>> b is a
True
>>> c, d, e = a.copy(), list(a), a[:]
>>> a[1] = "x"
>>> c, d == c, e is a
(['changed', 2, 3], True, False)
```

Copy everything with deepcopy()

`copy()`, `list()`, and `[:]` are *shallow*: they copy the outer list but reuse the inner objects. If an item is itself a list, both outer lists point at the *same* inner list. For a fully independent copy, use `copy.deepcopy()` from the standard library.



after `a[2][1] = 10`: `a` and `b` see `[8, 10]`; `c` still has `[8, 9]`

```
>>> import copy
>>> a = [1, 2, [8, 9]]
>>> b = a.copy()
>>> c = copy.deepcopy(a)
>>> a[2][1] = 10
>>> b, c
([1, 2, [8, 10]], [1, 2, [8, 9]])
>>> b[2] is a[2], c[2] is a[2]
(True, False)
```

■ BOSS FIGHT: two names, one list (and shared insides)

Almost every strange "my data changed by itself" bug is aliasing. `b = a` shares the list. Even `a.copy()` shares any *inner* lists, dicts, or sets. This applies to *every* mutable type.

Strategy: ask "is this one object or two?" with `is`. Copy when you need independence. Use `deepcopy`

when the data is nested. Prefer building new lists over editing a shared one.

> ⚠ **TRAP CARD: multiplying a list of lists repeats one inner list**
>
> Multiplying a list repeats *references*, so `[[0] * 3] * 2` holds the same inner list twice. The rows of the grid below are one list shown twice. Build each row separately with a comprehension.

```
>>> grid = [[0] * 3] * 2
>>> grid[0][0] = 1
>>> grid
[[1, 0, 0], [1, 0, 0]]
>>> grid = [[0] * 3 for _ in range(2)]
>>> grid[0][0] = 1
>>> grid
[[1, 0, 0], [0, 0, 0]]
```

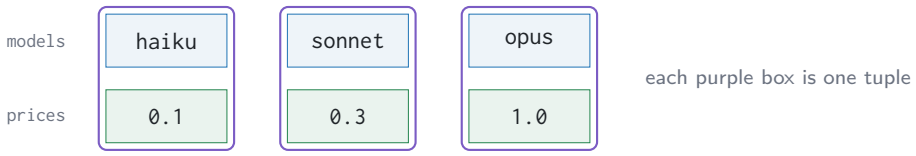
Compare and iterate

Lists are compared item by item from the left, like words in a dictionary. A shorter list that matches the start is smaller. Iteration is `for item in list`; `break`, `continue`, and `else` work as in Chapter 7.

```
>>> [7, 2] == [7, 2, 9], [7, 2] < [7, 2, 9], [3] < [2, 9]
(False, True, False)
>>> for cheese in ["brie", "gjetost", "havarti"]:
...     if cheese.startswith("g"):
...         print("skip", cheese)
...         break
...     print(cheese)
... else:
...     print("no g found")
brie
skip gjetost
```

Iterate several sequences with zip()

`zip(a, b, ...)` pairs up items at the same position and stops at the shortest input. It returns a lazy iterator of tuples, so wrap it in `list()` to see them, and you can unpack each tuple in the `for` line.



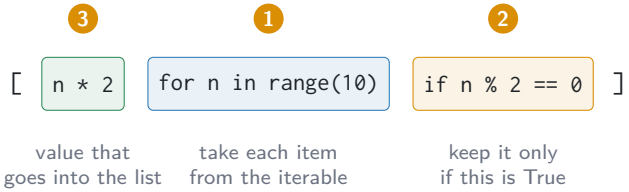

```
>>> models = ["haiku", "sonnet", "opus"]
>>> prices = [0.1, 0.3, 1.0]
>>> for name, price in zip(models, prices):
...     print(name, price)
haiku 0.1
sonnet 0.3
opus 1.0
>>> dict(zip(models, prices))
{'haiku': 0.1, 'sonnet': 0.3, 'opus': 1.0}
>>> list(zip("ab", [1, 2, 3]))
[('a', 1), ('b', 2)]
```

If the lengths differ, `zip()` silently drops the extras. Two ways to stay in control:

```
>>> from itertools import zip_longest
>>> list(zip_longest([1, 2], ["a", "b", "c"], fillvalue=0))
[(1, 'a'), (2, 'b'), (0, 'c')]
>>> list(zip([1], [1, 2], strict=True))
Traceback (most recent call last):
  ...
ValueError: zip() argument 2 is longer than argument 1
```

List comprehensions

A comprehension builds a list in one expression: `[expression for item in iterable if condition]`. The `if` part is optional. It reads as: for each item, if the condition is true, put the expression's value in the new list. The loop moves inside the brackets.




```
>>> [n * 2 for n in range(4)]
[0, 2, 4, 6]
>>> [n for n in range(10) if n % 3 == 0]
[0, 3, 6, 9]
>>> words = ["a", "bb", "ccc"]
>>> [len(w) for w in words]
[1, 2, 3]
>>> [(r, c) for r in range(2) for c in range(2)]
[(0, 0), (0, 1), (1, 0), (1, 1)]
```

With two `for` clauses, the first is the outer loop. The comprehension above equals a nested loop that appends `(r, c)` at the bottom.

> ⚠ **TRAP CARD: an `if` at the end filters; an `if/else` at the front transforms**
>
> These look alike but do different jobs. A trailing `if` (no `else`) *drops* items. A conditional expression in the value slot keeps every item and picks a value for each. Remember the order from Chapter 6: value, `if` test, `else` value.

```
>>> words = ["a", "bb", "ccc"]
>>> [w for w in words if len(w) > 1]
['bb', 'ccc']
>>> ["long" if len(w) > 1 else "short" for w in words]
['short', 'long', 'long']
```

Lists of lists

A list can hold lists. Use one index per level, left to right: the first picks the inner list, the next picks inside it.

```
>>> batches = [["a", "b"], ["c"], []]
>>> batches[0], batches[0][1]
(['a', 'b'], 'b')
>>> len(batches), [len(b) for b in batches]
(3, [2, 1, 0])
>>> [item for batch in batches for item in batch]
['a', 'b', 'c']
```

The last line *flattens* one level. The outer `for` comes first, just as in nested loops.

8.3 Tuples versus lists

You can often use either. The choice is about what you mean.

	tuple	**list**
Mutable?	no	yes
Methods	only `count` and `index`	many (`append`, `sort`, ...)
Can be a dict key?	yes, if all items are hashable	no
Typically holds	a fixed record of *different* things: `(model, price)`	many *similar* items: a list of prices
Size	a little smaller	over-allocates room to grow
Safe from accidents?	yes, nobody can edit it by mistake	no

> ➤ **RULE**
> Use a tuple for a small fixed record or a function's multiple results. Use a list for a collection that grows, shrinks, or gets sorted. In everyday code you will use lists (and dicts, Chapter 9) far more.

8.4 There are no tuple comprehensions

Lists, dicts, and sets have comprehensions. Tuples, strings, and bytes do not. Swapping the square brackets for parentheses does *not* give a tuple. It gives a *generator expression*: a lazy iterator that produces items one at a time and can be read only once. Wrap it in `tuple()` to get a tuple.

```
>>> g = (n * n for n in range(3))
>>> type(g)
<class 'generator'>
>>> tuple(g)
(0, 1, 4)
>>> tuple(g)
()
>>> sum(n * n for n in range(4))
14
```

Because a generator makes values only when asked, it is the right tool for big or endless data, for example processing a million documents without holding them all in memory. Chapter 10 covers generators fully.

> ➤ **PRACTICE**
> 1. Make a list `prices = [0.1, 0.3, 1.0, 0.6]`. Write the slices that give the first two, the last two, and the list reversed. What is each result's type?
> 2. What do `x = [3, 1, 2].sort()` and `y = sorted([3, 1, 2])` leave in `x` and `y`?
> 3. After `a = [1, [2, 3]]; b = a.copy(); b[1].append(4)`, what is `a`? Why? What one change makes `a` stay untouched?

4. Write a list comprehension that gives the lengths of only the words longer than 3 letters in a list of words.
5. What is the difference between `append` and `extend`? Show each with `[1, 2]` and `[3, 4]`.
6. Why does `("hello")` not make a tuple, and what makes the one-item tuple?

★ **ACHIEVEMENT UNLOCKED: List Keeper**

You can now:

- build tuples and lists, unpack them, and slice with start, stop, and step;
- add, change, find, and remove items, knowing which methods return `None`;
- sort with `sort`, `sorted`, and `key=`, and pair sequences with `zip`;
- tell aliasing, shallow copy, and deep copy apart and pick the right one;
- write list comprehensions with filters and conditional values.

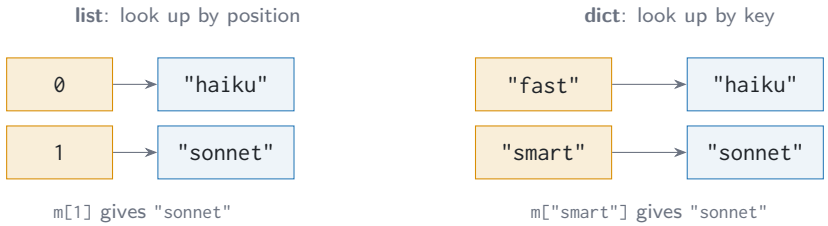
CHAPTER 09

Dictionaries and Sets

What this chapter gives you: the two containers that find things by *value* instead of by position. A `dict` maps keys to values and is the shape of nearly all JSON, API requests, model replies, and configuration. A `set` is a bag of unique items with fast membership tests and set algebra. After this chapter you can read any nested API response and pull out one number safely.

9.1 Dictionaries

A dictionary stores *key* → *value* pairs. You ask for a value by its key, not by a position. Keys are unique. Since Python 3.7 a dict also remembers the order in which keys were added.



Create with {} and dict()

```
>>> usage = {"input_tokens": 812, "output_tokens": 143}
>>> usage["input_tokens"]
812
>>> same = dict(input_tokens=812, output_tokens=143)
>>> same == usage
True
>>> type(usage), len(usage), {}
(<class 'dict'>, 2, {})
```

`dict(name=value)` is handy, but the names must be legal variable names (no spaces, no hyphens, no reserved words). For keys like `"max-tokens"`, use the brace form. A trailing comma after the last pair is allowed.

Convert with dict()

`dict()` also accepts a sequence of two-item sequences. `zip()` (Chapter 8) is the usual way to make them.

```
>>> dict([("a", 1), ("b", 2)])
{'a': 1, 'b': 2}
>>> dict(["ab", "cd"])
{'a': 'b', 'c': 'd'}
>>> dict(zip(["x", "y"], [10, 20]))
{'x': 10, 'y': 20}
>>> dict([("a", 1, 2)])
Traceback (most recent call last):
  ...
ValueError: dictionary update sequence element #0 has length 3; 2 is required
```

Add or change an item with [key]

Assigning to a key adds it when it is new and replaces the value when it exists. There is no out-of-range error as with lists. Duplicate keys in a literal keep the last value.

```
>>> cfg = {"model": "haiku"}
>>> cfg["max_tokens"] = 1000
>>> cfg["model"] = "sonnet"
>>> cfg
{'model': 'sonnet', 'max_tokens': 1000}
>>> {"a": 1, "a": 2}
{'a': 2}
>>> list(cfg)
['model', 'max_tokens']
```

■ **TYPE CHECK: what can be a key**

A key must be *hashable*, which in practice means immutable: `str`, `int`, `float`, `bool`, `None`, `bytes`, `frozenset`, and tuples whose items are hashable. A list, dict, or set cannot be a key. Values can be anything.

```
>>> {(1, 2): "tuple key works"}
{(1, 2): 'tuple key works'}
>>> {[1, 2]: "no"}
Traceback (most recent call last):
  ...
TypeError: unhashable type: 'list'
```

⚠ **TRAP CARD: keys that are equal count as the same key**

`1`, `1.0`, and `True` are equal, so they collide. The first key object stays, and the last value wins.

```
>>> {1: "a", True: "b", 1.0: "c"}
{1: 'c'}
```

Get an item by [key] or with get()

Expression	Key present	Key missing
`d[k]`	the value	raises `KeyError`
`d.get(k)`	the value	returns `None`
`d.get(k, dflt)`	the value	returns `dflt`
`k in d`	`True`	`False`
`d.pop(k)`	value, and removes the pair	raises `KeyError`
`d.pop(k, dflt)`	value, and removes the pair	returns `dflt`
`d.setdefault(k, dflt)`	the value	stores `dflt` under `k`, then returns it

```
>>> resp = {"model": "claude-haiku-5-5", "stop_reason": "end_turn"}
>>> resp["model"]
'claude-haiku-5-5'
>>> resp["usage"]
Traceback (most recent call last):
  ...
KeyError: 'usage'
>>> resp.get("usage") is None
True
>>> resp.get("usage", {})
{}
>>> "usage" in resp
False
```

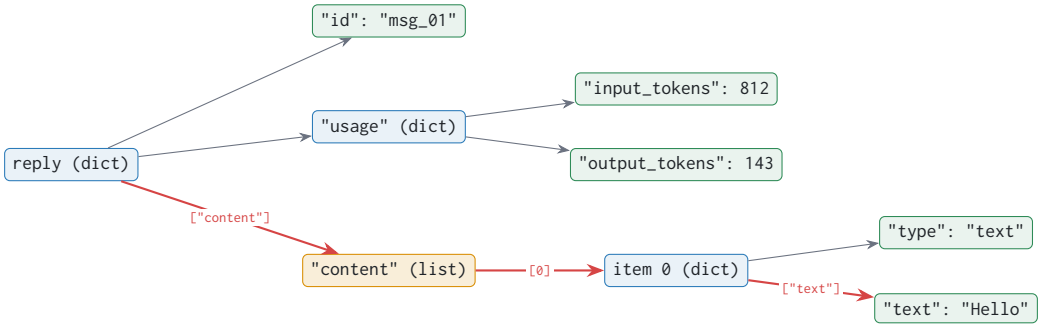
> ⚠ **TRAP CARD: `d.key` is not how dicts work**
>
> A dict is read with *brackets* and a key: `d["model"]`. Dot access (`d.model`) looks for an *attribute* and fails. SDK response objects are the opposite: they use dots (`response.usage`), while data parsed from JSON is a dict and uses brackets.

```
>>> resp = {"model": "claude-haiku-5-5"}
>>> resp.model
Traceback (most recent call last):
  ...
AttributeError: 'dict' object has no attribute 'model'
```

Syntax	Looks up	Fails with
`d["k"]`	a dict key	`KeyError`
`obj.x`	an attribute	`AttributeError`
`lst[0]`	a list position	`IndexError`
`d.get("k")`	a dict key, safely	never; gives `None`

Nested dictionaries: reading JSON

Values can be dicts and lists, so data nests as deeply as you like. Read it with one subscript per level, left to right. The picture shows how `reply["content"][0]["text"]` walks down the tree.



```
>>> reply = {
...     "id": "msg_01",
...     "usage": {"input_tokens": 812, "output_tokens": 143},
...     "content": [{"type": "text", "text": "Hello"}],
... }
>>> reply["usage"]["input_tokens"]
812
>>> reply["content"][0]["text"]
'Hello'
>>> reply["usage"]["total"]
Traceback (most recent call last):
  ...
KeyError: 'total'
>>> reply.get("error", {}).get("message", "no error")
'no error'
```

■ BOSS FIGHT: missing keys deep inside nested data

Real replies are not always complete. A key can be absent, or present with the value `None`, or a list can be empty. `reply["a"]["b"]["c"]` fails at the first gap with a `KeyError` or `TypeError` that does not say which level was wrong.

Strategy: decide per level. Use `[ ]` when absence is a bug (you want the loud error). Use `.get(key, {})` to step through levels that may be missing, then `.get(key, default)` at the last one. Check lists with `if items:` before `items[0]`. Remember `.get(k, d)` returns `d` only when the key is *missing*, not when its value is `None`.

```
>>> reply = {"usage": None, "content": []}
>>> reply.get("usage", {}).get("input_tokens", 0)
Traceback (most recent call last):
  ...
AttributeError: 'NoneType' object has no attribute 'get'
>>> (reply.get("usage") or {}).get("input_tokens", 0)
0
>>> first = reply["content"][0] if reply["content"] else None
>>> first is None
True
```

Iterate with for and in

Looping over a dict gives its *keys*, in insertion order. `.keys()`, `.values()`, and `.items()` return *views*: light objects that follow the dict and can be looped over or turned into lists.

```
>>> tokens = {"prompt": 812, "completion": 143}
>>> for name in tokens:
...     print(name)
prompt
completion
>>> list(tokens.keys()), list(tokens.values())
(['prompt', 'completion'], [812, 143])
>>> list(tokens.items())
[('prompt', 812), ('completion', 143)]
>>> for name, n in tokens.items():
...     print(name, "->", n)
prompt -> 812
completion -> 143
>>> sum(tokens.values())
955
```

■ TYPE CHECK: what the loop variable is

`for k in d` and `for k in d.keys()`: the key. `for v in d.values()`: the value. `for item in d.items()`: a `(key, value)` *tuple*, so unpack it in the `for` line (Chapter 8) with `for k, v in d.items()`.

⚠ TRAP CARD: do not add or remove keys while looping

Changing the number of keys during a loop raises `RuntimeError`. Loop over a copy of the keys (`list(d)`) when you must change the dict, or build a new dict.

```
>>> d = {"a": 1, "b": 2}
>>> for k in d:
...     d[k + "x"] = 0
Traceback (most recent call last):
  ...
RuntimeError: dictionary changed size during iteration
>>> for k in list(d):
...     d[k + "x"] = 0
>>> sorted(d)
['a', 'ax', 'axx', 'b', 'bx']
```

Combine dictionaries

When two dicts share a key, the one applied *last* wins. There are three ways to merge.

Way	Result	Changes the left dict?
`a | b`	new dict	no
`{**a, **b}`	new dict	no
`a.update(b)`	None	yes, in place
`a |= b`	(assignment)	yes, in place

```
>>> base = {"model": "haiku", "temperature": 0.7}
>>> override = {"temperature": 0.0, "max_tokens": 500}
>>> base | override
{'model': 'haiku', 'temperature': 0.0, 'max_tokens': 500}
>>> {**base, **override} == base | override
True
>>> base
{'model': 'haiku', 'temperature': 0.7}
>>> base.update(override)
>>> base["max_tokens"]
500
```

This "defaults, then overrides" pattern is how you combine default request settings with per-call settings. All three ways are shallow: nested values are shared, not copied.

Delete items

```
>>> d = {"a": 1, "b": 2, "c": 3}
>>> del d["a"]
>>> d.pop("b")
2
>>> d.pop("zzz", None) is None
True
>>> d.popitem()
('c', 3)
>>> d
{}
>>> del d["x"]
Traceback (most recent call last):
  ...
KeyError: 'x'
```

`popitem()` removes and returns the most recently added pair. `clear()` empties the dict in place; assigning `d = {}` makes a new empty dict and leaves other names pointing at the old one.

Test for a key with in

`in` on a dict checks *keys*, never values. Test values with `v in d.values()`, which scans.

```
>>> limits = {"input": 100, "output": 20}
>>> "input" in limits, 100 in limits, 100 in limits.values()
(True, False, True)
```

Assign, copy, deepcopy

Dicts are mutable, so everything from Chapter 8 applies. `b = a` is a second name for one dict. `a.copy()` is shallow: new outer dict, shared inner values. `copy.deepcopy(a)` copies all the way down.

```
>>> import copy
>>> a = {"model": "haiku", "stops": ["END"]}
>>> b = a
>>> c = a.copy()
>>> e = copy.deepcopy(a)
>>> a["stops"].append("STOP")
>>> c["stops"], e["stops"]
(['END', 'STOP'], ['END'])
>>> b is a, c is a, c["stops"] is a["stops"]
(True, False, True)
```

Compare dictionaries

== and != compare keys and values and ignore order. The ordering operators (<, >) are not defined for dicts.

```
>>> {1: 1, 2: 2, 3: 3} == {3: 3, 1: 1, 2: 2}
True
>>> {1: [1, 2]} == {1: [1, 1]}
False
>>> {1: 1} < {2: 2}
Traceback (most recent call last):
  ...
TypeError: '<' not supported between instances of 'dict' and 'dict'
```

Dictionary comprehensions

The form is `{key_expr: value_expr for item in iterable if condition}`. Use it to build or reshape dicts in one expression.

```
>>> words = ["haiku", "opus", "sonnet"]
>>> {w: len(w) for w in words}
{'haiku': 5, 'opus': 4, 'sonnet': 6}
>>> {w: len(w) for w in words if len(w) > 4}
{'haiku': 5, 'sonnet': 6}
>>> {v: k for k, v in {"a": 1, "b": 2}.items()}
{1: 'a', 2: 'b'}
```

Swapping keys and values drops data when values repeat, and fails when a value is not hashable.

Counting and grouping patterns

Two patterns appear constantly. Counting: `d.get(key, 0) + 1`. Grouping: `d.setdefault(key, [])` then `.append`. For counting alone, `collections.Counter` does it in one call (Chapter 12).

```
>>> counts = {}
>>> for ch in "letters":
...     counts[ch] = counts.get(ch, 0) + 1
>>> counts
{'l': 1, 'e': 2, 't': 2, 'r': 1, 's': 1}
>>> by_label = {}
>>> for doc, label in [("d1", "spam"), ("d2", "ok"), ("d3", "spam")]:
...     by_label.setdefault(label, []).append(doc)
>>> by_label
{'spam': ['d1', 'd3'], 'ok': ['d2']}
>>> from collections import Counter
>>> Counter("letters").most_common(1)
[('e', 2)]
```

9.2 Sets

A set is a dict with the values thrown away: a bag of *unique, hashable* items with no order and no positions. Use one when you only care whether something is in there, or when you need to remove duplicates or compare groups.

Create with set() or {}

```
>>> type({}), type(set())
(<class 'dict'>, <class 'set'>)
>>> seen = {3, 1, 3, 2}
>>> seen
{1, 2, 3}
>>> sorted(set("letters"))
['e', 'l', 'r', 's', 't']
>>> set((1,)), set([0, 2, 4]), set()
({1}, {0, 2, 4}, set())
```

> ⚠ **TRAP CARD: `{}` is an empty dict, not an empty set**
>
> Empty braces are a *dict*. Write `set()` for an empty set. Also `set()` takes *one* iterable, not several values, and a bare number is not iterable.

```
>>> set(0, 2, 4)
Traceback (most recent call last):
  ...
TypeError: set expected at most 1 argument, got 3
>>> set(5)
Traceback (most recent call last):
  ...
TypeError: 'int' object is not iterable
```

Because a set is unordered, you cannot index it, and the order of a printed set of strings can change between runs. Use `sorted(s)` when you need a stable order. Items must be hashable.

```
>>> {1, 2}[0]
Traceback (most recent call last):
  ...
TypeError: 'set' object is not subscriptable
>>> {[1, 2]}
Traceback (most recent call last):
  ...
TypeError: unhashable type: 'list'
```

Add, remove, length, membership

```
>>> s = {1, 2, 3}
>>> s.add(4)
>>> s.add(2)
>>> s.remove(3)
>>> s.discard(99)
>>> s
{1, 2, 4}
>>> len(s), 2 in s, 3 in s
(3, True, False)
>>> s.remove(99)
Traceback (most recent call last):
  ...
KeyError: 99
```

`remove()` raises `KeyError` when the item is absent. `discard()` stays quiet. Adding an item that is already there changes nothing. Testing `x in s` takes about the same time however big the set is; on a list it scans every item.

A common use is removing duplicates. A set loses the order; a dict keeps it:

```
>>> docs = ["b", "a", "b", "c", "a"]
>>> len(set(docs))
3
>>> list(dict.fromkeys(docs))
['b', 'a', 'c']
```

Set operations

Set algebra answers "what do these two groups share?" and "what is missing?". Each operator has a method twin. The methods accept any iterable; the operators need two sets.

with `a = {1, 2}` (left circle) and `b = {2, 3}` (right circle); the shaded part is the result



Operator	Method	Result is
`a | b`	`a.union(b)`	items in either
`a & b`	`a.intersection(b)`	items in both
`a - b`	`a.difference(b)`	items in `a` but not in `b`
`a ^ b`	`a.symmetric_difference(b)`	items in exactly one
`a <= b`	`a.issubset(b)`	`bool`: every item of `a` is in `b`
`a < b`	—	`bool`: proper subset (and not equal)
`a >= b`	`a.issuperset(b)`	`bool`: `a` contains all of `b`
—	`a.isdisjoint(b)`	`bool`: nothing in common

```
>>> a, b = {1, 2}, {2, 3}
>>> a | b, a & b, a - b, a ^ b
({1, 2, 3}, {2}, {1}, {1, 3})
>>> allowed = {"search", "calc", "email"}
>>> requested = {"calc", "shell"}
>>> sorted(requested & allowed), sorted(requested - allowed)
(['calc'], ['shell'])
>>> requested <= allowed, {1} <= {1, 2}, {1, 2} < {1, 2}
(False, True, False)
>>> a.union([5, 6]) == {1, 2, 5, 6}
True
>>> a | [5, 6]
Traceback (most recent call last):
  ...
TypeError: unsupported operand type(s) for |: 'set' and 'list'
```

The result of an operator is a new set, and an empty result is falsy, so `if requested - allowed:` reads as "is anything missing?".

Set comprehensions and frozenset

```
>>> {n % 3 for n in range(10)}
{0, 1, 2}
>>> sorted({w[0] for w in ["apple", "avocado", "banana"]})
['a', 'b']
>>> fs = frozenset([3, 2, 1])
>>> fs
frozenset({1, 2, 3})
>>> fs.add(4)
Traceback (most recent call last):
  ...
AttributeError: 'frozenset' object has no attribute 'add'
>>> {frozenset({1, 2}): "usable as a dict key"}
{frozenset({1, 2}): 'usable as a dict key'}
```

A `frozenset` is an immutable set. Because it is hashable it can be a dict key or a member of another set, which an ordinary set cannot.

■ **TYPE CHECK: the four comprehensions and the odd one out**

Syntax	Builds a	Duplicates	Note
`[x for ...]`	`list`	kept	ordered
`{x for ...}`	`set`	dropped	unordered
`{k: v for ...}`	`dict`	later key wins	the colon makes it a dict
`(x for ...)`	generator	kept	lazy, one pass; *not* a tuple

Which container?

	tuple	list	dict	set
Ordered?	✓	✓	✓(by insertion)	✗
Duplicates?	✓	✓	keys: ✗	✗
Mutable?	✗	✓	✓	✓
Find by	position	position	key	value (`in`)
Fast `x in`?	✗	✗	✓(keys)	✓

➤ **RULE**

Reach for a dict when you have named fields or a lookup table, a set when you ask "have I seen this?", a list when order and repeats matter, and a tuple for a small fixed record.

➤ **PRACTICE**

1. Build a dict `prices` mapping `"haiku"`, `"sonnet"`, `"opus"` to three prices. Print the price of `"sonnet"` two ways, one of which cannot raise `KeyError`.
2. Given `prices`, build the inverse dict (price to name). What goes wrong if two models share a price?
3. What is the difference between `d.get("k")` returning `None` and `d["k"]`?
4. Merge `{"a": 1, "b": 2}` and `{"b": 9}` into a new dict without changing either. Which value does `"b"` get?
5. Why does `{}` not create an empty set, and what do you write instead?
6. Given two sets of tool names, `allowed` and `requested`, write one expression for the tools that were requested but not allowed.

★ **ACHIEVEMENT UNLOCKED: Key Master**

You can now:

- create, read, update, merge, and delete dict entries, choosing between `[ ]`, `get`, and `in`;
- walk nested JSON-like data and handle missing keys safely;
- loop with `keys()`, `values()`, and `items()` without changing the dict during the loop;
- use sets for uniqueness, membership, and `|`, `&`, `-`, `^`;
- explain why a list cannot be a dict key and when to use `frozenset`.

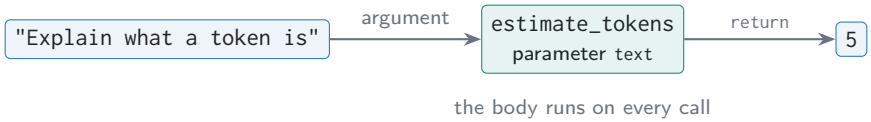
CHAPTER 10

Functions

A function is a named piece of code that you can run again and again with different inputs. This chapter shows how to define one, how arguments reach it, and how to treat functions as ordinary objects. It also covers closures, generators, decorators, scope and exceptions. These are the tools behind every API wrapper, retry loop and streaming loop you will write.

10.1 Define and call a function

`def` creates a function. Parentheses call it. The `return` statement hands a value back to the caller.



the body runs on every call

```
>>> def do_nothing():
...     pass
...
>>> do_nothing()          # nothing shown: the result is None
>>> print(do_nothing())
None
>>> def estimate_tokens(text):
...     return len(text) // 4
...
>>> estimate_tokens("Explain what a token is")
5
```

Parameters are the names in the `def` line. **Arguments** are the values you pass in the call. The value is bound to the parameter name for the duration of the call.

> ⚠ **TRAP CARD: print is not return**
>
> `print` shows text on the screen. `return` gives a value back to the code that called the function. A function that only prints returns `None`, and you cannot use its result.

```
>>> def show_tokens(text):
...     print(len(text) // 4)
...
>>> n = show_tokens("Explain what a token is")
5
>>> print(n)
None
```

A function can return only one object. If you write `return a, b`, that one object is a tuple, and you can unpack it.

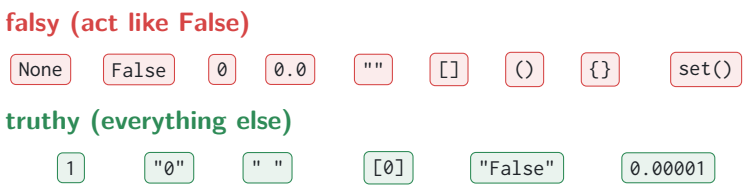
```
>>> def split_ratio(n):
...     return n * 0.8, n * 0.2
...
>>> split_ratio(100)
(80.0, 20.0)
>>> train, test = split_ratio(100)
>>> train
80.0
```

■ TYPE CHECK

Expression	Type	What you can do
`estimate_tokens`	`function`	pass it around, call it, read `.__name__`
`estimate_tokens(s)`	`int`	whatever the `return` gave back
`show_tokens(s)`	`NoneType`	nothing: no `return` means `None`
`split_ratio(100)`	`tuple`	unpack it, index it

10.2 None, truthiness and falsiness

`None` is a single special object that means "no value here". It is not the same as `False`, `0` or an empty string, but it behaves like `False` inside an `if`. Use `is` to test for `None`. `is` asks "same object?", while `==` asks "equal value?".




```
>>> bool(None), bool(0), bool(""), bool([]), bool({})
(False, False, False, False, False)
>>> bool(" "), bool([0]), bool("False")
(True, True, True)
>>> zero = 0
>>> zero == False       # equal values
True
>>> zero is False       # but two different objects
False
>>> x = None
>>> x is None
True
```

⚠ TRAP CARD: `if x:` cannot tell 0 from None

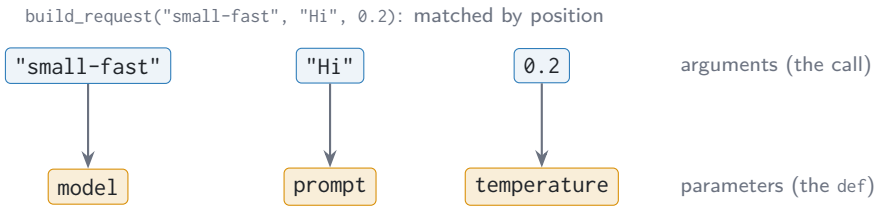
A setting of `0` is a real value. Testing it with a plain `if` treats it like "missing". Test for `None` explicitly when `0` or `""` are legal values.

```
>>> def limit_bad(max_tokens=None):
...     if max_tokens:               # 0 counts as missing
...         return max_tokens
...     return 1024
...
>>> def limit_ok(max_tokens=None):
...     if max_tokens is None:
...         return 1024
...     return max_tokens
...
>>> limit_bad(0), limit_ok(0)
(1024, 0)
```

10.3 Positional, keyword and default arguments

Positional and keyword arguments

Positional arguments are matched to parameters by position. Keyword arguments are matched by name, in any order. In a call, positional arguments must come first.




```
>>> def build_request(model, prompt, temperature=0.7):
...     return {"model": model, "prompt": prompt, "temperature": temperature}
...
>>> build_request("small-fast", "Hi")
{'model': 'small-fast', 'prompt': 'Hi', 'temperature': 0.7}
>>> build_request("Hi", "small-fast")     # swapped: no error, wrong meaning
{'model': 'Hi', 'prompt': 'small-fast', 'temperature': 0.7}
>>> build_request(prompt="Hi", model="small-fast", temperature=0)
{'model': 'small-fast', 'prompt': 'Hi', 'temperature': 0}
>>> build_request("small-fast", temperature=0.2, prompt="Hi")
{'model': 'small-fast', 'prompt': 'Hi', 'temperature': 0.2}
>>> build_request("small-fast")
Traceback (most recent call last):
  ...
TypeError: build_request() missing 1 required positional argument: 'prompt'
```

Putting a positional argument after a keyword argument is a syntax error:

```
build_request(prompt="Hi", "small-fast")
```

```
SyntaxError: positional argument follows keyword argument
```

Default values

A parameter with a default is optional. Parameters with defaults must come after those without. Prefer keyword arguments for optional settings: the call then explains itself.

```
def bad(a=1, b):
    pass
```

```
SyntaxError: parameter without a default follows parameter with a default
```

> ⚠ **TRAP CARD: a mutable default is created once**
> Python builds the default value *once*, when `def` runs. If it is a list or dict, every call that uses the default shares the same object and keeps adding to it.



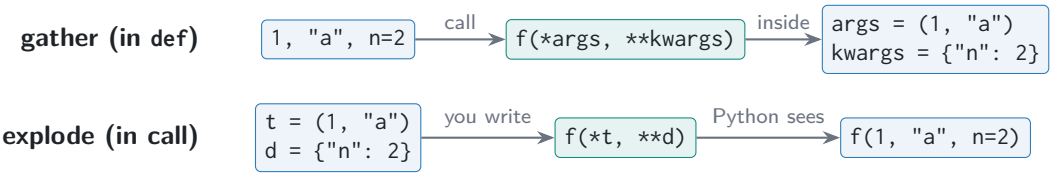

```
>>> def add_message(text, history=[]):
...     history.append(text)
...     return history
...
>>> add_message("hi")
['hi']
>>> add_message("again")
['hi', 'again']
>>> add_message.__defaults__      # the function object holds the one list
(['hi', 'again'],)
```

> ➤ **RULE: use `None` as the default**
> Default to `None`, then create the fresh list inside the function.

```
>>> def add_message(text, history=None):
...     if history is None:
...         history = []
...     history.append(text)
...     return history
...
>>> add_message("hi")
['hi']
>>> add_message("again")
['again']
```

10.4 Packing and unpacking: *args and **kwargs

In a `def` line, `*args` *gathers* extra positional arguments into a tuple, and `**kwargs` gathers extra keyword arguments into a dict. In a call, the same stars *explode* a tuple or dict back into separate arguments. The names `args` and `kwargs` are only a habit.



```
>>> def show(*args, **kwargs):
...     print(args, kwargs)
...
>>> show()
() {}
>>> show(1, "a", model="small-fast", n=2)
(1, 'a') {'model': 'small-fast', 'n': 2}
>>> nums = (1, 2)
>>> show(nums)           # one argument that happens to be a tuple
((1, 2),) {}
>>> show(*nums)          # two arguments
(1, 2) {}
>>> opts = {"model": "small-fast", "n": 2}
>>> show(**opts)
() {'model': 'small-fast', 'n': 2}
```

TYPE CHECK

Name	Type inside	Use it like
args	tuple	args[0], len(args), for a in args
kwargs	dict	kwargs["n"], kwargs.get("n", 1), .items()

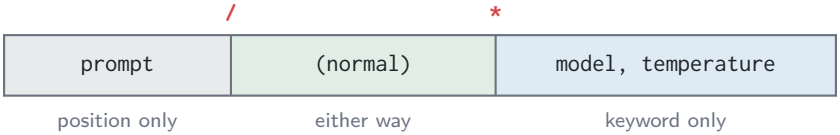
The usual order in a `def` line is: required parameters, then `*args`, then keyword parameters, then `**kwargs`. The stars only work in a definition or a call. A lone `*args` on a line by itself is a syntax error. (In a dict display, `{**a, **b}` merges two dicts.)

A typical use is a thin wrapper that forwards extra options:

```
>>> def ask(prompt, **options):
...     request = {"prompt": prompt, "temperature": 0.7}
...     request.update(options)       # caller's options win
...     return request
...
>>> ask("Hi", temperature=0, max_tokens=50)
{'prompt': 'Hi', 'temperature': 0, 'max_tokens': 50}
>>> defaults = {"a": 1, "b": 2}
>>> {**defaults, "b": 3}
{'a': 1, 'b': 3}
```

10.5 Keyword-only and position-only parameters

Two markers in the parameter list control how callers may pass arguments. A `/` means everything *before* it is position-only. A bare `*` means everything *after* it is keyword-only.

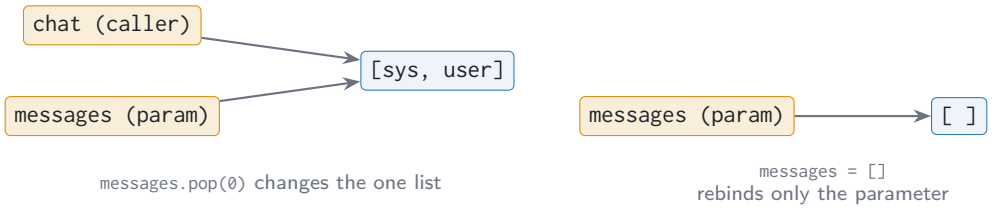


```
>>> def complete(prompt, /, *, model="small-fast", temperature=0.0):
...     return f"{model}@{temperature}: {prompt}"
...
>>> complete("Hi", model="large-slow")
'large-slow@0.0: Hi'
>>> complete("Hi", "large-slow")
Traceback (most recent call last):
  ...
TypeError: complete() takes 1 positional argument but 2 were given
>>> complete(prompt="Hi")
Traceback (most recent call last):
  ...
TypeError: complete() got some positional-only arguments passed as
keyword arguments: 'prompt'
```

> ➤ **RULE: make settings keyword-only**
>
> For functions with several optional settings, put a `*` before them. Callers must write `temperature=0`, never a bare `0`. Reordering parameters later cannot silently break old calls. SDK methods for LLM calls are written this way.

10.6 Mutable and immutable arguments

A parameter is just another name for the object the caller passed. If that object is mutable (list, dict, set), changes made through the parameter are visible to the caller. Assigning a new object to the parameter only moves the parameter's own name.




```
>>> chat = [{"role": "system", "content": "Be brief"},
...         {"role": "user", "content": "Hi"}]
>>> def drop_system(messages):          # changes the caller's list
...     messages.pop(0)
...
>>> def without_system(messages):       # returns a new list
...     return messages[1:]
...
>>> def rebind(messages):
...     messages = []                   # only the local name moves
...
>>> rebind(chat); len(chat)
2
>>> trimmed = without_system(chat)
>>> len(chat), len(trimmed)
(2, 1)
>>> drop_system(chat); len(chat)
1
```

> ➤ **RULE**
> Either return a new value, or say in the docstring that the argument is changed. Surprise mutation of the caller's data is a classic source of bugs. When you want a safe copy, use `messages.copy()` or `list(messages)`. Note that those are *shallow* copies: the dicts inside are still shared.

10.7 Docstrings

A string placed first in a function body is its docstring. Python stores it in `.__doc__`, and `help()` prints it with the signature. Write one sentence saying what the function returns.

```
>>> def cost_usd(tokens, price_per_million):
...     """Return the cost in dollars for a token count."""
...     return tokens * price_per_million / 1_000_000
...
>>> print(cost_usd.__doc__)
Return the cost in dollars for a token count.
>>> cost_usd(2_000_000, 0.10)
0.2
```

```
help(cost_usd)
```

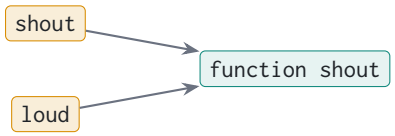
```
Help on function cost_usd in module __main__:

cost_usd(tokens, price_per_million)
    Return the cost in dollars for a token count.
```

Longer docstrings use triple quotes and may span many lines. Editors show them as hover help, and later chapters (type hints, testing) build on them.

10.8 Functions are first-class objects

Everything with a value is an object, and a function is a value. It has a type (`function`), you can bind it to other names, put it in lists and dicts, pass it to functions and return it from functions. The parentheses are what *run* it.



`loud = shout`: no parentheses, no call



a function stored as a dict value

```
>>> def shout(text):
...     return text.upper()
...
>>> loud = shout            # a second name for the same function
>>> loud is shout
True
>>> loud("hi")
'HI'
>>> type(shout), shout.__name__
(<class 'function'>, 'shout')
>>> callable(shout), callable(42)
(True, False)
```

Because functions are values, a dict of functions makes a clean "dispatch table", and a list of functions makes a pipeline:

```
>>> CLEANERS = {"upper": str.upper, "strip": str.strip, "title": str.title}
>>> def clean(text, steps):
...     for step in steps:
...         text = CLEANERS[step](text)     # look up a function, then call it
...     return text
...
>>> clean("  hello world ", ["strip", "title"])
'Hello World'
```

A function can also take a function and extra arguments, and forward them with the stars:

```
>>> def call_with(func, *args, **kwargs):
...     return func(*args, **kwargs)
...
>>> call_with(max, 3, 9, 4)
9
>>> call_with(sorted, ["b", "a"], reverse=True)
['b', 'a']
```

■ TYPE CHECK

Expression	Type	Meaning
`shout`	`function`	the function object itself
`shout("a")`	`str`	the function was *called*
`shout()`	—	`TypeError`: missing argument
`[shout, str.lower]`	`list`	a list holding two function objects

Arguments are not a tuple

Comma-separated arguments in a call are not one tuple. A function with one parameter and three arguments fails. To pass a single tuple, add a second pair of parentheses. Only `*args` on the receiving side collects arguments into a tuple.

```
>>> def one(thing):
...     return thing
...
>>> one((1, 2, 3))
(1, 2, 3)
>>> one(1, 2, 3)
Traceback (most recent call last):
  ...
TypeError: one() takes 1 positional argument but 3 were given
```

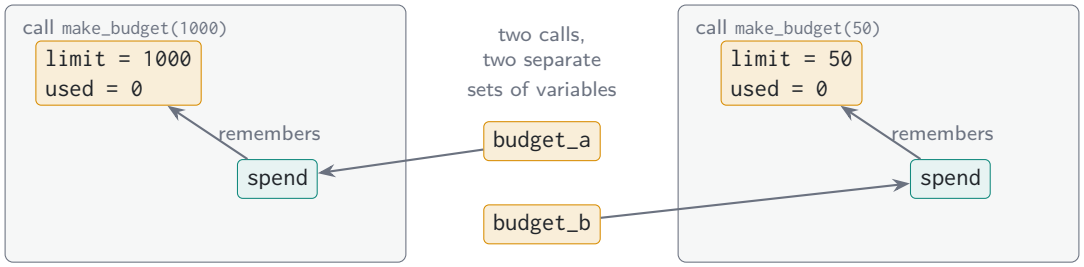
10.9 Inner functions

A function can be defined inside another function. The inner function exists only while the outer one runs, and it is a good place for a small helper that you use more than once.

```
>>> def format_usage(input_tokens, output_tokens):
...     def fmt(label, n):
...         return f"{label}: {n:,}"
...     return fmt("in", input_tokens) + " | " + fmt("out", output_tokens)
...
>>> format_usage(1200, 85)
'in: 1,200 | out: 85'
```

10.10 Closures

An inner function can read variables of the outer function. If the outer function *returns* the inner function, the inner function keeps those variables alive. That combination is a closure: a function plus the variables it remembers.




```
def make_budget(limit):
    used = 0
    def spend(tokens):
        nonlocal used          # rebind the outer variable
        used += tokens
        return limit - used
    return spend

budget_a = make_budget(1000)
budget_b = make_budget(50)
print(budget_a(300), budget_a(450))
print(budget_b(20))
```

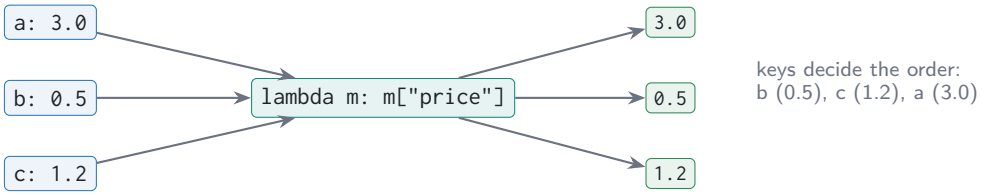
```
700 250
30
```

Each call to `make_budget` creates its own `limit` and `used`. Reading an outer variable needs nothing special. *Rebinding* it (`used += tokens`) needs `nonlocal`, otherwise Python assumes `used` is a new local variable and fails.

> ★ **KEY IDEA: a closure is a tiny object with state**
>
> Where you might write a small class that stores one value and has one method, a closure often does the same job. Reach for a class when you need several methods or want to inspect the state.

10.11 Lambda: anonymous functions

A `lambda` is a function written as one expression, with no name and no `return`. It is mostly used as a short argument, such as a sort key.



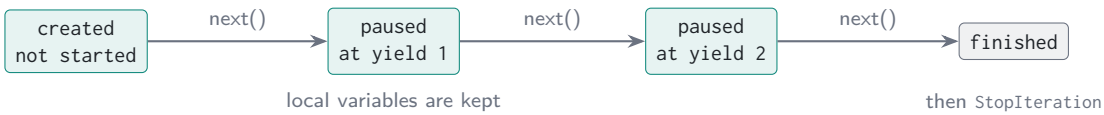

```
>>> models = [{"name": "a", "price": 3.0}, {"name": "b", "price": 0.5},
...           {"name": "c", "price": 1.2}]
>>> [m["name"] for m in sorted(models, key=lambda m: m["price"])]
['b', 'c', 'a']
>>> (lambda a, b: a + b)(2, 3)
5
>>> type(lambda: 0)
<class 'function'>
```

	def	**lambda**
Name	has one	none
Body	any number of statements	one expression only
`return`	explicit	the expression's value is returned
Best for	anything you reuse or explain	a one-off key or callback

> ➤ **RULE**
>
> If a lambda needs a name, or does not fit on one short line, write a `def` instead. Assigning a lambda to a variable gains nothing over `def`.

10.12 Generators

A generator produces values one at a time and pauses between them, remembering where it was. It never builds the whole sequence in memory. You make one by writing a function that uses `yield` instead of `return`. Calling that function does not run the body (the address in its repr will differ on your machine). It returns a *generator object*, and each `next()` runs the body until the next `yield`.



```
>>> def batches(items, size):
...     for start in range(0, len(items), size):
...         yield items[start:start + size]
...
>>> gen = batches(["a", "b", "c", "d", "e"], 2)
>>> gen
<generator object batches at 0x...>
>>> next(gen)
['a', 'b']
>>> list(gen)           # the rest
[['c', 'd'], ['e']]
>>> list(gen)           # a generator can be used only once
[]
```

To go again, call the generator function again for a fresh generator object. A *generator comprehension* is the same idea in one line, written with parentheses:

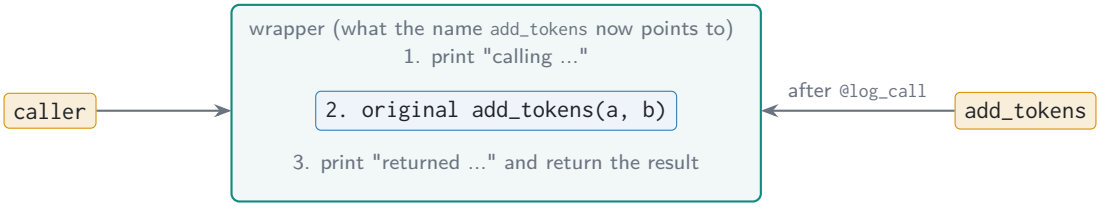
```
>>> texts = ["hi", "hello", "hey there"]
>>> lengths = (len(t) for t in texts)
>>> sum(lengths)
16
>>> sum(lengths)          # already used up
0
```

	list	**generator**
Values stored	all of them, in memory	none, made on demand
Reuse	as often as you like	once
Indexing, `len()`	yes	no
Good for	small data you revisit	big or endless streams, one pass

> ★ **KEY IDEA: streaming is a loop over a generator-like object**
>
> When an LLM API streams a reply, you loop over an iterator that hands you one chunk at a time. Chunks arrive as they are produced, and you print each one at once. It is the same "produce one, pause, produce the next" idea.

10.13 Decorators

A decorator is a function that takes a function and returns a function. It lets you wrap extra behavior (logging, timing, retrying) around a function without editing its body. The line `@log_call` above a `def` is shorthand for `add = log_call(add)`.



```
import functools

def log_call(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print("calling", func.__name__, args, kwargs)
        result = func(*args, **kwargs)
        print("returned", result)
        return result
    return wrapper

@log_call
def add_tokens(a, b):
    return a + b

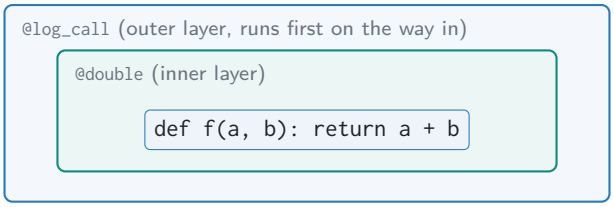
print(add_tokens(3, 5))
print(add_tokens.__name__)
```

```
calling add_tokens (3, 5) {}
returned 8
8
add_tokens
```

The wrapper takes `*args, **kwargs` so it works for any signature, and it returns the result of the original call. `functools.wraps` copies the original name and docstring onto the wrapper. Without it, `add_tokens.__name__` would be `"wrapper"`, which confuses logs and debuggers.

Stacking decorators

The decorator nearest the `def` is applied first. The one above wraps the result. Think of onion layers.




```
def double(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs) * 2
    return wrapper

@log_call
@double
def f(a, b):
    return a + b

@double
@log_call
def g(a, b):
    return a + b

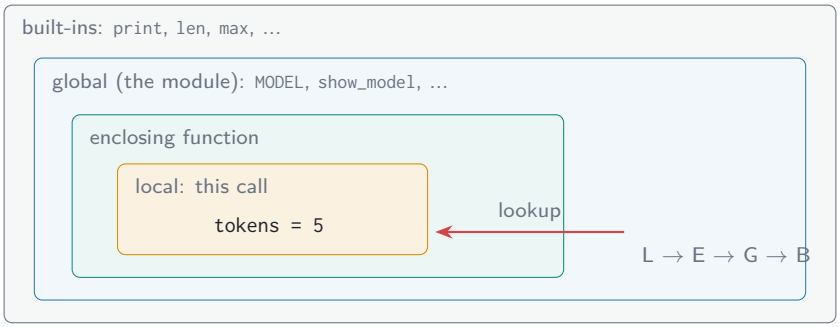
print(f(3, 5))
print(g(3, 5))
```

```
calling f (3, 5) {}
returned 16
16
calling g (3, 5) {}
returned 8
16
```

In `f`, logging sees the doubled value (16). In `g`, logging sees the original value (8) because doubling happens outside it. Order matters.

10.14 Namespaces and scope

A namespace is a table of names. Each function call gets its own. When Python meets a name, it searches: the function's own namespace first (**L**ocal), then any enclosing function (**E**nclosing), then the module (**G**lobal), then the built-ins (**B**uilt-in). This is the LEGB rule.




```
>>> MODEL = "small-fast"
>>> def show_model():
...     return MODEL                # reads the global
...
>>> def shadow():
...     MODEL = "large-slow"        # a new local, global untouched
...     return MODEL
...
>>> def broken():
...     print(MODEL)                # Python sees the assignment below ...
...     MODEL = "x"                 # ... so MODEL is local everywhere here
...
>>> def switch():
...     global MODEL
...     MODEL = "large-slow"
...
>>> show_model(), shadow(), MODEL
('small-fast', 'large-slow', 'small-fast')
>>> broken()
Traceback (most recent call last):
  ...
UnboundLocalError: cannot access local variable 'MODEL' where it is not
associated with a value
>>> switch(); MODEL
'large-slow'
```

> ⚠ **TRAP CARD: assigning makes a name local**
>
> If a function assigns to a name anywhere in its body, that name is local everywhere in that body, even in lines above the assignment. Reading a global is free. Rebinding one needs `global NAME`. Rebinding

an enclosing function's variable needs `nonlocal NAME`.

> **➤ RULE**
> Prefer parameters and return values over `global`. Functions that depend on hidden global state are hard to test. Module-level constants (named in capitals, never reassigned) are fine.

`locals()` and `globals()` return the current namespaces as dicts, which is handy for debugging:

```
>>> def peek():
...     tokens = 5
...     return locals()
...
>>> peek()
{'tokens': 5}
>>> "MODEL" in globals()
True
```

10.15 Dunder names

Names that start and end with two underscores are called dunder names (double underscore). Python reserves them. You read them, and you define some of them in classes, but you do not invent your own. Functions carry `__name__` and `__doc__`. A module carries `__name__` too. It is `"__main__"` when the file is run directly and the module's own name when it is imported.

```
# tool.py
def main():
    print("running as a script")

if __name__ == "__main__":
    main()
```

```
$ python tool.py
running as a script

$ python -c "import tool"
(no output: main() was not called)
```

The `if __name__ == "__main__":` guard lets one file act as both a script and an importable module. You will see it at the bottom of most scripts.

10.16 Recursion

A recursive function calls itself on a smaller version of the problem. It needs a *base case* that stops the calls. Each call gets its own frame on the call stack, and the frames unwind as results come back.



```
>>> def countdown(n):
...     if n == 0:                      # base case
...         return ["liftoff"]
...     return [n] + countdown(n - 1)    # smaller problem
...
>>> countdown(3)
[3, 2, 1, 'liftoff']
>>> def endless():
...     return endless()
...
>>> endless()
Traceback (most recent call last):
  ...
RecursionError: maximum recursion depth exceeded
>>> import sys
>>> sys.getrecursionlimit()
1000
```

Recursion suits uneven, nested data. A generator can walk it and `yield from` hands the work to a recursive call:

```
>>> def flatten(nested):
...     for item in nested:
...         if isinstance(item, list):
...             yield from flatten(item)      # same as looping and yielding
...         else:
...             yield item
...
>>> list(flatten(["a", ["b", ["c", "d"]], [], "e"]))
['a', 'b', 'c', 'd', 'e']
```

10.17 Async functions, briefly

Normally one line runs after another, and a slow call (a network request to a model) blocks everything. Functions defined with `async def` can pause at an `await` so that other work runs meanwhile. Many LLM SDKs offer async clients for this reason. Details come later in the concurrency chapter. Here is what to recognize:

	`def`	**`async def`**
Calling it	runs the body, returns the result	returns a coroutine; body has not run
To get the result	just call	`await` it, or `asyncio.run(...)`
`await` allowed inside	no	yes

```
import asyncio

async def fake_call(prompt):
    await asyncio.sleep(0.01)         # stands in for waiting on the network
    return prompt.upper()

async def main():
    return await asyncio.gather(fake_call("a"), fake_call("b"))

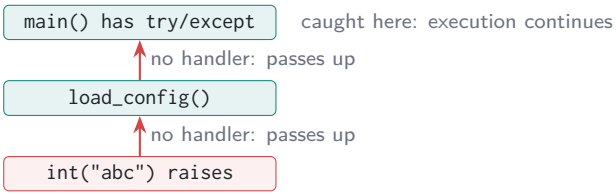
print(asyncio.run(main()))
```

```
['A', 'B']
```

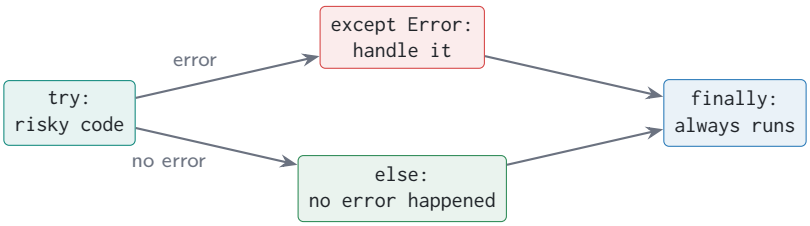
You do not need to write async code yet. Just know that `async def` and `await` mark it.

10.18 Exceptions

When something goes wrong, Python *raises an exception*. Unless some code catches it, the exception travels up the call stack, function by function, and finally stops the program with a traceback.



try, except, else, finally



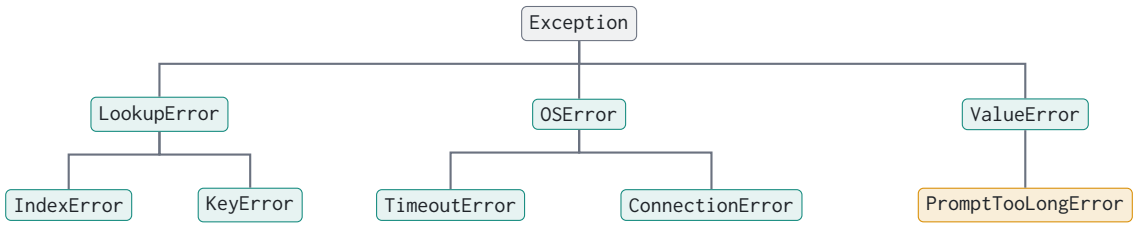

```
>>> def parse_count(text):
...     try:
...         n = int(text)
...     except ValueError as exc:           # exc is the exception object
...         print("bad number:", exc)
...         return None
...     else:
...         return n                        # only if no exception
...     finally:
...         print("parsed", repr(text))     # runs in every case
...
>>> parse_count("12")
parsed '12'
12
>>> parse_count("twelve")
bad number: invalid literal for int() with base 10: 'twelve'
parsed 'twelve'
```

Several `except` clauses are checked from top to bottom, and the first match wins. Put specific exception types before general ones. `except Exception as exc` catches almost everything that is an ordinary error and gives you the object to print or log.

> ⚠ **TRAP CARD: the bare `except:`**
>
> A bare `except:` also catches `KeyboardInterrupt`, so Ctrl-C stops working. Writing `except Exception: pass` hides real bugs. Catch the narrowest type you expect, wrap only the lines that can fail, and do something useful in the handler: log, retry, return a default, or re-raise.

Exception types form a tree

An `except` for a parent type also catches its children. That is why `LookupError` catches both `IndexError` and `KeyError`.



Raise and define your own exceptions

Use `raise` to signal a problem yourself. To make a new kind of error, write a class that inherits from `Exception` (or from a closer parent such as `ValueError`). The class body can be just `pass`: the parent supplies the behavior. Classes are the subject of the next chapter.

```
class PromptTooLongError(ValueError):
    pass

def check_prompt(text, limit=20):
    if len(text) > limit:
        raise PromptTooLongError(f"{len(text)} chars, limit {limit}")
    return text

try:
    check_prompt("x" * 50)
except PromptTooLongError as exc:
    print("caught:", exc)
    print(type(exc).__name__, exc.args)
    print(isinstance(exc, ValueError))
```

```
caught: 50 chars, limit 20
PromptTooLongError ('50 chars, limit 20',)
True
```

■ **TYPE CHECK**

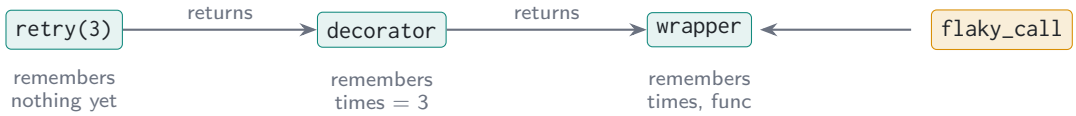
Name	Type	Notes
`PromptTooLongError`	a class	call it to make an exception object
`exc` (in `except ... as exc`)	an instance of that class	`str(exc)` is the message; `exc.args` is a tuple

■ **BOSS FIGHT: a decorator that takes arguments**

`@retry(3)` looks like one call, but it is three layers of function. Slow down and trace it.

Step	What runs	What it gives back
1	`retry(3)` runs when Python reads `@retry(3)`	`decorator`, a function that remembers `times=3`
2	`decorator(flaky_call)` runs at once	`wrapper`, which remembers `func` and `times`
3	the name `flaky_call` is rebound	now points to `wrapper`
4	each later call runs `wrapper(...)`	the result of `func`, or the last error

Three layers means three closures: `times` is remembered by `decorator` and `wrapper`, and `func` by `wrapper`.




```python
import functools

def retry(times):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except ConnectionError as exc:
                    print(f"attempt {attempt} failed: {exc}")
                    if attempt == times:
                        raise              # out of tries: pass the error on
        return wrapper
    return decorator

calls = {"n": 0}

@retry(3)
def flaky_call():
    calls["n"] += 1
    if calls["n"] < 3:
        raise ConnectionError("timeout")
    return "ok"

print(flaky_call())
```

```
attempt 1 failed: timeout
attempt 2 failed: timeout
ok
```

This is the shape of almost every retry helper you will meet when calling model APIs.

> ➤ **PRACTICE**
>
> 1. Write a function `count_words(text)` that *returns* the number of words. Then write a version that only prints the number. What does each give you when you assign the result to a variable?
> 2. Explain why `def log(msg, lines=[])` is a bug waiting to happen, and rewrite it so each call without `lines` starts with an empty list.
> 3. Write `chunk(items, n)` as a generator that yields lists of at most `n` items. Show what `list(...)` gives on a second use of the *same* generator object.
> 4. A function reads a global `DEBUG` and, a few lines later, assigns `DEBUG = True`. What error do you get, and what are two ways to fix it?
> 5. Write a decorator `timed` that prints how long a function took (use `time.perf_counter()`), and keep the original function name with `functools.wraps`.
> 6. Define your own exception `EmptyPromptError`. Write a function that raises it for an empty string, then catch it and print a friendly message while letting `finally` print "done".

★ **ACHIEVEMENT UNLOCKED: Function Wielder**

You can now:

- define functions with positional, keyword, default, `*args`, `**kwargs`, keyword-only and position-only parameters;
- pass functions around as values and build closures, generators and decorators;
- predict which variable a name refers to using the LEGB rule;
- handle and raise exceptions, including your own types.

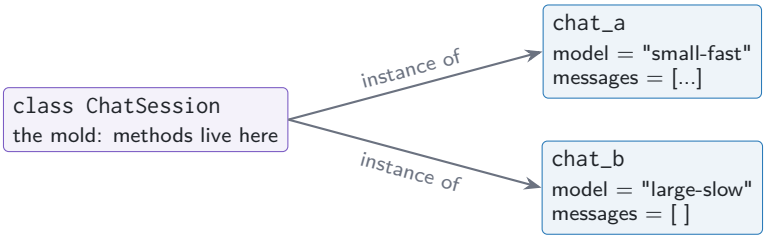
CHAPTER 11

Objects

Everything in Python is an object, but you only need to look inside objects when you build your own. This chapter explains classes, `__init__` and `self`, inheritance, properties, and the special methods (`__repr__`, `__eq__`, `__call__`) that make your objects behave like built-in ones. It ends with named tuples, dataclasses and nested objects, which is the shape of almost every API response you will handle.

11.1 What are objects?

An object bundles **data** (called *attributes*) with **code** (functions called *methods*). Think of objects as nouns and methods as verbs. The integer `7` is an object, `"hi"` is an object, and each belongs to a *class* (`int`, `str`) that says what such objects can do. A class is the mold, an object (an *instance*) is one thing made from it. Many instances of one class can exist at once, each with its own data.




```
>>> type(7), type("hi"), type([1])
(<class 'int'>, <class 'str'>, <class 'list'>)
>>> z = 3 + 4j
>>> z.real                  # an attribute: stored data
3.0
>>> z.conjugate()           # a method: a function that belongs to the object
(3-4j)
```

	attribute	**method**
What it is	a variable stored on the object (or class)	a function stored on the class
Read it	`obj.name`	`obj.name` gives the method, `obj.name()` calls it
Example	`z.real`	`z.conjugate()`

11.2 Simple objects

Define a class and create objects

The `class` keyword defines a new class. Calling the class like a function makes an instance.

```
>>> class Prompt:
...     pass
...
>>> p1 = Prompt()
>>> p2 = Prompt()
>>> p1
<__main__.Prompt object at 0x...>
>>> p1 is p2, p1 == p2       # two different objects
(False, False)
```

Assign attributes

An attribute is a variable that belongs to an object. You can attach one at any time with `obj.name = value`. Read it back with a dot. Reading one that was never set raises `AttributeError`.

```
>>> p1.text = "Say hi"
>>> p1.max_tokens = 50
>>> p1.text, p1.max_tokens
('Say hi', 50)
>>> vars(p1)                 # the object's attributes, stored as a dict
{'text': 'Say hi', 'max_tokens': 50}
>>> p2.text
Traceback (most recent call last):
  ...
AttributeError: 'Prompt' object has no attribute 'text'
```

Setting attributes one by one after creation is fragile: nothing guarantees every object gets the same ones. The fix is the initializer.

Methods, `__init__` and `self`

A method is a function defined inside a class. `__init__` is a special method that Python calls right after it creates a blank instance, so you can fill in its attributes. Every instance method receives the instance itself as its first parameter, named `self` by convention. `self.model = model` means "store the value on *this* object".



all of this happens when you write `chat = ChatSession("small-fast")`

```
>>> class ChatSession:
...     def __init__(self, model, system=None):
...         self.model = model
...         self.system = system
...         self.messages = []              # a fresh list for every instance
...
...     def add_user(self, text):
...         self.messages.append({"role": "user", "content": text})
...
...     def add_assistant(self, text):
...         self.messages.append({"role": "assistant", "content": text})
...
...     def turns(self):
...         return len(self.messages)
...
>>> chat = ChatSession("small-fast", system="Be brief")
>>> chat.add_user("Hello")
>>> chat.add_assistant("Hi!")
>>> chat.turns()
2
>>> chat.model
'small-fast'
>>> chat.messages
[{'role': 'user', 'content': 'Hello'},
 {'role': 'assistant', 'content': 'Hi!'}]
>>> ChatSession("large-slow").turns()       # a new, separate object
0
```

`__init__` is an *initializer*, not a constructor: Python has already built the object by the time it runs. It should set attributes and return nothing.

■ TYPE CHECK

Expression	Type	What you can do
`ChatSession`	`type` (a class)	call it to make instances; it is an object too
`chat`	`ChatSession`	read and set attributes, call methods
`chat.model`	`str`	any `str` method
`chat.messages`	`list`	`.append`, index, loop
`chat.add_user`	bound `method`	call it; `self` is already filled in
`ChatSession.add_user`	`function`	plain function; you must pass the instance yourself

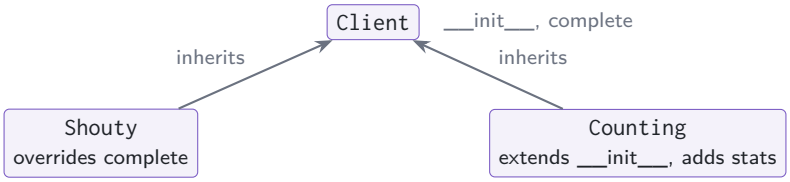
⚠ TRAP CARD: forgetting `self`

Two classic mistakes. Writing `model = model` in `__init__` only sets a local variable, so nothing is stored on the object. Leaving `self` out of a method's parameters makes Python pass one argument too many.

```
>>> class Broken:
...     def __init__(self, model):
...         model = model                   # missing "self."
...     def hello():                        # missing self
...         return "hi"
...
>>> Broken("x").model
Traceback (most recent call last):
  ...
AttributeError: 'Broken' object has no attribute 'model'
>>> Broken("x").hello()
Traceback (most recent call last):
  ...
TypeError: Broken.hello() takes 0 positional arguments but 1 was given
```

11.3 Inheritance

A subclass is a new class built from an existing one. It gets everything the parent has and may add or change things. The parent is also called the base or superclass. The relationship is *is-a*: a `Counting` client is a `Client`.



Inherit, override, add a method

```
>>> class Client:
...     def __init__(self, name):
...         self.name = name
...     def complete(self, prompt):
...         return f"[{self.name}] {prompt}"
...
>>> class Shouty(Client):                   # overrides complete
...     def complete(self, prompt):
...         return super().complete(prompt).upper()
...
>>> class Counting(Client):
...     def __init__(self, name):
...         super().__init__(name)          # let the parent set self.name
...         self.calls = 0
...     def complete(self, prompt):
...         self.calls += 1
...         return super().complete(prompt)
...     def stats(self):                    # a new method only Counting has
...         return f"{self.calls} calls"
...
```

Now use them:

```
>>> Client("a").complete("hi")
'[a] hi'
>>> Shouty("b").complete("hi")
'[B] HI'
>>> c = Counting("c")
>>> c.complete("x")
'[c] x'
>>> c.complete("y")
'[c] y'
>>> c.stats()
'2 calls'
>>> issubclass(Counting, Client), isinstance(c, Client), isinstance(c, Shouty)
(True, True, False)
>>> Client("a").stats()
Traceback (most recent call last):
  ...
AttributeError: 'Client' object has no attribute 'stats'
```

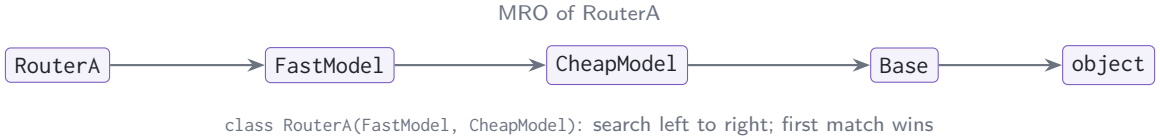
`super()` gives access to the parent's version of a method. Use it when the child does something its own way but still needs what the parent does. Defining `__init__` in a child *replaces* the parent's, so call `super().__init__(...)` yourself.

> ⚠ **TRAP CARD: a child `__init__` that skips the parent's**
>
> If the child's `__init__` never calls `super().__init__`, the attributes the parent would have set simply do not exist.

```
>>> class Forgetful(Client):
...     def __init__(self, name):
...         self.calls = 0                # forgot super().__init__(name)
...
>>> Forgetful("z").complete("hi")
Traceback (most recent call last):
  ...
AttributeError: 'Forgetful' object has no attribute 'name'
```

Multiple inheritance and method resolution order

A class can have several parents. When an attribute is missing from the object, Python looks through the *method resolution order* (MRO), a fixed list of classes. The first class that has the name wins. The order follows the parents as you wrote them.



```
>>> class Base:
...     def label(self):
...         return "base"
...
>>> class FastModel(Base):
...     def label(self):
...         return "fast"
...
>>> class CheapModel(Base):
...     def label(self):
...         return "cheap"
...
>>> class RouterA(FastModel, CheapModel):
...     pass
...
>>> class RouterB(CheapModel, FastModel):
...     pass
...
>>> RouterA().label(), RouterB().label()
('fast', 'cheap')
>>> [cls.__name__ for cls in RouterA.mro()]
['RouterA', 'FastModel', 'CheapModel', 'Base', 'object']
```

Mixins

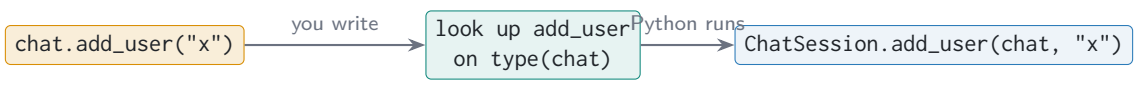
A mixin is a small parent class that adds one helper behavior and is not meant to stand alone, such as logging or JSON output. It avoids the confusion of overlapping method names because it shares no names with its siblings.

```
>>> import json
>>> class JsonMixin:
...     def to_json(self):
...         return json.dumps(vars(self))
...
>>> class Usage(JsonMixin):
...     def __init__(self, input_tokens, output_tokens):
...         self.input_tokens = input_tokens
...         self.output_tokens = output_tokens
...
>>> Usage(12, 5).to_json()
'{"input_tokens": 12, "output_tokens": 5}'
```

> ★ **KEY IDEA: inheritance is easy to overuse**
>
> Deep inheritance trees couple children tightly to their parents. Prefer composition (an object *has* other objects, see below) unless the child truly *is* a more specific parent.

11.4 In self defense

Python makes `self` visible, which many newcomers find noisy. The reason is simple: a method is an ordinary function stored in the class, and `self` is just its first argument. When you write `chat.add_user("x")`, Python looks up `add_user` on the class of `chat` and passes `chat` as the first argument.



```
>>> chat.add_user("Another")
>>> ChatSession.add_user(chat, "Third")     # exactly what Python does for you
>>> chat.turns()
4
>>> type(chat.add_user), type(ChatSession.add_user)
(<class 'method'>, <class 'function'>)
>>> chat.add_user.__self__ is chat          # bound method keeps its instance
True
```

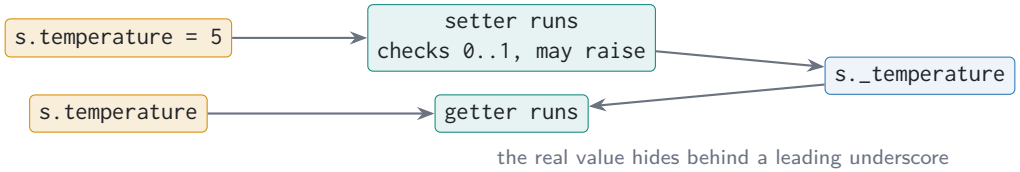
11.5 Attribute access

Python has no truly private attributes. The culture is "consenting adults": a leading underscore (`_name`) says "internal, please do not touch". Here is the ladder of options.

Style	Reading / writing	When
Direct attribute	`obj.name`, `obj.name = v`	the default; start here
Getter / setter methods	`obj.get_name()`, `obj.set_name(v)`	avoid in Python
Property	looks like `obj.name`, runs code	validation or computed values
Name mangling	`self.__name`	rarely; avoids clashes in subclasses

Getters, setters and properties

Languages like Java need getter and setter methods from day one. In Python you start with a plain attribute and switch to a *property* later if you need control. Callers keep writing `obj.name`, so no other code changes. A property is a method you access like an attribute.



```
>>> class Sampler:
...     def __init__(self, temperature=0.7):
...         self.temperature = temperature     # goes through the setter
...
...     @property
...     def temperature(self):
...         return self._temperature
...
...     @temperature.setter
...     def temperature(self, value):
...         if not 0.0 <= value <= 1.0:
...             raise ValueError(f"temperature must be 0..1, got {value}")
...         self._temperature = value
...
>>> s = Sampler()
>>> s.temperature
0.7
>>> s.temperature = 0.2
>>> s.temperature = 5
Traceback (most recent call last):
  ...
ValueError: temperature must be 0..1, got 5
>>> vars(s)
{'_temperature': 0.2}
```

The older spelling builds the same thing from two ordinary methods. The decorator form above is the one to write.

```
>>> class TokenBudget:
...     def __init__(self, tokens):
...         self._tokens = tokens
...     def get_tokens(self):
...         return self._tokens
...     def set_tokens(self, value):
...         self._tokens = value
...     tokens = property(get_tokens, set_tokens)
...
>>> b = TokenBudget(100)
>>> b.tokens = 250
>>> b.tokens
250
```

Properties for computed values

A property does not have to store anything. It can compute a value from other attributes each time. Without a setter it is read-only.

```
>>> class Usage:
...     def __init__(self, input_tokens, output_tokens):
...         self.input_tokens = input_tokens
...         self.output_tokens = output_tokens
...     @property
...     def total(self):
...         return self.input_tokens + self.output_tokens
...
>>> u = Usage(120, 30)
>>> u.total
150
>>> u.output_tokens = 80
>>> u.total                        # recomputed
200
>>> u.total = 1
Traceback (most recent call last):
  ...
AttributeError: property 'total' of 'Usage' object has no setter
```

Name mangling

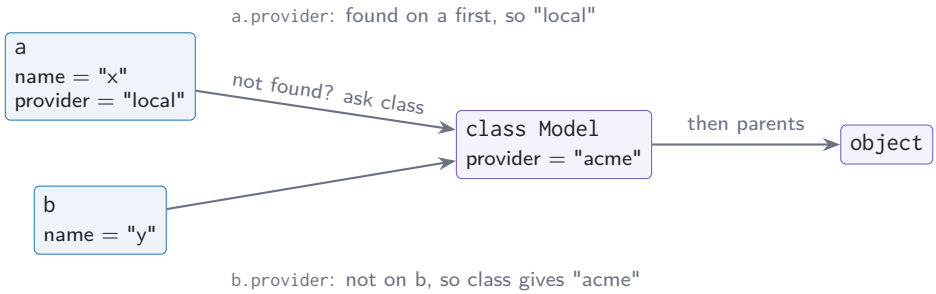
An attribute named with two leading underscores gets its name rewritten to `_ClassName__name`. This stops accidental clashes, but it is not security.

```
>>> class Client2:
...     def __init__(self, key):
...         self.__key = key
...
>>> c2 = Client2("sk-test")
>>> c2.__key
Traceback (most recent call last):
  ...
AttributeError: 'Client2' object has no attribute '__key'
>>> c2._Client2__key
'sk-test'
```

> ➤ **RULE**
> Never put real secrets in objects you print or log. A default `repr` does not show attributes, but a dataclass `repr` does. Keep API keys in environment variables and hide them from `repr` (see the dataclass section).

11.6 Class attributes and object attributes

A variable assigned in the class body belongs to the class and is shared. Instances can read it. Assigning through an instance creates a new attribute on *that instance* that hides the class one (it does not change the class). Python looks up a name on the instance first, then the class, then the parent classes.



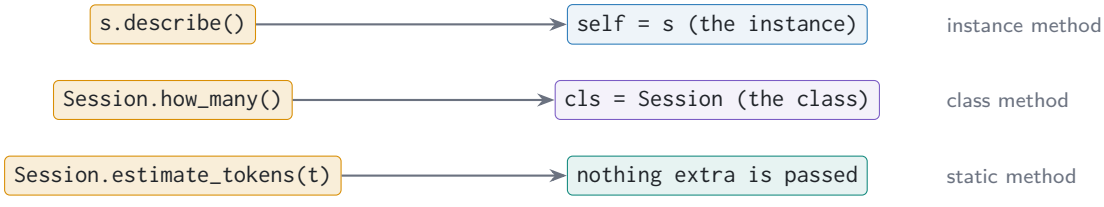

```
>>> class Model:
...     provider = "acme"                  # class attribute
...     def __init__(self, name):
...         self.name = name               # instance attribute
...
>>> a, b = Model("x"), Model("y")
>>> Model.provider, a.provider
('acme', 'acme')
>>> a.provider = "local"                  # makes an instance attribute
>>> a.provider, b.provider, Model.provider
('local', 'acme', 'acme')
>>> Model.provider = "other"              # hits instances without their own
>>> a.provider, b.provider
('local', 'other')
>>> vars(a)
{'name': 'x', 'provider': 'local'}
```

> ⚠ **TRAP CARD: a mutable class attribute is shared by all instances**
>
> `history = []` in the class body creates one list for the whole class. Every instance that appends to it changes the same list. Create per-instance lists in `__init__`.

```
>>> class Shared:
...     history = []                       # one list for everyone
...     def add(self, text):
...         self.history.append(text)
...
>>> x, y = Shared(), Shared()
>>> x.add("hi")
>>> y.history                             # y never added anything
['hi']
```

11.7 Method types

Kind	Decorator	First parameter	Use it for
Instance method	none	`self`, the object	work on one object's data
Class method	`@classmethod`	`cls`, the class	class-wide data; alternative constructors
Static method	`@staticmethod`	nothing special	a helper that lives in the class for tidiness

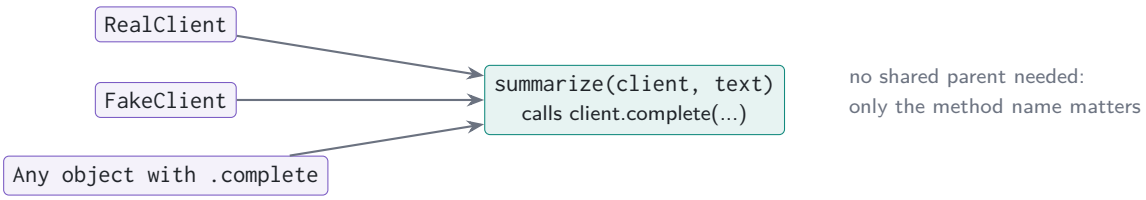


```
>>> class Session:
...     count = 0                          # class attribute: sessions made
...     def __init__(self, model):
...         self.model = model
...         Session.count += 1
...     def describe(self):                # instance method
...         return f"session on {self.model}"
...     @classmethod
...     def how_many(cls):                 # class method
...         return cls.count
...     @classmethod
...     def from_dict(cls, data):          # alternative constructor
...         return cls(data["model"])
...     @staticmethod
...     def estimate_tokens(text):         # static method
...         return len(text) // 4
...
>>> s1 = Session("small-fast")
>>> s2 = Session.from_dict({"model": "large-slow"})
>>> Session.how_many()
2
>>> s2.describe()
'session on large-slow'
>>> Session.estimate_tokens("Explain what a token is")
5
```

> ➤ **RULE**
> `@classmethod def from_dict(cls, data)` is the standard way to build an object from parsed JSON. Use `cls(...)` rather than the class name so subclasses build themselves.

11.8 Duck typing

Python does not check an object's class when you call a method. It only asks: does the object have that method? If it walks like a duck and quacks like a duck, treat it as a duck. Two unrelated classes that both have `complete(prompt)` are interchangeable for any code that only calls `complete`.



```
>>> class RealClient:
...     def complete(self, prompt):
...         return "(would call the network here)"
...
>>> class FakeClient:
...     def complete(self, prompt):
...         return "canned reply"
...
>>> def summarize(client, text):
...     return client.complete("Summarize: " + text)
...
>>> summarize(FakeClient(), "a long article")   # handy in tests: no network
'canned reply'
>>> summarize(object(), "a long article")
Traceback (most recent call last):
  ...
AttributeError: 'object' object has no attribute 'complete'
```

11.9 Magic methods

Operators and built-in functions call special methods on your object. These methods have names that start and end with two underscores ("dunder" methods). Define them and your class works with `==`, `+`, `print`, `len`, `[]` and even `()`.

You write	Python calls	Typical return
`a == b`, `a != b`	`a.__eq__(b)`, `a.__ne__(b)`	`bool`
`a < b`, `>`, `<=`, `>=`	`a.__lt__(b)`, `__gt__`, `__le__`, `__ge__`	`bool`
`a + b`, `a - b`, `a * b`	`a.__add__(b)`, `__sub__`, `__mul__`	a new object
`a / b`, `a // b`, `a % b`, `a ** b`	`__truediv__`, `__floordiv__`, `__mod__`, `__pow__`	a new object
`str(a)`, `print(a)`	`a.__str__()`	`str`, for people
`repr(a)`, REPL echo	`a.__repr__()`	`str`, for developers
`len(a)`	`a.__len__()`	`int`
`a[i]`	`a.__getitem__(i)`	the item
`a(x)`	`a.__call__(x)`	anything

Equality, ordering and math

By default `==` on your objects means "the very same object". Define `__eq__` to compare by value. If the other object is a different type, return the special value `NotImplemented` instead of crashing, and Python tries other options.

```
>>> class TokenCount:
...     def __init__(self, n):
...         self.n = n
...     def __eq__(self, other):
...         if not isinstance(other, TokenCount):
...             return NotImplemented
...         return self.n == other.n
...     def __lt__(self, other):
...         return self.n < other.n
...     def __add__(self, other):
...         return TokenCount(self.n + other.n)
...     def __repr__(self):
...         return f"TokenCount({self.n})"
...     def __str__(self):
...         return f"{self.n} tokens"
...
>>> TokenCount(3) == TokenCount(3), TokenCount(3) == 3
(True, False)
>>> TokenCount(2) + TokenCount(3)
TokenCount(5)
>>> sorted([TokenCount(5), TokenCount(2), TokenCount(9)])     # uses __lt__
[TokenCount(2), TokenCount(5), TokenCount(9)]
```

> ⚠ **TRAP CARD: `__eq__` without `__hash__`**
>
> When you define `__eq__`, Python removes the default `__hash__`, so the objects can no longer go in sets or be dict keys. If you need that, define `__hash__` as well (on objects that never change), or use a frozen dataclass.

```
>>> {TokenCount(1)}
Traceback (most recent call last):
  ...
TypeError: unhashable type: 'TokenCount'
```

`__repr__` and `__str__`

`__repr__` is the developer view. The REPL, debuggers and containers use it, and ideally it looks like code that could rebuild the object. `__str__` is the friendly view for `print` and f-strings. If you define only one, define `__repr__`: `str()` falls back to it.

```
>>> t = TokenCount(7)
>>> t                          # REPL uses repr
TokenCount(7)
>>> print(t)                   # print uses str
7 tokens
>>> f"{t}", repr(t)
('7 tokens', 'TokenCount(7)')
>>> print([t])                 # inside a list, elements use repr
[TokenCount(7)]
```

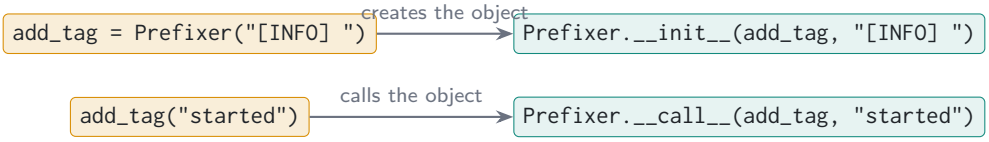
`__len__` and `__getitem__`: subscript syntax

`obj[i]` is not magic of the language, it just calls `__getitem__`. Dicts and lists define it. Your classes can too. Defining `__len__` also makes an empty object falsy.

```
>>> class Transcript:
...     def __init__(self, messages):
...         self.messages = list(messages)
...     def __len__(self):
...         return len(self.messages)
...     def __getitem__(self, i):
...         return self.messages[i]
...
>>> tr = Transcript(["hi", "yo", "bye"])
>>> len(tr), tr[0], tr[-1], tr[0:2]
(3, 'hi', 'bye', ['hi', 'yo'])
>>> [m.upper() for m in tr]            # looping works through __getitem__
['HI', 'YO', 'BYE']
>>> bool(Transcript([]))
False
```

`__call__`: objects you can call

If a class defines `__call__`, then writing `obj(args)` runs `type(obj).__call__(obj, args)`. The object behaves like a function that also remembers state in its attributes. (A closure from the previous chapter does a similar job, but an object lets you inspect and change the state.)



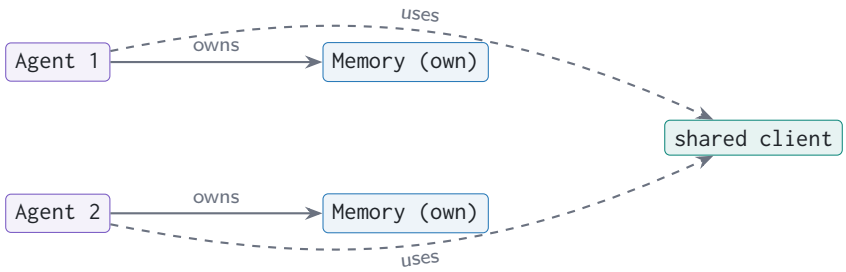

```
>>> class Prefixer:
...     def __init__(self, prefix):
...         self.prefix = prefix
...         self.calls = 0
...     def __call__(self, text):
...         self.calls += 1
...         return self.prefix + text
...
>>> add_tag = Prefixer("[INFO] ")
>>> add_tag("started")
'[INFO] started'
>>> add_tag("done")
'[INFO] done'
>>> add_tag.calls
2
>>> callable(add_tag), add_tag.__call__("manual")
(True, '[INFO] manual')
>>> list(map(add_tag, ["a", "b"]))        # works anywhere a function does
['[INFO] a', '[INFO] b']
```

The same idea gives a class-based decorator: the object is built once with the function, and every later call of the decorated name runs `__call__`.

```
>>> class CountCalls:
...     def __init__(self, func):
...         self.func = func
...         self.count = 0
...     def __call__(self, *args, **kwargs):
...         self.count += 1
...         return self.func(*args, **kwargs)
...
>>> @CountCalls
... def ping():
...     return "pong"
...
>>> ping(), ping()
('pong', 'pong')
>>> ping.count
2
```

11.10 Composition and aggregation

Inheritance says *is-a*. Composition says *has-a*: an object holds other objects as attributes and delegates work to them. In composition the parts belong to the whole (a memory created for one agent). In aggregation the whole merely uses something that exists independently (a client shared by several agents).




```
>>> class Memory:
...     def __init__(self):
...         self.items = []
...     def remember(self, text):
...         self.items.append(text)
...
>>> class Agent:
...     def __init__(self, client, memory):
...         self.client = client              # passed in: can be shared
...         self.memory = memory
...     def run(self, prompt):
...         self.memory.remember(prompt)
...         return self.client.complete(prompt)
...
>>> shared = FakeClient()
>>> a1 = Agent(shared, Memory())
>>> a2 = Agent(shared, Memory())
>>> a1.run("hi")
'canned reply'
>>> a1.memory.items, a2.memory.items
(['hi'], [])
>>> a1.client is a2.client
True
```

11.11 When to use objects or something else

Use the simplest structure that does the job. A dict, list or tuple is simpler than a class. Reach for a class when many pieces of data travel together and have behavior of their own.

Tool	Good when	Watch out
`dict`	ad-hoc, keys vary, JSON in/out	typos in keys, no structure
`tuple` / named tuple	fixed small record, never changes	positions or no behavior
`dataclass`	mostly data, you want `repr`/`==` for free	types are hints only
pydantic model	data from outside (JSON, API) that must be validated	third-party package
plain class	behavior plus state, several methods	more code to write
module	you want exactly one of something	no inheritance

> ★ **KEY IDEA**
> You can write plenty of useful Python without defining a class. But you will *use* classes constantly: API clients, response objects and data models are all classes written by others.

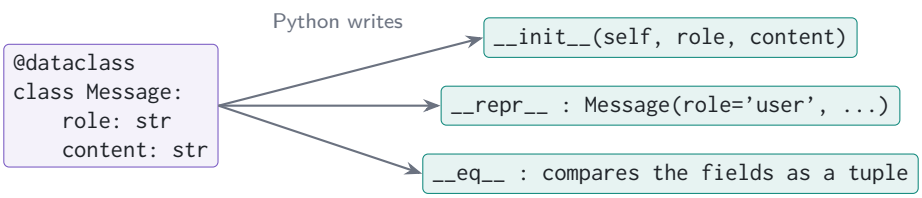
11.12 Named tuples

A named tuple is a tuple whose positions also have names. It is immutable, lightweight, and works as a dict key. You get it from `collections.namedtuple`.

```
>>> from collections import namedtuple
>>> Usage = namedtuple("Usage", "input_tokens output_tokens")
>>> u = Usage(120, 30)
>>> u
Usage(input_tokens=120, output_tokens=30)
>>> u.input_tokens, u[1]                    # by name or by position
(120, 30)
>>> inp, out = u                            # unpacks like any tuple
>>> u._replace(output_tokens=99)            # "changes" it by making a new one
Usage(input_tokens=120, output_tokens=99)
>>> u._asdict()
{'input_tokens': 120, 'output_tokens': 30}
>>> Usage(**{"input_tokens": 1, "output_tokens": 2})
Usage(input_tokens=1, output_tokens=2)
>>> u.input_tokens = 5
Traceback (most recent call last):
  ...
AttributeError: can't set attribute
```

11.13 Dataclasses

A dataclass is a normal class where Python writes `__init__`, `__repr__` and `__eq__` for you from a list of annotated fields. You write the fields as `name: type` or `name: type = default`. The decorator does the rest.



```
>>> from dataclasses import dataclass, field, asdict, replace
>>> @dataclass
... class Message:
...     role: str
...     content: str
...
>>> m = Message("user", "Hi")
>>> m
Message(role='user', content='Hi')
>>> m.content = "Yo"                        # mutable by default
>>> m == Message("user", "Yo")              # equal fields, so equal
True
>>> m is Message("user", "Yo")              # but a different object
False
>>> asdict(m)
{'role': 'user', 'content': 'Yo'}
>>> replace(m, content="Bye")               # a modified copy
Message(role='user', content='Bye')
```

To see what the decorator saved you, here is the hand-written equivalent of the first two methods. This is how `__init__`, `__repr__` and `__eq__` work in any class.

```
>>> class MessageByHand:
...     def __init__(self, role, content):
...         self.role = role
...         self.content = content
...     def __repr__(self):
...         return (f"MessageByHand(role={self.role!r}, "
...                 f"content={self.content!r})")
...     def __eq__(self, other):
...         if other.__class__ is not self.__class__:
...             return NotImplemented
...         return (self.role, self.content) == (other.role, other.content)
...
>>> MessageByHand("user", "Hi")
MessageByHand(role='user', content='Hi')
>>> MessageByHand("user", "Hi") == MessageByHand("user", "Hi")
True
```

Defaults, options and validation

> ⚠ **TRAP CARD: mutable defaults need `default_factory`**
>
> Writing `messages: list = []` would share one list between all instances, so dataclasses refuse it. Use `field(default_factory=list)` to build a fresh list per instance.

```
>>> @dataclass
... class Chat:
...     model: str
...     system: str = ""
...     messages: list = field(default_factory=list)
...
>>> Chat("small-fast")
Chat(model='small-fast', system='', messages=[])
>>> @dataclass
... class Oops:
...     items: list = []
...
Traceback (most recent call last):
  ...
ValueError: mutable default <class 'list'> for field items is not allowed:
use default_factory
```

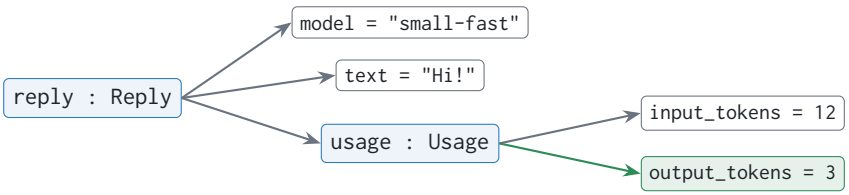
`@dataclass(frozen=True)` makes instances immutable (and hashable). `order=True` adds `<` and friends that compare the fields in order. `__post_init__` runs right after the generated `__init__`, a good place for checks. `field(repr=False)` keeps a value out of the `repr`.

```
>>> @dataclass(frozen=True, order=True)
... class Version:
...     major: int
...     minor: int
...     def __post_init__(self):
...         if self.minor < 0:
...             raise ValueError("minor must be >= 0")
...
>>> Version(1, 2) < Version(1, 10)
True
>>> v = Version(1, 2)
>>> v.minor = 3
Traceback (most recent call last):
  ...
dataclasses.FrozenInstanceError: cannot assign to field 'minor'
>>> {v: "ok"}                              # frozen means hashable
{Version(major=1, minor=2): 'ok'}
>>> @dataclass
... class Credentials:
...     user: str
...     api_key: str = field(repr=False)   # never shows up in logs
...
>>> Credentials("ana", "sk-secret")
Credentials(user='ana')
```

> ⚠ **TRAP CARD: annotations are not enforced**
> The `str` in `role: str` is only a hint. A dataclass accepts `Message(1, None)` without complaint. Enforcing types is what validation libraries such as pydantic do.

11.14 Nested objects, and attribute versus subscript access

Real API data is nested: a reply holds usage numbers, content blocks and so on. You can model that by letting one dataclass field hold another dataclass. Reading a deep value is a chain of dots.



path to the green box: `reply.usage.output_tokens` (one dot per level)

```
>>> @dataclass
... class Usage:
...     input_tokens: int
...     output_tokens: int
...
>>> @dataclass
... class Reply:
...     model: str
...     text: str
...     usage: Usage                        # a field that holds another object
...
>>> reply = Reply("small-fast", "Hi!", Usage(12, 3))
>>> reply.usage.output_tokens
3
>>> asdict(reply)                           # nested objects become dicts
{'model': 'small-fast', 'text': 'Hi!',
 'usage': {'input_tokens': 12, 'output_tokens': 3}}
>>> reply["text"]
Traceback (most recent call last):
  ...
TypeError: 'Reply' object is not subscriptable
```

	attribute: `obj.x`	**subscript: `d["x"]`**
Works on	instances of classes, dataclasses, models	dicts, lists, tuples, anything with `__getitem__`
Missing name	`AttributeError`	`KeyError` (dict) or `IndexError` (list)
Name or key is a variable	`getattr(obj, name)`	`d[name]`
Typical source	your own classes, SDK response objects	parsed JSON (`json.loads`)
Nested read	`reply.usage.output_tokens`	`data["usage"]["output_tokens"]`

From a dict to nested objects

JSON gives you nested dicts. Passing it straight to a dataclass with `**` only converts the *top* level. The inner dict stays a dict, and dot access then fails.

```
>>> data = {"model": "small-fast", "text": "Hi!",
...         "usage": {"input_tokens": 12, "output_tokens": 3}}
>>> data["usage"]["output_tokens"]
3
>>> half = Reply(**data)                    # usage is still a plain dict
>>> half.usage.output_tokens
Traceback (most recent call last):
  ...
AttributeError: 'dict' object has no attribute 'output_tokens'
>>> full = Reply(data["model"], data["text"], Usage(**data["usage"]))
>>> full.usage.output_tokens
3
>>> full == reply
True
```

Writing that conversion for every nested level is tedious. A validation library does it for you. **pydantic** (a third-party package, installed with `pip install pydantic`) builds nested models from nested dicts, converts compatible values, and raises a clear error otherwise. Many LLM SDKs return pydantic objects, so `response.usage.input_tokens` is exactly this pattern.

```
from pydantic import BaseModel, ValidationError

class UsageM(BaseModel):
    input_tokens: int
    output_tokens: int

class ReplyM(BaseModel):
    model: str
    text: str
    usage: UsageM

data = {"model": "small-fast", "text": "Hi!",
        "usage": {"input_tokens": "12", "output_tokens": 3}}
r = ReplyM.model_validate(data)        # nested dict -> nested objects
print(r.usage.input_tokens, type(r.usage).__name__)
print(r.model_dump())                  # back to plain dicts
try:
    ReplyM.model_validate({"model": "m", "text": "t"})
except ValidationError as exc:
    print(exc.error_count(), "error:", exc.errors()[0]["loc"])
```

```
12 UsageM
{'model': 'small-fast', 'text': 'Hi!',
 'usage': {'input_tokens': 12, 'output_tokens': 3}}
1 error: ('usage',)
```

Note the string `"12"` became the integer `12`, and a missing `usage` was reported by name. This block needs the `pydantic` package.

■ TYPE CHECK

Expression	Type	Note
`r`	`ReplyM`	attributes, no `["..."]`
`r.usage`	`UsageM`	a nested model object
`r.usage.input_tokens`	`int`	converted from `"12"`
`r.model_dump()`	`dict`	plain nested dicts, ready for `json.dumps`

11.15 attrs

`attrs` is a third-party library that predates dataclasses. It offers the same "write the fields, get the methods" idea plus validators and converters. It is optional reading: dataclasses cover most needs, and pydantic covers validation. If you meet `@attrs.define` in someone's code, read it as a dataclass with extras.

```
# needs: pip install attrs   (not run here: package not installed)
import attrs

@attrs.define
class Msg:
    role: str
    content: str = attrs.field(validator=attrs.validators.instance_of(str))
```

■ BOSS FIGHT: three different "calls" that look alike

`Cls(...)`, `obj.method(...)` and `obj(...)` all use parentheses, but Python does a different thing each time. If you can say which row you are in, `self`, `__init__` and `__call__` stop being confusing.

You write	Python does	`self` is
`Cls(a)`	builds a blank object, then runs `Cls.__init__(new, a)`	the new blank object
`obj.m(a)`	finds `m` on the class, runs `Cls.m(obj, a)`	`obj`
`obj(a)`	runs `Cls.__call__(obj, a)`	`obj`

```
class Tracer:
    def __init__(self, label):
        print("__init__ runs, self is the new object, label =", label)
        self.label = label

    def method(self, x):
        print("method runs, self.label =", self.label, "| x =", x)

    def __call__(self, x):
        print("__call__ runs, self.label =", self.label, "| x =", x)

t = Tracer("T")          # row 1
t.method(1)              # row 2
t(2)                     # row 3
print(t.method.__self__ is t)
```

```
__init__ runs, self is the new object, label = T
method runs, self.label = T | x = 1
__call__ runs, self.label = T | x = 2
True
```

➤ **PRACTICE**

1. Write a class `PromptTemplate` whose `__init__` takes a template string with `{name}` placeholders, and a method `render(**values)` that fills them in. Then make instances callable with `__call__` so `tpl(name="Ana")` works too.
2. Explain in your own words what `self` is, and show two ways to call the same method (one through the instance, one through the class).
3. Given `class A: items = []`, why do two instances share the list? Rewrite the class so each instance gets its own.
4. Make a `Temperature` class with a property `celsius` (stored) and a read-only property `fahrenheit` (computed). What error do you get when you assign to `fahrenheit`?
5. Write two dataclasses, `Author` and `Book`, where `Book` has an `Author` field. Build a `Book` from a nested dict, then print `asdict(book)`. Why does `Book(**data)` alone not give you an `Author` object?
6. Add `__eq__` and `__repr__` to a small class by hand, then rewrite it as a dataclass. What do you get for free, and what is still up to you?

★ **ACHIEVEMENT UNLOCKED: Object Builder**

You can now:

- write classes with `__init__`, `self`, methods, properties and class attributes, and say how each lookup works;
- reuse code with inheritance, `super()` and composition, and know when to prefer which;
- make objects work with `==`, `print`, `len`, `[]` and `()`;
- model data with named tuples and dataclasses, including nested objects and validation with pydantic.

CHAPTER 12

Modules and Packages

Until now your code lived in one file. This chapter shows how to split it across files (modules), group files into folders (packages), and reuse the large standard library. These are the skills you need before you can organise a real LLM project, or install the `anthropic` package and import it.

12.1 Modules and the import statement

A **module** is any Python file. There is nothing to declare: if a file is called `pricing.py`, then `import pricing` makes its names available.

```
# pricing.py  (the module)
print("pricing.py is running")

PRICES = {"small": (0.10, 0.50), "large": (4.00, 20.00)}  # USD per 1M tokens

def cost_usd(model, tokens_in, tokens_out):
    price_in, price_out = PRICES[model]
    return tokens_in / 1e6 * price_in + tokens_out / 1e6 * price_out
```

```
# app1.py  (same folder)
import pricing

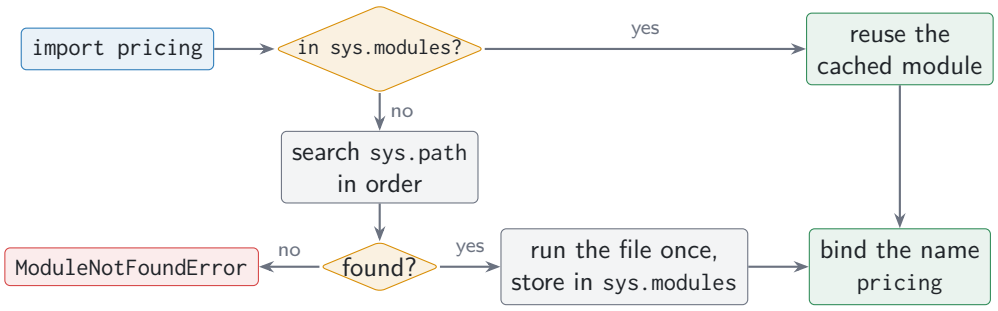
print(type(pricing))
print(pricing.__name__)
print(f"{pricing.cost_usd('small', 22, 73):.6f}")
import pricing           # second import: file is NOT run again
import sys
print("pricing" in sys.modules)
```

```
pricing.py is running
<class 'module'>
pricing
0.000039
True
```

Note that `pricing.py is running` printed only once, even though we imported twice.

> ■ **TYPE CHECK**
> `import pricing` binds the name `pricing` to a **module object** (`<class 'module'>`). Names inside it are attributes: `pricing.PRICES`, `pricing.cost_usd`. The *file* runs once, the first time it is imported.

How an import is resolved



The three import forms

Form	You get	Use when
`import pricing`	module object; write `pricing.cost_usd(...)`	default choice, no name clashes
`import pricing as p`	the same module, shorter name	long names, or two modules with the same name
`from pricing import cost_usd`	the *function itself*, no prefix	you use one or two names a lot
`from pricing import cost_usd as price`	the function under a new name	name clash with your own code
`from pricing import *`	every public name	almost never

```python
# app2.py
from pricing import cost_usd as price_it
import pricing as p

print(type(price_it))
print(price_it("large", 1000, 500))
print(p.PRICES["small"])
```

```
pricing.py is running
<class 'function'>
0.014
(0.1, 0.5)
```

■ **TYPE CHECK**

`import x` gives you a `module`. `from x import y` gives you whatever `y` is (a function, a dict, a class). Even with `from`, the whole module still runs, as the first output line shows.

⚠ **TRAP CARD**

`from module import *` dumps unknown names into your file. A later line can silently overwrite one of your own variables, and you cannot tell where a name came from. Import the names you need, or the module.

➤ **RULE: where to put imports**

Put imports at the top of the file, standard library first, then third-party (such as `anthropic`), then your own modules. Everyone can see the dependencies at a glance. Import inside a function only when it is rarely used or slow to load.

`__name__` and running a module as a script

Every module has a `__name__`. If you import it, the name is the module name. If you run the file directly, the name is `"__main__"`. This lets one file be both a library and a script.

```
# greet.py
def hello(name):
    return f"hello {name}"

if __name__ == "__main__":      # only true when run directly
    print(hello("world"))
```

12.2 Packages

A **package** is a folder of modules. You import from it with a dotted name. Put an (often empty) `__init__.py` in the folder: it marks the folder as a regular package and its code runs when the package is first imported.

```
chatapp
├── main.py
└── chatkit                 the package
    ├── __init__.py         runs on import chatkit
    ├── tokens.py           import chatkit.tokens
    └── costs.py            import chatkit.costs
```

Here are the three small files. They are plain Python.

```
# chatkit/tokens.py
def rough_tokens(text):
    """Very rough estimate: about 4 characters per token."""
    return max(1, len(text) // 4)
```

```
# chatkit/costs.py
from .tokens import rough_tokens      # relative import (same package)
from chatkit import tokens            # absolute import (also fine)

RATE_IN = 0.10  # USD per 1M input tokens (made up)

def prompt_cost(text):
    return rough_tokens(text) / 1e6 * RATE_IN

def same_function():
    return rough_tokens is tokens.rough_tokens
```

```
# main.py  (run from the chatapp folder)
from chatkit import costs, tokens

print(tokens.rough_tokens("Hello, how many tokens am I?"))
print(f"{costs.prompt_cost('x' * 4000):.8f}")
print(costs.same_function())
```

```
7
0.00010000
True
```

■ **TYPE CHECK**

`chatkit` is a package object, `chatkit.costs` is a module object inside it. Both have type `module`. `from chatkit import costs` works because `costs` is a submodule; `costs.prompt_cost` is then a plain function.

➤ **RULE: be explicit in `__init__.py`**

A one-line docstring is enough for an `__init__.py`. If you want users to write `from chatkit import prompt_cost`, re-export it there with `from .costs import prompt_cost`. Do not put heavy logic in it.

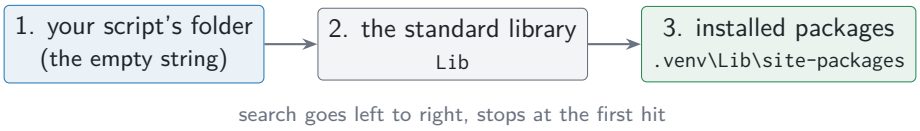
12.3 The module search path

Python looks for a module in the folders listed in `sys.path`, in order. The **first match wins**.

```python
import sys
for place in sys.path:
    print(repr(place))
```

```
''
'C:\\...\\Python312\\python312.zip'
'C:\\...\\Python312\\DLLs'
'C:\\...\\Python312\\Lib'
'C:\\...\\Python312'
'C:\\...\\ai-engineering\\.venv'
'C:\\...\\ai-engineering\\.venv\\Lib\\site-packages'
```

(Output from `python -c` in a project with a `.venv`; paths shortened. When you run a script file, the first entry is the script's folder instead of the empty string.)



search goes left to right, stops at the first hit

You can add a folder at run time, but prefer a proper install or running from the project root.

```python
import sys
sys.path.insert(0, r"C:\my\modules")   # searched first
```

⚠ **TRAP CARD**

Shadowing. A file in your folder with the same name as a library hides the library. Name a file `random.py` (or `json.py`, or, in an AI project, `anthropic.py`) and `import random` loads *your* file.

```python
# random.py (yours)
def choice(items):
    return items[0]
```

```
# pick.py
import random
print(random.__file__)
print(random.randint(1, 6))
```

```
C:\...\shadow\random.py
Traceback (most recent call last):
  File "...\shadow\pick.py", line 3, in <module>
    print(random.randint(1, 6))
AttributeError: module 'random' has no attribute 'randint'
```

Clue: the error says the *module* lacks the name. Print `module.__file__` to see which file was really loaded, then rename your file.

12.4 Relative and absolute imports

Style	Example	Meaning
absolute	`from chatkit import tokens`	start at a folder on `sys.path`
relative, same folder	`from . import tokens`	start at the current package
relative, same folder, one name	`from .tokens import rough_tokens`	the same, picking one name
relative, parent	`from .. import config`	one package up
relative, sibling package	`from ..utils import clean`	up, then down into `utils`

Relative imports only work *inside a package*. They need to know the parent package. That causes the most common beginner error:

```
> python chatkit\costs.py
Traceback (most recent call last):
  File "...\chatkit\costs.py", line 1, in <module>
    from .tokens import rough_tokens
ImportError: attempted relative import with no known parent package
```

Run the module *as part of its package* with `-m`, from the project root:

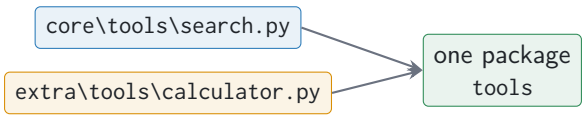
```
> python -m chatkit.costs
running as a script, __name__ = __main__
1.4999999999999999e-05
```

(That ends with the two-line `if __name__ == "__main__":` block added to `costs.py`.)

> ➤ **RULE**
> In an application, prefer absolute imports (`from chatkit import tokens`); they read the same everywhere. Use relative imports for tight, internal links in a package you may rename or move. Run package code with `python -m package.module` from the project root.

12.5 Namespace packages

If a folder has *no* `__init__.py`, Python can treat it as a **namespace package**. The useful trick: one package name can be spread across several folders on `sys.path`, and Python merges them. This lets separately shipped plug-ins share one name.




```
# demo.py  (folders core/tools/search.py and extra/tools/calculator.py,
#           no __init__.py anywhere)
import sys
sys.path.insert(0, "core")
sys.path.insert(0, "extra")

from tools import search, calculator    # two files, two folders, one name
import tools
print(type(tools), len(list(tools.__path__)), tools.__file__)
```

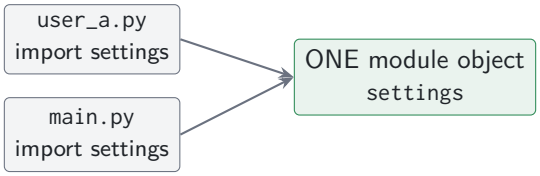
```
<class 'module'> 2 None
```

A namespace package has no single file (`__file__` is `None`) and its `__path__` lists all the folders. For your own small project, just use a normal package with `__init__.py`.

12.6 Modules versus objects

Modules and objects look alike: both give you `thing.name`. The difference is the number of copies.

	Module	Class / object
Copies in a program	exactly one (a *singleton*)	as many instances as you create
Hiding data	none; every name is public by convention	`_name`, properties, methods
Good for	shared constants, config, helper functions	many things with their own state




```
# settings.py:   MODEL = "small"
# user_a.py:     import settings
#                def show(): return settings.MODEL
import math
import settings
import user_a

print(user_a.show())
settings.MODEL = "large"        # change it in one place...
print(user_a.show())            # ...every importer sees it
print(settings is user_a.settings)

math.pi = 3.0                   # legal! but only inside this process
print(math.pi)
```

```
small
large
True
3.0
```

Restarting Python restores `math.pi` to `3.141592653589793`. You changed the copy in memory, not the file on disk.

> ⚠ **TRAP CARD**
> Because a module is shared, changing its variables is a hidden side effect for every other importer. Use module-level *constants* freely. For state that varies per user or per conversation, make a class (Chapter 11) so each one gets its own instance.

12.7 Goodies in the standard library

"Batteries included": before writing a helper, check whether the standard library has it. Five tools you will use constantly follow.

Missing keys: `setdefault` and `defaultdict`

`d[key]` on a missing key raises `KeyError`. `d.get(key, default)` returns a default but does not store it. `setdefault` returns the default *and stores it*.

```
usage = {"haiku": 3}
print(usage.get("opus", 0))           # 0, dict unchanged
print(usage)
print(usage.setdefault("opus", 0))    # 0, and now stored
print(usage)
print(usage.setdefault("haiku", 99))  # key exists: old value wins
```

```
0
{'haiku': 3}
0
{'haiku': 3, 'opus': 0}
3
```

A `defaultdict` sets the default up front. Pass it a *function with no arguments* (a "factory"); it is called when a key is missing. The factory is the *function*, so write `list`, not `list()`.

```
from collections import defaultdict

by_model = defaultdict(list)
by_model["haiku"].append("hi")        # no KeyError: starts as []
by_model["opus"].append("yo")
by_model["haiku"].append("again")
print(dict(by_model))

tally = defaultdict(int)              # int() returns 0
for m in ["haiku", "opus", "haiku"]:
    tally[m] += 1
print(tally)

plain = {}
try:
    plain["haiku"] += 1               # a normal dict has no default
except KeyError as e:
    print("KeyError:", e)
```

```
{'haiku': ['hi', 'again'], 'opus': ['yo']}
defaultdict(<class 'int'>, {'haiku': 2, 'opus': 1})
KeyError: 'haiku'
```

■ TYPE CHECK

`defaultdict` is a subclass of `dict`. The argument is a **callable** that takes no arguments: `int` gives `0`, `list` gives `[]`, `dict` gives `{}`, and `lambda: "n/a"` gives any value you like. With no argument it behaves like a normal dict.

⚠ TRAP CARD

Reading a missing key in a `defaultdict` *creates* it. After `by_model["ghost"]`, the test `"ghost" in by_model` is `True`. Use `.get()` when you only want to look.

Counting: `Counter`

```
from collections import Counter

words = "the cat saw the dog and the cat ran".split()
c = Counter(words)
print(c)
print(c.most_common(2))     # top two, as (item, count) tuples
print(c["unicorn"])         # missing -> 0, no KeyError
```

```
Counter({'the': 3, 'cat': 2, 'saw': 1, 'dog': 1, 'and': 1, 'ran': 1})
[('the', 3), ('cat', 2)]
0
```

Counters combine with operators, like sets of items with counts.

Operator	Meaning	a = x:2, y:1, b = x:1, z:1
a + b	add counts	x:3, y:1, z:1
a - b	subtract, drop zero and negative	x:1, y:1
a & b	lower count of common items	x:1
a | b	higher count of any item	x:2, y:1, z:1

```
a = Counter(["x", "x", "y"])
b = Counter(["x", "z"])
print(a + b, a - b, a & b, a | b, sep="\n")
print(isinstance(c, dict))
```

```
Counter({'x': 3, 'y': 1, 'z': 1})
Counter({'x': 1, 'y': 1})
Counter({'x': 1})
Counter({'x': 2, 'y': 1, 'z': 1})
True
```

`OrderedDict`: mostly history

Since Python 3.7 a normal `dict` remembers insertion order, so you rarely need `OrderedDict`. Two differences remain: it compares order too, and it has `move_to_end`.

```
from collections import OrderedDict

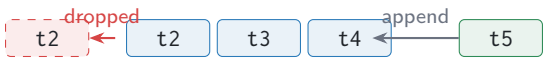
d1 = {"a": 1, "b": 2}
d2 = {"b": 2, "a": 1}
print(d1 == d2)                                 # order ignored
print(OrderedDict(d1) == OrderedDict(d2))       # order matters
od = OrderedDict(a=1, b=2, c=3)
od.move_to_end("a")
print(list(od))
```

```
True
False
['b', 'c', 'a']
```

Stack, queue, and `deque`

A **stack** is last-in, first-out (a pile of plates). A **queue** is first-in, first-out (a line). A `deque` ("deck", double-ended queue) is fast at both ends, so it can be either. A `list` is slow when you remove from the front.

Its best feature for chat apps is `maxlen`: old items fall off automatically, a ready-made sliding window of recent messages.



`deque(maxlen=3)` keeps only the newest 3

```python
from collections import deque

history = deque(maxlen=3)
for turn in ["t1", "t2", "t3", "t4"]:
    history.append(turn)           # t1 falls off when t4 arrives
print(history)
history.appendleft("t0")           # t4 falls off the right instead
print(history)
print(history.popleft(), history.pop(), history)
```

```
deque(['t2', 't3', 't4'], maxlen=3)
deque(['t0', 't2', 't3'], maxlen=3)
t0 t3 deque(['t2'], maxlen=3)
```

`itertools`: loops without the loop code

Each function returns an **iterator**: it produces one item at a time and remembers where it is. Wrap in `list(...)` to see everything. Never call `list()` on an infinite one such as `cycle`.

```python
import itertools

print(list(itertools.chain([1, 2], ["a"])))             # one long sequence
print(list(itertools.islice(itertools.cycle("ab"), 5))) # cap an endless one
print(list(itertools.accumulate([10, 20, 5])))          # running total
print(list(itertools.accumulate([1, 2, 3, 4], lambda a, b: a * b)))
print(list(itertools.batched(range(7), 3)))             # Python 3.12+
print(list(itertools.combinations("abc", 2)))
print(list(itertools.product("ab", [1, 2])))
```

```
[1, 2, 'a']
['a', 'b', 'a', 'b', 'a']
[10, 30, 35]
[1, 2, 6, 24]
[(0, 1, 2), (3, 4, 5), (6,)]
[('a', 'b'), ('a', 'c'), ('b', 'c')]
[('a', 1), ('a', 2), ('b', 1), ('b', 2)]
```

`batched` is handy for sending documents to an API in chunks of n. It exists from Python 3.12, so older interpreters will not have it.

`random`

Call	Returns
`choice(seq)`	one item from a sequence
`sample(seq, k)`	`k` different items (no repeats of the same position)
`randint(a, b)`	integer, **both ends included**
`randrange(a, b, step)`	like `range`: `b` is excluded
`random()`	float in $[0.0, 1.0)$
`shuffle(list)`	shuffles the list *in place*, returns `None`
`seed(n)`	makes the sequence repeatable

```python
import random

random.seed(7)
first = random.choice(["a", "b", "c"])
random.seed(7)                          # same seed ...
second = random.choice(["a", "b", "c"])
print(first == second)                  # ... same result
print(random.randint(1, 6) in range(1, 7))
```

```
True
True
```

> ➤ **RULE**
>
> Seed the generator in tests and experiments (for example when sampling an evaluation set), so a run can be reproduced. `random` is not secure; for tokens or secrets use the `secrets` module.

12.8 More batteries: third-party code

When the standard library is not enough, look on **PyPI** (the Python Package Index; what `pip` searches), on GitHub, and on Read the Docs for documentation. The `anthropic` SDK you use is one such package. Before installing an unknown one, check the maintainer, the release date, the download count and the source repository. Installation is the topic of Chapter 13.

> ■ **BOSS FIGHT: why did `import` give me the wrong thing?**
>
> Three facts explain nearly every import bug: (1) Python searches `sys.path` in order and takes the **first** match; (2) the result is cached in `sys.modules`, so the file runs only once; (3) the first entry of `sys.path` is your script's folder, so your own files can hide library modules. When an import misbehaves, print `module.__file__` and `sys.path[:3]`. If the path is not what you expect, you have a shadowing or a working-directory problem, not a bug in the library.

> ➤ **PRACTICE**
>
> 1. Create `tokens.py` with a function `count(text)` that returns the number of words. Import it in four ways (module, alias, one name, one name with alias) and call it each time.
> 2. You run `python app.py` and get `AttributeError: module 'json' has no attribute 'loads'`. What do you check first, and with which line of code?
> 3. Why does `python helpers/util.py` fail on `from . import config`, and what command fixes it?
> 4. Write a loop that counts how many times each `stop_reason` string appears in a list, once with `defaultdict(int)` and once with `Counter`.
> 5. Keep only the last 5 messages of a chat using one data structure. Which one, and why not a list?
> 6. Use `itertools.batched` to split 10 file names into groups of 4. What does the last group look like?

> ★ **ACHIEVEMENT UNLOCKED: Module Mapper**
>
> - Split a program into modules and a package, and import from it four ways.
> - Explain how Python finds a module and why a file called `random.py` breaks things.
> - Choose `defaultdict`, `Counter` or `deque` instead of hand-written loops.
> - Run package code with `python -m`, and know why relative imports need it.

CHAPTER 13

Development Environment

Writing code is half the job. The other half is the setup around it: where to get packages, how to keep each project's packages separate, which editor to use, and how to save your history safely. This chapter is the setup you need for every Python and `anthropic` project.

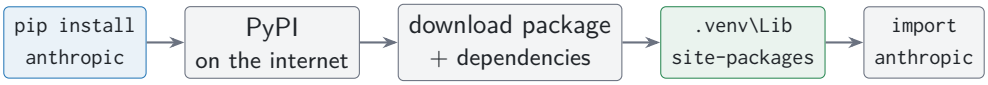
13.1 Find Python code

Before writing something, check if it already exists. Look in this order:

Source	What it is	Notes
Standard library	ships with Python, no install	start here (Chapter 12)
PyPI	the Python Package Index; what `pip` downloads from	more than half a million packages
GitHub	source code hosting	sort by stars; check the last commit date
Recipe collections	short snippets for one task	good for ideas, read before you paste

> ➤ **RULE: judge a package before installing**
>
> Open its PyPI page and check: recent releases, a source link, a reasonable download count, a maintainer you can identify. Copy the name *exactly*: fake packages with near-identical names (`anthropic` versus a typo) exist to steal API keys.

13.2 Install packages



Tool	What it does	When to pick it
`pip`	installs packages from PyPI; comes with Python	the default; learn it first
`venv`	standard-library module that makes virtual environments	every project
`virtualenv`	third-party ancestor of `venv`	old projects, extra features
Pipenv	pip + virtualenv + a lock file	legacy teams
Poetry	dependency management through `pyproject.toml`	publishing packages
Conda	installs Python *and* non-Python libraries (science stack)	data science, GPU libraries
uv	very fast tool written in Rust; replaces pip, venv and more	new projects, if you want speed

pip

On Windows, prefer `python -m pip`. It runs pip *for the exact interpreter* on the left, so you cannot install into one Python and run another.

```
> python -m pip install anthropic              latest version
> python -m pip install "anthropic>=0.40"      minimum version
> python -m pip install pytest==8.3.4          exact version
> python -m pip install --upgrade anthropic    update
> python -m pip uninstall anthropic            remove
> python -m pip list                           what is installed here
> python -m pip freeze                         the same, in requirements format
```

(The text on the right is a note, not part of the command. Not run here: these download from the internet and change an environment. The version numbers are only examples of the syntax.)

> ⚠ **TRAP CARD**
> Quote any specifier with >, < or = signs. In both PowerShell and cmd the shell reads > as "redirect output to a file", so `pip install anthropic>=0.40` creates a file called `=0.40`.

Specifier	Meaning
pkg	any version, usually the newest
pkg==1.4.2	exactly this version
pkg>=1.4	this version or newer
pkg>=1.4,<2	newer than 1.4 but not 2.x
pkg~=1.4.2	compatible: at least 1.4.2, below 1.5

Requirements file. A text file listing the packages a project needs, one per line. Anyone can rebuild the same setup with one command.

```
# requirements.txt
anthropic>=0.40
pytest>=8
python-dotenv

> python -m pip install -r requirements.txt
```

> ➤ **RULE**
> Keep `requirements.txt` in the project and commit it to Git. Use `pip freeze > requirements.txt` to record the *exact* versions that work today, so a teammate (or you in six months) gets the same ones.

Other ways to install

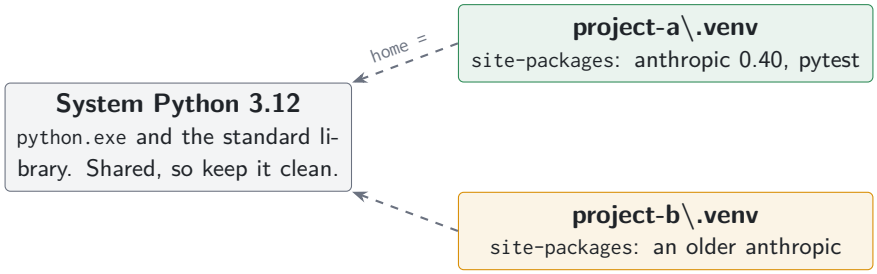
- **Native package managers** install programs, not just Python packages: Homebrew on macOS, `apt` on Linux, and on Windows the MSI installer or `winget`. Python itself on Windows usually comes this way, along with the `py` launcher.
- **From source.** If a project is not on PyPI, download or `git clone` it, then run `python -m pip install .` inside its folder. (The book mentions `python setup.py install`; that old command is no longer recommended.) You can also install straight from Git with `pip install git+https://...`.

> ⚠ **TRAP CARD**
> Installing code means running someone else's code on your machine, with access to your files and environment variables, including your `ANTHROPIC_API_KEY`. Pure-Python packages are readable, but malware in them has happened. Install only what you trust.

13.3 Virtual environments

Why you need one

Project A may need `anthropic` 0.40, project B an older release. If both install into the one system Python, they collide. A **virtual environment** is a private folder holding a Python interpreter link plus its own `site-packages`. Each project gets its own.



Create and use one with `venv`

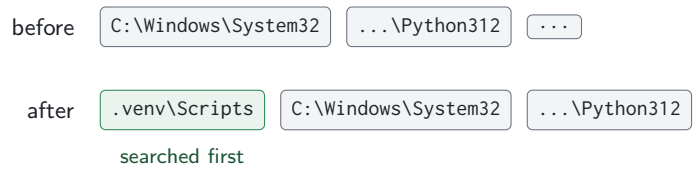
```
> cd my-chat-app
> python -m venv .venv                  create the folder .venv
> .venv\Scripts\activate                cmd.exe
> .venv\Scripts\Activate.ps1            PowerShell  (see the trap below)
(.venv) > python -m pip install anthropic
(.venv) > deactivate                    leave the environment
```

On macOS and Linux the activation command is `source .venv/bin/activate`, and the folder is `bin` instead of `Scripts`. This is what the folder contains (real listing on Windows):

```
.venv
├─ pyvenv.cfg              records the base Python (home =)
├─ Include                 C headers (rarely used)
├─ Lib\site-packages       packages for THIS project
└─ Scripts                 python.exe  pip.exe  activate scripts
```

What "activate" really does

Your shell finds programs by searching the folders in the `PATH` environment variable, left to right. Activation puts `.venv\Scripts` at the *front*, so typing `python` or `pip` finds the project's copies first. Nothing else changes, and only in that terminal window.



> ⚠ **TRAP CARD**
>
> **PowerShell may refuse the script.** Windows can block `Activate.ps1` with an error about "running scripts is disabled". You do not need activation at all: call the interpreter directly, `.venv\Scripts\python.exe -m pip install anthropic`. VS Code selects the interpreter for you (command palette, *Python: Select Interpreter*) and activates it in new terminals.

Am I inside the venv?

Two attributes tell you. `sys.prefix` is the environment you run in; `sys.base_prefix` is the Python it was built from. They differ only inside a venv.

```python
# envcheck.py
import os
import sys

print(sys.executable.endswith("python.exe"))
print("in a venv:", sys.prefix != sys.base_prefix)

key = os.environ.get("DEMO_API_KEY")        # None if the variable is missing
print(type(key))
print("key is set" if key else "key is missing")

os.environ["DEMO_PORT"] = "8080"            # values are always str
port = os.environ["DEMO_PORT"]
print(type(port), int(port) + 1)

try:
    os.environ["NOPE_NOT_SET"]
except KeyError as err:
    print("KeyError:", err)
```

```
True
in a venv: True
<class 'NoneType'>
key is missing
<class 'str'> 8081
KeyError: 'NOPE_NOT_SET'
```

(Run with the interpreter of a venv created by `python -m venv .venv`.)

> ■ **TYPE CHECK**
>
> `os.environ` behaves like a `dict[str, str]`: every value is a **string**, so convert with `int(...)` yourself. `os.environ["X"]` raises `KeyError` when missing; `os.environ.get("X")` returns `None`. That is why you check an `Optional[str]` before using an API key.

> ➤ **RULE: keep secrets out of code**
>
> Put `ANTHROPIC_API_KEY` in an environment variable or a `.env` file, never in a `.py` file. List `.env` and `.venv/` in `.gitignore` so neither is ever committed.

The other tools, in a sentence each

- **virtualenv**: the older third-party tool that `venv` was modelled on; same idea.
- **Pipenv**: runs pip and a virtualenv for you and writes a lock file of exact versions.
- **Poetry**: manages dependencies and packaging from a `pyproject.toml` file; needs a recent Python 3.
- **Conda**: a general package manager (not only Python), common for scientific stacks.
- **uv**: a fast modern replacement for most of the above. Same ideas, different commands: `uv venv`, `uv pip install`.

13.4 Integrated development environments

An IDE adds to a plain editor: autocomplete, inline docs, formatting and style checks, a file tree, source control, test and debug buttons. Pick one and learn it well.

Tool	Kind	Comment
Notepad++, vi/Vim, Neovim	text editors	fast and light; Vim keys take weeks to learn
Sublime Text, Zed	fast code editors	Zed has built-in AI assistant features
IDLE	Python's own small IDE	ships with Python; fine for first steps
PyCharm	full Python IDE	free Community edition; heavy but complete
VS Code	editor plus extensions	**free, most popular; you use it**; install the Python extension
Cursor	VS Code fork with AI built in	same look and settings as VS Code
IPython	improved interactive shell	`?` help, Tab completion, history lists `In` and `Out`
Jupyter, JupyterLab	notebooks: code, output and text in cells	data and experiments; also inside VS Code

IPython and Jupyter

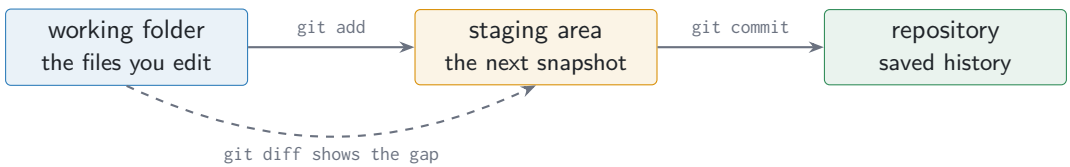
IPython is a nicer version of the plain interactive Python prompt. Install it with `python -m pip install ipython`. Typing `name?` shows the type, value and docstring of any object, and the *Tab* key completes names. Jupyter wraps this in a browser notebook (`pip install jupyterlab`, run `jupyter lab`): you run small cells and see results below each one. Notebooks are excellent for trying out prompts and looking at data.

> ⚠ **TRAP CARD**
> A notebook keeps variables alive between cells, in the order you *ran* them, not the order they appear. A result can depend on a cell you deleted. Before trusting a notebook, restart the kernel and run all cells from the top. For application code, use plain `.py` files.

13.5 Source control

Source control records every saved version of your project, so you can see what changed, undo a mistake or work with others. **Git** is the standard; **Mercurial** (written in Python) is similar but less common. Both are *distributed*: every copy holds the full history. **GitHub** is a website that hosts Git repositories.

The three places a change lives



A first repository, step by step

Real output from a scratch folder (Git 2.x on Windows):

```
> git init -b main
Initialized empty Git repository in .../repo/.git/

> echo print("Oops") > hello.py
> git add hello.py
> git status
On branch main
No commits yet
Changes to be committed:
        new file:   hello.py

> git commit -m "add hello script"
[main (root-commit) 8357896] add hello script
 1 file changed, 1 insertion(+)
 create mode 100644 hello.py
```

Now edit `hello.py` (change `Oops` to `Hi`) and look at the change before saving it:

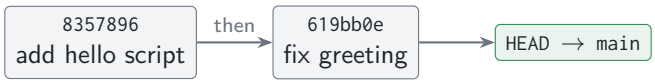
```
> git status
Changes not staged for commit:
        modified:   hello.py

> git diff
--- a/hello.py
+++ b/hello.py
@@ -1 +1 @@
-print("Oops")
+print("Hi")

> git add hello.py
> git commit -m "fix greeting"
[main 619bb0e] fix greeting
 1 file changed, 1 insertion(+), 1 deletion(-)

> git log --oneline
619bb0e fix greeting
8357896 add hello script
```

(Git on Windows may also print `warning: LF will be replaced by CRLF`. It is about line endings and is harmless here.)



Command	What it does
`git init`	turn the current folder into a repository
`git status`	show what is changed, staged, or untracked
`git add file` / `git add .`	put a file (or all changes) into the staging area
`git commit -m "msg"`	save the staged snapshot with a message
`git diff`	show line changes not yet staged
`git log` / `git log --oneline`	show history, newest first
`git clone url`	copy a whole repository from a host such as GitHub

⚠ TRAP CARD

`git commit` saves only what you staged. If you edit a file and commit without `git add`, Git answers "no changes added to commit" and nothing is saved. Also: `git add .` stages *everything*, including a `.env` file with a key. Write the `.gitignore` first:

```
.venv/
__pycache__/
.env
```

➤ **RULE**

Commit small and often, with a message that says *what* changed ("fix greeting"), not "stuff". Commit when the tests pass. Review with `git diff` before every commit.

■ **BOSS FIGHT: "which Python am I running?"**

Most setup bugs are a mismatch: you installed a package into one Python and run your code with another (`ModuleNotFoundError: No module named 'anthropic'` right after a successful install). Ask three questions: (1) `import sys; print(sys.executable)`: which interpreter is running? (2) `sys.prefix != sys.base_prefix`: is it a venv? (3) `python -m pip --version`: does pip belong to the same interpreter? Always install with `python -m pip` and make VS Code use the `.venv` interpreter, and the mismatch disappears.

➤ **PRACTICE**

1. Create a project folder with a `.venv`. Which two commands activate it in PowerShell and in cmd, and how do you leave it?
2. You ran `pip install anthropic` but Python says the module is missing. List three causes you would check.
3. Write a `requirements.txt` that needs any `anthropic`, `pytest` 8 or newer, and nothing else. How does a teammate install it?
4. What does `git add` do that `git commit` does not? Why is there a staging area at all?
5. Write the lines for a `.gitignore` in an LLM project and say why each line is there.
6. Read an API key from an environment variable. What is the type of the result when the variable is missing, and what do you do about it?

★ **ACHIEVEMENT UNLOCKED: Environment Engineer**

- Create a virtual environment, install packages into it, and prove which Python you are using.
- Record and rebuild a project's dependencies with `requirements.txt`.
- Keep secrets out of code and out of Git with environment variables and `.gitignore`.
- Make, review and list commits with Git.

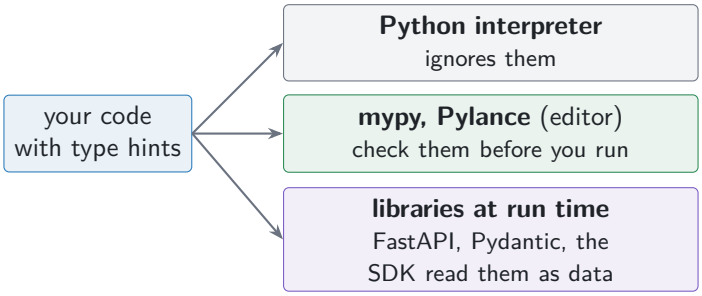
CHAPTER 14

Type Hints and Documentation

Python variables are just names attached to objects, so nothing in the language says what kind of object a name should hold. Type hints are optional notes that say it. This chapter shows how to write them, how a checker (mypy) uses them to find bugs before you run the code, and how comments, docstrings and README files explain your code to the next reader, which may be you.

14.1 Type hints

A **type hint** (also *type annotation*) says what type a variable, parameter or return value is *supposed* to have. Python itself ignores hints. They are for people and for tools.



Variable hints

The form is `name: type` or `name: type = value`.

```
from typing import Any

model: str = "small-model"
temperature: float = 0.7
max_tokens: int                      # hint only: no value, no variable yet
stop: list[str] = ["END"]            # a list of str
usage: dict[str, int] = {"in": 22, "out": 73}      # keys str, values int
pair: tuple[int, str] = (1, "a")     # exactly: one int, then one str
price: int | float = 4               # either type (Python 3.10+)
alias: str | None = None             # a str, or missing
anything: Any = object()             # opt out: could be anything

print(__annotations__["model"], __annotations__["usage"])
print(sorted(__annotations__))
try:
    print(max_tokens)
except NameError as e:
    print("NameError:", e)
```

```
<class 'str'> dict[str, int]
['alias', 'anything', 'max_tokens', 'model', 'pair', 'price', 'stop', 'temperature', 'usage']
NameError: name 'max_tokens' is not defined
```

■ TYPE CHECK

The hint is stored in `__annotations__`, a plain `dict` mapping names to types. It is only a record. `max_tokens: int` *does not create* the variable, so reading it raises `NameError`. `list[str]` means "list whose items are `str`"; `tuple[int, str]` means a pair of exactly those types.

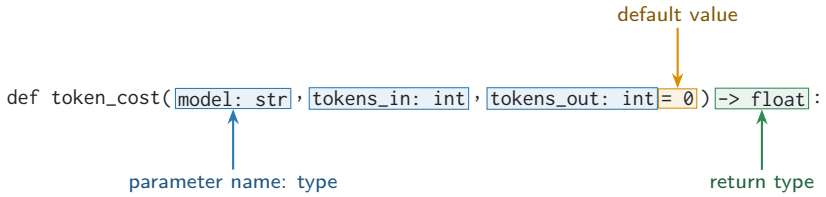
Hint	Meaning	Python
`int`, `str`, `float`, `bool`	the plain type	any
`list[str]`, `dict[str, int]`, `set[int]`	container and what is inside	3.9+
`tuple[int, str]` / `tuple[int, ...]`	fixed pair / any number of ints	3.9+
`int | float`	one of several types	3.10+
`str | None`	a value, or `None` (same as `Optional[str]`)	3.10+
`Any` (from `typing`)	anything; switches checking off for it	any
`Union[int, float]`, `Optional[float]`	older spellings of the two lines above	any

⚠ TRAP CARD

The book's older spelling is `Union[int, float]` with square brackets. Writing `Union(int, float)` with round brackets is an error. On Python 3.10+ prefer the shorter `int | float`.

Function hints

Parameters get `name: type`; the return type goes after `->`. A function that returns nothing is `-> None`.




```python
def token_cost(model: str, tokens_in: int, tokens_out: int = 0) -> float:
    rates = {"small": (0.10, 0.50), "large": (4.0, 20.0)}   # USD per 1M
    rate_in, rate_out = rates[model]
    return (tokens_in * rate_in + tokens_out * rate_out) / 1e6

def no_return(n: int) -> None:
    print("got", n)

print(token_cost.__annotations__)
print(token_cost("small", 1000, 500))
x = no_return(3)
print(x)
```

```
{'model': <class 'str'>, 'tokens_in': <class 'int'>, 'tokens_out': <class 'int'>, 'return': <class 'float'>}
0.00035
got 3
None
```

Hints are not enforced

```
def double(n: int) -> int:
    return n * 2

print(double("ab"))
```

```
abab
```

Python happily multiplied a string. The hint said `int`, but nobody checked. Only a tool such as mypy would have flagged the call.

Optional values: `str | None`

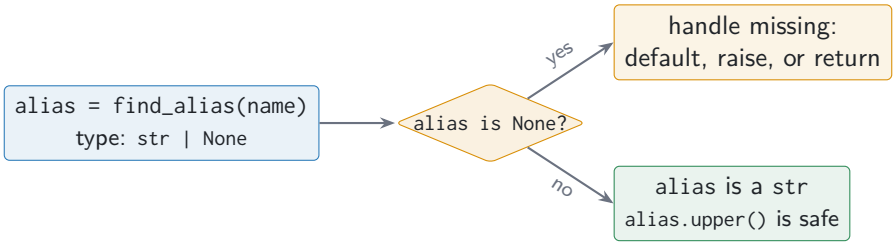
When a function may have nothing to return, say so. The checker then forces you to handle `None` before you use the value.

```
def find_alias(name: str) -> str | None:
    aliases = {"fast": "small"}
    return aliases.get(name)          # .get returns None when missing

r = find_alias("slow")
print(r, type(r))
try:
    print(r.upper())                  # a checker flags this line
except AttributeError as e:
    print("AttributeError:", e)

r2 = find_alias("fast")
if r2 is not None:                    # narrowing: inside, r2 is a plain str
    print(r2.upper())
```

```
None <class 'NoneType'>
AttributeError: 'NoneType' object has no attribute 'upper'
SMALL
```



■ TYPE CHECK

`str | None` is a *union*: a value of this type is either a `str` or `None`. You may use string methods only after you prove it is not `None`. Every `dict.get(...)`, `os.environ.get(...)` and `re.match(...)` returns this kind of value.

Messages, `Literal` and `TypedDict`

LLM code passes around lists of small dicts. Two helpers from `typing` describe them precisely. `Literal` lists the allowed exact values; `TypedDict` describes a dict with fixed keys.

```python
from typing import Literal, TypedDict

class Message(TypedDict):
    role: Literal["user", "assistant"]
    content: str

def last_user(history: list[Message]) -> str | None:
    for m in reversed(history):
        if m["role"] == "user":
            return m["content"]
    return None

msgs: list[Message] = [{"role": "user", "content": "hi"},
                       {"role": "assistant", "content": "hello"}]
print(last_user(msgs), type(msgs[0]))
```

```
hi <class 'dict'>
```

At run time a `TypedDict` is just a `dict`; only the checker knows the rules. A checker would reject `{"role": "robot", "content": "x"}`. The SDK uses the same trick: you can read the allowed `stop_reason` values off its types.

```python
from anthropic.types import Message
print(Message.model_fields["stop_reason"].annotation)
```

```
typing.Optional[typing.Literal['end_turn', 'max_tokens', 'stop_sequence', 'tool_use', 'pause_turn', 'refusal',
    ...]]
```

(Needs the `anthropic` package; the list shown is shortened and depends on your installed version.)

> ➤ **RULE**
> Hint every function signature (parameters and return). Hint a variable only when the type is not obvious from the value. Use `X | None` for "maybe missing" and handle it. Use `Any` rarely: it turns checking off.

14.2 Mypy

mypy reads your code without running it and reports places where a hint is violated. Install and run it inside your `.venv`:

```
> python -m pip install mypy
> python -m mypy errs.py
```

Here is a file with three type mistakes and one value mistake:

```python
# errs.py
def int_to_str(num: int) -> str:
    return str(num)

def str_to_int(txt: str) -> int:
    return int(txt)

a = int_to_str(19)        # fine
b = int_to_str(47.5)      # float, expected int
c = int_to_str("")        # str, expected int
d = str_to_int("56")      # fine
e = str_to_int("")        # type ok, value bad
f = str_to_int(98)        # int, expected str
```

Typical mypy report (*not run here: mypy is not installed in this environment; the file does compile*):

```
errs.py:10: error: Argument 1 to "int_to_str" has incompatible type "float"; expected "int"  [arg-type]
errs.py:11: error: Argument 1 to "int_to_str" has incompatible type "str"; expected "int"  [arg-type]
errs.py:14: error: Argument 1 to "str_to_int" has incompatible type "int"; expected "str"  [arg-type]
Found 3 errors in 1 file (checked 1 source file)
```

Running the same file with plain Python gets only as far as the first *runtime* failure, and it is the one line mypy does *not* report (this is real output, last lines):

```
    return int(txt)
           ^^^^^^^^
ValueError: invalid literal for int() with base 10: ''
```

	mypy (static)	**Python (run time)**
When	before the program runs, on the whole file	only when that line executes
Catches	wrong argument or return *types*, missing `None` checks	whatever actually fails on this run
Misses	wrong *values*: `int("")`, empty list, bad JSON	type mistakes on lines that never run

> ⚠ **TRAP CARD**
> Types do not describe values. `txt: str` accepts `""`, so mypy is silent about `int("")`. A checker removes a class of bugs; you still need tests (Chapter 15) for values and logic.

> ➤ **RULE**
> Run the checker as part of your routine, next to tests. In VS Code, the Pylance extension shows the same kind of errors as you type, with no command. Start with the default settings; add `–strict` later.

14.3 Documentation

Code says *what happens*. Documentation says *why*, and *how to use it*. Hints cover part of that; text covers the rest.

Comments

A comment starts with # and is for the reader. Explain *why*, not what the code already says.

```
retries = 3                    # bad: says what the code says
retries = 3                    # 429 errors come in bursts; 3 tries was enough
```

Docstrings

A **docstring** is a string literal as the first statement of a function, class or module. Python stores it in `__doc__` and `help()` prints it. IDEs show it as a tooltip when you hover.

```
def echo(text):
    "Return the input unchanged."
    return text

print(echo.__doc__)
help(echo)
```

```
Return the input unchanged.
Help on function echo in module __main__:

echo(text)
    Return the input unchanged.
```

For anything longer, use a multi-line docstring: a one-line summary, then details. The common "Google style" lists arguments, return value and errors.

```
def token_cost(model: str, tokens_in: int, tokens_out: int = 0) -> float:
    """Return the cost in USD of one API call.

    Args:
        model: model name, "small" or "large".
        tokens_in: number of input tokens.
        tokens_out: number of output tokens (default 0).

    Returns:
        Cost in US dollars as a float.

    Raises:
        KeyError: if the model name is unknown.
    """
    rates = {"small": (0.10, 0.50), "large": (4.0, 20.0)}
    rate_in, rate_out = rates[model]
    return (tokens_in * rate_in + tokens_out * rate_out) / 1e6

print(token_cost.__doc__.splitlines()[0])
```

```
Return the cost in USD of one API call.
```

Because the types are already in the signature, the docstring does not need to repeat them; describe meaning, units (USD, tokens) and errors.

Markup files

Project-level documentation lives in text files beside the code. **Markdown** (`README.md`) is the standard on GitHub; AsciiDoc and reStructuredText are alternatives. A README says what the project is, how to install it (the `requirements.txt` and venv steps from Chapter 13), and how to run it.

Tool	Answers the question	Read by
type hint	what type goes in and out?	checker, IDE, library
comment #	why is this line odd?	someone reading the source
docstring	what does this function do, and how do I call it?	`help()`, IDE tooltips
`README.md`	what is this project and how do I run it?	new users and teammates

■ **BOSS FIGHT: hints are a promise, not a guard**

Python never checks hints. They are a contract that a *tool* verifies before running, and that *libraries* may read while running. Hold two things apart: (1) the interpreter runs even when the hints are wrong; (2) a clean mypy report says the *types* line up, not that the *values* are valid. If data comes from outside your program (JSON from an API, user input), the hint on that variable is your *belief*. Validate the data, or use a library such as Pydantic that checks it at run time using the hints.

➤ **PRACTICE**

1. Write `most_used(counts)` that takes a `dict[str, int]` of tokens per model and returns the model with the largest count, or `None` if the dict is empty. Add complete hints and a docstring.
2. Why does a type checker complain about `find_alias("x").upper()` when `find_alias` returns `str | None`? Show two ways to fix it.
3. Python runs `double("ab")` without error though `n: int` is hinted. Who *would* complain, and when?
4. What is the difference between a comment, a docstring and a README? Give one thing each should say about a `summarize(text)` function.
5. Hint a variable holding a list of chat messages, where each message is a dict with a `role` and a `content` string.
6. Name one kind of mistake mypy cannot find and the tool from Chapter 15 that can.

★ **ACHIEVEMENT UNLOCKED: Signature Writer**

- Annotate variables and function signatures, including `None`, unions and containers.
- Explain that hints are ignored at run time and checked by mypy or your editor.
- Read a mypy error and say which line and which type are wrong.
- Write a useful docstring and know where `help()` and your editor find it.

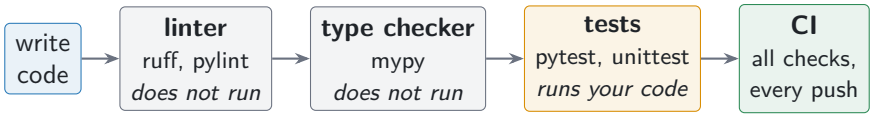
CHAPTER 15

Testing

Even a tiny change can break code that used to work. A test is a small program that calls your code with known input and checks the output, so a break shows up in seconds, not in production. This chapter covers the tools that check code without running it (linters), the two standard-library test tools, `pytest` (the one you will use), plus fakes for API calls, property tests and automation.

15.1 The ladder of checks

Printing values and eyeballing them does not scale. Use layers, cheapest first:



Layer	Finds	Misses
Linter	undefined names, unused imports, style problems	wrong logic
Type checker	wrong types passed or returned (Chapter 14)	wrong values
Tests	wrong behaviour on the inputs you thought of	inputs you did not think of

15.2 Linters: Pylint, Pyflakes and Ruff

A **linter** reads source code and reports likely errors and style problems without running it. Take this three-line file. It has a typo: `totl` instead of `total`.

```
# style1.py
import os
total = 1
print(totl)
```

Running it would crash with `NameError` only when line 3 executes. A linter reports it at once.

Ruff (real output)

Ruff is a very fast linter and formatter written in Rust. Install it with `python -m pip install ruff` (or `uv`). Real output, with its default rule set for this version:

```
> ruff check --output-format concise style1.py
style1.py:1:1: I001 [*] Import block is un-sorted or un-formatted
style1.py:1:8: F401 [*] `os` imported but unused
style1.py:3:7: F821 Undefined name `totl`
Found 3 errors.
[*] 2 fixable with `--fix`.
```

Read it as `file:line:column: CODE message`. `F821` is the real bug. The `[*]` marks problems Ruff can fix itself:

```
> ruff check --fix style1.py          (removes the unused import, keeps the bug)
```

Ruff also formats code, like a strict spell-checker for whitespace. Given an untidy file:

```
def add(a,b):
    return a+b
x=add( 1,2 )
```

`ruff format messy.py` rewrites it to:

```
def add(a, b):
    return a + b


x = add(1, 2)
```

Pylint, Pyflakes, pycodestyle

Tool	Checks	Character
`pyflakes`	errors only: undefined names, unused imports	quick, few opinions
`pycodestyle` (was `pep8`)	layout against PEP 8: spacing, blank lines	style only
`pylint`	errors, style, naming, design smells, and a score out of 10	thorough and noisy; configure it
`ruff`	most of the three above plus a formatter and import sorting	fast; the modern default

Pylint prefixes each message with a letter: `E` error, `W` warning, `C` convention (style), `R` refactor, `F` fatal. For the file above its report has the shape below (*typical form; Pylint is not installed here, so not run*). An `E` line is the one to fix first.

```
************* Module style1
style1.py:1:0: C0114: Missing module docstring (missing-module-docstring)
style1.py:3:6: E0602: Undefined variable 'totl' (undefined-variable)
```

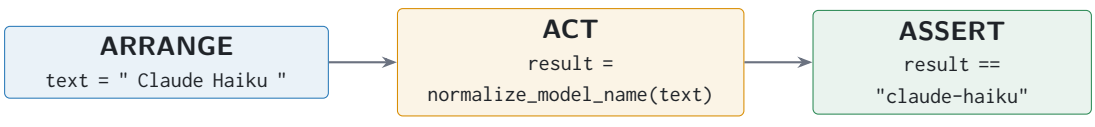
> ➤ **RULE**
>
> Fix errors (`E`, `F`) first, style later. Do not chase a perfect score. Run `ruff check` and `ruff format` before every commit; your editor can do it on save.

15.3 Unittest

Linters cannot know what your function *should* return. For that, write a test. `unittest` is in the standard library and needs no install.

Arrange, act, assert

Every test has three steps. The *assertion* is the claim that must be true; if it is not, the test fails.



Here is the function under test, a first and naive version. It makes a model name safe for file names.

```
# names.py
def normalize_model_name(name):
    return name.lower().replace(" ", "-")
```

A test class derives from `unittest.TestCase`. Each method whose name starts with `test` is a test.

```
# test_names.py
import unittest

import names


class TestNormalize(unittest.TestCase):
    def setUp(self):
        self.fn = names.normalize_model_name   # runs before EACH test

    def tearDown(self):
        pass                                   # runs after EACH test

    def test_one_word(self):
        self.assertEqual(self.fn("Haiku"), "haiku")

    def test_two_words(self):
        self.assertEqual(self.fn("Claude Haiku"), "claude-haiku")

    def test_extra_spaces(self):
        self.assertEqual(self.fn("  Claude   Haiku "), "claude-haiku")

    def test_rejects_none(self):
        with self.assertRaises(AttributeError):
            self.fn(None)


if __name__ == "__main__":
    unittest.main()
```

```
> python test_names.py
F...
======================================================================
FAIL: test_extra_spaces (__main__.TestNormalize.test_extra_spaces)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "...\test_names.py", line 20, in test_extra_spaces
    self.assertEqual(self.fn("  Claude   Haiku "), "claude-haiku")
AssertionError: '--claude---haiku-' != 'claude-haiku'
- --claude---haiku-
? --       --     -
+ claude-haiku

----------------------------------------------------------------------
Ran 4 tests in 0.001s

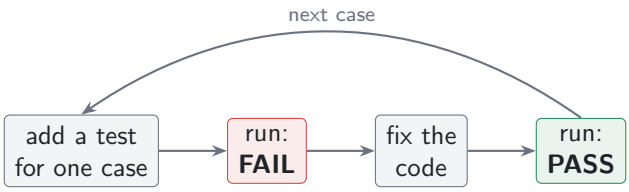
FAILED (failures=1)
```

Read the first line: one character per test, `.` passed and `F` failed. The `-` and `+` lines show actual and expected, and the `?` line points to the differences. The test found a real bug. Fix the function, then rerun:

```python
# names.py, fixed
def normalize_model_name(name):
    return "-".join(name.lower().split())    # split() drops all extra spaces
```

```
> python test_names.py
....
----------------------------------------------------------------------
Ran 4 tests in 0.000s

OK
```



TYPE CHECK

A test class is a *class* (Chapter 11) inheriting `TestCase`; `self.assertEqual(a, b)` is an inherited method. It prints the difference when `a != b`. Plain `assert a == b` also works in unittest, but its failure message is poor.

Method	Passes when
`assertEqual(a, b)`	`a == b`
`assertTrue(x)` / `assertFalse(x)`	`x` is truthy / falsy
`assertIn(a, seq)`	`a in seq`
`assertIsNone(x)`	`x is None`
`assertAlmostEqual(a, b)`	floats equal to 7 decimal places
`assertRaises(Err)`	the block raises `Err` (use `with`)

Run all tests in a folder with `python -m unittest` (it finds files named `test*.py`); add `-v` for one line per test.



15.4 Doctest

`doctest` runs the examples you put *inside a docstring*. They look like a session at the >>> prompt. The test is also documentation that cannot go out of date.

```
# names2.py
def normalize_model_name(name):
    """Make a model name safe for file names.

    >>> normalize_model_name("Haiku")
    'haiku'
    >>> normalize_model_name("  Claude   Haiku ")
    'claude-haiku'
    >>> normalize_model_name("")
    ''
    """
    return "-".join(name.lower().split())


if __name__ == "__main__":
    import doctest
    doctest.testmod()
```

Run it. No output means every example passed. Add `-v` to see them all:

```
> python names2.py
> python names2.py -v
Trying:
    normalize_model_name("Haiku")
Expecting:
    'haiku'
ok
...
3 tests in 2 items.
3 passed and 0 failed.
Test passed.
```

(Middle lines omitted.) You can also run it without the `if` block: `python -m doctest names2.py`. A wrong expectation shows what Python really returned:

```
Failed example:
    twice(2)
Expected:
    5
Got:
    4
***Test Failed*** 1 failures.
```

> ⚠ **TRAP CARD**
> Doctest compares the *printed text* exactly. The result `'haiku'` must be written with the quotes, and a float such as `0.30000000000000004` must match digit for digit. Use doctest for short examples; use pytest for real test suites.

15.5 Pytest

pytest is the most popular test tool. Install it with `python -m pip install pytest`. Tests are plain functions with plain `assert`; there is no class and no `self.assertEqual`.

```
> python -m pytest -q
```

Feature	What you get
Discovery	finds `test_*.py` files and `test_*` functions below the current folder
Rich failures	a failing `assert` shows both sides and a diff, no special method needed
Fixtures	functions that prepare test data or resources, injected by name
Parametrization	run one test over a table of inputs
Built-ins	`tmp_path`, `monkeypatch`, `capsys`, `pytest.raises`, `pytest.approx`

A first test file

```
# costs.py
RATES = {"small": (0.10, 0.50), "large": (4.0, 20.0)}   # USD per 1M tokens

def token_cost(model, tokens_in, tokens_out=0):
    if tokens_in < 0 or tokens_out < 0:
        raise ValueError("token counts must be >= 0")
    rate_in, rate_out = RATES[model]
    return (tokens_in * rate_in + tokens_out * rate_out) / 1e6
```

```
# test_costs_pytest.py   (needs: pytest)
import pytest

from costs import token_cost


def test_zero_tokens_cost_nothing():
    assert token_cost("small", 0, 0) == 0


def test_known_value():
    assert token_cost("small", 1_000_000, 0) == pytest.approx(0.10)


def test_negative_tokens_rejected():
    with pytest.raises(ValueError, match="must be >= 0"):
        token_cost("small", -1)


def test_unknown_model():
    with pytest.raises(KeyError):
        token_cost("huge", 1)
```

Typical pytest report for a passing run (*pytest is not installed in this environment, so this is the usual format, not a captured run*; the file itself was syntax-checked and its logic verified by calling the functions):

```
============================= test session starts ==============================
collected 4 items

test_costs_pytest.py ....                                                [100%]

============================== 4 passed in 0.03s ===============================
```

On a failure, pytest prints the test source, a `>` on the failing line, and `E` lines with the values.

⚠ TRAP CARD

Never compare floats with ==. Binary floats are approximate. Compare with `pytest.approx(x)` (or `math.isclose` in plain code).

```
import math
print(0.1 + 0.2 == 0.3, math.isclose(0.1 + 0.2, 0.3))
```

```
False True
```

Parametrization: one test, many cases

`parametrize` takes a string of argument names and a list of tuples; pytest runs the test once per tuple and reports each separately.

```
import pytest
from costs import token_cost

@pytest.mark.parametrize(
    "model, tin, tout, expected",
    [
        ("small", 1_000_000, 0, 0.10),
        ("small", 0, 1_000_000, 0.50),
        ("large", 1_000, 500, 0.014),
    ],
)
def test_token_cost_table(model, tin, tout, expected):
    assert token_cost(model, tin, tout) == pytest.approx(expected)
```

Fixtures

A **fixture** is a function marked `@pytest.fixture`. A test that lists its name as a parameter receives its return value. Fixtures replace `setUp` and `tearDown`.

```
# test_fixtures.py   (needs: pytest)
import sqlite3

import pytest


@pytest.fixture                      # scope="function" is the default
def history():
    return [{"role": "user", "content": "hi"}]

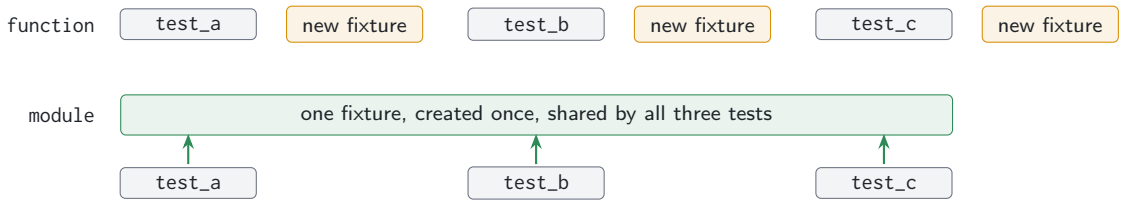

@pytest.fixture(scope="module")      # created once for this file
def calls_db():
    conn = sqlite3.connect(":memory:")
    conn.execute("create table calls (model text, cost real)")
    yield conn                       # the test runs here
    conn.close()                     # teardown, after the last test
```

The tests then list the fixtures they need by name:

```python
def test_insert(calls_db, history):
    calls_db.execute("insert into calls values ('small', 0.0001)")
    assert calls_db.execute("select count(*) from calls").fetchone()[0] == 1


def test_same_connection_reused(calls_db):
    assert calls_db.execute("select count(*) from calls").fetchone()[0] == 1


def test_save_reply(tmp_path):
    out = tmp_path / "reply.txt"     # tmp_path: a fresh temp folder
    out.write_text("hello")
    assert out.read_text() == "hello"
```



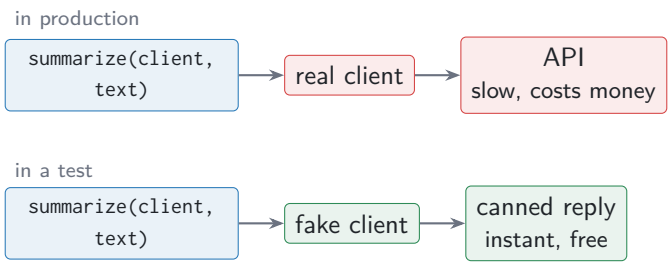
> ➤ **RULE**
>
> Keep the default `function` scope so each test starts clean. Raise the scope (`module`, `session`) only for something slow to build, like a database connection, and only if tests do not change it in ways that leak between them.

Handy flags

Command	Effect
`python -m pytest -q`	quiet: one dot per test
`python -m pytest -v`	one line per test, with names
`python -m pytest -x`	stop at the first failure
`python -m pytest -k normalize`	run only tests whose name contains *normalize*
`pytest test_costs_pytest.py::test_unknown_model`	run one test (start with `python -m`)
`python -m pytest -s`	show `print` output (otherwise captured)

Testing code that calls the API

A test must be fast, free and repeatable. A real API call is slow, costs money and gives a different answer each time. So **pass the client in** as a parameter and give the test a fake one.



```
# summarize.py
def summarize(client, text):
    """Ask the model for a one-line summary. Returns '' if no text block."""
    response = client.messages.create(
        model="small-model",
        max_tokens=200,
        messages=[{"role": "user", "content": f"Summarize in one line: {text}"}],
    )
    for block in response.content:
        if block.type == "text":
            return block.text.strip()
    return ""
```

`unittest.mock.Mock` (standard library) creates an object that accepts any attribute or call. We set what `create` returns and later check how it was called.

```
# test_summarize.py
import unittest
from types import SimpleNamespace
from unittest.mock import Mock

from summarize import summarize


def fake_client(*blocks):
    client = Mock()
    client.messages.create.return_value = SimpleNamespace(content=list(blocks))
    return client
```

Each test builds a fake client with the blocks the "model" should return:

```
class TestSummarize(unittest.TestCase):
    def test_returns_text(self):
        client = fake_client(SimpleNamespace(type="text", text=" Short. "))
        self.assertEqual(summarize(client, "long text"), "Short.")

    def test_skips_thinking_block(self):
        client = fake_client(SimpleNamespace(type="thinking", thinking="..."),
                             SimpleNamespace(type="text", text="Answer"))
        self.assertEqual(summarize(client, "x"), "Answer")

    def test_no_text_block(self):
        client = fake_client(SimpleNamespace(type="thinking", thinking="..."))
        self.assertEqual(summarize(client, "x"), "")

    def test_sends_one_request(self):
        client = fake_client(SimpleNamespace(type="text", text="ok"))
        summarize(client, "hello")
        client.messages.create.assert_called_once()
        kwargs = client.messages.create.call_args.kwargs
        self.assertEqual(kwargs["max_tokens"], 200)
        self.assertIn("hello", kwargs["messages"][0]["content"])


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

```
test_no_text_block (__main__.TestSummarize.test_no_text_block) ... ok
test_returns_text (__main__.TestSummarize.test_returns_text) ... ok
test_sends_one_request (__main__.TestSummarize.test_sends_one_request) ... ok
test_skips_thinking_block (__main__.TestSummarize.test_skips_thinking_block) ... ok

----------------------------------------------------------------------
Ran 4 tests in 0.001s

OK
```

Note that the same fake lets us test the "thinking block first" case without paying for a real model. In pytest, the `monkeypatch` fixture does a similar job for environment variables, for example `monkeypatch.setenv(...)` with a dummy value for `ANTHROPIC_API_KEY`.

> **⚠ TRAP CARD**
> A test that passes only when the network is up, or when today's date is right, or when the model answers a certain way, is a *flaky* test. Fake the clock, the network and the model. Never put a real API key in a test or commit it.

15.6 Hypothesis: property-based tests

Instead of listing inputs, describe the *kind* of input and a rule that must always hold; **Hypothesis** generates many cases (100 by default), including odd ones you would not write, and shrinks a failure to a minimal example.

```python
# test_props.py   (needs: hypothesis)
from hypothesis import given
from hypothesis import strategies as st

from costs import token_cost

counts = st.integers(min_value=0, max_value=10**7)


@given(counts, counts, counts)
def test_more_output_never_cheaper(tokens_in, tokens_out, extra):
    cheaper = token_cost("small", tokens_in, tokens_out)
    dearer = token_cost("small", tokens_in, tokens_out + extra)
    assert dearer >= cheaper
```

(Install with `python -m pip install hypothesis`; not installed here. I checked the rule itself against 100 000 random inputs with plain `random`: it held.) If you let `counts` include negative numbers, Hypothesis would report a counterexample at once, since `token_cost` raises `ValueError` for them. That is the point: it finds the cases you forgot.

15.7 Nox

Nox runs your checks in fresh environments, for several Python versions or tool combinations. You describe the sessions in a Python file named `noxfile.py` and run `nox`.

```python
# noxfile.py   (needs: nox)
import nox

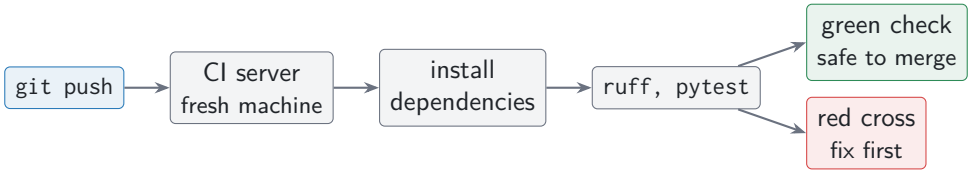

@nox.session(python=["3.11", "3.12"])
def tests(session):
    session.install("-r", "requirements.txt", "pytest")
    session.run("pytest", "-q")


@nox.session
def lint(session):
    session.install("ruff")
    session.run("ruff", "check", ".")
```

Nox is the successor of `tox`. Use it when your package must work on more than one Python version; for a personal project, a plain `pytest` is enough.

15.8 Continuous integration

CI runs your checks automatically on a server each time code is pushed, so a break is caught before it is merged.



A minimal GitHub Actions workflow (file `.github/workflows/tests.yml`), not run here:

```yaml
name: tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: python -m pip install -r requirements.txt pytest ruff
      - run: ruff check .
      - run: python -m pytest -q
```

System	Note
GitHub Actions	built into GitHub; free for public repositories; the easy start
Jenkins	self-hosted, written in Java; common in companies
Buildbot	self-hosted, written in Python
Travis CI, CircleCI	hosted services; free tiers for open source

> ➤ **RULE: CI and secrets**
> CI machines are not yours. Store an API key, if a job truly needs one, in the host's *secrets* setting, never in the repository. Most tests should not need a key: use fakes.

■ **BOSS FIGHT: testing what you do not control**

The hard part of testing is not `assert`; it is *isolation*. Code that reads the clock, the environment, a random number or a web API gives a different answer on each run or costs money. The fix is one idea: do not reach out from inside the function; **pass the dependency in** (the client, the current time, a random seed) and give the test a fake or a fixed value. Then the test depends only on its inputs, which is what makes it fast, free and repeatable. Practise it on `summarize(client, text)`: the function never creates its own client.

➤ **PRACTICE**

1. Run `ruff check` on a file that uses a variable before defining it. What is the code of the error, and does the linter run your program?
2. Write three `unittest` tests for a function `clip(text, n)` that cuts text to at most `n` characters. Which edge case would you add first?
3. What do `setUp` and `tearDown` do, and which pytest feature replaces them?
4. Rewrite your three tests as one `parametrize` test. What do you gain?
5. Why is `assert cost == 0.014` a bad test, and what do you write instead?
6. Your function calls `client.messages.create`. Describe how you test it without internet or an API key.

★ **ACHIEVEMENT UNLOCKED: Green Build**

- Lint and format code with Ruff, and read an error code and its location.
- Write unit tests in `unittest` and `pytest`, including edge cases and exceptions.
- Use fixtures and `parametrize` to remove repeated test code.
- Test API-calling code with a fake client, with no network and no key.
- Describe how CI runs the same checks on every push.

CHAPTER 16

Debugging

Bugs happen even with good tests. Debugging is the skill of finding out *why* the program does what it does, quickly and without guessing. This chapter goes from cheap tools (reading a traceback, `print`, f-strings) to better ones (`assert`, `logging`) to the real debugger (`pdb`, `breakpoint()`).

16.1 Start here: read the traceback

Debug from the bottom up first: you or a recent change broke it, so look at what you changed last. If that fails, question what you trusted (a library, the data from an API); that is the slower, top-down search.

A **traceback** lists the chain of function calls that led to the error. Here is a program that fails when the API returns an error payload instead of a normal reply (paths shortened with `...`):

```
# usage_report.py
import json

def get_tokens(payload):
    return payload["usage"]["output_tokens"]

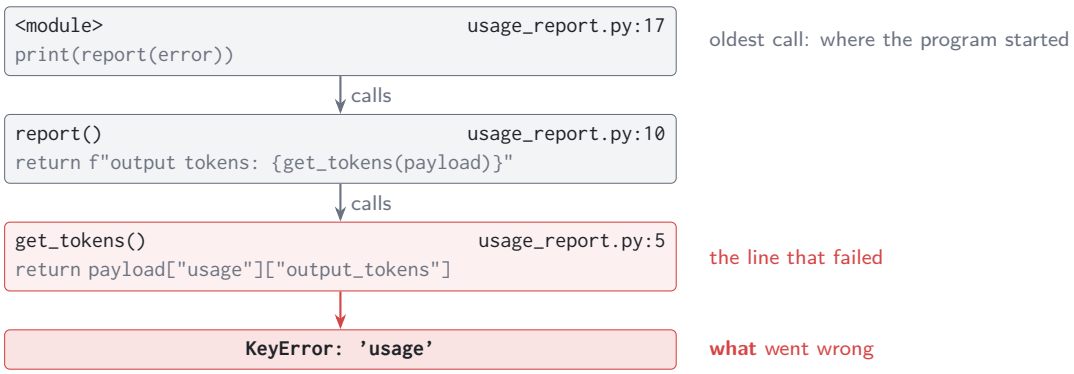
def report(raw):
    payload = json.loads(raw)
    return f"output tokens: {get_tokens(payload)}"

ok = '{"usage": {"output_tokens": 73}}'
error = '{"type": "error", "error": {"message": "overloaded"}}'

print(report(ok))
print(report(error))
```

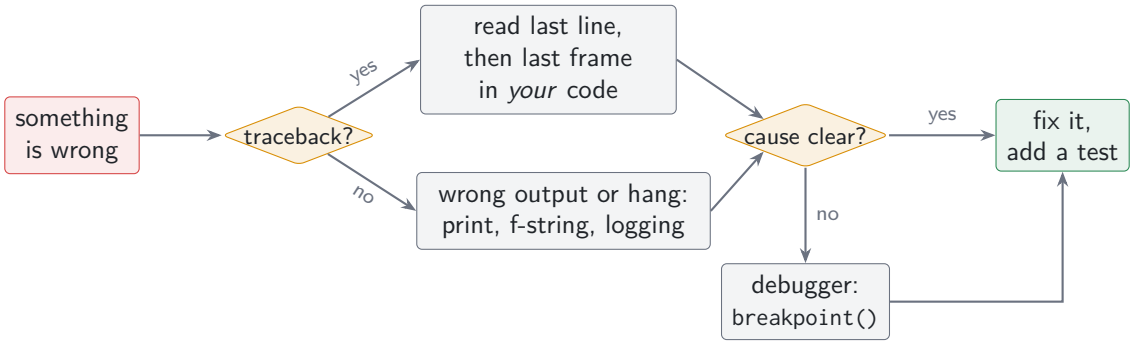
```
output tokens: 73
Traceback (most recent call last):
  File ".../usage_report.py", line 17, in <module>
    print(report(error))
  File ".../usage_report.py", line 10, in report
    return f"output tokens: {get_tokens(payload)}"
  File ".../usage_report.py", line 5, in get_tokens
    return payload["usage"]["output_tokens"]
           ~~~~~~~^^^^^^^^^^
KeyError: 'usage'
```

(Real output. The `~` and `^` marker lines under the first two frames are left out.)



★ **KEY IDEA: how to read a traceback**

Read from the **bottom**. The last line names the error and its message. The frame just above it is the line that failed. Walk upward until you find the first frame in *your own* code; library frames are rarely the cause. Here the real question is not "why a `KeyError`" but "why did the API return no usage?": the answer is an error payload.



16.2 Assert

`assert condition, message` states something that *must* be true. If it is false, Python raises `AssertionError` on the spot, with your message. Use it to catch impossible data early, before it causes a confusing failure ten lines later.

```
# assert_demo.py
def avg_tokens(counts):
    assert len(counts) > 0, "counts must not be empty"
    assert all(isinstance(c, int) for c in counts), f"ints only: {counts!r}"
    return sum(counts) / len(counts)

print(avg_tokens([10, 20]))
print(avg_tokens([]))
```

```
15.0
Traceback (most recent call last):
  ...
AssertionError: counts must not be empty
```

⚠ **TRAP CARD**

Run Python with `-O` (`python -O assert_demo.py`) and every `assert` is *skipped*. The same call then fails later with a less helpful error:

```
ZeroDivisionError: division by zero
```

So use `assert` for internal sanity checks and tests. For input you do not control (user input, API data), check explicitly with `if ...: raise ValueError(...)`.

16.3 Print

`print` is the fastest debugger. Print the values that matter, with labels. Three built-ins show variables: `vars()` (or `locals()`) gives the local variables as a dict; `globals()` gives the module's.

```
import sys

def func(*args, **kwargs):
    print(vars())
    print(sorted(locals()))

func(1, 2, model="small")
print("x" in globals())
print("debug info", file=sys.stderr)       # to the error stream
```

```
{'args': (1, 2), 'kwargs': {'model': 'small'}}
['args', 'kwargs']
True
debug info
```

Debug output sent to `sys.stderr` stays separate from normal output: you can redirect your program's real output to a file and still see the debug lines in the terminal.

> ⚠ **TRAP CARD**
> Print debugging leaves clutter. Remove the prints before committing, or move them to `logging` (below), which you can switch off.

16.4 F-strings

An f-string inserts the value of any expression in `{}`. Adding `=` inside the braces prints the *expression text and its value*, so you never mislabel a debug line.

```
x = "thing"
print(f"{x=}", f"{ x = }", f"{10/2=}")
tokens = 1234
print(f"{tokens=:,}")                    # with a format spec after the =
print(f"{type(tokens)=}")
```

```
x='thing'  x = 'thing' 10/2=5.0
tokens=1,234
type(tokens)=<class 'int'>
```

Note `x='thing'`: the value is shown with `repr`, so strings have quotes, which makes stray spaces and `None` visible.

16.5 Pprint

`pprint` ("pretty print") lays out nested data over several lines, indented. Use it for JSON-like API replies.

```
from pprint import pprint

text = "Hello there, how can I help you today?"
reply = {"type": "message", "role": "assistant",
         "content": [{"type": "text", "text": text}],
         "usage": {"input_tokens": 22, "output_tokens": 73}}
pprint(reply)
```

```
{'content': [{'text': 'Hello there, how can I help you today?',
              'type': 'text'}],
 'role': 'assistant',
 'type': 'message',
 'usage': {'input_tokens': 22, 'output_tokens': 73}}
```

By default `pprint` sorts keys. Pass `sort_dicts=False` to keep insertion order, and `width=60` to wrap sooner. For real JSON you may prefer `json.dumps(obj, indent=2)`.

16.6 IceCream

IceCream is a third-party package (`python -m pip install icecream`; not installed here, so the output below is the documented style, not a captured run). Its `ic()` prints the expression, its value, and optionally the file, line and function, and it *returns* its argument, so you can wrap any sub-expression.

```
from icecream import ic

tokens = 73
ic(tokens)                # prints: ic| tokens: 73
ic(tokens * 2)            # prints: ic| tokens * 2: 146


def cost(n):
    ic()                  # prints file:line and the function name
    return n * 2


total = ic(cost(5)) + 1   # ic returns 10, so the expression still works
```

It is a nice shortcut, but `print(f"{x=}")` gives you most of the benefit with no install.

16.7 Decorators

A decorator wraps a function to run code before or after it, without editing the function. That makes it a good place for debug output. Here `trace` prints the arguments and the result of every call:

```python
import functools


def trace(func):
    """Print arguments and result of every call."""
    @functools.wraps(func)              # keep the original name and docstring
    def wrapper(*args, **kwargs):
        print(f"call {func.__name__} args={args} kwargs={kwargs}")
        result = func(*args, **kwargs)
        print(f"  -> {result!r}")
        return result
    return wrapper


@trace
def token_cost(model, tokens_in, tokens_out=0):
    rates = {"small": (0.10, 0.50)}
    r_in, r_out = rates[model]
    return (tokens_in * r_in + tokens_out * r_out) / 1e6


token_cost("small", 1000, tokens_out=500)
print(token_cost.__name__)
```

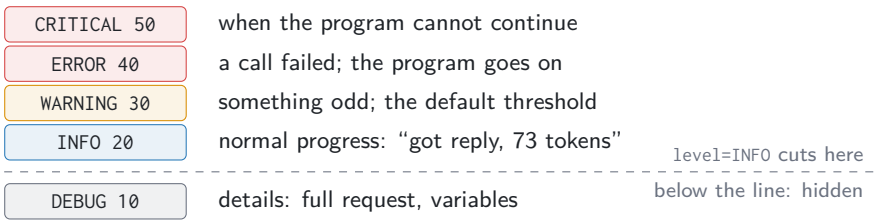
```
call token_cost args=('small', 1000) kwargs={'tokens_out': 500}
  -> 0.00035
token_cost
```

> **■ TYPE CHECK**
> After `@trace`, the name `token_cost` refers to `wrapper`, a new function. `*args` collects positional arguments into a tuple and `**kwargs` collects keyword arguments into a dict, so the wrapper accepts any call. Without `functools.wraps` the name would print as `wrapper`.

16.8 Logging

`print` output vanishes when the terminal closes, and has no timestamp or importance. The standard `logging` module records messages with a **level**, a time, and a name, and sends them where you choose. Use it in anything that runs longer than a quick script.

Levels



Before any setup, only `WARNING` and above appear. This explains the book's puzzle of `debug` and `info` "doing nothing":

```
import logging

logging.debug("not shown: default level is WARNING")
logging.info("not shown either")
logging.warning("shown")
logging.error("shown too")
```

```
WARNING:root:shown
ERROR:root:shown too
```

(The functions `logging.warn` and `logging.fatal` are old, deprecated spellings; use `warning` and `critical`.)

Configure once, with named loggers

Call `basicConfig` *once*, at the start of the program. Then each module asks for its own logger by name. Names with dots form a family tree: `"chat.api"` is a child of `"chat"`, which is a child of the root logger.

```
# log2.py
import logging

logging.basicConfig(level=logging.DEBUG,
                    format="%(levelname)-8s %(name)s:%(lineno)d %(message)s")
log = logging.getLogger("chat.api")

log.debug("sending request, model=%s", "small")
log.info("got reply, output_tokens=%d", 73)
log.warning("retrying after 429")
try:
    {}["usage"]
except KeyError:
    log.exception("could not read usage")     # message + the traceback
print("parent:", log.parent.name)
```

```
DEBUG    chat.api:7 sending request, model=small
INFO     chat.api:8 got reply, output_tokens=73
WARNING  chat.api:9 retrying after 429
ERROR    chat.api:13 could not read usage
Traceback (most recent call last):
  File ".../log2.py", line 11, in <module>
    {}["usage"]
    ~~^^^^^^^^^
KeyError: 'usage'
parent: root
```

Two habits are visible here. Pass values as arguments (`"model=%s"`, `"small"`) instead of building the string yourself, so Python formats it only if the message is actually emitted. Use `log.exception` inside `except`: it logs the traceback too.

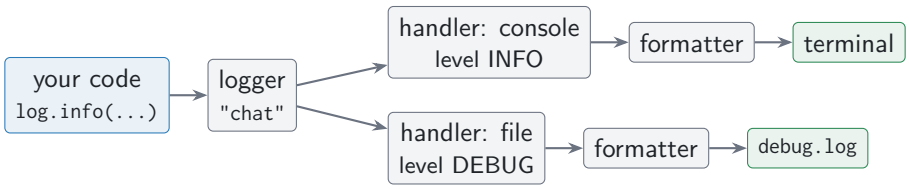
> ⚠ **TRAP CARD**
>
> `basicConfig` does nothing if the root logger already has handlers (an earlier call, or a library did it). Your level or format silently stays as before. Pass `force=True` to reset. Real output of three calls:

```
import logging
logging.basicConfig(level=logging.INFO, format="A %(message)s")
logging.basicConfig(level=logging.DEBUG, format="B %(message)s")   # ignored
logging.debug("debug?")                  # level is still INFO: hidden
logging.info("info")
logging.basicConfig(level=logging.DEBUG, format="C %(message)s", force=True)
logging.debug("debug now")
```

```
A info
C debug now
```

Handlers and formatters

Logging has four parts. A **logger** is what your code calls. A **handler** decides *where* a record goes (screen, file, email, web service). A **formatter** decides how it looks. A **filter** can drop records.




```
# log4.py
import logging
import sys

log = logging.getLogger("chat")
log.setLevel(logging.DEBUG)                  # logger lets everything through

console = logging.StreamHandler(sys.stdout)
console.setLevel(logging.INFO)               # but the screen shows INFO+
console.setFormatter(logging.Formatter("%(levelname)s %(message)s"))

logfile = logging.FileHandler("debug.log", encoding="utf-8")
logfile.setLevel(logging.DEBUG)              # the file keeps everything
logfile.setFormatter(logging.Formatter("%(levelname)s %(name)s %(message)s"))

log.addHandler(console)
log.addHandler(logfile)

log.debug("prompt has 12 messages")
log.info("reply received")
log.error("rate limited")
```

```
INFO reply received
ERROR rate limited
```

The file `debug.log` then contains all three:

```
DEBUG chat prompt has 12 messages
INFO chat reply received
ERROR chat rate limited
```

For a simple one-file setup, `basicConfig(filename="app.log", level=..., format=...)` is enough. A

format such as `"%(asctime)s %(levelname)s %(message)s"` adds a time stamp, which looks like `2026-11-02 13:02:01,655 ERROR boom`. Other useful fields: `%(name)s`, `%(lineno)d`, `%(funcName)s`.

> ➤ **RULE: logging in an LLM app**
>
> Log the model name, token counts, latency and the request id, at `INFO`. Log the full prompt only at `DEBUG`, because prompts can contain private data. **Never log an API key.** Keep the file out of Git (`.gitignore`).

16.9 Pdb: the debugger

A debugger runs your program under your control: you pause it, look at variables, and step through lines. Python's built-in debugger is `pdb`. Here is a data-dependent bug. The function should total tokens per model from lines like `"small,100"`, but the total for `small` comes out wrong.

```python
# totals.py
def total_usage(lines):
    totals = {}
    for line in lines:
        model, tokens = line.split(",")
        totals[model] = int(tokens)
    return totals


if __name__ == "__main__":
    data = ["small,100", "large,40", "small,50"]
    print(total_usage(data))
```

```
{'small': 50, 'large': 40}
```

No exception, so there is no traceback to read: it is a *logic* error. `small` should be 150. Start the program under pdb (`-m pdb`), set a breakpoint on line 5 with `b 5`, and continue with `c`:

```
> python -m pdb totals.py
> ...\totals.py(1)<module>()
-> def total_usage(lines):
(Pdb) b 5
Breakpoint 1 at ...\totals.py:5
(Pdb) c
> ...\totals.py(5)total_usage()
-> totals[model] = int(tokens)
(Pdb) c
> ...\totals.py(5)total_usage()
-> totals[model] = int(tokens)
(Pdb) c
> ...\totals.py(5)total_usage()
-> totals[model] = int(tokens)
(Pdb) p line
'small,50'
(Pdb) p totals
{'small': 100, 'large': 40}
(Pdb) n
> ...\totals.py(3)total_usage()
-> for line in lines:
(Pdb) p totals
{'small': 50, 'large': 40}
```

(Real session; only the long path is shortened.) After the third stop, `totals` holds `small: 100`. The next

single line replaces it with 50: the code *assigns* (=) where it should *add*. Fix it, and turn the bug into a regression test:

```
from collections import defaultdict


def total_usage(lines):
    totals = defaultdict(int)
    for line in lines:
        model, tokens = line.split(",")
        totals[model] += int(tokens)        # add, do not overwrite
    return dict(totals)

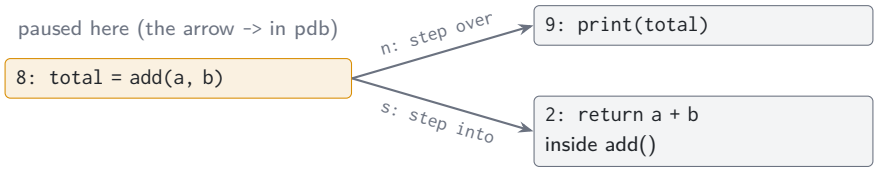

def test_repeated_model_is_added():
    data = ["small,100", "large,40", "small,50"]
    assert total_usage(data) == {"small": 150, "large": 40}


test_repeated_model_is_added()
print(total_usage(["small,100", "large,40", "small,50"]))
```

```
{'small': 150, 'large': 40}
```

Commands you will use

pdb	Name	Does	VS Code
`n`	next	run this line, *step over* any call	F10
`s`	step	*step into* the call on this line	F11
`r`	return	run until the current function returns	Shift+F11
`c`	continue	run until the next breakpoint or the end	F5
`b 5` / `b`	breakpoint	set one on line 5 / list them all	click the gutter
`l` / `ll`	list	show code around the line / the whole function	the editor
`p expr` / `pp expr`	print	show a value / pretty-print it	Variables panel
`w`, `u`, `d`	where, up, down	show the call stack, move between frames	Call Stack panel
`q`	quit	stop the program and leave	Shift+F5



> ➤ **RULE**
> Use `n` when you trust a function (library code, code you tested) and `s` when you suspect it. Do not step line by line from the top: set a breakpoint near the suspect and `c` to it.

Post-mortem: debug a crash where it happened

If a program crashes, run it under `python -m pdb script.py` and type `c`. On the uncaught exception pdb stops at the failing frame ("post mortem"), with all variables intact. For `usage_report.py` that looks like this (condensed from a real session):

```
(Pdb) c
Uncaught exception. Entering post mortem debugging
> ...\usage_report.py(5)get_tokens()
-> return payload["usage"]["output_tokens"]
(Pdb) p payload
{'type': 'error', 'error': {'message': 'overloaded'}}
(Pdb) up
> ...\usage_report.py(10)report()
(Pdb) p raw
'{"type": "error", "error": {"message": "overloaded"}}'
```

Now you can *see* that the payload is an error object. The fix is to check before reading it:

```
def get_tokens(payload):
    if payload.get("type") == "error":
        raise RuntimeError(payload["error"]["message"])
    return payload["usage"]["output_tokens"]
```

Another useful flag is `python -i script.py`: after the script ends or crashes, you stay in the interactive prompt and can inspect the variables.

16.10 `breakpoint()`

Add `breakpoint()` to your code and Python opens pdb *at that line*, with no setup. It is the quickest way into the debugger.

```
# bp.py
def total_usage(lines):
    totals = {}
    for line in lines:
        model, tokens = line.split(",")
        breakpoint()                        # pause here on every loop
        totals[model] = totals.get(model, 0) + int(tokens)
    return totals

print(total_usage(["small,100", "small,50"]))
```

```
> ...\bp.py(6)total_usage()
-> totals[model] = totals.get(model, 0) + int(tokens)
(Pdb) p model, tokens
('small', '100')
(Pdb) c
> ...\bp.py(6)total_usage()
-> totals[model] = totals.get(model, 0) + int(tokens)
(Pdb) p totals
{'small': 100}
(Pdb) c
{'small': 150}
```

The environment variable `PYTHONBREAKPOINT` chooses what `breakpoint()` does. Setting it to `0` turns every breakpoint off without editing the code (real output of the same run):

```
> set PYTHONBREAKPOINT=0               (cmd.exe)
> $env:PYTHONBREAKPOINT = "0"          (PowerShell)
> python bp.py
{'small': 150}
```

Setting it to the name of another debugger function (for example `web_pdb.set_trace`, from a package you install) switches debugger.

> ⚠ **TRAP CARD**
>
> Never commit a `breakpoint()` call: a server or a CI job would stop and wait for input forever. Search for it before every commit (`git grep breakpoint`), or let your linter flag it.

> ■ **BOSS FIGHT: find the cause, do not guess**
>
> Programmers lose hours changing code at random. A better loop has four steps: (1) **reproduce** the bug with the smallest input that still fails (three lines of data, not the whole file); (2) **state a hypothesis** about the cause ("the dict value is overwritten, not added"); (3) **test the hypothesis** by observing, not changing: `print`, a log line, or `p totals` in pdb at the right moment; (4) **fix one thing, then add a test** that fails without the fix. If the observation does not match your hypothesis, your mental model is wrong; revise it instead of editing more code. Most hard bugs are a wrong assumption about the data, not a wrong line of code.

> ➤ **PRACTICE**
>
> 1. A traceback ends with `KeyError: 'usage'`. Which line of the traceback do you read first, and what two questions do you ask next?
> 2. Use an f-string with `=` to print the expression `len(messages) * 2` and its value, and the type of the variable `reply`.
> 3. Why does `logging.debug("x")` print nothing in a fresh script? Give two different ways to make it appear.
> 4. Set up logging so that INFO and above go to the screen but DEBUG and above also go to a file. Which objects do you create?
> 5. You run `python -O` and a check vanishes. Which statement was it, and what should you use for checking user input instead?
> 6. In pdb, you are on a line that calls `summarize(client, text)`. What is the difference between typing `n` and `s`?

> ★ **ACHIEVEMENT UNLOCKED: Bug Hunter**
>
> - Read a traceback from the bottom and locate the failing line in your own code.
> - Debug with f-string `=`, `pprint`, `assert` and a tracing decorator.
> - Configure `logging` with levels, named loggers, a file handler and a format.
> - Step through code in `pdb`, set breakpoints, inspect variables, and debug a crash post-mortem.
> - Turn a fixed bug into a regression test.

CHAPTER 17

Text Data

What this chapter gives you: a clear picture of what a Python `str` really is (a sequence of Unicode code points), how text turns into bytes and back, and how to pull fields out of messy text with regular expressions. You will need both skills every time you read model output, log files, scraped pages or JSON from an API.

17.1 Text Strings: Unicode

Python Unicode strings

A Python 3 `str` is a sequence of *code points*. A code point is just a number that Unicode assigns to one character. ASCII covers only 128 of them. Unicode covers more than 150,000: accented letters, other alphabets, math symbols, emoji. You can write any character with an escape sequence, so your source file can stay plain ASCII.

Escape	Example	Meaning
`\u` + 4 hex digits	`"\u00e9"`	code point up to `FFFF`
`\U` + 8 hex digits	`"\U0001f47b"`	any code point, pad with zeros
`\N{name}`	`"\N{SNOWMAN}"`	the official Unicode name (any case)

The `unicodedata` module converts between names and characters. The built-ins `ord()` and `chr()` convert between a character and its number.

```
import unicodedata

a = "caf\u00e9"
b = "caf\N{LATIN SMALL LETTER E WITH ACUTE}"
print(a == b, len(a))                       # same text, 4 characters
print(unicodedata.name("\u00e9"))
print(hex(ord("\u00e9")), ascii(chr(233)))
print(unicodedata.lookup("snowman") == "\u2603")
print(len("\U0001f47b"))                    # one emoji is one character
```

```
True 4
LATIN SMALL LETTER E WITH ACUTE
0xe9 '\xe9'
True
1
```

■ TYPE CHECK

`len(s)` counts *code points*, not bytes and not what you see on screen. One emoji is length 1. A `str` has no encoding at all. It is only when you turn it into `bytes` that an encoding appears.

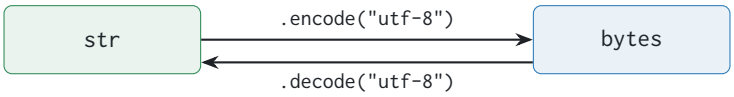
⚠ TRAP CARD

Your terminal or font may not be able to *show* a character that Python holds correctly. A box or question mark on screen does not mean the string is broken. Use `ascii(s)` or `s.encode("unicode-escape")`

to see what is really inside. On Windows, `print()` of non-ASCII text can even raise an error if the console uses an old code page.

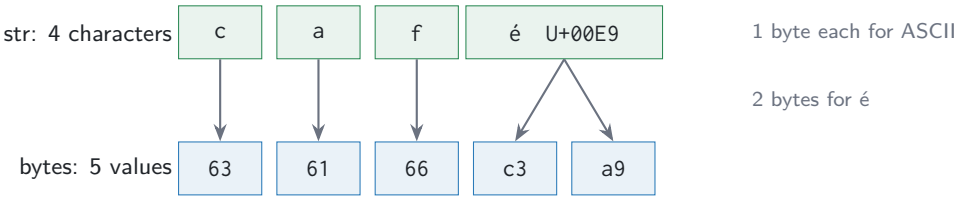
UTF-8: encode and decode

Files, sockets and APIs carry `bytes`, not `str`. Two operations cross the border:



UTF-8 uses one to four bytes per character. ASCII characters take one byte, so plain English text looks the same in ASCII and UTF-8. Accented Latin letters take two, most other scripts three, emoji four. It is the default almost everywhere, so use it unless you are forced not to.

The picture below shows what happens to `"caf\u00e9"`: four characters become five bytes, because the last character needs two.




```
s = "na\u00efve \u2603"          # naive with a diaeresis, plus a snowman
data = s.encode("utf-8")
print(data, len(s), len(data))
print(type(s), type(data))
print(data.decode("utf-8") == s)
```

```
b'na\xc3\xafve \xe2\x98\x83' 7 10
<class 'str'> <class 'bytes'>
True
```

Encode: what if the character does not fit?

Encodings other than UTF-8 cannot hold every character. The default error handler is `"strict"`, which raises `UnicodeEncodeError`. The second argument of `encode()` picks another policy.

```
for enc in ("ascii", "latin-1"):
    try:
        print(enc, s.encode(enc))
    except UnicodeEncodeError as e:
        print(enc, "failed:", e.reason)
print(s.encode("ascii", "ignore"))
print(s.encode("ascii", "replace"))
print(s.encode("ascii", "backslashreplace"))
print(s.encode("ascii", "xmlcharrefreplace"))
```

```
ascii failed: ordinal not in range(128)
latin-1 failed: ordinal not in range(256)
b'nave '
b'na?ve ?'
b'na\\xefve \\u2603'
b'na&#239;ve &#9731;'
```

Handler	What it does with a character that does not fit
`strict`	raise an error (default)
`ignore`	drop it silently
`replace`	write ?
`backslashreplace`	write a Python-style escape such as `\u2603`
`xmlcharrefreplace`	write an HTML/XML numeric entity such as `☃`

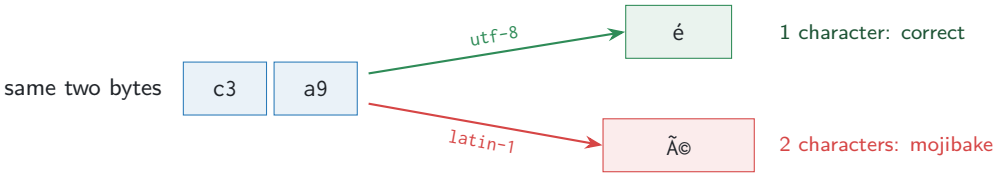
Decode: the encoding is a guess you must get right

Bytes carry no label saying which encoding made them. You must know it from the source: a file header, an HTTP `Content-Type`, a spec. Decode with the wrong one and you get either an error or, worse, wrong text without any error.

```
raw = "caf\u00e9".encode("utf-8")
print(raw)
print(ascii(raw.decode("latin-1")))     # no error, but wrong text
try:
    raw.decode("ascii")
except UnicodeDecodeError as e:
    print("ascii failed:", e.reason, "at", e.start)
print(ascii(b"caf\xe9".decode("utf-8", errors="replace")))
```

```
b'caf\xc3\xa9'
'caf\xc3\xa9'
ascii failed: ordinal not in range(128) at 3
'caf\ufffd'
```

The second line is the classic garbled text (called mojibake): two strange characters where one é should be. The last line shows the `"replace"` handler on decode: bad bytes become the replacement character `U+FFFD`.



➤ RULE: UTF-8 everywhere

Decode bytes to `str` as soon as they enter your program. Work only with `str` inside. Encode to bytes as late as possible, when you write or send. Always name the encoding explicitly (`"utf-8"`) instead of trusting the platform default.

⚠ TRAP CARD

`errors="ignore"` and `errors="replace"` hide data loss. They are fine for a log viewer. They are dangerous in a data pipeline, because corrupted training or eval text goes in without any warning. Prefer `strict` and fix the cause.

■ BOSS FIGHT: the str/bytes border

Most Unicode bugs come from one mistake: not knowing whether a value is `str` or `bytes` at this point in the program. Keep a mental label on every variable. Bytes arrive from outside (`.decode` them), text goes out (`.encode` it). Never mix: `"a" + b"a"` raises `TypeError`, and `"a" == b"a"` is just `False`, not an error. If you see `UnicodeDecodeError`, ask first: *which encoding made these bytes?*

HTML entities

Web text often hides characters behind entities such as `é`, `é` or `é`. The `html` module converts them both ways. This matters when you scrape pages or clean model output that was copied from a web page.

```
import html

print(ascii(html.unescape("&eacute; &#233; &#xe9; &amp; &lt;b&gt;")))
print(html.escape('<a href="x">Q&A</a>'))
print(ascii("caf\u00e9".encode("ascii", "xmlcharrefreplace").decode()))
```

```
'\xe9 \xe9 \xe9 & <b>'
&lt;a href=&quot;x&quot;&gt;Q&amp;A&lt;/a&gt;
'caf&#233;'
```

> **■ TYPE CHECK**
> `html.unescape(str)` returns `str`. For the other direction on a whole string, you encode to ASCII with `xmlcharrefreplace` and get `bytes`; call `.decode()` to get a `str` again. For the markup characters `< > & "` use `html.escape()`. The module `html.entities` also holds dictionaries of names such as `html5` and `codepoint2name`.

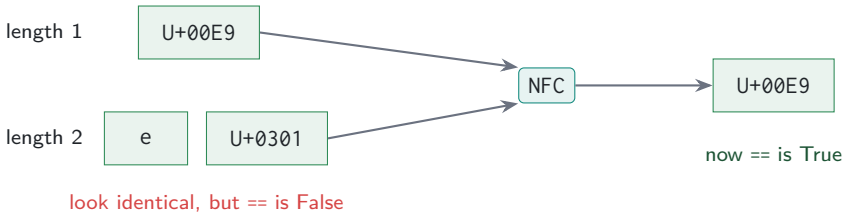
Normalization

Some visible characters can be built in more than one way. The letter é can be one code point (`U+00E9`) or a plain `e` followed by a combining accent (`U+0301`). They look identical. They are not equal.

```
import unicodedata

one = "\u00e9"
two = "e\u0301"
print(one == two, len(one), len(two))
n = unicodedata.normalize("NFC", two)
print(n == one, len(n))
print(len(unicodedata.normalize("NFD", one)))
print(unicodedata.normalize("NFKC", "\ufb01"), len("\ufb01"))
```

```
False 1 2
True 1
2
fi 1
```



`NFC` composes characters into the shortest form, `NFD` splits them apart. The `NFKC` and `NFKD` forms also fold look-alikes (the one-character ligature *fi* becomes the two letters `fi`).

> ➤ **RULE**
> Normalize text with `unicodedata.normalize("NFC", s)` before you compare it, hash it, use it as a dict key, or deduplicate a dataset. Two strings from different sources can look the same and still be different keys.

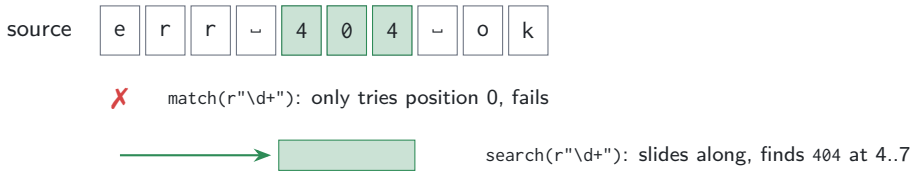
17.2 Text Strings: Regular Expressions

A regular expression (regex) is a small pattern language for describing text. The standard module is `re`. You give it a pattern and a source string. The pattern is itself a `str`.

The five basic functions

Function	Returns	Meaning
`re.match(p, s)`	`Match or None`	pattern must match at the *start* of `s`
`re.search(p, s)`	`Match or None`	first match anywhere in `s`
`re.findall(p, s)`	`list`	every non-overlapping match
`re.split(p, s)`	`list`	pieces of `s` between matches
`re.sub(p, repl, s)`	`str`	copy of `s` with matches replaced

The difference between the first two functions is where they are willing to look:



Our sample is one log line from a training run.

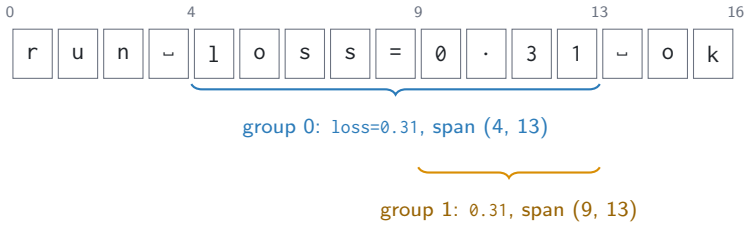
```
import re

line = "2025-03-14 12:01:07 INFO run=a17 model=small-7b loss=0.3125 tokens=1520"

print(re.match(r"\d{4}", line))      # starts with 4 digits
print(re.match(r"INFO", line))       # INFO is not at the start
print(re.search(r"INFO", line))      # but it is somewhere
m = re.search(r"loss=(\d+\.\d+)", line)
print(type(m))
print(m.group(), m.group(1), m.groups(), m.span())
print(float(m.group(1)) + 1)
print(re.search(r"nope", line))
if m := re.search(r"tokens=(\d+)", line):
    print(int(m.group(1)) * 2)
```

```
<re.Match object; span=(0, 4), match='2025'>
None
<re.Match object; span=(20, 24), match='INFO'>
<class 're.Match'>
loss=0.3125 0.3125 ('0.3125',) (48, 59)
1.3125
None
3040
```

Here is what the groups of the `loss` pattern cover. Group 0 is the whole match, group 1 is only the part in parentheses.



■ TYPE CHECK

`match` and `search` return a `re.Match` object or `None`, never a plain string. `m.group()` is the whole match, `m.group(1)` is the first parenthesised part, `m.groups()` is a tuple of all of them. Every group is a `str`, even if it looks like a number. Convert with `float()` or `int()`.

⚠ TRAP CARD

Calling `m.group()` when `m` is `None` raises `AttributeError`. Always test first (`if m:`) or use the walrus form shown above. And remember: `match` only tries the start of the string. People expect it to behave like `search`.

`findall`, `split` and `sub` work on the whole string:

```
print(re.findall(r"\d+", line))
print(re.findall(r"(\w+)=(\S+)", line))
print(dict(re.findall(r"(\w+)=(\S+)", line)))
print(re.split(r"\s*,\s*", "a , b,c ,d"))
print(re.sub(r"\d", "#", "key 12345 here"))
print(re.sub(r"(\w+)=(\S+)", r"\2:\1", "a=1 b=2"))
print(re.sub(r"\d+", lambda m: str(int(m.group()) * 2), "3 apples 10 pears"))
```

```
['2025', '03', '14', '12', '01', '07', '17', '7', '0', '3125', '1520']
[('run', 'a17'), ('model', 'small-7b'), ('loss', '0.3125'), ('tokens', '1520')]
{'run': 'a17', 'model': 'small-7b', 'loss': '0.3125', 'tokens': '1520'}
['a', 'b', 'c', 'd']
key ##### here
1:a 2:b
6 apples 20 pears
```

Notes on the output:

- The first `findall` grabs digits from inside `a17`, `small-7b` and `0.3125` too. A pattern that is too loose finds too much.
- `findall` returns a list of strings if the pattern has no groups, a list of strings if it has one group, and a list of *tuples* if it has two or more.
- In `sub`, `\1`, `\2` refer to groups. The replacement can also be a *function* that receives the match and returns a string.

Other tools you will use daily: `re.finditer()` (like `findall` but yields `Match` objects with positions), `re.fullmatch()` (the whole string must match), and `re.compile(p)` (build the pattern once, reuse it; the compiled object has the same methods).

```
for mm in re.finditer(r"\d+", "x12 y345"):
    print(mm.group(), mm.start(), mm.end())
print(re.fullmatch(r"\d+", "123"))
print(re.fullmatch(r"\d+", "123a"))
pat = re.compile(r"error", re.IGNORECASE)
print(pat.findall("Error ERROR error"))
```

```
12 1 3
345 5 8
<re.Match object; span=(0, 3), match='123'>
None
['Error', 'ERROR', 'error']
```

Patterns

Special characters

Pattern	Matches
`\d` / `\D`	a digit / a non-digit
`\w` / `\W`	a letter, digit or underscore / anything else
`\s` / `\S`	a whitespace character / a non-whitespace character
`\b` / `\B`	a word boundary / not a word boundary
`.`	any single character except newline

For `str` patterns these classes are Unicode-aware. `\d` matches digits from other scripts, and `\w` matches accented letters. Add the flag `re.ASCII` if you want plain ASCII only.

```
print(ascii(re.findall(r"\d", "a1\u0663")))
print(ascii(re.findall(r"\d", "a1\u0663", re.ASCII)))
x = "abc-/*\u00ea\u0115"
print(ascii(re.findall(r"\w", x)))
```

```
['1', '\u0663']
['1']
['a', 'b', 'c', '\xea', '\u0115']
```

Specifiers

Pattern	Meaning
`abc`	the literal text `abc`
`(expr)`	group: treat `expr` as a unit and capture it
`a|b`	`a` or `b`
`^` `$`	start / end of the string (anchors)
`x?` `x*` `x+`	zero or one / zero or more / one or more `x`
`x{m}` `x{m,n}`	exactly `m` / between `m` and `n` `x`
`x*?` `x+?` `x??`	the same, but match as *little* as possible (lazy)
`[abc]` `[^abc]`	one of these characters / any character *not* listed
`x(?=y)` `x(?!y)`	`x` only if followed / not followed by `y`
`(?<=y)x` `(?<!y)x`	`x` only if preceded / not preceded by `y`

```
src = "I wish I may, I wish I might\nHave a dish of fish tonight."
print(re.findall(r"wish|fish", src))
print(re.findall(r"^wish", src), re.findall(r"^I wish", src))
print(re.findall(r"fish$", src), re.findall(r"fish tonight\.$", src))
print(re.findall(r"[wf]ish", src))
print(re.findall(r"[^aeiou\s]+", "hello world"))
print(re.findall(r"I (?=wish)", src))
print(re.findall(r"(?<=I) wish", src))
print(re.findall(r"I (?!wish)", src))
```

```
['wish', 'wish', 'fish']
[] ['I wish']
[] ['fish tonight.']
['wish', 'wish', 'fish']
['h', 'll', 'w', 'rld']
['I ', 'I ']
[' wish', ' wish']
['I ', 'I ']
```

By default `^` and `$` look only at the start and end of the *whole* string. Add `re.MULTILINE` to make them work per line, and `re.DOTALL` so that `.` also matches a newline.

```
print(re.findall(r"^\w+", "ab cd\nef gh"))
print(re.findall(r"^\w+", "ab cd\nef gh", re.MULTILINE))
print(re.findall(r"a.b", "a\nb"), re.findall(r"a.b", "a\nb", re.DOTALL))
```

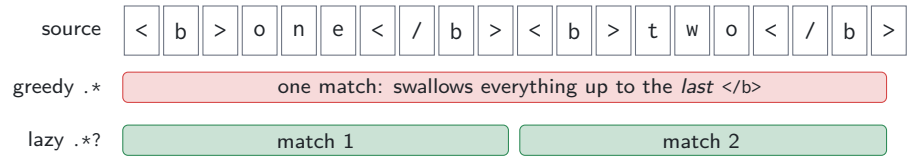
```
['ab']
['ab', 'ef']
[] ['a\nb']
```

Greedy versus lazy

Quantifiers grab as much as they can. That surprises people when they extract text between two markers.

```
t = "<b>one</b><b>two</b>"
print(re.findall(r"<b>.*</b>", t))      # greedy: one big match
print(re.findall(r"<b>.*?</b>", t))     # lazy: stops at the first </b>
print(re.findall(r"a{2,3}", "a aa aaa aaaa"))
```

```
['<b>one</b><b>two</b>']
['<b>one</b>', '<b>two</b>']
['aa', 'aaa', 'aaa']
```



Always use raw strings for patterns

In a normal string, `"\b"` is the backspace character. In a regex, `\b` means word boundary. Put an `r` before the quote so Python leaves backslashes alone.

```
print(len("\b"), len(r"\b"))
print(re.findall("\bcat", "cat"), re.findall(r"\bcat", "cat"))
print(re.findall(r"\bcat\b", "cat concat cat."))
print(re.escape("a.b*c"))
print(re.findall("price: $5", "price: $5"))
print(re.findall(re.escape("price: $5"), "price: $5"))
```

```
1 2
[] ['cat']
['cat', 'cat']
a\.b\*c
[]
['price: $5']
```

> ⚠ **TRAP CARD**
> Forgetting the `r` prefix often *works by accident* (`"\d"` is left alone by Python) and then fails on `\b`. Make the `r` a habit. Also escape special characters when you want them literally: use `\.` for a dot, or let `re.escape()` do it for user-supplied text. The unescaped `$` in the last example means "end of string", so it found nothing.

Specifying match output

Parentheses make groups. Add `?P<name>` inside to name a group; then you read it by name, or get a ready-made dict.

```
m = re.search(r"run=(?P<run>\w+) .*loss=(?P<loss>[\d.]+)", line)
print(m.group("run"), m["loss"], m.groupdict())
out = "Answer: 42. Confidence: 0.87"
print(re.search(r"Confidence:\s*([\d.]+)", out).group(1))
```

```
a17 0.3125 {'run': 'a17', 'loss': '0.3125'}
0.87
```

> ➤ **RULE**
> Use named groups for any pattern with more than one group, and compile it once at module level with a clear name (`LOSS_RE`). Regexes are write-only if you do not name their parts. When the text is JSON, parse it with `json`, not regex. Use regex for text that has no better structure.

17.3 Review and what comes next

You now know that a `str` is Unicode code points, that `.encode()` and `.decode()` are the only doors between `str` and `bytes`, and that the `re` module finds, splits and rewrites text by pattern. The next chapter looks at the `bytes` side: binary data.

> ➤ **PRACTICE**
> 1. Build a string holding the Unicode character with code point `U+1F984` using only an escape sequence. Print its official name and its length.
> 2. Encode that string to UTF-8. How many bytes do you get? Decode them back and compare with the original using `==`.
> 3. Why does `"e\u0301" == "\u00e9"` give `False`? Which call makes the comparison true?
> 4. Write a pattern, with a raw string, that finds every number with a decimal point in the text `"lr 0.001, wd 0.01, epochs 3"`. What does `findall` return?
> 5. Given `"[A] x [B] y"`, extract only the letters in brackets. First write it greedy so it fails, then fix it.
> 6. You get `b"caf\xc3\xa9"` from an API. Which line turns it into a `str`, and what goes wrong if you pass `"latin-1"`?

★ **ACHIEVEMENT UNLOCKED: Text Wrangler**

- Tell `str` from `bytes` and convert between them with an explicit encoding.
- Handle encoding errors on purpose, and recognise mojibake.
- Normalize Unicode text before comparing it.
- Extract fields from a log line with `search`, `findall` and named groups.
- Write regex patterns as raw strings and avoid the greedy trap.

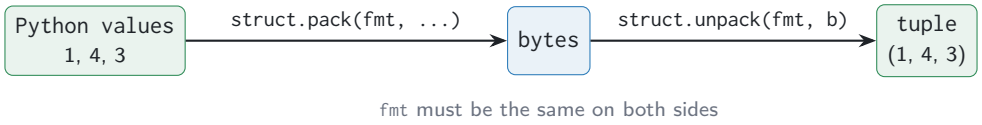
CHAPTER 18

Binary Data

What this chapter gives you: the tools to read and write raw bytes with fixed layouts (`struct`), to show bytes as hex or base64 (`binascii`, `base64`), and to work on single bits with Python's bit operators. You need this for model weight files, embedding dumps, image bytes in API payloads and network packets.

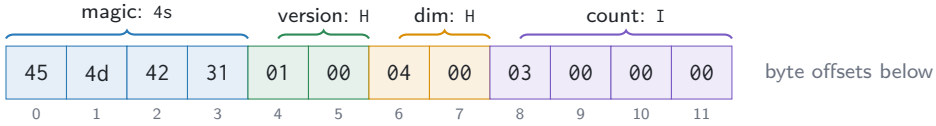
18.1 Convert Binary Data with struct

Python has many text tools and few binary ones. The standard `struct` module is the main one. It converts between Python values and a `bytes` object laid out like a C `struct`: fixed-size fields, one after another, no separators, no labels.



`fmt` must be the same on both sides

Suppose a small file starts with a 12-byte header: a 4-byte tag, a 2-byte version, a 2-byte vector size and a 4-byte count. The format string `"<4sHHI"` describes exactly that.




```
import struct

header = struct.pack("<4sHHI", b"EMB1", 1, 4, 3)
print(header)
print(len(header), struct.calcsize("<4sHHI"))
magic, version, dim, count = struct.unpack("<4sHHI", header)
print(magic, version, dim, count)
print(type(header), type(struct.unpack("<4sHHI", header)))
```

```
b'EMB1\x01\x00\x04\x00\x03\x00\x00\x00'
12 12
b'EMB1' 1 4 3
<class 'bytes'> <class 'tuple'>
```

> ■ **TYPE CHECK**
>
> `pack` returns `bytes`. `unpack` always returns a *tuple*, even when there is one value, so write `(n,) = struct.unpack("<I", b)` or use `[0]`. A `4s` field comes back as `bytes`, not `str`. Decode it yourself if it holds text.

The format string

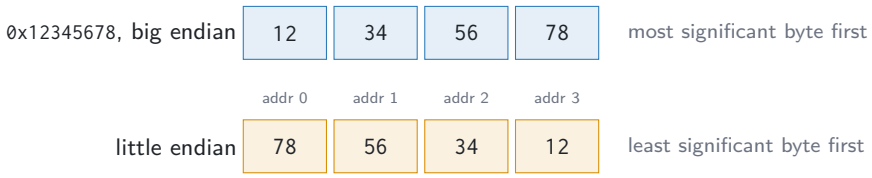
The *first* character sets byte order (and whether sizes are standard). Then one letter per field, optionally with a count in front: `"4f"` means four floats, `"2H"` means two unsigned shorts.

Char	Meaning	Python type
`< >`	little / big endian, standard sizes	(prefix)
`= @`	native order; `@` also adds alignment padding	(prefix)
`b B`	signed / unsigned byte (1)	`int`
`h H`	signed / unsigned short (2)	`int`
`i I`	signed / unsigned int (4)	`int`
`q Q`	signed / unsigned long long (8)	`int`
`f d`	float32 (4) / float64 (8)	`float`
`?`	bool (1)	`bool`
`Ns`	`N` raw bytes, padded with zero bytes	`bytes`
`x`	one padding byte, skipped	(no value)

> ➤ **RULE**
> Always start the format with `<` or `>`. Without a prefix Python uses your machine's native order *and* may insert padding bytes between fields, so `@bI` is 8 bytes while `<bI` is 5. A file format is defined by its spec, not by your CPU.

Endianness

A number bigger than one byte must be stored in some order. *Big endian* puts the most significant byte first, *little endian* the least significant first. Most CPUs (x86, ARM in normal mode) are little endian. Network protocols and many file headers are big endian.




```
print(struct.pack(">I", 0x12345678).hex(" "))
print(struct.pack("<I", 0x12345678).hex(" "))
print((0x12345678).to_bytes(4, "big").hex(" "))
print(int.from_bytes(b"\x01\x00", "little"), int.from_bytes(b"\x01\x00", "big"))
print(int.from_bytes(b"\xff\xff", "big", signed=True))
```

```
12 34 56 78
78 56 34 12
12 34 56 78
1 256
-1
```

For a single integer, `int.to_bytes()` and `int.from_bytes()` are simpler than `struct`. The same two bytes `01 00` are 1 in little endian and 256 in big endian, so a wrong guess gives a wrong number, not an error.

Reading from the middle of a buffer

You rarely have just the header. `unpack_from(fmt, buf, offset)` reads at an offset without slicing. `iter_unpack` walks over a repeated record. The `x` code skips bytes you do not care about.

```
vec = struct.pack("<4f", 0.5, -1.25, 3.0, 0.1)
blob = header + vec
print(struct.unpack_from("<4f", blob, 12))
print(struct.unpack_from("<4sHHI", blob))
for (x,) in struct.iter_unpack("<f", vec):
    print(x)
print(struct.unpack("<8xHH2x", b"12345678\x07\x00\x08\x00.."))
```

```
(0.5, -1.25, 3.0, 0.10000000149011612)
(b'EMB1', 1, 4, 3)
0.5
-1.25
3.0
0.10000000149011612
(7, 8)
```

Look at the last float: `0.1` stored as float32 comes back as `0.10000000149011612`. Float32 holds about seven decimal digits. This is why model weights saved as float32 and read as Python `float` (which is float64) show these long tails. It is not a bug.

Typical errors

```
for bad in (lambda: struct.unpack("<I", b"\x01\x02"),
            lambda: struct.pack("<B", 300),
            lambda: struct.pack("<4s", "abcd")):
    try:
        bad()
    except struct.error as e:
        print(e)
print(struct.pack("<8s", b"abc"))
print(struct.calcsize("@bI"), struct.calcsize("<bI"))
```

```
unpack requires a buffer of 4 bytes
'B' format requires 0 <= number <= 255
argument for 's' must be a bytes object
b'abc\x00\x00\x00\x00\x00'
8 5
```

> ⚠ **TRAP CARD**
>
> Three classic mistakes: giving `unpack` a buffer of the wrong length (it must match `calcsize` exactly), passing a `str` to an `s` field (you need `bytes`), and packing a number that does not fit the type. All raise `struct.error`.

A real-world shape: length, then JSON, then data

Many ML file formats start with a small fixed header that tells you how long the next, flexible part is. The pattern below (a length, a JSON block, then raw floats) imitates this idea.

```
import json

meta = json.dumps({"w": {"dtype": "F32", "shape": [2]}}).encode("utf-8")
blob = struct.pack("<Q", len(meta)) + meta + struct.pack("<2f", 1.5, -2.0)
(n,) = struct.unpack_from("<Q", blob)
info = json.loads(blob[8:8 + n])
print(n, info)
print(struct.unpack_from("<2f", blob, 8 + n))
print(type(info))
```

```
37 {'w': {'dtype': 'F32', 'shape': [2]}}
(1.5, -2.0)
<class 'dict'>
```

BOSS FIGHT: nothing in the bytes tells you the layout

Binary data has no field names, no types, no separators. The same 4 bytes can be an `int`, a `float`, four tiny numbers or text. The meaning lives in a spec, and your format string is your copy of that spec. To decode safely: (1) check a magic value first, (2) pick byte order explicitly, (3) pick the exact sizes and signedness, (4) check the total length with `calcsize`. When a number comes out absurd (millions where you expect 3), suspect byte order first.

18.2 Extraction with Binary Data Tools

`struct` gets clumsy when formats are nested or conditional. Third-party packages describe the layout declaratively instead: `construct`, `bitstring`, `hachoir` and Kaitai Struct are the ones to know. You describe the fields once and the library parses (and sometimes builds) the bytes. Below is the same header with `construct`. It needs `pip install construct`; I did not run it here, so check the library docs for the exact API of your version.

```
# needs: pip install construct   (not run in these notes)
from construct import Struct, Const, Int16ul, Int32ul

Header = Struct(
    "magic" / Const(b"EMB1"),
    "version" / Int16ul,      # ul = unsigned, little endian
    "dim" / Int16ul,
    "count" / Int32ul,
)
h = Header.parse(header)         # header = the 12 bytes built above
print(h.dim, h.count)
```

	struct	**declarative library**
Install	built in	`pip install`
Style	format string, positions by hand	named fields, nested records
Good for	flat, fixed records	complex or conditional formats
Readability	terse, easy to mis-type	self-documenting

18.3 Convert Bytes/Strings with binascii

Raw bytes print as a mix of letters and `\x` escapes. To *see* them, convert to hex. To *send* them through a text-only channel such as JSON, convert to base64.

```python
import binascii, base64

print(binascii.hexlify(b"\x89PNG"))
print(binascii.unhexlify(b"89504e47"))
print(b"\x89PNG".hex(), bytes.fromhex("89 50 4e 47"))
print(type(b"ab".hex()), type(binascii.hexlify(b"ab")))
```

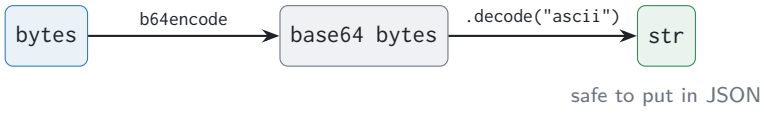
```
b'89504e47'
b'\x89PNG'
89504e47 b'\x89PNG'
<class 'str'> <class 'bytes'>
```

> ■ **TYPE CHECK**
>
> `bytes.hex()` returns a `str`, and `bytes.fromhex(str)` takes one. The older `binascii.hexlify` returns `bytes`. The methods on `bytes` are usually nicer. Pass `hex(" ")` to put a space between bytes.

Base64 turns any bytes into plain ASCII letters, digits, `+`, `/` and `=`. APIs that accept an image inside JSON expect exactly this.

```python
raw = bytes(range(250, 256)) + b"hi"
b64 = base64.b64encode(raw)
print(b64, type(b64))
print(b64.decode("ascii"))
print(base64.b64decode(b64) == raw)
print(hex(binascii.crc32(b"hello")))
```

```
b'+vv8/f7/aGk=' <class 'bytes'>
+vv8/f7/aGk=
True
0x3610a686
```

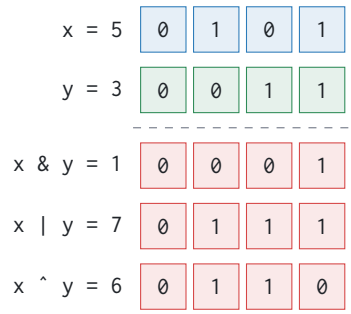


> ⚠ **TRAP CARD**
>
> `b64encode` returns `bytes`, and `json.dumps` cannot serialize `bytes`. Call `.decode("ascii")` to get a `str` first. Base64 is an *encoding*, not encryption, and it makes the data about one third larger.

18.4 Use Bit Operators

Integers are stored as bits. Python lets you work on those bits with six operators. They work on `int` only.

Operator	Name	Result bit is 1 when ...	Example
`&`	and	both bits are 1	`5 & 3 = 1`
`|`	or	at least one bit is 1	`5 | 3 = 7`
`^`	xor	exactly one bit is 1	`5 ^ 3 = 6`
`~`	not	flips every bit (`~x` is `-x-1`)	`~5 = -6`
`<<`	left shift	bits move left, zeros fill in (times 2)	`5 << 1 = 10`
`>>`	right shift	bits move right (floor-divide by 2)	`5 >> 1 = 2`




```
x, y = 0b0101, 0b0011
print(x & y, x | y, x ^ y, ~x, x << 1, x >> 1)
print(bin(x), format(x, "08b"), f"{x:#06b}", f"{255:#x}")
print(~x, ~x & 0xF, f"{~x & 0xFF:08b}")
print(-1 & 0xFF, -6 & 0xFF)
```

```
1 7 6 -6 10 2
0b101 00000101 0b0101 0xff
-6 10 11111010
255 250
```

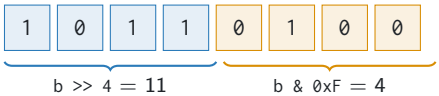
> ■ **TYPE CHECK**
> Python's `int` has no fixed width, so `~5` is `-6`, not a big positive number. Use a mask to see a fixed width: `~5 & 0xFF` is `250`. Negative numbers behave as if they had infinitely many leading 1 bits (two's complement), which is why `-6 & 0xFF` gives 250.

Using bits as flags and fields

A common use is packing several yes/no options into one integer, or pulling a field out of a byte with a shift and a mask.

```
READ, WRITE, EXEC = 0b100, 0b010, 0b001
perm = READ | WRITE              # set two flags
print(perm, bool(perm & WRITE), bool(perm & EXEC))
perm |= EXEC                     # set a flag
perm &= ~WRITE                   # clear a flag
print(f"{perm:03b}")
perm ^= READ                     # toggle a flag
print(f"{perm:03b}")
b = 0b10110100
print((b >> 4) & 0xF, b & 0xF)   # high nibble, low nibble
print((255).bit_count(), (0b1011).bit_length())
```

```
6 True False
101
001
11 4
8 4
```



⚠ **TRAP CARD**

Do not confuse `&` with `and`. `6 and 3` is `3` (logical: the last truthy value), `6 & 3` is `2` (bit by bit). And `<<` with a huge shift quietly builds a huge integer. It never overflows.

18.5 Review and what comes next

You can now describe a binary layout with a format string, read and write it with `struct`, display bytes as hex, move them through JSON as base64, and flip single bits. The next chapter turns to dates and times.

➤ **PRACTICE**

1. Write the hex string `"4749463839"` to a `bytes` variable with `bytes.fromhex`. Does it start with the ASCII letters `GIF`? Show how you check.
2. Pack the three integers 7, 300 and 70000 as unsigned 1-, 2- and 4-byte little-endian values. What is the length of the result, and which format string did you use?
3. The bytes `b"\x00\x00\x01\x00"` are an unsigned 4-byte integer. What are its values read big endian and little endian?
4. Why does `struct.unpack("<I", data)` raise an error when `data` has 5 bytes? Show two ways to read only the first four.
5. Store the flags `ADMIN=1`, `BETA=2`, `DARK=4` in one integer. Turn on `BETA`, turn off `ADMIN`, and test for `DARK`.
6. Take 10 random bytes, convert to base64, put it in a dict, dump it with `json`, load it back and recover the original bytes.

★ **ACHIEVEMENT UNLOCKED: Byte Wrangler**

- Read and write fixed binary layouts with `struct`, choosing byte order on purpose.
- Explain endianness and spot it as the cause of absurd numbers.
- Show bytes as hex, and ship them inside JSON as base64.
- Set, clear, toggle and test bits, and extract a bit field with shift and mask.

CHAPTER 19

Dates and Times

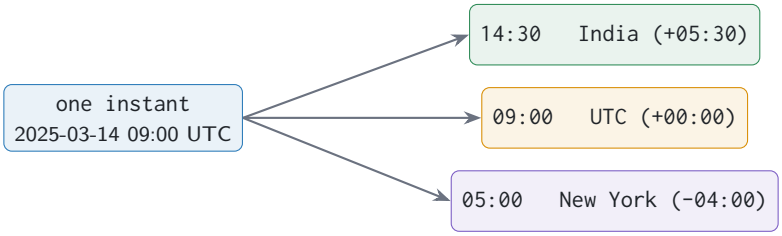
What this chapter gives you: the four types you need from `datetime` (`date`, `time`, `datetime`, `timedelta`), the difference between a `str`, a `datetime` and a float of seconds, and a few rules that stop time zone bugs. In AI work you meet this in log timestamps, API `created` fields, run durations and dated dataset files.

19.1 Why dates and times are hard

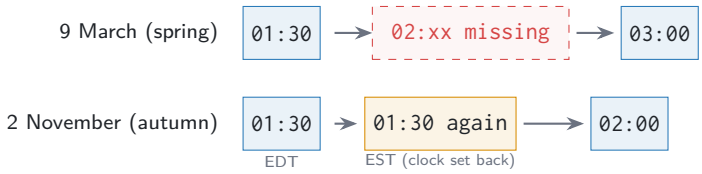
The same day can be written many ways, and some are ambiguous.

Text	Problem
`21 Jul 1987`	month name depends on the language
`7/21/1987`	only the US order; 21 cannot be a month
`03/04/2025`	4 March or 3 April? Depends on the country
`2025-03-04`	ISO 8601: year, month, day. Unambiguous, and it sorts as text

Times have a worse problem: time zones follow politics, not geometry, and daylight saving time (DST) moves the clock on different days in different countries. One moment in time has many local clock readings.



DST makes it worse. In spring one local hour does not exist. In autumn one local hour happens twice. Here is New York in 2025:

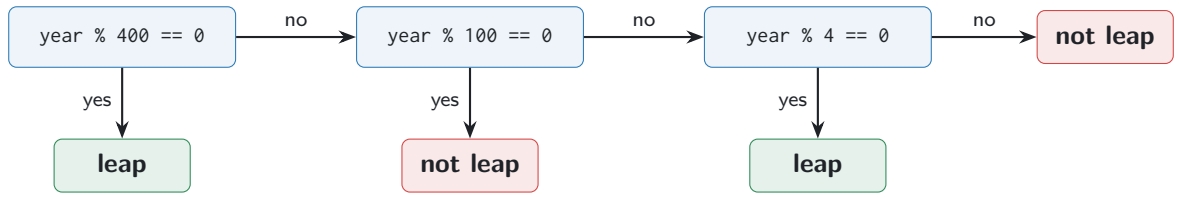


> ➤ **RULE: store UTC, show local**
>
> Keep every stored or computed timestamp in UTC (or as an epoch number). Convert to a local zone only at the edge, when a human reads it. Servers should run on UTC. Local wall-clock times with DST create hours that do not exist and hours that happen twice.

Python spreads dates over several modules: `datetime` (the main one), `time` (epoch seconds and clocks) and `calendar` (calendar facts). They overlap, which is confusing at first.

19.2 Leap Year

A year has about 365.2422 days, so calendars add a day now and then. The rule has three layers, checked from the most special case outward.




```
import calendar

print([(y, calendar.isleap(y)) for y in (1900, 1996, 2000, 2023, 2024)])
print(calendar.monthrange(2024, 2)[1], calendar.monthrange(2023, 2)[1])
print(calendar.day_name[0], calendar.month_name[2])
```

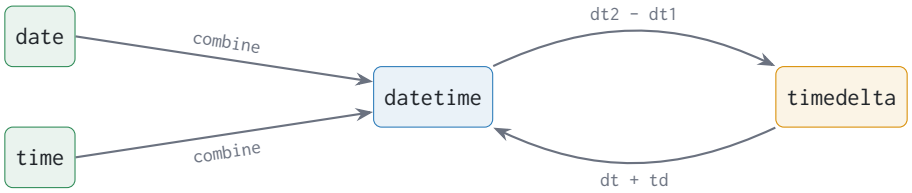
```
[(1900, False), (1996, True), (2000, True), (2023, False), (2024, True)]
29 28
Monday February
```

`monthrange(year, month)` returns the weekday of the first day and the number of days. Do not write your own leap rule. Use `calendar.isleap`, or better, let `datetime` do the date arithmetic so leap days are handled for you.

19.3 The datetime Module

`datetime` has four main classes.

Class	Holds	Looks like
`date`	year, month, day	`2025-03-14`
`time`	hour, minute, second, microsecond	`09:30:00`
`datetime`	a date and a time together	`2025-03-14T09:30:00`
`timedelta`	a length of time	`1 day, 3:00:00`



date and timedelta

```
from datetime import date, datetime, timedelta, timezone
from datetime import time as dtime   # avoid a clash with the time module

d = date(2025, 3, 14)
print(d, repr(d))
print(d.year, d.month, d.day, d.weekday(), d.strftime("%A"))
print(d + timedelta(days=30), d - timedelta(weeks=2))
gap = date(2025, 12, 25) - d
print(gap, type(gap), gap.days)
print(date.min, date.max)
two = timedelta(days=2)
print(date(2024, 2, 28) + two, date(2025, 2, 28) + two)
```

```
2025-03-14 datetime.date(2025, 3, 14)
2025 3 14 4 Friday
2025-04-13 2025-02-28
286 days, 0:00:00 <class 'datetime.timedelta'> 286
0001-01-01 9999-12-31
2024-03-01 2025-03-02
```

`weekday()` counts Monday as 0, so Friday is 4. `isoweekday()` counts Monday as 1. The leap day in 2024 shows up with no extra code.

> ■ **TYPE CHECK**
> `print(d)` shows `2025-03-14`, but `d` is a `date` object, not a string. `repr(d)` shows its real nature. Subtracting two dates gives a `timedelta`, not an `int`: use `.days` or `.total_seconds()` to get a number.

A `timedelta` stores days, seconds and microseconds. Careful with `.seconds`: it is only the seconds part after whole days, not the full length.

```
a = datetime(2025, 3, 14, 12, 0, 0)
b = datetime(2025, 3, 14, 12, 0, 7, 250000)
td = b - a
print(td, td.total_seconds(), td.seconds, td.microseconds)
print(timedelta(days=1, hours=3), timedelta(minutes=90).total_seconds())
print(timedelta(days=1.5), timedelta(hours=3) / timedelta(hours=2))
print(timedelta(hours=25) // timedelta(hours=2), timedelta(hours=1) * 3)
print(timedelta(seconds=-5))
```

```
0:00:07.250000 7.25 7 250000
1 day, 3:00:00 5400.0
1 day, 12:00:00 1.5
12 3:00:00
-1 day, 23:59:55
```

> ⚠ **TRAP CARD**
> Negative timedeltas print strangely: five seconds backward is `-1 day, 23:59:55`. It is correct, just stored as days plus a positive remainder. Use `total_seconds()` when you want `-5.0`. Also, `.seconds` of a 2-day delta is `0`, not 172800.

```
for bad in (lambda: a - d, lambda: d + d):
    try:
        bad()
    except TypeError as e:
        msg = str(e).replace("unsupported operand type(s) for ", "")
        print("TypeError:", msg)
```

```
TypeError: -: 'datetime.datetime' and 'datetime.date'
TypeError: +: 'datetime.date' and 'datetime.date'
```

Expression	Result type	Note
`date - date`	`timedelta`	also `datetime - datetime`
`date + timedelta`	`date`	`datetime + timedelta` gives `datetime`
`timedelta * int`	`timedelta`	
`timedelta / timedelta`	`float`	`//` gives an `int`
`date + date`	`TypeError`	adding two points in time makes no sense
`datetime - date`	`TypeError`	do not mix the two types

time and datetime

```
t = dtime(9, 30)
print(repr(t), t.hour, t.isoformat())
dt = datetime(2025, 1, 2, 3, 4, 5, 6)   # y, m, d, h, min, s, microsecond
print(repr(dt))
print(dt.isoformat())
print(dt.isoformat(sep=" ", timespec="seconds"))
full = datetime.combine(d, t)
print(full, repr(full.date()), full.time())
print(full.replace(hour=0, minute=0), full.replace(year=2026))
print(type(datetime.now()), type(date.today()))
```

```
datetime.time(9, 30) 9 09:30:00
datetime.datetime(2025, 1, 2, 3, 4, 5, 6)
2025-01-02T03:04:05.000006
2025-01-02 03:04:05
2025-03-14 09:30:00 datetime.date(2025, 3, 14) 09:30:00
2025-03-14 00:00:00 2026-03-14 09:30:00
<class 'datetime.datetime'> <class 'datetime.date'>
```

Objects of these classes are *immutable*. `replace()` returns a new object. The `now()` and `today()` calls give the current moment, so their results change each time you run them; I show only their types above.

> ⚠ **TRAP CARD**
>
> `datetime.now()` returns local time *without* any time zone attached. Two machines in different zones produce different values for the same instant, and Python cannot tell which is which. See the section on aware datetimes below.

Naive versus aware

A `datetime` is *naive* when it has no `tzinfo`, and *aware* when it carries a UTC offset. Naive values mean "some wall clock, somewhere". Aware values mean one exact instant.

```
naive = datetime(2025, 3, 14, 9, 0)
aware = datetime(2025, 3, 14, 9, 0, tzinfo=timezone.utc)
print(naive.tzinfo, aware.tzinfo, aware.isoformat())
try:
    naive < aware
except TypeError as e:
    print("TypeError:", e)
ist = timezone(timedelta(hours=5, minutes=30), "IST")
local = aware.astimezone(ist)
print(local.isoformat(), local == aware)
print(datetime.now(timezone.utc).tzinfo)
```

```
None UTC 2025-03-14T09:00:00+00:00
TypeError: can't compare offset-naive and offset-aware datetimes
2025-03-14T14:30:00+05:30 True
UTC
```

`astimezone` changes how the instant is *shown*, not which instant it is, so `local == aware` is `True`. For real zones with DST rules, use the standard `zoneinfo` module. On Windows it needs the `tzdata` package (`pip install tzdata`); I could not run that here.

```
# needs: tzdata on Windows (pip install tzdata); not run in these notes
from zoneinfo import ZoneInfo

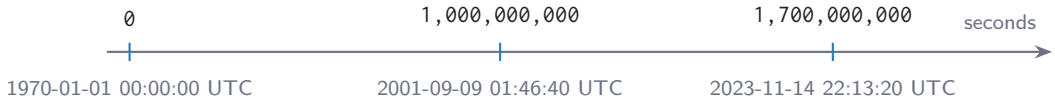
ny = ZoneInfo("America/New_York")
instant = datetime(2025, 3, 14, 9, 0, tzinfo=timezone.utc)
print(instant.astimezone(ny))      # the same instant on a New York wall clock
```

■ BOSS FIGHT: naive versus aware

Most date bugs in real systems are naive/aware bugs. A naive `datetime` is just numbers; it does not say which zone it is in. An aware one is an exact instant. You cannot compare or subtract the two kinds (Python raises `TypeError`, which is the helpful outcome). The safe pattern: create aware UTC values (`datetime.now(timezone.utc)`), store them as ISO strings with an offset or as epoch numbers, and convert with `astimezone` only for display.

19.4 The time Module

The `time` module is *not* the same as the `datetime.time` class. It works with *epoch seconds*: the number of seconds since midnight UTC on 1 January 1970. This is the simplest way to pass a moment between systems, such as JavaScript, databases or API fields (`"created": 1700000000`).




```
import calendar, time

st = time.gmtime(1_700_000_000)          # UTC, as a struct_time
print(type(st).__name__, tuple(st))
print(time.asctime(st))
print(st.tm_year, st[0], st.tm_yday, st.tm_wday)
print(time.strftime("%Y-%m-%d %H:%M:%S", st))
print(calendar.timegm(st))               # back to epoch seconds (UTC)
print(type(time.time()), type(time.perf_counter()))
```

```
struct_time (2023, 11, 14, 22, 13, 20, 1, 318, 0)
Tue Nov 14 22:13:20 2023
2023 2023 318 1
2023-11-14 22:13:20
1700000000
<class 'float'> <class 'float'>
```

`time.time()` returns a `float` of epoch seconds. `localtime(secs)` gives a `struct_time` in your zone, `gmtime(secs)` in UTC. A `struct_time` is a named tuple of nine fields: year, month, day, hour, minute, second, weekday, day of year, DST flag. Weekday counts from 0 (Monday), day of year from 1.

Function	Returns	Use it for
`time.time()`	`float`, epoch seconds	timestamps to store or send
`time.gmtime(s)` / `localtime(s)`	`struct_time`	breaking seconds into parts
`time.mktime(st)`	`float`	`struct_time` in local zone back to seconds
`calendar.timegm(st)`	`int`	`struct_time` in UTC back to seconds
`time.perf_counter()`	`float`	measuring how long code takes
`time.monotonic()`	`float`	timeouts that must never go backward
`time.sleep(s)`	None	pause for `s` seconds

> ➤ **RULE**
>
> To time code, use `time.perf_counter()`, not `time.time()`. The wall clock can jump (clock sync, DST), while `perf_counter` only moves forward and has high resolution. The absolute value is meaningless; subtract two readings.

```
t0 = time.perf_counter()
time.sleep(0.05)
elapsed = time.perf_counter() - t0
print(0.04 < elapsed < 0.2)
```

```
True
```

19.5 Read and Write Dates and Times

ISO 8601: the default choice

`isoformat()` writes it, `fromisoformat()` reads it. In Python 3.11 and later it accepts a trailing `Z` too. Use ISO for logs, JSON, JSONL eval files and file names: it is unambiguous and sorts correctly as plain text.

```
print(date.fromisoformat("2025-03-14") == d)
print(sorted(["2025-03-14", "2025-01-09", "2024-12-31"]))
print(datetime.fromisoformat("2025-02-14T15:05:09+00:00"))
print(repr(datetime.fromisoformat("2025-02-14T15:05:09Z")))
```

```
True
['2024-12-31', '2025-01-09', '2025-03-14']
2025-02-14 15:05:09+00:00
datetime.datetime(2025, 2, 14, 15, 5, 9, tzinfo=datetime.timezone.utc)
```

strftime: object to string

`strftime` ("string from time") formats a date or time with percent codes. `strptime` ("string parsed to time") goes the other way with the same codes.



Code	Meaning	Example	Note
%Y %y	year 4 / 2 digits	2025 25	
%m	month	02	zero-padded
%d	day of month	14	zero-padded
%H %I %p	hour 24 / 12, AM or PM	15 03 PM	
%M %S	minute, second	05 09	
%f	microseconds	123456	6 digits
%b %B	month name short / long	Feb February	locale dependent
%a %A	weekday short / long	Fri Friday	locale dependent
%j	day of year	045	
%z %Z	UTC offset, zone name	+0000 UTC	aware values only

```
dt = datetime(2025, 2, 14, 15, 5, 9)
fmt = "It's %A, %B %d, %Y, %I:%M:%S%p"
print(dt.strftime(fmt))
print(dt.strftime("%Y%m%d-%H%M%S"))
print(date(2025, 2, 14).strftime("%d/%m/%Y %H:%M"))
print(dtime(10, 35).strftime("%H:%M"), dtime(10, 35).strftime("%Y-%m-%d"))
```

```
It's Friday, February 14, 2025, 03:05:09PM
20250214-150509
14/02/2025 00:00
10:35 1900-01-01
```

> ⚠ **TRAP CARD**
> A `date` has no time, so `%H:%M` prints midnight (`00:00`). A `time` has no date, so `%Y-%m-%d` prints the meaningless `1900-01-01`. Format only the parts the object really has.

strptime: string to object

`datetime.strptime(text, fmt)` returns a `datetime`. It is not a regex: the characters that are not percent codes (dashes, spaces, slashes) must match exactly. It also checks that the date exists.

```
p = datetime.strptime("2025-02-14 15:05:09", "%Y-%m-%d %H:%M:%S")
print(repr(p), p == dt, type(p))
print(datetime.strptime("14 Feb 2025", "%d %b %Y").date())
print(datetime.strptime("03/04/2025", "%d/%m/%Y").date())
print(datetime.strptime("03/04/2025", "%m/%d/%Y").date())
for text in ("2025 02 14", "2025-02-30"):
    try:
        datetime.strptime(text, "%Y-%m-%d")
    except ValueError as e:
        print("ValueError:", e)
```

```
datetime.datetime(2025, 2, 14, 15, 5, 9) True <class 'datetime.datetime'>
2025-02-14
2025-04-03
2025-03-04
ValueError: time data '2025 02 14' does not match format '%Y-%m-%d'
ValueError: day is out of range for month
```

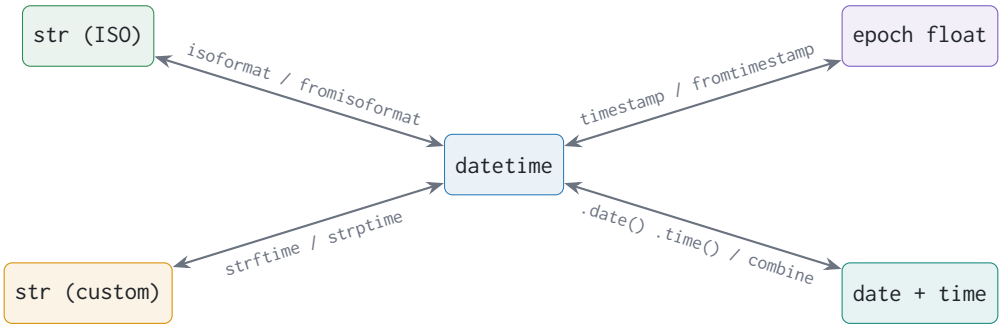
> ⚠ **TRAP CARD**
> The same text `03/04/2025` gives 3 April or 4 March depending on the format you choose, and Python will not warn you. Never guess the order. Ask the data source, or demand ISO. Also, `strptime` returns a *naive* datetime unless the text contains an offset parsed with `%z`.

Locale and names

Month and weekday names come from the operating system's locale. `locale.setlocale(locale.LC_TIME, "fr_FR")` switches them. That works only if the OS has that locale installed (names vary, such as `fr_FR.UTF-8` on Linux). It changes process-wide state, so avoid it in servers. For storing and exchanging data, use numeric or ISO formats, which do not depend on language.

19.6 All the Conversions

You end up converting among a handful of forms all the time. The `datetime` object is the hub.




```
a = datetime(2025, 3, 14, 9, 0, tzinfo=timezone.utc)
print(a.timestamp())
print(datetime.fromtimestamp(1_700_000_000, tz=timezone.utc).isoformat())
print(a.strftime("%Y-%m-%dT%H:%M:%SZ"))
print(a.timetuple().tm_yday)
```

```
1741942800.0
2023-11-14T22:13:20+00:00
2025-03-14T09:00:00Z
73
```

> ⚠ **TRAP CARD**
> `datetime.fromtimestamp(secs)` *without* `tz=` gives a naive datetime in your local zone. For API timestamps, always pass `tz=timezone.utc`. Likewise `timestamp()` on a naive datetime assumes local time, which is a silent source of off-by-hours errors.

A short AI-flavored example: duration of a run from ISO log timestamps.

```
lines = [
    "2025-03-14T09:00:01Z start run=a",
    "2025-03-14T09:00:03.500Z first token run=a",
    "2025-03-14T09:02:11Z done run=a",
]
stamps = [datetime.fromisoformat(l.split()[0]) for l in lines]
print([str(y - x) for x, y in zip(stamps, stamps[1:])])
print((stamps[-1] - stamps[0]).total_seconds())
print(date(1990, 5, 17) + timedelta(days=10000))
```

```
['0:00:02.500000', '0:02:07.500000']
130.0
2017-10-02
```

The last line answers a typical "when am I 10,000 days old" question for someone born on 17 May 1990.

19.7 Alternative Modules

If the standard library feels clumsy, these packages exist. None is needed for this series.

Package	What it is good for
`python-dateutil`	parses almost any date text; relative dates such as "next month"
`arrow`	one friendly object for create, shift, format
`whenever`	newer, fast library (written in Rust) with clear naive/aware types
`iso8601`	fills gaps in ISO 8601 parsing
`astropy.time`, `TAI64`	astronomy-size dates, atomic time

> ➤ **RULE**
> Before you add a date library, try `datetime` with `timezone.utc` and `fromisoformat`. It covers most log and API work. Use `dateutil.parser` only for messy human-written dates you cannot control.

19.8 Review and what comes next

You can now tell a date, a time, a datetime and a timedelta apart, convert between ISO strings, custom strings, epoch numbers and objects, and you know to keep UTC at the core. The next chapter opens files: reading, writing and moving data on disk.

> ➤ **PRACTICE**
> 1. Which of the years 1800, 2000, 2023 and 2100 are leap years? Check with code.
> 2. Create the date of your birthday, print its ISO form and its weekday name, and compute the date on which you turn 10,000 days old.
> 3. Why does `date(2025, 3, 1) - date(2025, 2, 1)` have type `timedelta` and not `int`? Which attribute gives a number of days?
> 4. Parse `"07/08/2025"` two times, once as day-first and once as month-first. Print both results in ISO format.
> 5. Write one line that turns the API field `1700000000` into an ISO string in UTC.
> 6. What happens if you subtract `datetime.now()` from `datetime.now(timezone.utc)`? Why is that error a good thing?

> ★ **ACHIEVEMENT UNLOCKED: Time Keeper**
> - Choose the right type among `date`, `time`, `datetime` and `timedelta`.
> - Parse and format timestamps with ISO, `strptime` and `strftime`.
> - Convert between epoch seconds, aware datetimes and strings.
> - Explain naive versus aware datetimes and why you store UTC.
> - Time a piece of code correctly with `perf_counter`.

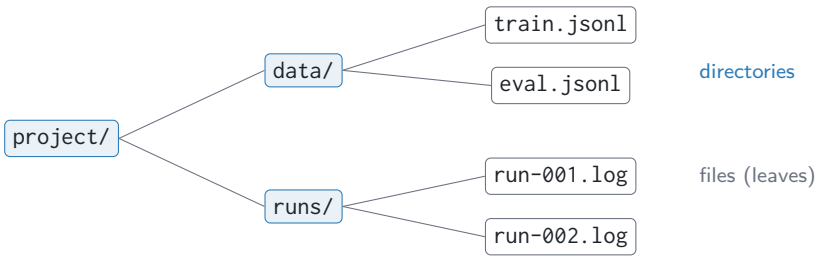
CHAPTER 20

Files

What this chapter gives you: how to open, read, write and close files safely, how to jump around inside a file, and how to find, copy, move and name files and folders. You will use this for JSONL eval sets, logs, model outputs, checkpoints and config files. You also meet `StringIO` and `BytesIO`, which make memory behave like a file.

20.1 Files and directories

A *file* is a sequence of bytes stored under a name. A *directory* (folder) holds files and other directories, so the whole thing forms a tree. Text and bytes are the same on disk. The only difference is whether Python decodes the bytes into `str` for you.



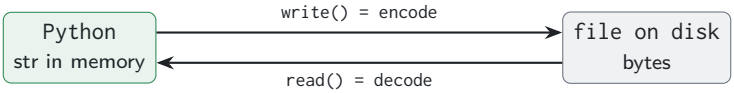
20.2 File Input and Output

Create or open with open()

`open(name, mode)` returns a *file object*. It is your handle on the file: you call methods on it to read and write. When you are done you must close it, so that buffered writes reach the disk.

Mode	Meaning	File must exist?	Erases old content?	Pointer starts
`"r"`	read (default)	yes	no	start
`"w"`	write	no, created	**yes**	start
`"x"`	write, but only if new	must *not* exist	–	start
`"a"`	append	no, created	no	end
`"r+"`	read and write	yes	no	start

A second letter picks the kind: `t` for text (the default) or `b` for bytes. So `"wb"` is write-bytes and `"rt"` is the same as `"r"`.



text mode does the encoding for you; binary mode skips it

```
f = open("notes.txt", "w", encoding="utf-8", newline="\n")
print(type(f))
n = f.write("line one\nline two\nline three\n")
print(n)
f.close()
print(f.closed)
```

```
<class '_io.TextIOWrapper'>
29
True
```

■ TYPE CHECK

`open(..., "w")` returns a `TextIOWrapper`, and `open(..., "wb")` returns a `BufferedWriter`. They are not strings. `write()` returns how many characters (text mode) or bytes (binary mode) it wrote, and it adds nothing: no space, no newline. A closed file object still exists, but using it raises `ValueError`.

➤ RULE

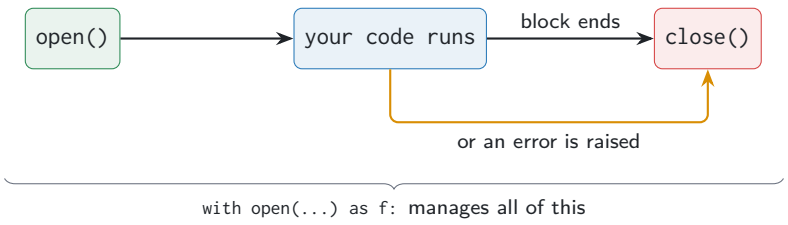
Always pass `encoding="utf-8"` in text mode. If you leave it out, Python uses the system default, which on many Windows machines is a legacy code page, so the same script can write different bytes on different computers.

⚠ TRAP CARD

Mode `"w"` destroys the old content the moment you open the file, before you write anything. A typo in the filename can wipe an important file. Use `"x"` when the file must be new, and `"a"` to add to the end (for example, a log or a JSONL results file).

with: close files automatically

Writing `f.close()` by hand is easy to forget, and it is skipped if an error happens in the middle. A `with` block closes the file for you when the block ends, normally or by an exception.




```
with open("notes.txt", "a", encoding="utf-8", newline="\n") as f:
    print("line four", file=f)
    f.write("line five\n")
print(f.closed)
with open("notes.txt", encoding="utf-8") as f:
    text = f.read()
print(type(text), len(text))
print(text, end="")
```

```
True
<class 'str'> 49
line one
line two
line three
line four
line five
```

Two ways to write text: `f.write(s)` writes exactly `s`. `print(s, file=f)` is `print` aimed at a file, so it adds a newline unless you pass `end=""`. It also joins several arguments with spaces unless you change `sep`.

Read a text file

Call	Returns	Memory use
`f.read()`	one `str`, the whole file	whole file in memory
`f.read(n)`	next `n` characters	small, you choose
`f.readline()`	one line, with its `\n`	one line
`f.readlines()`	`list` of all lines	whole file in memory
`for line in f:`	one line at a time	one line; best for big files

At the end of the file, `read()` and `readline()` return an empty string `""`. That is how you detect the end. A blank line in the middle is `"\n"`, which is not empty, so it is still truthy.

```
with open("notes.txt", encoding="utf-8") as f:
    print(repr(f.readline()))
    print(repr(f.read(4)))
    print(repr(f.read()))
    print(repr(f.read()), repr(f.readline()))
with open("notes.txt", encoding="utf-8") as f:
    lines = f.readlines()
print(len(lines), repr(lines[0]), repr(lines[-1]))
with open("notes.txt", encoding="utf-8") as f:
    for i, line in enumerate(f, 1):
        print(i, line.rstrip("\n"))
```

```
'line one\n'
'line'
' two\nline three\nline four\nline five\n'
'' ''
5 'line one\n' 'line five\n'
1 line one
2 line two
3 line three
4 line four
5 line five
```

■ TYPE CHECK

A file object is its own iterator: `iter(f) is f`. Looping over it yields `str` lines *including* the trailing newline, so you often write `line.rstrip("\n")`. Because it is an iterator, you can use it once. A second loop over the same open file yields nothing, since the pointer is already at the end.

➤ RULE

For anything that could be large (logs, datasets), loop over the file object. Do not call `read()` or `readlines()`: a 5 GB file would need 5 GB of memory.

Example: JSONL, one JSON object per line

JSONL is the common format for eval sets and fine-tuning data: each line is a complete JSON document. Write with `json.dumps(obj) + "\n"`, read with `json.loads(line)`.

```
import json

rows = [{"id": 1, "score": 0.5}, {"id": 2, "score": 0.75}]
with open("out.jsonl", "w", encoding="utf-8", newline="\n") as f:
    for r in rows:
        f.write(json.dumps(r) + "\n")
with open("out.jsonl", "rb") as f:
    print(f.read())
with open("out.jsonl", encoding="utf-8") as f:
    back = [json.loads(line) for line in f]
print(back == rows, type(back[0]))
with open("out.jsonl", encoding="utf-8") as f:
    print(sum(json.loads(line)["score"] for line in f if line.strip()))
```

```
b'{"id": 1, "score": 0.5}\n{"id": 2, "score": 0.75}\n'
True <class 'dict'>
1.25
```

The last loop streams the file and keeps one line in memory at a time, so it scales to files much bigger than RAM.

Errors you will meet

```
try:
    open("notes.txt", "x", encoding="utf-8")
except FileExistsError as e:
    print("FileExistsError", e.errno)
try:
    open("missing.txt", encoding="utf-8")
except FileNotFoundError as e:
    print("FileNotFoundError:", e.strerror)
with open("cafe.txt", "w", encoding="utf-8") as f:
    f.write("caf\u00e9\n")
try:
    with open("cafe.txt", encoding="ascii") as f:
        f.read()
except UnicodeDecodeError as e:
    print("UnicodeDecodeError:", e.reason)
```

```
FileExistsError 17
FileNotFoundError: No such file or directory
UnicodeDecodeError: ordinal not in range(128)
```

The error text for a missing file differs between operating systems, but the exception class does not. Catch the class. The third error is the str/bytes border from the Text Data chapter again: you told Python the bytes were ASCII, but the file is UTF-8.

Newlines

Lines end with `"\n"` on Linux and macOS and `"\r\n"` on Windows. In text mode Python translates for you: reading turns any ending into `"\n"`, and writing turns `"\n"` into the platform's ending unless you pass `newline`.

```
with open("a.txt", "w", newline="") as f:        # no translation
    f.write("a\nb\n")
with open("b.txt", "w", newline="\r\n") as f:    # force Windows endings
    f.write("a\nb\n")
print(open("a.txt", "rb").read(), open("b.txt", "rb").read())
print(open("b.txt", encoding="utf-8").read().splitlines())
```

```
b'a\nb\n' b'a\r\nb\r\n'
['a', 'b']
```

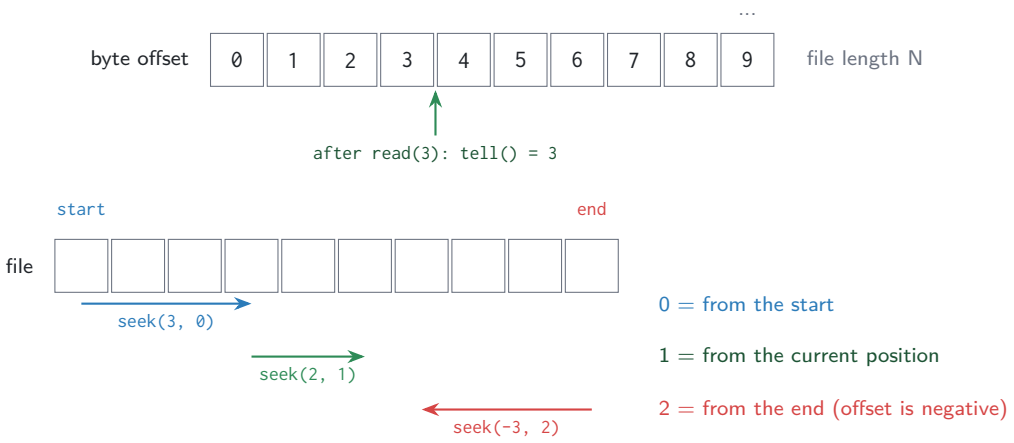
Binary files and seek()

Add `b` to the mode and you get `bytes` back, with no decoding and no newline translation. Use it for images, model weights, pickles and anything that is not text.

```
data = bytes(range(256))
with open("b.bin", "wb") as f:
    print(f.write(data))
with open("b.bin", "rb") as f:
    raw = f.read()
print(type(raw), len(raw), raw[:4])
```

```
256
<class 'bytes'> 256 b'\x00\x01\x02\x03'
```

A file object remembers a *position* (a pointer). Every `read` and `write` starts at the pointer and moves it forward. `tell()` reports it, `seek(offset, origin)` moves it. The position counts *bytes* from the start of the file.



```python
import os

with open("b.bin", "rb") as f:
    print(f.tell(), f.seek(255), f.read(), f.tell())
    print(f.seek(-1, os.SEEK_END), f.read(1))
    print(f.seek(10), f.seek(5, os.SEEK_CUR), f.read(3))
print(os.SEEK_SET, os.SEEK_CUR, os.SEEK_END)
```

```
0 255 b'\xff' 256
255 b'\xff'
10 15 b'\x0f\x10\x11'
0 1 2
```

`seek` returns the new position. Reading the last byte of a huge file this way is instant, because nothing before it is read.

> ⚠ **TRAP CARD**
> In text mode, `seek` and `tell` work with opaque numbers. Only `seek(0)` and values that `tell()` gave you are safe. With UTF-8, one character can be several bytes, so "go to character 100" is not "go to byte 100". For arbitrary jumps, open the file in binary mode.

> ■ **BOSS FIGHT: the file pointer**
> A file object is not the file. It is a cursor plus a buffer. Reading moves the cursor, so the *same* file object returns different data on each call, and returns nothing once it is at the end. Writing in mode `"w"` erases first, then moves forward. Most confusing file bugs come from forgetting where the cursor is: a second `read()` that returns `""`, a loop over a file that runs only once, or a write that overwrote the start because of `"r+"` and no `seek`. When in doubt, print `f.tell()`.

Memory mapping

The standard `mmap` module can map a file into memory so that it looks like a `bytearray`: you slice it and search it, and the operating system loads pages on demand. It helps for very large binary files you need random access to. For everyday work, plain `open` is enough.

20.3 File Operations

Python's names come from Unix. The `os`, `os.path` and `shutil` modules hold the functions. The newer `pathlib` module (later in this chapter) does the same jobs with objects.

Task	Function	Returns / note
exists?	`os.path.exists(p)`	`bool`; works for files and directories
is a file?	`os.path.isfile(p)`	`bool`
is a directory?	`os.path.isdir(p)`	`bool`
is it absolute?	`os.path.isabs(p)`	`bool`; does not check the disk
size	`os.path.getsize(p)`	`int` bytes
copy	`shutil.copy(src, dst)`	returns the destination path
move or rename	`os.rename(a, b)`, `shutil.move`	`move` also works across disks
delete a file	`os.remove(p)`	permanent; no recycle bin
hard / soft link	`os.link`, `os.symlink`	Unix concepts; `islink()` tests
permissions	`os.chmod(p, mode)`	Unix; `0o400` = owner can read only
owner	`os.chown(p, uid, gid)`	Unix only

```
import shutil

with open("oops.txt", "w", encoding="utf-8", newline="\n") as f:
    f.write("hello\n")
print(os.path.exists("oops.txt"), os.path.exists("waffles"))
print(os.path.exists("."))
print(os.path.isfile("oops.txt"), os.path.isdir("oops.txt"), os.path.isdir("."))
print(os.path.isabs("oops.txt"), os.path.isabs(os.path.abspath("oops.txt")))
print(shutil.copy("oops.txt", "copy.txt"))
os.rename("copy.txt", "renamed.txt")
print(sorted(x for x in os.listdir(".") if x.endswith(".txt")))
os.remove("renamed.txt")
print(os.path.exists("renamed.txt"), os.path.getsize("oops.txt"))
```

```
True False
True
True False True
False True
copy.txt
['a.txt', 'b.txt', 'cafe.txt', 'notes.txt', 'oops.txt', 'renamed.txt']
False 6
```

Links. A *hard link* is a second name for the same bytes on disk. A *symbolic link* is a small file that points to another path. `os.symlink` needs special rights on Windows, so it is not run here. `chmod` and `chown` are for Unix-like systems. A typical call is `os.chmod("secrets.txt", 0o600)` so that only you can read your key file.

> ⚠ **TRAP CARD**
>
> `os.remove` and `shutil.rmtree` delete for good. `os.rename` silently overwrites an existing destination on Unix. Check with `os.path.exists` first, or write to a temporary name and then `Path.replace()` into place, which swaps the file in one step.

20.4 Directory Operations

Task	Function	Note
create	`os.mkdir(d)`	one level; `FileExistsError` if present
create nested	`os.makedirs(d, exist_ok=True)`	all missing levels, no error if present
delete empty	`os.rmdir(d)`	fails if not empty
delete tree	`shutil.rmtree(d)`	deletes everything inside: careful
list names	`os.listdir(d)`	`list` of `str`, unsorted
current dir	`os.getcwd()`, `os.chdir(d)`	changes for the whole program
match names	`glob.glob(pattern)`	shell-style patterns, not regex

```
import glob

os.makedirs("data/runs/a1", exist_ok=True)
print(os.listdir("data"), os.path.isdir("data/runs/a1"))
try:
    os.rmdir("data")
except OSError as e:
    print("OSError:", type(e).__name__)
shutil.rmtree("data")
print(os.path.exists("data"))
os.mkdir("runs")
for name in ("a.jsonl", "b.jsonl", "notes.md"):
    with open("runs/" + name, "w", encoding="utf-8") as f:
        f.write("x\n")
os.chdir("runs")
print(sorted(os.listdir(".")), os.path.basename(os.getcwd()))
os.chdir("..")
```

```
['runs'] True
OSError: OSError
False
['a.jsonl', 'b.jsonl', 'notes.md'] runs
```

The glob patterns are *not* regular expressions.

Pattern	Matches	Regex equivalent
*	any number of any characters	.*
?	exactly one character	.
[abc]	one of a, b, c	same
[!abc]	one character that is not a, b, c	[^abc]

```
print(sorted(glob.glob("*.txt")))
print(glob.glob("ru??"), glob.glob("[rz]*"))
print(sorted(os.path.basename(p) for p in glob.glob("runs/*.jsonl")))
```

```
['a.txt', 'b.txt', 'cafe.txt', 'notes.txt', 'oops.txt']
['runs'] ['runs']
['a.jsonl', 'b.jsonl']
```

> ⚠ **TRAP CARD**
> `os.listdir` gives names only, not paths, and in no fixed order. Wrap it in `sorted()` if order matters, and join the folder name back on before opening a file. Also `os.chdir` changes the whole program's idea of "here"; avoid it in library code and use full paths.

20.5 Pathnames

A *pathname* says where a file is: *absolute* from the root, or *relative* to the current directory. Unix, macOS and URLs separate parts with `/`. Windows uses `\`. Python accepts `/` on Windows too, so you can often use slashes everywhere. If you write backslashes yourself, double them or use a raw string (`r"eek\urk"`); `"eek\urk"` contains an escape sequence, not a path.

os.path

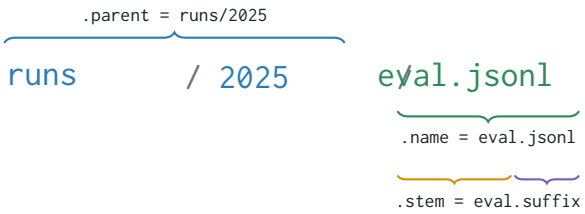
```
p = os.path.join("eek", "urk", "snort.txt")        # uses the OS separator
print(p.replace(os.sep, "/"))
print(os.path.splitext("a/b/model.tar.gz"))
print(os.path.basename("a/b/c.txt"), os.path.dirname("a/b/c.txt"))
print(os.path.isabs(os.path.abspath("x")))
```

```
eek/urk/snort.txt
('a/b/model.tar', '.gz')
c.txt a/b
True
```

`os.path.abspath(p)` turns a relative path into an absolute one. `os.path.realpath(p)` also follows symbolic links to the real file. Never build paths with `"a" + "/" + "b"`: let `join` or `pathlib` insert the right separator.

pathlib

`pathlib.Path` treats a path as an *object*, not a string. The `/` operator joins parts, and the object has handy attributes and methods.




```
from pathlib import Path, PurePosixPath, PureWindowsPath

p = Path("eek") / "urk" / "snort.txt"
print(p.as_posix(), p.name, p.stem, p.suffix)
print(p.parent.as_posix(), p.parts)
print(p.with_suffix(".jsonl").as_posix(), p.with_name("x.md").as_posix())
print(PureWindowsPath(PurePosixPath("eek/urk/snort.txt")))
print(type(p).__name__ in ("WindowsPath", "PosixPath"))
```

```
eek/urk/snort.txt snort.txt snort .txt
eek/urk ('eek', 'urk', 'snort.txt')
eek/urk/snort.jsonl eek/urk/x.md
eek\urk\snort.txt
True
```

`print(p)` shows the path in your platform's style, and `repr(p)` shows `WindowsPath(...)` or `PosixPath(...)` depending on the system. The `Pure...` classes only manipulate path text and never touch the disk, so you can build a Windows path on a Mac.

`Path` objects also do the file work:

```
runs = Path("runs")
body = '{"x": 2}\n{"x": 3}\n'
(runs / "c.jsonl").write_text(body, encoding="utf-8", newline="\n")
print(sorted(x.name for x in runs.iterdir()))
print(sorted(x.name for x in runs.glob("*.jsonl")))
print(sorted(x.as_posix() for x in Path(".").rglob("*.md")))
print((runs / "c.jsonl").read_text(encoding="utf-8").splitlines())
print((runs / "c.jsonl").exists(), (runs / "c.jsonl").is_file(), runs.is_dir())
print((runs / "c.jsonl").stat().st_size, runs.resolve().is_absolute())
tmp = Path("final.json.tmp")
tmp.write_text("{}", encoding="utf-8")
tmp.replace("final.json")
print(sorted(x for x in os.listdir(".") if x.startswith("final")))
```

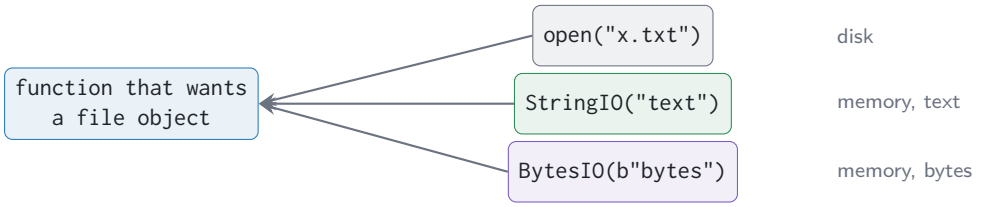
```
['a.jsonl', 'b.jsonl', 'c.jsonl', 'notes.md']
['a.jsonl', 'b.jsonl', 'c.jsonl']
['runs/notes.md']
['{"x": 2}', '{"x": 3}']
True True True
18 True
['final.json']
```

Job	os / os.path style	pathlib style
join	`os.path.join(a, b)`	`Path(a) / b`
file name	`os.path.basename(p)`	`p.name`
extension	`os.path.splitext(p)[1]`	`p.suffix`
exists	`os.path.exists(p)`	`p.exists()`
read all text	`open(p).read()`	`p.read_text(encoding=...)`
list folder	`os.listdir(d)`	`d.iterdir()`
find by pattern	`glob.glob("d/*.py")`	`d.glob("*.py")`

> ➤ **RULE**
> Prefer `pathlib` in new code. Paths become objects with a clear type, the `/` join is portable, and one-line helpers like `read_text` and `write_text` close the file for you. You can pass a `Path` anywhere a filename string is accepted, including `open()`.

20.6 BytesIO and StringIO

Many functions want a file object, but your data is already in memory. `io.StringIO` wraps a `str` and `io.BytesIO` wraps `bytes`. They have the same methods as real files (`read`, `write`, `seek`, iteration) but no disk is touched.



```
import io, csv

s = io.StringIO()
s.write("a,b\n")
print("x", file=s)
print(repr(s.getvalue()), type(s).__name__)
s.seek(0)
print(s.readline(), end="")
print(list(io.StringIO("p\nq\n")))
bio = io.BytesIO(b"\x89PNG\r\n")
print(bio.read(4), bio.tell(), bio.getvalue())
bio.seek(0)
bio.write(b"AB")
print(bio.getvalue())
print(list(csv.DictReader(io.StringIO("name,score\nx,1\ny,2\n"))))
```

```
'a,b\nx\n' StringIO
a,b
['p\n', 'q\n']
b'\x89PNG' 4 b'\x89PNG\r\n'
b'ABNG\r\n'
[{'name': 'x', 'score': '1'}, {'name': 'y', 'score': '2'}]
```

> ■ **TYPE CHECK**
>
> `StringIO` holds `str` and `BytesIO` holds `bytes`. Do not mix them: writing a `str` into a `BytesIO` raises `TypeError`. `getvalue()` returns the whole content regardless of the pointer, while `read()` starts from the pointer. After you write into a `StringIO`, call `seek(0)` before reading it back.

Typical uses: unit tests for code that reads a file, capturing what a function prints into a string, building a CSV or an image in memory before sending it over the network, and converting an image format with the Pillow library (`Image.open(BytesIO(data))`, which needs `pip install pillow`; not run here).

20.7 File Formats: Determination

You may meet a file and not know what it is. The file extension is only a hint: anyone can rename `.png` to `.txt`. Better checks look at the first bytes, the *magic number* (chapter on binary data: a PNG starts with the bytes `\x89PNG`).

Tool	What it does
`file` (Unix command)	identifies files by magic bytes and other rules
`python-magic`	Python wrapper around the `libmagic` library used by `file`
`magika`	Google's learned file-type detector; `pip install magika`
your own check	compare `f.read(8)` with a known header such as `b"\x89PNG\r\n\x1a\n"`

```
sig = b"\x89PNG\r\n\x1a\n"
with open("fake.png", "wb") as f:
    f.write(sig + b"rest of image")
with open("fake.png", "rb") as f:
    print(f.read(8) == sig)
```

```
True
```

20.8 Review and what comes next

You can now open files in the right mode, read them line by line without loading them whole, use `with` so that they close, move the file pointer with `seek`, and manage files and folders with `os`, `shutil` and `pathlib`. The next chapter is about concurrency: doing several things at once.

➤ **PRACTICE**

1. Write the string `"This is a test of the emergency text system"` to `test.txt` with a `with` block and UTF-8. Read it back into `test2`. Are they equal?
2. List all `.txt` files in the current folder, sorted, using `pathlib` and then using `glob`.
3. Why does a second `for line in f:` over the same open file print nothing? Show two ways to fix it.
4. You have a 4 GB JSONL file of eval rows. Write the loop that counts rows with `"score" > 0.5` without loading the file into memory.
5. Write 100 bytes, then use `seek` to read the last 10 bytes in two different ways.
6. What goes wrong if you open an existing file with mode `"w"` just to look at it? Which mode would you use to append a line to a log?

★ **ACHIEVEMENT UNLOCKED: File Handler**

- Choose the correct open mode and always close files with `with`.
- Stream large text files line by line and read or write JSONL.
- Move around a binary file with `seek` and `tell`.
- Build, split and search paths with `pathlib` and `glob`.
- Use `StringIO` and `BytesIO` to treat memory like a file.

CHAPTER 21

Data in Time: Concurrency

What this chapter gives you: a map of the ways Python can do more than one thing at a time (processes, threads, a thread pool, `asyncio`, queues), and a rule for choosing among them. The core question is always: is my program *waiting* (on an API, a disk, a network) or *computing*? Calling many LLM APIs at once is the typical case, and this is where it pays off. All examples use `sleep` to fake slow network calls, so you can run them offline.

21.1 Programs and Processes

When you run a program, the operating system creates a *process*: its own memory, its own open files, its own ID. Processes are isolated. One cannot read another's variables. The OS gives each a short slice of CPU time and switches between them so fast that they seem to run together.

```
import os, sys

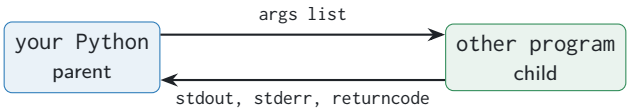
print(type(os.getpid()), type(os.cpu_count()), type(os.getcwd()))
```

```
<class 'int'> <class 'int'> <class 'str'>
```

`os.getpid()` is the process ID (an `int`), `os.cpu_count()` the number of logical CPUs, `os.getcwd()` the current directory (a `str`). Other `os` helpers such as `os.uname()`, `os.getloadavg()` and `os.getuid()` exist only on Unix-like systems. The third-party `psutil` package (`pip install psutil`) gives CPU, memory, disk and per-process information on every platform; it is not needed here.

Create a process with subprocess

`subprocess` starts *another program* and talks to it. The modern entry point is `subprocess.run`. It waits for the child to finish and returns a `CompletedProcess`.



To stay portable, the examples start the Python interpreter itself (`sys.executable`).

```
import subprocess

r = subprocess.run([sys.executable, "-c", "print('hi from child')"],
                   capture_output=True, text=True)
print(type(r).__name__, r.returncode, repr(r.stdout), repr(r.stderr))
r3 = subprocess.run([sys.executable, "-c", "import sys; sys.exit(3)"])
print(r3.returncode)
try:
    subprocess.run([sys.executable, "-c", "raise SystemExit(2)"], check=True)
except subprocess.CalledProcessError as e:
    print("failed with", e.returncode)
out = subprocess.check_output([sys.executable, "-c", "print(1 + 1)"])
print(out.strip(), type(out))
```

```
CompletedProcess 0 'hi from child\n' ''
3
failed with 2
b'2' <class 'bytes'>
```

Call	What it gives you
`run(args, capture_output=True, text=True)`	`CompletedProcess` with `.stdout` and `.stderr` as `str`, and `.returncode`
`run(..., check=True)`	raises `CalledProcessError` if the exit code is not 0
`check_output(args)`	only stdout, as `bytes` unless `text=True`; raises on failure
`getoutput(cmd)`	shell command string in, output `str` out (stdout and stderr mixed)
`getstatusoutput(cmd)`	tuple `(status, output)`
`call(args)`	runs it, output goes to your terminal, returns the exit code

■ **TYPE CHECK**

Without `text=True`, `.stdout` is `bytes`. With it, `str`. The exit status is an `int`: `0` means success, anything else means failure. Pass the command as a *list* of strings: `["git", "status"]`.

⚠ **TRAP CARD**

`shell=True` (or `getoutput`) hands a single string to a shell. If any part of that string comes from user input or model output, a crafted value such as `x; rm -rf ~` can run extra commands. Prefer the list form without a shell. Never build shell strings from untrusted text.

Create a process with multiprocessing

`multiprocessing` runs a *Python function* in a new process. Each process has its own interpreter and its own memory, so it can use a separate CPU core. A `Pool` hands a list of inputs to a fixed set of worker processes.

```python
# file: mp_demo.py
import multiprocessing as mp


def count_primes(limit):                    # CPU-bound work
    n = 0
    for x in range(2, limit):
        if all(x % d for d in range(2, int(x ** 0.5) + 1)):
            n += 1
    return n


def square_to(q, n):
    q.put((n, n * n))


if __name__ == "__main__":                  # required on Windows and macOS
    with mp.Pool(2) as pool:
        print(pool.map(count_primes, [100, 1000, 10000]))
    q = mp.Queue()
    ps = [mp.Process(target=square_to, args=(q, n)) for n in range(3)]
    for p in ps:
        p.start()
    got = sorted(q.get() for _ in ps)
    for p in ps:
        p.join()
    print(got, [p.exitcode for p in ps])
```

```
[25, 168, 1229]
[(0, 0), (1, 1), (2, 4)] [0, 0, 0]
```

Save this as a file and run it; processes do not work well in a notebook or the REPL. The numbers are the counts of primes below 100, 1000 and 10000.

> ➤ **RULE**
> Always put the start-up code under `if __name__ == "__main__":`. On Windows and macOS a new process *re-imports* your file. Without the guard it would start new processes again and again.

> ⚠ **TRAP CARD**
> A child process cannot change your variables. Whatever it computes must come back through a `Queue`, a `Pipe`, a return value (`Pool.map`) or a file. Arguments and results are pickled (serialized) on the way, so they must be picklable. Lambdas and open files are not.

Kill a process with terminate()

A process you started can be stopped with `p.terminate()`, followed by `p.join()` to collect it. `p.is_alive()` tells you whether it still runs, and `p.exitcode` is `None` until it ends. Threads have no equivalent, which is one reason to use processes for work that you may need to cancel hard.

```python
# file: mp_kill.py
import multiprocessing as mp, time


def forever():
    while True:
        time.sleep(0.1)


if __name__ == "__main__":
    p = mp.Process(target=forever)
    p.start()
    print(p.is_alive())
    p.terminate()
    p.join()
    print(p.is_alive(), p.exitcode is not None)
```

```
True
False True
```

21.2 Command Automation

For repeatable command-line tasks, third-party tools wrap `subprocess` with a nicer interface. `invoke` turns decorated Python functions into command-line tasks.

```python
# needs: pip install invoke   (not run in these notes)
# tasks.py
from invoke import task

@task
def clean(ctx):               # ctx is required, invoke fills it in
    ctx.run("echo cleaning")
```

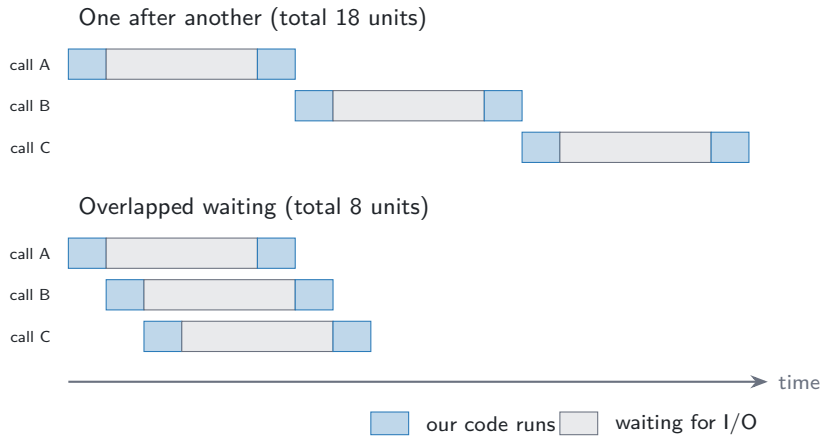
You then type `invoke clean` in a shell, and `invoke -l` lists the tasks. Related tools: `click` (build command-line programs), `doit`, `sh` (call commands like functions) and `fabric` (remote commands over SSH). Reach for them only when plain `subprocess.run` and a small script stop being enough.

21.3 Concurrency

A program is usually slow for one of two reasons.

	I/O bound	CPU bound
The program is ...	waiting for network, disk, a database, an API	busy calculating
Typical AI examples	calling LLM APIs, downloading files, reading many files	tokenizing a huge corpus, parsing, local math
The CPU is ...	mostly idle	fully used
What helps	overlap the waiting	use more CPU cores

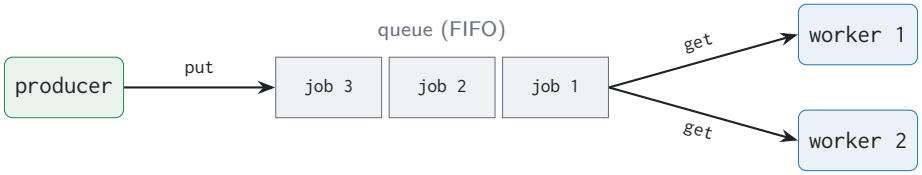
Synchronous code does one thing, finishes, then starts the next. *Asynchronous* code lets other work proceed while one task waits. The picture shows why that matters. Every call needs a little of our code (blue) and a long wait (gray).



The CPU is idle during every gray block. Overlapping the waits cuts total time from 18 units to 8, without any extra CPU. If the work were *computing* instead, the blue blocks would be long and overlap would not help on one core. That case needs several cores, hence several processes.

Queues

A queue holds items in order: you add at one end and take from the other. *FIFO* means first in, first out. Queues let one part of a program hand work to another without the two having to move at the same speed. The producer puts jobs in. The workers take jobs out when free.



There is one queue type per world: `multiprocessing.Queue` (between processes), `queue.Queue` (between threads), `asyncio.Queue` (between coroutines), and external servers such as Redis (between machines). Their methods look the same: `put`, `get`, `task_done`, `join`.

A common trick is a *sentinel*: a special item such as `None` that means "no more work". Each worker stops when it sees it.

```python
import queue, threading

q = queue.Queue()
results = []

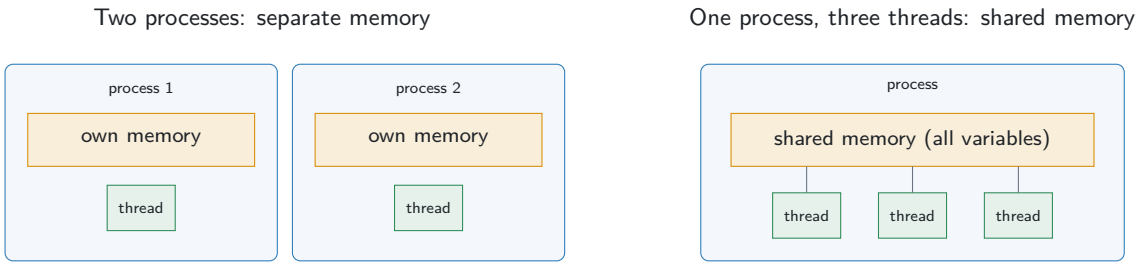
def consumer():
    while True:
        item = q.get()                # blocks until an item arrives
        if item is None:              # sentinel: stop
            q.task_done()
            break
        results.append(item.upper())
        q.task_done()

c = threading.Thread(target=consumer)
c.start()
for w in ["salad", "bread", "soup"]:
    q.put(w)
q.put(None)
q.join()                              # wait until every item is processed
c.join()
print(results)
```

```
['SALAD', 'BREAD', 'SOUP']
```

Threads

A *thread* is a path of execution inside a process. All threads of a process share the same memory. That is what makes them light and handy, and also what makes them dangerous.




```python
import threading, time

def fake_call(i):
    time.sleep(0.2)               # pretend to wait for a network reply
    return i * i

t = threading.Thread(target=fake_call, args=(1,))
t.start()
t.join()                          # wait for the thread to finish
print(threading.current_thread().name, threading.active_count())
```

```
MainThread 1
```

`Thread(target=f, args=(...))` makes a thread, `start()` launches it, `join()` waits for it. A thread's return value is lost, so give it a queue or a list to write into (or use a thread pool, below). There is no `terminate()` for threads.

Shared data and locks

When two threads change the same variable, the order of tiny steps can interleave and results go wrong (a *race condition*). A `Lock` lets only one thread into a block at a time.

```
counter = 0
lock = threading.Lock()

def work():
    global counter
    for _ in range(50000):
        with lock:                # only one thread at a time in here
            counter += 1

ts = [threading.Thread(target=work) for _ in range(4)]
for t in ts:
    t.start()
for t in ts:
    t.join()
print(counter)
```

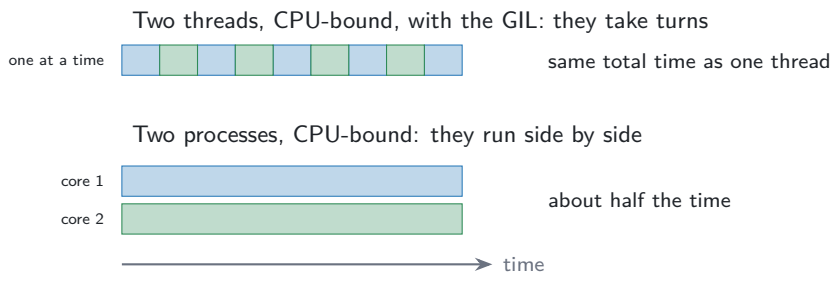
```
200000
```

> ⚠ **TRAP CARD**
>
> Race conditions are hard bugs: the code works nine times and fails on the tenth, and adding a `print` can hide it. `counter += 1` looks like one step but is read, add, write. Keep shared mutable state to a minimum, protect it with a `with lock:` block, or avoid sharing by passing data through a `Queue`. Forgetting to release a lock (or two locks waiting on each other, a *deadlock*) freezes the program; `with` releases it for you.

The GIL

CPython has a *Global Interpreter Lock*: only one thread at a time may run Python bytecode. The consequences are:

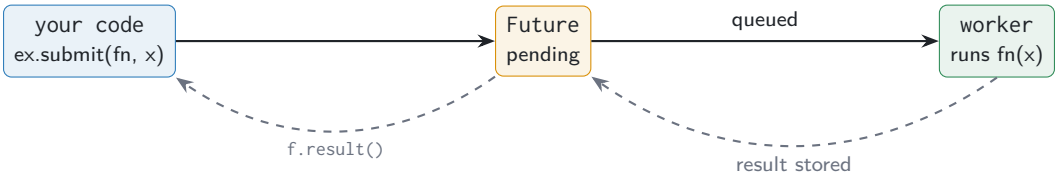
- Threads waiting on I/O release the lock, so threads *do* help I/O-bound programs.
- Threads doing pure-Python calculation take turns, so they give *no speed-up*, and can even be slower than one thread.
- Processes each have their own interpreter and lock, so they use several cores.
- C extensions such as NumPy release the lock inside long computations.
- Python 3.13 has an experimental build without the GIL. Do not count on it yet.



> ➤ **RULE**
>
> Threads for I/O-bound work. Processes for CPU-bound work. For most AI-engineering scripts (calling APIs) threads or `asyncio` are right. Use processes when you crunch data in pure Python.

concurrent.futures

Managing threads and processes by hand is fiddly. `concurrent.futures` gives one simple interface over both: a *pool* of workers, and a *future* for each job that you submitted. A future is a receipt: it will hold the result later (or the exception that was raised).




```
from concurrent.futures import ThreadPoolExecutor, as_completed

def timed(workers):
    t0 = time.perf_counter()
    with ThreadPoolExecutor(max_workers=workers) as ex:
        out = list(ex.map(fake_call, range(6)))      # keeps input order
    return out, time.perf_counter() - t0

t0 = time.perf_counter()
serial = [fake_call(i) for i in range(6)]
t_serial = time.perf_counter() - t0
out3, t3 = timed(3)
out6, t6 = timed(6)
print(serial == out3 == out6)
print(round(t_serial, 1), round(t3, 1), round(t6, 1))
```

```
True
1.2 0.4 0.2
```

Six calls of 0.2 s each take about 1.2 s one by one, 0.4 s with three workers (two rounds) and 0.2 s with six (one round). Your numbers will differ slightly. `ex.map` returns the results *in input order*. `submit` returns a `Future` at once, and `as_completed` yields futures in the order they *finish*.

```
with ThreadPoolExecutor(max_workers=3) as ex:
    futs = [ex.submit(fake_call, i) for i in range(4)]
    print(type(futs[0]).__name__, futs[0].done())
    print(sorted(f.result() for f in as_completed(futs)))

def boom(x):
    raise ValueError("bad input %d" % x)

with ThreadPoolExecutor(2) as ex:
    f = ex.submit(boom, 7)
    try:
        f.result()                # the worker's exception is raised here
    except ValueError as e:
        print("caught:", e)
```

```
Future False
[0, 1, 4, 9]
caught: bad input 7
```

> **TYPE CHECK**
> `ex.submit(...)` returns a `Future`, not the result. `f.result()` returns the function's return value and *re-raises* any exception from the worker, so errors are not lost; they appear where you ask for the result. `ProcessPoolExecutor` has the same API, but the function and its arguments must be picklable, and the start-up needs the `__main__` guard.

Green threads and gevent

Another idea is *event-based* programming: one loop watches many waiting tasks and runs whichever is ready. The `gevent` library does this with *greenlets* ("green threads") and rewrites standard library calls such as `socket` ("monkey patching") so that ordinary-looking blocking code yields to others instead of freezing. It is third-party (`pip install gevent`), not run here, and it only works with pure-Python code, not C libraries that block. It lost ground to `asyncio`, but you will see it in older servers.

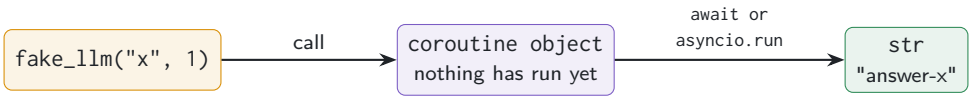
Twisted

`Twisted` is an older event-driven networking framework built on *callbacks*: you register a function to run when "data arrives" or "connection closed". It is powerful and covers many protocols, but callback code can get tangled as a program grows (*callback hell*). New code usually chooses `asyncio`. Install with `pip install twisted`; not needed in this series.

asyncio

`asyncio` (standard library) runs many tasks in *one thread*. Tasks cooperate: a task runs until it reaches an `await`, which says "I am waiting, someone else can go now". A single *event loop* decides who runs next.

- `async def f(): ...` defines a *coroutine function*.
- Calling `f()` does *not* run it. It returns a *coroutine object*.
- `await x` runs a coroutine to completion (inside another coroutine) and gives its result.
- `asyncio.run(f())` starts the event loop from normal code and runs `f()`.
- `asyncio.create_task(f())` starts `f()` in the background, concurrently.
- `asyncio.gather(a, b, c)` runs several and returns their results as a list, in order.




```python
import asyncio

async def fake_llm(name, delay):
    print("start", name)
    await asyncio.sleep(delay)        # a fake network wait that lets others run
    print("done", name)
    return f"answer-{name}"

c = fake_llm("x", 0)
print(type(c).__name__)               # nothing has run yet
print(asyncio.run(c))
```

```
coroutine
start x
done x
answer-x
```

> ■ **TYPE CHECK**
>
> `fake_llm("x", 0)` is a `coroutine` object, *not* a `str`. The `str` appears only after `await` (or `asyncio.run`). Forgetting the `await` is the number one async bug: Python warns `coroutine ... was never awaited` and your code silently did nothing.

Now three fake API calls with different delays, started together.

```
async def main():
    t0 = time.perf_counter()
    out = await asyncio.gather(fake_llm("a", 0.3), fake_llm("b", 0.1),
                               fake_llm("c", 0.2))
    return out, time.perf_counter() - t0

out, elapsed = asyncio.run(main())
print(out, round(elapsed, 1))
```

```
start a
start b
start c
done b
done c
done a
['answer-a', 'answer-b', 'answer-c'] 0.3
```

All three started at once. They finished in the order of their delays (b, c, a). The total time is about the *longest* delay (0.3 s), not the sum (0.6 s). Yet `gather` returned the results in the order of the arguments (a, b, c), not the order of finishing.

The event loop is one lane. It runs whoever is ready:



each letter: that task runs until its next `await`; `a b c` then resume after their waits

Tasks, timeouts and limits

```
async def with_timeout():
    try:
        await asyncio.wait_for(fake_llm("slow", 1.0), timeout=0.1)
    except asyncio.TimeoutError:
        print("timed out")

async def limited():
    sem = asyncio.Semaphore(2)          # at most 2 calls in flight
    active = peak = 0
    async def one(i):
        nonlocal active, peak
        async with sem:
            active += 1
            peak = max(peak, active)
            await asyncio.sleep(0.05)
            active -= 1
    await asyncio.gather(*(one(i) for i in range(6)))
    return peak

async def collect_errors():
    async def ok():
        return 1
    async def bad():
        raise RuntimeError("api down")
    return await asyncio.gather(ok(), bad(), return_exceptions=True)

asyncio.run(with_timeout())
print(asyncio.run(limited()))
print(asyncio.run(collect_errors()))
```

```
start slow
timed out
2
[1, RuntimeError('api down')]
```

Tool	Use it to ...
`asyncio.gather(*coros)`	run many coroutines at once; results in argument order
`asyncio.create_task(coro)`	start one in the background, `await` it later
`asyncio.TaskGroup()`	structured version (3.11+): all tasks are awaited, and one failure cancels the rest
`asyncio.wait_for(coro, t)`	give up after `t` seconds (raises `TimeoutError`)
`asyncio.Semaphore(n)`	cap how many run at once, for example to respect an API rate limit
`asyncio.Queue()`	hand work between coroutines
`gather(..., return_exceptions=True)`	get exceptions as values instead of failing the whole batch

> ⚠ **TRAP CARD**
>
> A normal blocking call inside a coroutine freezes the *whole* event loop, because there is only one thread. `time.sleep(0.2)` in three coroutines takes 0.6 s in total, not 0.2 s. Inside `async def`, use `await asyncio.sleep`, an async library (for example async SDK clients such as `AsyncAnthropic`, `httpx.AsyncClient`, `aiohttp`), or push the blocking call to a thread with `await asyncio.to_thread(fn, x)`.

```
async def blocking():
    async def bad(n):
        time.sleep(0.2)               # blocks the loop: no overlap
        return n
    t0 = time.perf_counter()
    await asyncio.gather(bad(1), bad(2), bad(3))
    return time.perf_counter() - t0

async def threaded():
    t0 = time.perf_counter()
    await asyncio.gather(*(asyncio.to_thread(fake_call, n) for n in range(3)))
    return time.perf_counter() - t0

print(round(asyncio.run(blocking()), 1), round(asyncio.run(threaded()), 1))
```

```
0.6 0.2
```

■ BOSS FIGHT: await means "let someone else run"

Concurrency in `asyncio` is *cooperative*. A coroutine runs without interruption until it hits an `await` on something that actually waits. Three ideas must click together. (1) Calling an `async def` gives a coroutine object, not a result. (2) `await` is both "give me the result" and "you may switch away now". (3) Only `awaited` waits overlap; anything that blocks the thread, such as `time.sleep` or a slow loop, stops everyone. If your async program is no faster than the plain one, look for a blocking call or a forgotten `gather`: `await a(); await b()` runs them *one after the other*.

```
async def sequential():
    t0 = time.perf_counter()
    await fake_llm("a", 0.3)
    await fake_llm("b", 0.1)          # starts only after a is finished
    return time.perf_counter() - t0

print(round(asyncio.run(sequential()), 1))
```

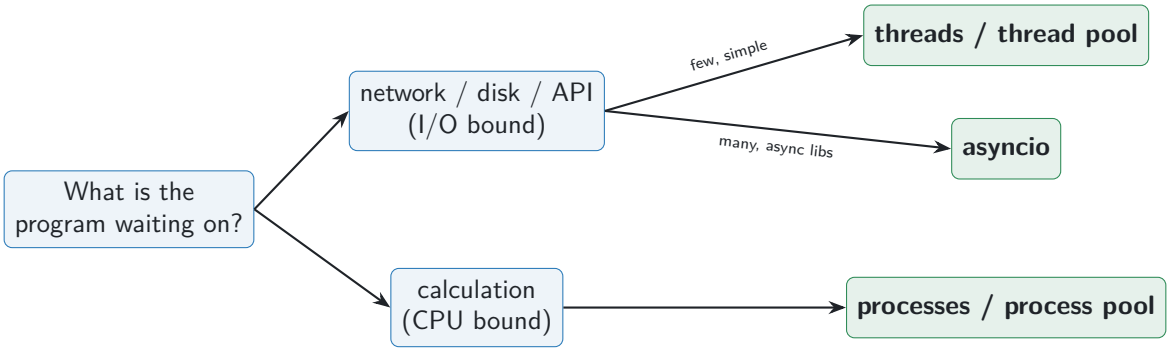
```
start a
done a
start b
done b
0.4
```

Asyncio alternatives

`async/await` is part of the language, and `asyncio` is just one event-loop library. `Trio` and `Curio` are alternatives with different designs. `Trio` uses a "nursery" to group tasks, an idea that influenced `TaskGroup`. `AnyIO` offers one API on top of `asyncio` and `Trio`. Third-party, not needed here.

Async versus ...

Approach	Strength	Cost or catch
Processes	use all CPU cores; real isolation	heavy, slow start, data must be pickled
Threads	easy; fine for I/O waits; shared memory	GIL limits CPU work; races and locks
Coroutines (`asyncio`)	tiny per task: thousands or more at once; no data races between awaits	needs async libraries; one blocking call stalls all
Green threads (`gevent`)	looks like normal code	needs monkey patching; not for C libraries
Callbacks (`Twisted`)	mature; fine for network servers	"inside-out" code that is hard to follow
Queues (Redis, Celery)	spread work over machines; durable	more moving parts to run



Async frameworks and servers

Async support is spreading through the Python web world. *ASGI* is the async successor of WSGI, the standard between web servers and apps. This is where it shows up when you serve a model behind an API.

Kind	Examples
ASGI servers	Uvicorn, Hypercorn
async web frameworks	FastAPI (uses type hints), Starlette, Sanic, Tornado, aiohttp, Quart
async HTTP clients	`httpx` (async mode), `aiohttp`
async database drivers	asyncpg (Postgres), aiomysql, redis-py's async client

Redis

When one machine is not enough, put the queue *outside* the programs, on a server that all of them can reach over the network. A Redis list works as a queue: producers `rpush` messages, workers `blpop` (a pop that blocks until a message arrives, with an optional timeout). The workers can run on other computers. It needs a Redis server and `pip install redis`, so it is not run here.

```python
# needs: a Redis server and  pip install redis   (not run in these notes)
import redis

conn = redis.Redis()

# producer
for job in ["salad", "bread", "soup"]:
    conn.rpush("jobs", job.encode("utf-8"))   # Redis stores bytes
conn.rpush("jobs", b"quit")                     # sentinel

# worker
while True:
    item = conn.blpop("jobs", timeout=20)   # None on timeout, else tuple
    if item is None or item[1] == b"quit":
        break
    print("processed", item[1].decode("utf-8"))
```

■ **TYPE CHECK**

Redis returns `bytes`, so you `.decode("utf-8")` what comes out, exactly as in the Text Data chapter. `blpop` returns either `None` (timed out) or a tuple `(key, value)`, both `bytes`.

Two ways to end workers: a sentinel (`quit`), or a timeout (no message for N seconds means stop). Several workers can `blpop` from the same list; each message goes to exactly one of them.

Beyond queues

More moving parts mean more ways to fail. Typical strategies:

Strategy	Meaning
fire and forget	send the job and do not check. Simple, but lost jobs go unnoticed
request and reply	the receiver confirms each job (an acknowledgment)
back pressure (throttling)	a slow stage tells fast stages to slow down, so queues do not grow forever
retry with limits	try again after a failure, a few times, with growing delays

Ready-made task systems add tracking of pending, running, finished and failed jobs: `Celery` (distributed tasks, many back ends) and `RQ` (simple, Redis-based). If you only need to cap how many API calls run at once, an `asyncio.Semaphore` is often enough.

21.4 Review and what comes next

Processes isolate and use many cores. Threads share memory and suit waiting. `asyncio` runs many waiting tasks in one thread and needs `await`. Pools and queues organize the work. The next chapter moves from time to space: networks.

➤ **PRACTICE**

1. You must call an LLM API for 200 prompts. Is the program I/O bound or CPU bound? Name two tools that would speed it up and one that would not.
2. Rewrite `[fake_call(i) for i in range(6)]` with a `ThreadPoolExecutor` with four workers. In which order do results arrive with `map`? With `as_completed`?
3. Why does `await a(); await b()` not run the two coroutines at the same time? Show the one-line change that does.
4. What is the type of `fake_llm("q", 1)`, and what do you have to do to get a `str`?
5. Three threads each run `counter += 1` 100000 times on a shared global without a lock. Why can the final value be wrong, and what fixes it?

6. Start a child process that sleeps for 10 seconds, then stop it after one second. Which calls do you need, and what does `exitcode` tell you?
7. A program using `asyncio.gather` on 50 calls is rate-limited by the API. How do you allow at most 5 calls at a time?

★ **ACHIEVEMENT UNLOCKED: Time Bender**

- Tell I/O-bound from CPU-bound work and pick processes, threads or `asyncio`.
- Run commands and Python functions in other processes safely.
- Fan out work with `concurrent.futures` and collect results and errors.
- Write `async`/`await` code with `gather`, timeouts and a semaphore.
- Pass work between workers with queues and sentinels.

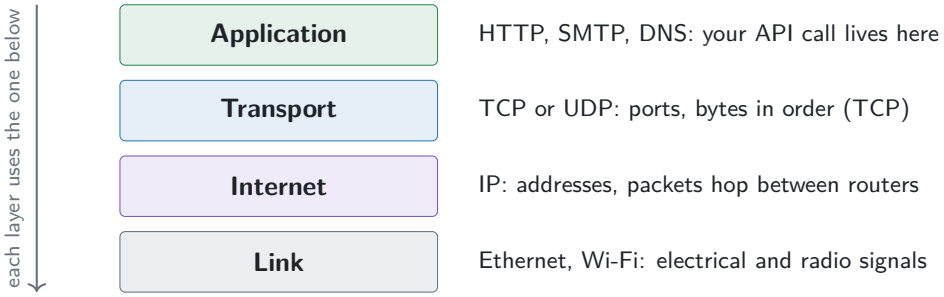
CHAPTER 22

Data in Space: Networks

Chapter 21 was about doing many things at once. This chapter is about doing things in many places: programs on different machines that exchange bytes. You get the layers under every web and LLM API call (TCP/IP, sockets), the common message patterns, and the tools that sit on top (serialization, RPC, cloud basics).

22.1 TCP/IP

Networks work because everyone follows layered rules called *protocols*. A layer only needs to know the layer directly below it.



Two protocols matter at the transport layer.

	UDP	TCP
Unit of data	datagram (one small packet)	stream of bytes
Setup	none, just send	handshake first (connect)
Delivery	may be lost, duplicated, reordered	retried, in order, no duplicates
Speed	fast, light	slower
Typical use	voice, games, DNS lookups	web, databases, most APIs

HTTP/3 runs on QUIC, which itself sits on UDP, so the choice is not always black and white.

Your own machine is always `127.0.0.1`, also called `localhost` or the *loopback* interface. Two programs on one machine can talk over it without any real network. Every example below uses it.

22.2 Sockets

A *socket* is the lowest-level network tool: an endpoint you send bytes into and read bytes from. It comes from C and Unix. Its address is a tuple `(host, port)`: the host is like a building, the port like a door number.

UDP: send one datagram

This example runs server and client in one script. The server runs in a thread (Chapter 21). Port `0` asks the OS to pick any free port, so the example never clashes with another program.

```
import socket
import threading

ready = threading.Event()
addr = {}

def server():
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)   # DGRAM = UDP
    s.bind(("127.0.0.1", 0))          # port 0: OS picks a free port
    addr["value"] = s.getsockname()
    ready.set()
    data, client = s.recvfrom(4096)   # waits for one datagram
    print("server got:", data)
    s.sendto(data.upper(), client)    # reply to whoever sent it
    s.close()

t = threading.Thread(target=server)
t.start()
ready.wait()

c = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
c.sendto("ping from eval worker".encode("utf-8"), addr["value"])
reply, who = c.recvfrom(4096)
print("client got:", reply)
print(type(reply), who == addr["value"])
c.close()
t.join()
```

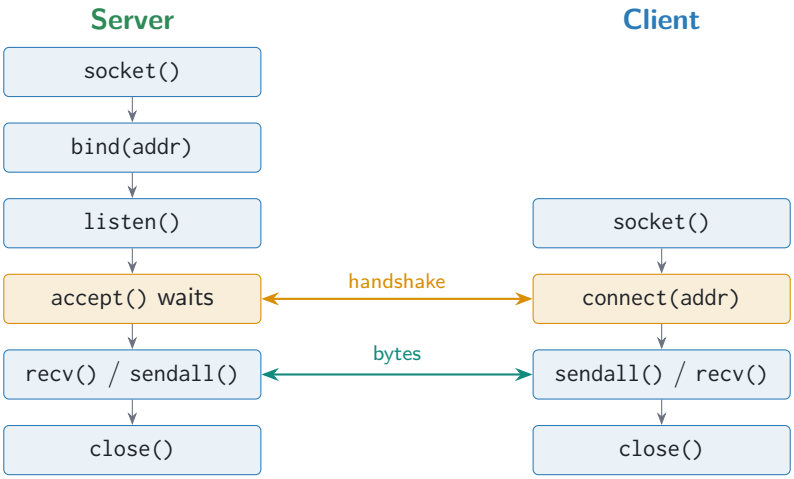
```
server got: b'ping from eval worker'
client got: b'PING FROM EVAL WORKER'
<class 'bytes'> True
```

> ■ **TYPE CHECK**
> Sockets move `bytes`, never `str`. `sendto("text", addr)` raises `TypeError`. Encode on the way out (`.encode("utf-8")`) and decode on the way in (`.decode("utf-8")`). `AF_INET` means an IP socket; `SOCK_DGRAM` means UDP.

The client never calls `bind`. The OS gives it a random port, and the server learns that port from `recvfrom`, which returns `(bytes, (host, port))`.

TCP: a connection

TCP adds a handshake, so the roles are more visible: the server *listens* and *accepts*; the client *connects*.

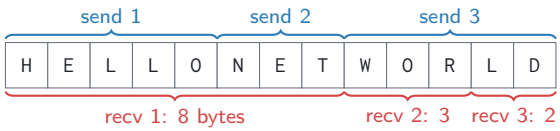


TCP code looks easy until you meet one problem: where does one message end?

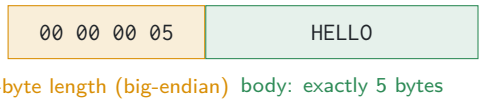
TCP is a stream, not messages

BOSS FIGHT: no message boundaries

TCP delivers a *stream of bytes*. If you call `sendall` three times, the other side may get all three pieces in one `recv`, or one piece split into two. The number of `send` calls and the number of `recv` calls are unrelated.



To exchange whole messages you must add structure yourself: a fixed size, a delimiter character, or a length prefix. The length prefix is the most robust. Write a 4-byte number first, then the body.



```
import socket
import struct
import threading

ready = threading.Event()
box = {}

def recv_exact(sock, n):
    """Read exactly n bytes, however many recv() calls it takes."""
    chunks = []
    while n > 0:
        chunk = sock.recv(n)
        if not chunk:                       # b"" means the peer closed
            raise ConnectionError("peer closed early")
        chunks.append(chunk)
        n -= len(chunk)
    return b"".join(chunks)

def send_msg(sock, text):
    body = text.encode("utf-8")
    sock.sendall(struct.pack(">I", len(body)) + body)  # header + body

def recv_msg(sock):
    (size,) = struct.unpack(">I", recv_exact(sock, 4))
    return recv_exact(sock, size).decode("utf-8")

def server():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as srv:
        srv.bind(("127.0.0.1", 0))
        srv.listen()
        box["addr"] = srv.getsockname()
        ready.set()
        conn, peer = srv.accept()          # blocks until a client connects
        with conn:
            while True:
                try:
                    msg = recv_msg(conn)
                except ConnectionError:
                    break
                send_msg(conn, "echo: " + msg)

t = threading.Thread(target=server)
t.start()
ready.wait()

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as cli:
    cli.connect(box["addr"])
    for prompt in ["hello", "second request", "x" * 5000]:
        send_msg(cli, prompt)
        reply = recv_msg(cli)
        print(len(prompt), "->", reply[:22], len(reply))
t.join()
print(struct.pack(">I", 5))
```

```
5 -> echo: hello 11
14 -> echo: second request 20
5000 -> echo: xxxxxxxxxxxxxxx 5006
b'\x00\x00\x00\x05'
```

The 5000-byte message needs several `recv` calls underneath. Because `recv_exact` loops, the caller never notices.

> ⚠ **TRAP CARD: `recv(n)` may return fewer than n bytes**
>
> `recv(1000)` means *at most* 1000. Assuming you got a whole message is the classic socket bug: it works on localhost with tiny messages and fails in production. An empty result `b""` is not "no data yet"; it means the other side closed.

> ■ **TYPE CHECK**
>
> Length is counted in *bytes*, not characters. `len("caf\u00e9")` is 4 but `len("caf\u00e9".encode("utf-8"))` is 5. Compute the header from the encoded bytes, as `send_msg` does. `struct.pack(">I", 5)` gives the `bytes` value `b'\x00\x00\x00\x05'`.

> ⚠ **TRAP CARD: "Address already in use"**
>
> A fixed port such as 6789 stays busy while an old server (or a recently closed connection) holds it. Use port 0 for tests, or kill the old process.

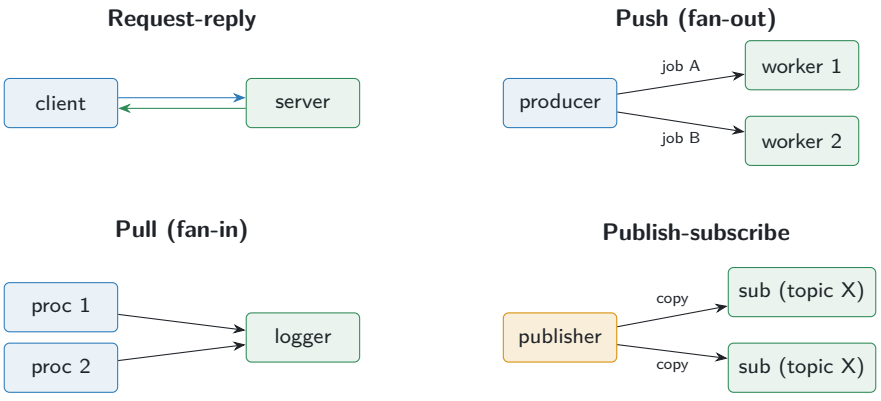
> ➤ **RULE: bind to localhost while learning**
>
> Use `"127.0.0.1"` in `bind`. Binding to `"0.0.0.0"` or `"*"` exposes the port to every network the machine is on. Do that only on purpose.

Scapy and Netcat

Tool	What it is for
Scapy (`pip install scapy`)	build, send and inspect raw packets from Python. Like a programmable Wireshark. Do not confuse it with Scrapy (web scraping).
Netcat (`nc`)	command-line tool to open a connection to a host and port and type bytes by hand. Handy to check that a port answers.

22.3 Networking Patterns

Real systems are built from a few repeated shapes.



Pattern	Who gets a message	Example
Request-reply	the one server asked; client waits	browser to web server, call to an LLM API
Push (fan-out)	exactly one worker from a pool	job queue, load balancer
Pull (fan-in)	one collector reads many sources	log collector
Pub-sub	*every* subscriber of the topic; none means dropped	event notifications

Push and pub-sub look similar but differ in one question: does the message go to one receiver or to all of them? Both shapes in plain Python, with no network:

```
import queue
import threading

# push / pull: one queue, several workers; each job goes to ONE worker
jobs = queue.Queue()
done = queue.Queue()

def worker(name):
    while True:
        job = jobs.get()
        if job is None:
            break
        done.put((name, job, job * job))

pool = [threading.Thread(target=worker, args=(f"w{i}",)) for i in range(2)]
for t in pool:
    t.start()
for n in range(4):
    jobs.put(n)
for _ in pool:
    jobs.put(None)               # one stop signal per worker
for t in pool:
    t.join()
results = sorted(done.get()[1:] for _ in range(4))
print(results)

# pub-sub: every subscriber of a topic gets its OWN copy
class Broker:
    def __init__(self):
        self.subs = {}           # topic -> list of callbacks

    def subscribe(self, topic, callback):
        self.subs.setdefault(topic, []).append(callback)

    def send(self, topic, message):
        for callback in self.subs.get(topic, []):   # no subscriber: dropped
            callback(message)

bus = Broker()
bus.subscribe("eval.done", lambda m: print("logger  :", m))
bus.subscribe("eval.done", lambda m: print("alerter :", m))
bus.subscribe("eval.failed", lambda m: print("pager   :", m))
bus.send("eval.done", {"run": 7, "score": 0.91})
bus.send("eval.cost", {"usd": 1.2})        # nobody listens: vanishes
```

```
[(0, 0), (1, 1), (2, 4), (3, 9)]
logger  : {'run': 7, 'score': 0.91}
alerter : {'run': 7, 'score': 0.91}
```

22.4 The Request-Reply Pattern

Request-reply is the everyday pattern: you ask DNS, a web server or an LLM API, and you get an answer or an error.

ZeroMQ

ZeroMQ (package `pyzmq`) is a library, not a server. It wraps sockets and gives you what plain sockets do not: whole messages (no framing code), automatic reconnects, and queued messages when the peer is late. The "zero" means no broker is needed. You can start client and server in either order.

Socket type	Pattern	Note
`REQ` / `REP`	synchronous request / reply	strict send, receive, send, receive
`DEALER` / `ROUTER`	asynchronous request / reply	build brokers between many clients and servers
`PUSH` / `PULL`	fan-out / fan-in	"ventilator": farm out tasks, gather results
`PUB` / `SUB`	publish / subscribe	subscribers filter by topic prefix

Needs `pip install pyzmq`. I could not run these here (no package installed); the files were syntax-checked only.

```
# server.py   (needs: pip install pyzmq)
import zmq

ctx = zmq.Context()
server = ctx.socket(zmq.REP)
server.bind("tcp://127.0.0.1:5555")
while True:
    request = server.recv()                    # bytes, one whole message
    text = request.decode("utf-8")
    server.send(("tokens: %d" % len(text.split())).encode("utf-8"))
```

```
# client.py
import zmq

ctx = zmq.Context()
client = ctx.socket(zmq.REQ)
client.connect("tcp://127.0.0.1:5555")         # connect, not bind
client.send("count my words please".encode("utf-8"))
print(client.recv().decode("utf-8"))           # tokens: 4
```

The server *binds* (it is the stable side); the client *connects*. Many REQ clients can connect to one REP server; it handles them one at a time and ZeroMQ queues the rest. Strings must be encoded: ZeroMQ messages are bytes. For structured data, send JSON bytes.

ZeroMQ scales by changing only the address scheme:

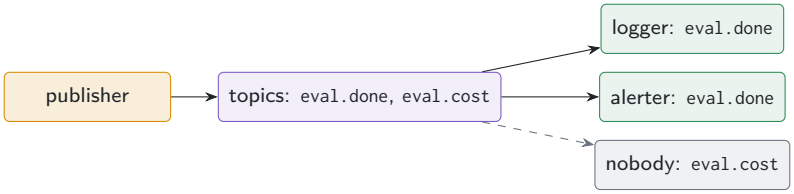
Transport	Connects
`tcp://host:port`	processes, same or different machines
`ipc://name`	processes on one machine
`inproc://name`	threads inside one process (no locks needed)

Other messaging tools

Tool	Notes
RabbitMQ (Python client: Pika)	popular message broker with queues and routing
ActiveMQ (STOMP protocol)	Apache broker, several Python clients
NATS	fast, light messaging; request-reply, pub-sub, queues

22.5 The Publish-Subscribe Pattern

Pub-sub is a broadcast, not a queue. Publishers announce messages on a *topic*; each subscriber says which topics it wants. A message can be processed once, many times or never. A publisher does not know who listens.



Redis

Redis (Chapter 23 covers it as a data store) also has pub-sub. Start the subscriber *first*: messages sent while nobody is subscribed are lost. Needs `pip install redis` and a running Redis or Valkey server; syntax-checked only.

```
# needs: pip install redis, and a Redis (or Valkey) server on localhost
import redis

r = redis.Redis(decode_responses=True)         # return str, not bytes

sub = r.pubsub()
sub.subscribe("eval.done")
for msg in sub.listen():
    if msg["type"] == "message":
        print(msg["channel"], msg["data"])
        break

# in another process:  r.publish("eval.done", "run 7")

# push / pull with a blocking pop: waits up to 2 s for a job
r.lpush("jobs", "prompt-1", "prompt-2")
item = r.brpop("jobs", timeout=2)              # (list_name, value) or None
print(item)
```

> **■ TYPE CHECK**
> Without `decode_responses=True`, redis-py returns `bytes` (`b'run 7'`), so comparisons with `str` silently fail. `brpop` returns a tuple `(key, value)`, or `None` on timeout.

ZeroMQ

ZeroMQ pub-sub has no central server: each publisher writes to its connected subscribers. Messages are *multipart*: the first part is the topic.

```python
# needs: pip install pyzmq
import time
import zmq

ctx = zmq.Context()
pub = ctx.socket(zmq.PUB)
pub.bind("tcp://127.0.0.1:5556")
time.sleep(1)                                  # let subscribers connect
pub.send_multipart([b"eval.done", b'{"run": 7}'])

sub = ctx.socket(zmq.SUB)
sub.connect("tcp://127.0.0.1:5556")
sub.setsockopt(zmq.SUBSCRIBE, b"eval.")        # topic prefix, as bytes
topic, body = sub.recv_multipart()
print(topic.decode("utf-8"), body.decode("utf-8"))
```

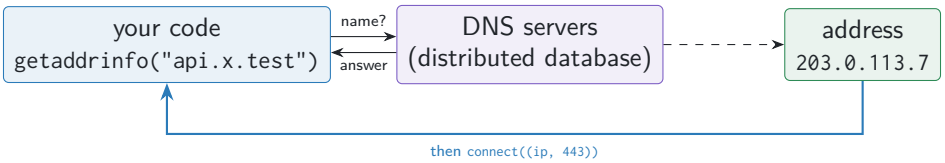
> ⚠ **TRAP CARD: a SUB socket receives nothing by default**
>
> You must subscribe. To get everything, subscribe to the empty bytes `b""`. Also, a PUB that sends right after `bind` may lose the first messages because subscribers have not connected yet (the `sleep` above).

Other options: RabbitMQ with Pika, NATS, PyPubSub (in-process), PubSubHubbub (webhook style).

22.6 Internet Services

Domain Name System (DNS)

DNS turns names into IP addresses. When a request hangs on "looking up host", DNS is the first suspect.



The `socket` module has the functions. Names below resolve locally, so the output is stable; the `getaddrinfo` tuple shape is the same for real hosts.

```python
import socket

print(socket.gethostbyname("localhost"))            # name -> IPv4 str
print(socket.getservbyname("http"), socket.getservbyport(443))
for info in socket.getaddrinfo("localhost", 80, socket.AF_INET,
                               socket.SOCK_STREAM):
    print(info)
```

```
127.0.0.1
80 https
(<AddressFamily.AF_INET: 2>, <SocketKind.SOCK_STREAM: 1>, 0, '', ('127.0.0.1', 80))
```

Function	Returns
`gethostbyname(name)`	one IPv4 address as `str`
`gethostbyname_ex(name)`	`(canonical name, [aliases], [addresses])`
`getaddrinfo(host, port, ...)`	list of tuples with everything needed to build a socket; filter with family and socket type
`getservbyname("http")` / `getservbyport(80)`	service name to port number and back (IANA list)

Python email modules

Module	Protocol	Use
`email`	–	build and parse message objects
`smtplib`	SMTP	send mail
`poplib`	POP3	download mail
`imaplib`	IMAP	read mail kept on the server

Building a message needs no server, so it can be run here:

```
from email.message import EmailMessage

msg = EmailMessage()
msg["From"] = "bot@example.com"
msg["To"] = "me@example.com"
msg["Subject"] = "Nightly eval finished"
msg.set_content("Score: 0.91")
print(msg.as_string())
```

```
From: bot@example.com
To: me@example.com
Subject: Nightly eval finished
Content-Type: text/plain; charset="utf-8"
Content-Transfer-Encoding: 7bit
MIME-Version: 1.0

Score: 0.91
```

Sending uses `smtplib.SMTP(host, 587)`, then `starttls()`, `login(...)`, `send_message(msg)`. I did not run that: it needs a real mail server (syntax-checked only). Never hard-code the password; read it from an environment variable.

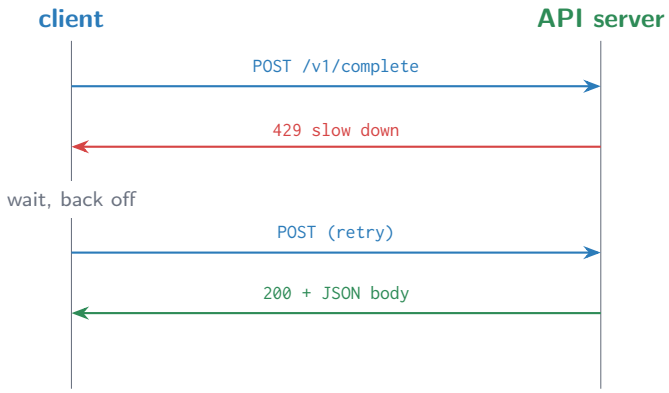
Other protocols

`ftplib` (FTP) moves files; the standard library has modules for most other internet protocols too. The point: before installing a package, check whether the standard library already speaks the protocol.

22.7 Web Services and APIs

Websites are made for human eyes. An *API* serves the same data to programs in a structured format, usually JSON. A scraper breaks whenever the page layout changes; an API changes much less often. Most APIs need a registered *key* (a long token) that identifies you and lets the provider limit your traffic.

An LLM API call is exactly this: an HTTPS request with a key in a header, a JSON body, and a JSON reply. The part beginners miss is the *status code*.



Code	Meaning	What your code should do
200	OK	parse the JSON body
400	bad request	your payload is wrong; fix it, do not retry
401 / 403	bad key / not allowed	stop; check the key
404	not found	check the URL or model name
429	too many requests	wait (see a `retry-after` header), then retry
500 / 503	server error / overloaded	retry with exponential backoff

To keep this runnable offline, here is a tiny fake API on localhost using only the standard library, and a client that returns the status code and parsed body:

```
import json
import threading
import urllib.error
import urllib.request
from http.server import BaseHTTPRequestHandler, HTTPServer

class FakeLLM(BaseHTTPRequestHandler):
    def do_POST(self):
        size = int(self.headers.get("Content-Length", 0))
        body = json.loads(self.rfile.read(size))
        if self.headers.get("x-api-key") != "test-key":
            code, out = 401, {"error": "bad key"}
        elif len(body["prompt"]) > 20:
            code, out = 429, {"error": "slow down"}
        else:
            code, out = 200, {"text": body["prompt"][::-1]}
        data = json.dumps(out).encode("utf-8")
        self.send_response(code)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(data)))
        self.end_headers()
        self.wfile.write(data)

    def log_message(self, *args):          # silence request logging
        pass

srv = HTTPServer(("127.0.0.1", 0), FakeLLM)
threading.Thread(target=srv.serve_forever, daemon=True).start()
url = f"http://127.0.0.1:{srv.server_address[1]}/v1/complete"

def call(prompt, key="test-key"):
    req = urllib.request.Request(
        url,
        data=json.dumps({"prompt": prompt}).encode("utf-8"),
        headers={"Content-Type": "application/json", "x-api-key": key},
        method="POST",
    )
    try:
        with urllib.request.urlopen(req) as resp:
            return resp.status, json.loads(resp.read())
    except urllib.error.HTTPError as err:   # 4xx and 5xx raise
        return err.code, json.loads(err.read())

print(call("hello"))
print(call("hello", key="wrong"))
print(call("a very long prompt that is over the limit"))
srv.shutdown()
```

```
(200, {'text': 'olleh'})
(401, {'error': 'bad key'})
(429, {'error': 'slow down'})
```

■ TYPE CHECK

The request body is `bytes` (`json.dumps(...).encode("utf-8")`). The reply body is also `bytes`; `json.loads` accepts bytes and returns a `dict`. With the popular `requests` package the same idea is shorter (`pip install requests`; syntax-checked only): `resp.status_code` is an `int`, `resp.json()` is

a `dict`, and `resp.raise_for_status()` turns 4xx/5xx into an exception.

```
# needs: pip install requests
import os
import requests

resp = requests.post(
    "https://api.example.test/v1/complete",
    headers={"x-api-key": os.environ["MY_API_KEY"]},
    json={"prompt": "hello"},
    timeout=30,
)
if resp.status_code == 429:
    print("rate limited; retry after", resp.headers.get("retry-after"))
else:
    resp.raise_for_status()                    # 4xx/5xx -> HTTPError
    data = resp.json()                         # dict
```

➤ **RULE: always set a timeout, keep keys out of code**

A network call without a timeout can hang forever. Put API keys in environment variables, never in source files or git.

22.8 Data Serialization

Memory holds objects; a network or a file holds bytes. Converting one to the other is *serialization* (also *marshaling*); the reverse is *deserialization*.



JSON vs pickle

```
import datetime
import json
import pickle

record = {"model": "m1", "tokens": 120, "ok": True, "tags": ["a", "b"]}
wire = json.dumps(record).encode("utf-8")
print(type(wire), wire)
print(json.loads(wire.decode("utf-8")) == record)

stamp = datetime.datetime(2026, 1, 5, 9, 30)
try:
    json.dumps({"at": stamp})
except TypeError as err:
    print("TypeError:", err)
print(json.dumps({"at": stamp}, default=str))

blob = pickle.dumps(stamp)
print(type(blob), pickle.loads(blob) == stamp)

class Tiny:
    def __init__(self, n):
        self.n = n

copy = pickle.loads(pickle.dumps(Tiny(3)))
print(type(copy).__name__, copy.n)

print(json.loads(json.dumps((1, 2))))
try:
    json.dumps({1, 2})
except TypeError as err:
    print("TypeError:", err)
print(json.dumps({1: "a"}), json.loads(json.dumps({1: "a"})))
```

```
<class 'bytes'> b'{"model": "m1", "tokens": 120, "ok": true, "tags": ["a", "b"]}'
True
TypeError: Object of type datetime is not JSON serializable
{"at": "2026-01-05 09:30:00"}
<class 'bytes'> True
Tiny 3
[1, 2]
TypeError: Object of type set is not JSON serializable
{"1": "a"} {'1': 'a'}
```

■ TYPE CHECK

JSON only knows: `dict` (string keys), `list`, `str`, `int`/`float`, `True`/`False`, `None`. Everything else fails or changes shape: a `tuple` comes back as a `list`, a `set` or `datetime` raises `TypeError`, an `int` dict key comes back as `str`. Fix unknown types with `default=str` or by converting yourself. `pickle.dumps` returns `bytes`, `json.dumps` returns `str`.

⚠ TRAP CARD: never unpickle data you do not trust

`pickle.loads` can run arbitrary code. Data from a network, an upload or a shared cache must never go through pickle. Use JSON (or another data-only format) at trust boundaries. Pickle is fine for your own temporary files; `multiprocessing` uses it internally.

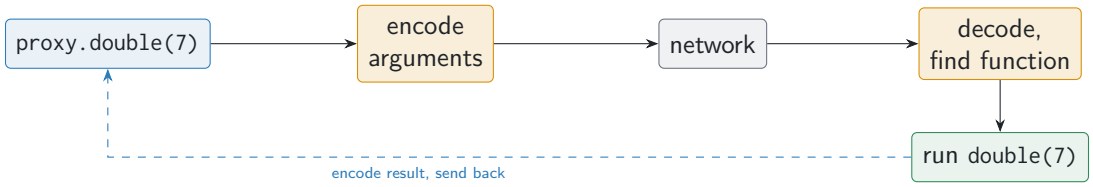
Other formats

Format	Text or binary	Notes
JSON	text	human-readable, universal on the web, few types
pickle	binary	any Python object; Python-only; unsafe for untrusted data
MessagePack, msgspec	binary	JSON-like data, smaller and faster
Protocol Buffers, Avro, Thrift	binary	schema-defined, used with RPC

Packages such as **Pydantic** and marshmallow go the other way: they turn raw data into typed objects and *validate* it (types, ranges, required fields). Web servers use them so bytes from the wire land in the right structures. Pydantic is the one you will meet most in AI work (structured LLM output).

22.9 Remote Procedure Calls

An RPC makes a function on another machine look like a local call. Your side serializes the arguments, sends bytes, and the remote side decodes, runs the function, and sends the encoded result back.



XML-RPC

The standard library has one RPC: XML-RPC over HTTP. This runs fully on localhost:

```
import threading
import xmlrpc.client
from xmlrpc.server import SimpleXMLRPCServer

def cost_usd(tokens_in, tokens_out):
    return tokens_in * 1e-7 + tokens_out * 5e-7

server = SimpleXMLRPCServer(("127.0.0.1", 0), logRequests=False)
port = server.server_address[1]
server.register_function(cost_usd, "cost_usd")
t = threading.Thread(target=server.serve_forever, daemon=True)
t.start()

proxy = xmlrpc.client.ServerProxy(f"http://127.0.0.1:{port}/")
result = proxy.cost_usd(2000, 500)       # looks like a local call
print(result, type(result))
try:
    proxy.nope()                         # no such function on the server
except xmlrpc.client.Fault as err:
    print("Fault:", err.faultCode)
server.shutdown()
```

```
0.00045 <class 'float'>
Fault: 1
```

`proxy.cost_usd` does not exist in the client; the proxy turns any attribute access into a remote call. Remote errors arrive as `xmlrpc.client.Fault`.

Other RPC systems

System	Package	Notes
XML-RPC	stdlib `xmlrpc`	XML over HTTP, simple, verbose
JSON-RPC	`jsonrpcserver`, `jsonrpcclient`	same idea in JSON
MessagePack RPC	`msgpack-rpc-python`	binary, uses Tornado
zerorpc	`zerorpc`	ZeroMQ + MessagePack; methods of a class become endpoints
gRPC	`grpcio`, `grpcio-tools`	Google; Protocol Buffers; streaming; fast

gRPC works from a schema. You describe the service in a `.proto` file, and a tool generates Python client and server stubs:

```
service Pricing {
  rpc Cost (CostRequest) returns (CostReply);
}
message CostRequest { int32 tokens_in = 1; int32 tokens_out = 2; }
message CostReply   { double usd = 1; }
```

A rule of thumb: REST/JSON for public APIs, gRPC for fast service-to-service calls inside your own system.

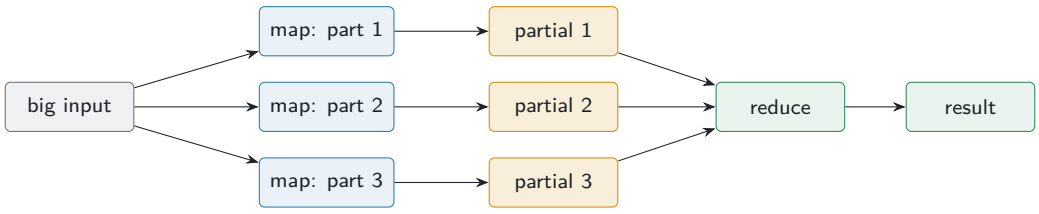
22.10 Remote Management

Tools that run commands on many machines at once.

Tool	Notes
Salt	written in Python; ZeroMQ based; scales to thousands of machines
Ansible	written in Python; agentless, uses SSH
Puppet, Chef	popular, tied to Ruby

22.11 Big Fat Data

One machine, even a big one, hits limits in disk, memory and CPU. Spreading data across many machines can be faster than a single fast one. The core idea is *MapReduce*: split the data, process each piece independently (map), then combine the partial results (reduce).



Tool	Notes
Hadoop	Java; copies data between machines and writes to disk at each step, so batch jobs are slow. Hadoop streaming lets any language, including Python, plug in
Spark	much faster (in memory); Python API (PySpark); the usual choice today
Disco	Python + Erlang MapReduce; little maintained
Dask	written in Python; works with NumPy, pandas, scikit-learn; scales from a laptop to clusters

"Big data" often just means "too big for my machine". Try a bigger machine or a smarter format first; use a cluster only when you must.

22.12 Clouds

Servers fail, and with thousands of them, failure is routine. Peter Deutsch's "fallacies of distributed computing" list the false assumptions that cause bugs.

Things that are NOT true on a network	
The network is reliable.	Latency is zero.
Bandwidth is infinite.	The network is secure.
Topology does not change.	There is one administrator.
Transport cost is zero.	The network is homogeneous.

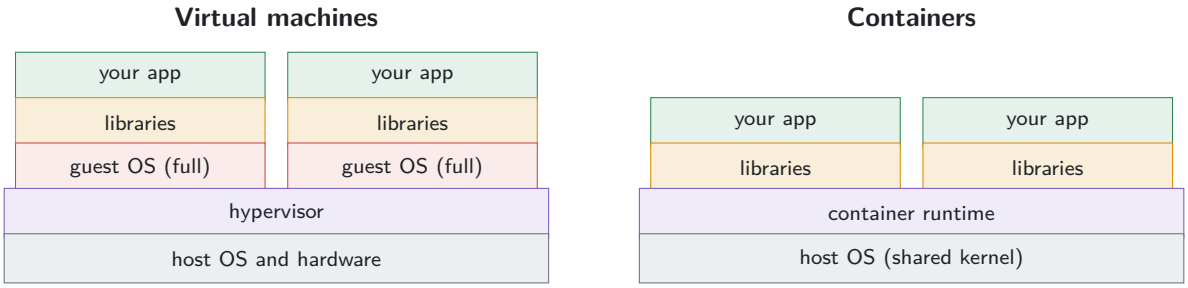
This is why calls to remote APIs need timeouts, retries and error handling (the status code table above).

At scale, servers are treated as interchangeable ("livestock"): replace broken ones rather than nursing them. Renting machines in the cloud turns that into an API call.

Service	Python tool	Notes
Amazon Web Services	`boto3`	the market leader; replaces the web console with code
Google Cloud	Google client libraries	Google uses Python heavily
Microsoft Azure	Azure SDK for Python	third big vendor
OpenStack	its REST APIs	open source cloud framework

Docker and Kubernetes

A *container* packages your app with its libraries, sharing the host's operating system kernel. It is lighter than a virtual machine and heavier than a virtualenv. *Kubernetes* runs many containers for you: restarts failures, balances load, scales up and down.



Python clients: `docker` and `kubernetes` (`pip install docker kubernetes`). Docker is how most AI services get shipped.

22.13 Practice

➤ **PRACTICE**

1. Build a TCP time service on localhost: when a client sends `time`, the server replies with the current time as an ISO string. Use the length-prefix helpers and port 0.
2. Extend the fake API above: make it return 503 for the first two calls, then 200. Write a client that retries with doubling waits and gives up after four tries.
3. Add prefix matching to the in-process `Broker`, so a subscriber to `"eval."` also receives `"eval.done"`.
4. Write an XML-RPC server with one function that returns a dict. Call it from a client. What happens if the function returns `None`?
5. Explain in two sentences why `pickle.loads` on bytes from a socket is dangerous, and which format you would use instead.
6. (Needs Redis or Valkey.) Push ten job strings to a list, then write a worker that loops on `brpop`,

sleeps 0.5 s per job, and prints how many jobs remain.

★ **ACHIEVEMENT UNLOCKED: Networking Basics**

You can now:

- explain the difference between UDP and TCP and write a localhost echo with sockets;
- frame messages on a TCP stream and keep `bytes` and `str` apart;
- pick request-reply, push or pub-sub for a given problem;
- read HTTP status codes from an API and decide whether to retry;
- choose between JSON and pickle, and explain the pickle danger.

CHAPTER 23

Data in a Box: Persistent Storage

Variables vanish when a program ends. This chapter shows how to keep data: plain files, structured text (CSV, JSON, XML, TOML), binary files, SQL databases and key-value stores. The SQL part matters most for a working engineer: you will store eval runs, costs and cached prompts in tables like the ones here.

23.1 Text Files

RAM is fast but forgets everything without power. A disk is slower but keeps data. *Persistence* means writing data to disk (or a server) and reading it back later. The simplest form is a flat text file (file reading and writing are in Chapter 20).

A *record* is one chunk of related data made of *fields*. In a *padded* text file, every field has a fixed width, so every line is the same length. Then `seek()` can jump to record n at byte $n\times$ line length without reading the rest.

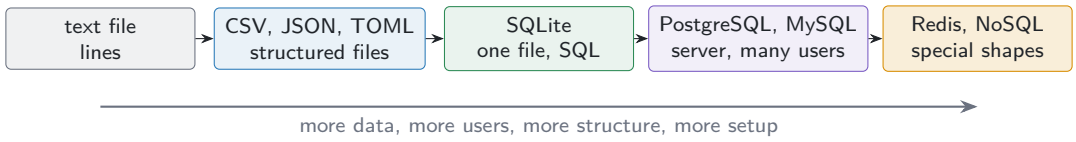
```
import os
import tempfile

os.chdir(tempfile.mkdtemp())           # scratch folder for the examples
with open("fixed.txt", "w", encoding="utf-8", newline="\n") as f:
    for name, score in [("alpha", 91), ("beta", 78), ("gamma", 85)]:
        f.write(f"{name:<8}{score:>4}\n")      # 8 + 4 + newline = 13 chars
with open("fixed.txt", "r", encoding="utf-8", newline="") as f:
    f.seek(13)                                 # skip record 0
    print(repr(f.read(13)))
```

```
'beta      78\n'
```

Fixed-width files are rare today. They are shown because they explain what databases do for you: find a row without scanning everything.

The picture to keep in mind is a ladder. Move up only when the lower rung hurts.



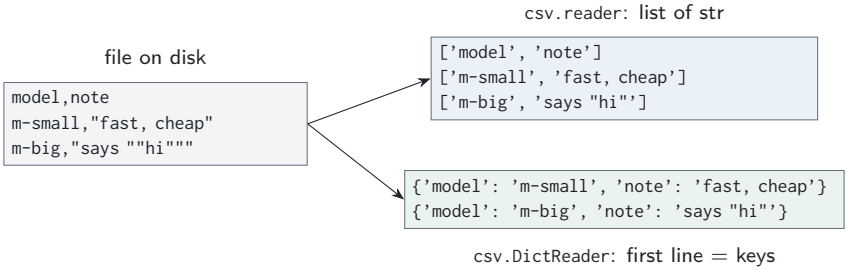
23.2 Tabular and Delimited Text Files

Lines are the only structure in plain text. These formats add more:

Format	Structure by	Good for
CSV, TSV	delimiter (comma, tab, `|`)	tables, spreadsheet exchange
XML, HTML	`<tags>`	documents, feeds, old enterprise data
JSON	braces, brackets, quotes	APIs, nested data, configs
YAML	indentation	human-edited configs
TOML	`[sections]`, `key = value`	configs (stdlib reader)
INI (`configparser`)	`[sections]`	simple settings

CSV

Do not split lines on commas yourself. Fields may contain commas, quotes or newlines, and only the `csv` module handles all of that. It also works with other delimiters (`delimiter="\t"`).




```python
import csv
from pathlib import Path

rows = [["model", "note"], ["m-small", "fast, cheap"], ["m-big", 'says "hi"']]
with open("runs.csv", "w", newline="", encoding="utf-8") as f:
    csv.writer(f).writerows(rows)
print(Path("runs.csv").read_text(encoding="utf-8"))

with open("runs.csv", newline="", encoding="utf-8") as f:
    for row in csv.reader(f):
        print(row)

with open("runs.csv", newline="", encoding="utf-8") as f:
    data = list(csv.DictReader(f))
print(data[0], type(data[0]))

with open("scores.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.DictWriter(f, fieldnames=["model", "score"])
    w.writeheader()
    w.writerow({"model": "m-small", "score": 0.82})
with open("scores.csv", newline="", encoding="utf-8") as f:
    r = next(csv.DictReader(f))
print(r, type(r["score"]))
```

```
model,note
m-small,"fast, cheap"
m-big,"says ""hi"""

['model', 'note']
['m-small', 'fast, cheap']
['m-big', 'says "hi"']
{'model': 'm-small', 'note': 'fast, cheap'} <class 'dict'>
{'model': 'm-small', 'score': '0.82'} <class 'str'>
```

■ TYPE CHECK

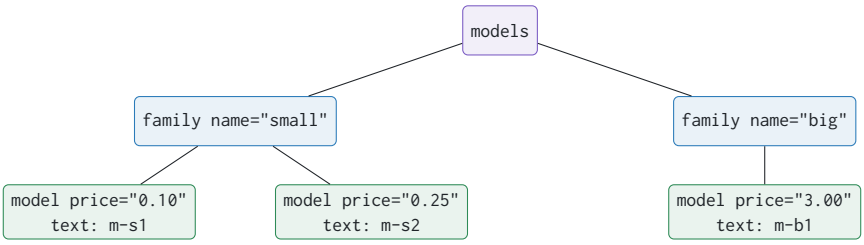
CSV has no types. Every field comes back as `str`, even `0.82`. Convert yourself: `float(r["score"])`. A row is a `list[str]` from `reader`, a `dict[str, str]` from `DictReader` (plain `dict` in modern Python).

⚠ TRAP CARD: open CSV files with `newline=""`

Without it, Windows can write blank lines between rows and can break quoted fields that contain newlines. Also pass `encoding="utf-8"` so the file means the same on every machine.

XML

XML nests data in tags. A tag can have attributes, text and child tags. Python's `xml.etree.ElementTree` module parses it into a tree.




```
import xml.etree.ElementTree as ET

xml_text = """<models>
  <family name="small">
    <model price="0.10">m-s1</model>
    <model price="0.25">m-s2</model>
  </family>
  <family name="big">
    <model price="3.00">m-b1</model>
  </family>
</models>"""
root = ET.fromstring(xml_text)          # parse from a str
print(root.tag, len(root))
for fam in root:
    print(fam.tag, fam.attrib, len(fam))
for m in root.iter("model"):             # every <model> at any depth
    print(m.text, m.attrib["price"], type(m.attrib["price"]))
print(root.find("family[@name='big']/model").text)
```

```
models 2
family {'name': 'small'} 2
family {'name': 'big'} 1
m-s1 0.10 <class 'str'>
m-s2 0.25 <class 'str'>
m-b1 3.00 <class 'str'>
m-b1
```

■ TYPE CHECK

`.tag` is a `str`, `.attrib` is a `dict[str, str]`, `.text` is a `str` or `None`, and `len(element)` counts child elements. Numbers inside XML are text until you convert them. `ET.parse(path)` reads a file; `ET.fromstring(s)` reads a string.

Other standard parsers: `xml.dom` loads the whole tree and lets you move around freely (like a browser's DOM); `xml.sax` streams through the file without holding it all in memory, so it suits very large files.

> ⚠ **TRAP CARD: untrusted XML can attack you**
>
> The "billion laughs" attack nests entity definitions so a few lines expand to gigabytes in memory. For XML you did not write, use the `defusedxml` package (a drop-in safe version of these parsers) instead of the plain standard-library ones.

HTML looks like XML but is usually messy and is meant for display, not data exchange. It is handled with scraping tools in Chapter 24.

JSON

JSON is the standard exchange format for APIs and the closest match to Python data. The module is `json`: `dumps`/`loads` work with strings, `dump`/`load` with files.

```
import json

cfg = {"model": "m-small", "temp": 0.2, "stop": ["\n"],
       "seed": None, "ok": True}
text = json.dumps(cfg)
print(text)
print(json.dumps(cfg, indent=2, sort_keys=True))
back = json.loads(text)
print(back == cfg, back["seed"], back["ok"])
with open("cfg.json", "w", encoding="utf-8") as f:
    json.dump(cfg, f)
with open("cfg.json", encoding="utf-8") as f:
    print(json.load(f)["temp"])
try:
    json.loads("{'a': 1}")
except json.JSONDecodeError as e:
    print("JSONDecodeError:", e)
```

```
{"model": "m-small", "temp": 0.2, "stop": ["\n"], "seed": null, "ok": true}
{
  "model": "m-small",
  "ok": true,
  "seed": null,
  "stop": [
    "\n"
  ],
  "temp": 0.2
}
True None True
0.2
JSONDecodeError: Expecting property name enclosed in double quotes: line 1 column 2 (char 1)
```

> ■ **TYPE CHECK**

Python	JSON
`dict` (str keys)	object `{...}`
`list`, `tuple`	array `[...]` (a tuple comes back as a list)
`str`	string (double quotes only)
`int`, `float`	number
`True`, `False`, `None`	`true`, `false`, `null`

`dumps` returns `str`. `set`, `datetime`, `Decimal` and your own classes are not JSON serializable and raise `TypeError`. JSON strings need double quotes: `{'a': 1}` is invalid JSON even though it is valid

Python.

Teach `json` about extra types with a `default` function. It is called only for objects the module cannot handle, and must return something serializable:

```
import datetime
import json

def to_json(obj):
    if isinstance(obj, datetime.datetime):
        return obj.isoformat()
    raise TypeError(f"cannot serialize {type(obj).__name__}")

stamp = datetime.datetime(2026, 1, 5, 9, 30)
print(json.dumps({"at": stamp}, default=to_json))
print(json.dumps({"at": stamp}, default=str))      # quick and lazy
```

```
{"at": "2026-01-05T09:30:00"}
{"at": "2026-01-05 09:30:00"}
```

Prefer ISO 8601 (`isoformat()`) for dates: it sorts correctly and every language reads it. By default `dumps` writes non-ASCII characters as `\uXXXX` escapes; pass `ensure_ascii=False` to keep readable text.

> ➤ **RULE: JSON for data, never pickle, across trust boundaries**
>
> JSON can only carry data, not code. Use it for configs, API payloads and caches.

YAML

YAML is like JSON but uses indentation and allows comments and more types (dates). It is very common in config files (Docker Compose, CI, Kubernetes). The reader is third party: `pip install pyyaml`. This was syntax-checked only.

```
# needs: pip install pyyaml
import yaml

text = """
model: m-small
params:
  temperature: 0.2
  stops: [END, STOP]
debug: false
"""
conf = yaml.safe_load(text)            # dict; false -> False
print(conf["params"]["stops"])         # ['END', 'STOP']
```

> ⚠ **TRAP CARD: `yaml.load` on untrusted text can run code**
>
> Always call `yaml.safe_load`. Plain `load` can build arbitrary Python objects.

TOML

TOML is a clear config format with comments, in the standard library since Python 3.11 (`tomllib`, read-only). `pyproject.toml` uses it.

```
import tomllib

toml_text = """
title = "eval config"
[model]
name = "m-small"
temperature = 0.2
stops = ["END", "STOP"]
[limits]
max_tokens = 512
"""
conf = tomllib.loads(toml_text)         # loads takes a str
print(conf)
print(conf["limits"]["max_tokens"] + 1)

# not run: from a file, open in BINARY mode
# with open("pyproject.toml", "rb") as f:
#     conf = tomllib.load(f)
```

```
{'title': 'eval config', 'model': {'name': 'm-small', 'temperature': 0.2, 'stops': ['END', 'STOP']}, 'limits':
    {'max_tokens': 512}}
513
```

> ■ **TYPE CHECK**
> TOML keeps types: strings, `int`, `float`, `bool`, lists, dates, tables (nested `dict`). `tomllib.load` needs a file opened with `"rb"`; `tomllib.loads` takes a `str`. It cannot write TOML.

Tablib

Tablib (`pip install tablib`) imports, edits and exports tabular data between CSV, JSON, YAML, Excel and DataFrames. In Chapter 25 pandas does the same job and more.

Configuration files

Do not invent your own config format. You would have to maintain both writer and parser. Use one of the formats above. The standard `configparser` reads Windows-style `.ini` files with sections:

```
import configparser

ini = """[api]
base = https://example.test
timeout = 30
[paths]
home = /srv/app
logs = %(home)s/logs
"""
cp = configparser.ConfigParser()
cp.read_string(ini)                     # cp.read("file.ini") for a file
print(cp["api"]["timeout"], type(cp["api"]["timeout"]))
print(cp["api"].getint("timeout") + 1)
print(cp["paths"]["logs"])              # %(home)s was filled in
```

```
30 <class 'str'>
31
/srv/app/logs
```

> ■ **TYPE CHECK**
> Every `configparser` value is a `str`. Use `getint`, `getfloat`, `getboolean`. Only two levels (section, key); for deeper nesting use JSON, YAML or TOML.

> ➤ **RULE: secrets do not go in config files you commit**
> Keep API keys in environment variables or a secrets manager, not in `.json`, `.toml` or `.ini` files in git.

23.3 Binary Files

Some formats are neither relational nor NoSQL; they store a particular shape of data in bytes.

Padded binary files and memory mapping

Like the padded text file, but with fixed-size binary records. The `struct` module packs values into bytes with a format string; `mmap` makes a file look like one big `bytes`-like object so you can slice it without reading everything.

```python
import mmap
import struct

rec = struct.Struct("<8sIf")            # 8-byte name, uint32, float32
print(rec.size)
with open("fixed.bin", "wb") as f:
    records = [(b"alpha", 1, 0.5), (b"beta", 2, 1.5), (b"gamma", 3, 2.5)]
    for name, n, x in records:
        f.write(rec.pack(name, n, x))
with open("fixed.bin", "rb") as f:
    with mmap.mmap(f.fileno(), 0, access=mmap.ACCESS_READ) as mm:
        chunk = mm[rec.size:2 * rec.size]      # record 1, no full read
        print(rec.unpack(chunk))
```

```
16
(b'beta\x00\x00\x00\x00', 2, 1.5)
```

> ■ **TYPE CHECK**
> Everything here is `bytes`. A fixed-width string field comes back padded with `b"\x00"`; strip it and decode: `raw.rstrip(b"\x00").decode("utf-8")`. `"<"` in the format means little-endian with no alignment padding.

This is low-level work, used close to hardware or in file formats you must match. Chapter 18 covers `struct` in depth.

Spreadsheets, HDF5 and TileDB

Format	How to use it from Python
Excel (`.xlsx`)	export to CSV and use `csv`; or use a library (`openpyxl`, Tablib, pandas `read_excel`)
HDF5	binary format for huge numeric arrays and hierarchies, fast random access; write once, read many. Packages `h5py` (low level), PyTables (database-like)
TileDB	successor to HDF5 for dense or sparse arrays (`pip install tiledb`); science oriented

23.4 Relational Databases

A relational database stores data in *tables* and lets you combine tables. It gives you things files do not:

- many users at once, without corrupting the data;
- a *schema*: columns with types and rules (*constraints*);
- fast lookups with *indexes*;
- *joins*: combine tables by shared values;
- a declarative query language, SQL: you say *what* you want, not how to get it.

Tables, keys and indexes

table `runs`

primary key: unique →

id	task	score
1	qa	0.71
2	code	0.55
3	qa	NULL

← column: name and type

← row: one record

← NULL: value missing

- **Primary key**: a column (or group) whose values are unique in the table. It identifies a row and is indexed automatically.
- **Index**: a side structure, like a book index, that finds rows by a column without scanning the whole table. Without one the database does a *table scan*.
- **Foreign key**: a column holding another table's primary key. It links rows across tables.
- **NULL**: "no value". It is not zero and not an empty string.

"Database" is used for three things: the server program, a named container of tables, and the stored data. Say "database server", "database" and "data" when it matters.

23.5 SQL

SQL is a text language sent to the database, which decides how to run it. It is not case-sensitive, but keywords are written in capitals by tradition. Dialects differ slightly between vendors; the core below works on SQLite, PostgreSQL and MySQL.

Group	Statements
DDL (data definition)	`CREATE TABLE`, `DROP TABLE`, `ALTER TABLE`, `CREATE INDEX`: shape of the data
DML (data manipulation)	`INSERT`, `SELECT`, `UPDATE`, `DELETE`: the data itself. Known as CRUD: Create, Read, Update, Delete

```
CREATE TABLE t (id INTEGER PRIMARY KEY, label TEXT NOT NULL);   -- DDL
INSERT INTO t (label) VALUES ('first');                          -- create
SELECT id, label FROM t WHERE id > 0;                            -- read
UPDATE t SET label = 'one' WHERE id = 1;                         -- update
DELETE FROM t WHERE id = 1;                                      -- delete
DROP TABLE t;                                                    -- DDL
```

> ⚠ **TRAP CARD: `UPDATE` or `DELETE` without `WHERE` hits every row**
>
> Write the `WHERE` first. Run it as a `SELECT` to see which rows it matches, then change `SELECT` to `DELETE`.

A small dataset

We use SQLite from the standard library (details in the next section). Two tables: models and their eval runs. `runs.model_id` is a foreign key to `models.id`. The helper `q` runs a query and returns the rows.

```
import sqlite3

conn = sqlite3.connect(":memory:")       # lives in RAM; gone at exit
conn.execute("""
CREATE TABLE models (
    id        INTEGER PRIMARY KEY,
    name      TEXT NOT NULL UNIQUE,
    vendor    TEXT NOT NULL,
    usd_per_m REAL NOT NULL
)""")
conn.execute("""
CREATE TABLE runs (
    id       INTEGER PRIMARY KEY,
    model_id INTEGER NOT NULL REFERENCES models(id),
    task     TEXT NOT NULL,
    score    REAL,
    tokens   INTEGER NOT NULL
)""")
conn.executemany(
    "INSERT INTO models (name, vendor, usd_per_m) VALUES (?, ?, ?)",
    [("m-small", "acme", 0.10), ("m-big", "acme", 3.00),
     ("m-open", "libre", 0.0)],
)
conn.executemany(
    "INSERT INTO runs (model_id, task, score, tokens) VALUES (?, ?, ?, ?)",
    [(1, "qa", 0.71, 1200), (1, "code", 0.55, 3000), (2, "qa", 0.90, 1100),
     (2, "code", 0.84, 2900), (2, "qa", 0.88, 1000), (3, "qa", None, 800)],
)
conn.commit()

def q(sql, params=()):
    return conn.execute(sql, params).fetchall()

print(q("SELECT * FROM models"))
```

```
[(1, 'm-small', 'acme', 0.1), (2, 'm-big', 'acme', 3.0), (3, 'm-open', 'libre', 0.0)]
```

Notice that the last run has `score = None`: it becomes SQL `NULL`.

SELECT, WHERE, ORDER BY, LIMIT

`SELECT` picks columns, `FROM` the table, `WHERE` filters rows, `ORDER BY` sorts (`DESC` for descending), `LIMIT` caps the count.

```
print(q("SELECT task, score FROM runs WHERE score > 0.7 AND task = 'qa'"))
print(q("""SELECT id, score FROM runs WHERE score IS NOT NULL
           ORDER BY score DESC LIMIT 3"""))
print(q("SELECT id FROM runs WHERE score = NULL"))    # wrong
print(q("SELECT id FROM runs WHERE score IS NULL"))   # right
```

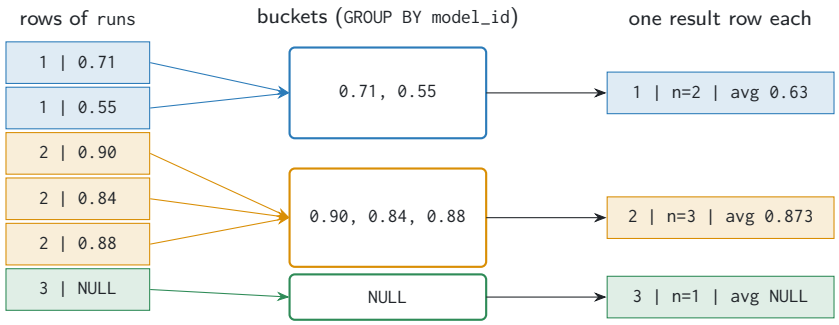
```
[('qa', 0.71), ('qa', 0.9), ('qa', 0.88)]
[(3, 0.9), (5, 0.88), (4, 0.84)]
[]
[(6,)]
```

> ⚠ **TRAP CARD: `= NULL` never matches**
> `NULL` means "unknown", and nothing equals unknown. Use `IS NULL` and `IS NOT NULL`. Also note that `AVG` and `MAX` skip NULLs.

Other filters: `IN ('a','b')`, `BETWEEN 1 AND 5`, `LIKE 'm-%'` (`%` means any text), `OR`, `NOT`.

Aggregates and GROUP BY

Aggregate functions collapse many rows into one value: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`. `GROUP BY` makes one bucket per distinct value and runs the aggregate once per bucket. `HAVING` filters buckets (`WHERE` filters rows *before* grouping).



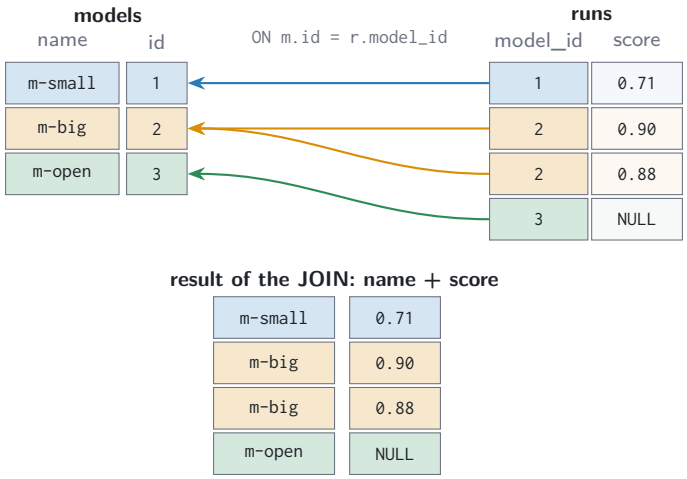

```
print(q("SELECT COUNT(*), AVG(score), MAX(tokens), SUM(tokens) FROM runs"))
print(q("""SELECT model_id, COUNT(*) AS n, ROUND(AVG(score), 3) AS avg_score
           FROM runs GROUP BY model_id ORDER BY model_id"""))
print(q("""SELECT model_id, COUNT(*) AS n FROM runs
           GROUP BY model_id HAVING COUNT(*) >= 2"""))
```

```
[(6, 0.776, 3000, 10000)]
[(1, 2, 0.63), (2, 3, 0.873), (3, 1, None)]
[(1, 2), (2, 3)]
```

`AS` gives a column an alias. `COUNT(*)` counts rows; `COUNT(score)` skips NULLs. Model 3 has an average of `None` because its only score is NULL.

JOIN

A join combines rows of two tables where the `ON` condition is true. Here: each run row is matched with the model row whose `id` equals the run's `model_id`.




```
print(q("""SELECT m.name, r.task, r.score
           FROM runs AS r JOIN models AS m ON m.id = r.model_id
           WHERE r.task = 'qa' ORDER BY r.score DESC"""))
for row in q("""SELECT m.name, SUM(r.tokens) AS toks,
                       SUM(r.tokens) * m.usd_per_m / 1000000.0 AS usd
                FROM models m JOIN runs r ON r.model_id = m.id
                GROUP BY m.id ORDER BY usd DESC"""):
    print(row)
```

```
[('m-big', 'qa', 0.9), ('m-big', 'qa', 0.88), ('m-small', 'qa', 0.71),
 ('m-open', 'qa', None)]
('m-big', 5000, 0.015)
('m-small', 4200, 0.00042)
('m-open', 800, 0.0)
```

The second query is the shape you will use daily: join, group by, aggregate. It computes cost per model from token counts and a price per million tokens.

`JOIN` (inner join) drops rows with no partner. `LEFT JOIN` keeps every row of the left table and fills missing right-side columns with NULL:

```
conn.execute("INSERT INTO models (name, vendor, usd_per_m) "
             "VALUES ('m-new', 'acme', 1.0)")
print(q("""SELECT m.name, COUNT(r.id) FROM models m
           LEFT JOIN runs r ON r.model_id = m.id GROUP BY m.id"""))
print(q("""SELECT m.name, COUNT(r.id) FROM models m
           JOIN runs r ON r.model_id = m.id GROUP BY m.id"""))
```

```
[('m-small', 2), ('m-big', 3), ('m-open', 1), ('m-new', 0)]
[('m-small', 2), ('m-big', 3), ('m-open', 1)]
```

`m-new` has no runs. The `LEFT JOIN` still lists it, with count 0; the inner `JOIN` loses it.

The order SQL really runs

You write `SELECT` first, but the database starts at `FROM`. This explains most "why can't I use that here?" errors, for instance why an alias from `SELECT` cannot be used in `WHERE`.



pick tables, filter rows, make buckets, filter buckets, choose columns, sort, cut

UPDATE, DELETE and indexes

```
cur = conn.execute("UPDATE runs SET score = 0.5 WHERE score IS NULL")
print(cur.rowcount)                      # rows changed
cur = conn.execute("DELETE FROM runs WHERE score < 0.6")
print(cur.rowcount)                      # rows removed
conn.commit()
```

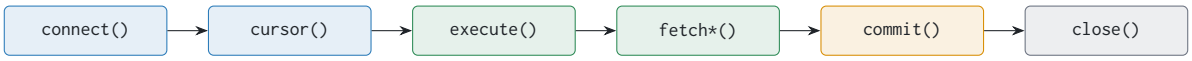
```
1
2
```

An index speeds up lookups and joins on a column but slows writes slightly and uses space. Index columns you filter or join on often (foreign keys are good candidates):

```
CREATE INDEX idx_runs_model ON runs (model_id);
```

23.6 DB-API and SQLite

DB-API is Python's standard interface to relational databases. Drivers for SQLite, PostgreSQL and MySQL all follow it, so the shape of your code stays the same.



connection = the line to the database cursor = holds one query and its results

Call	What it does
`connect(...)`	open a connection (file name for SQLite; host, user, password for servers)
`conn.cursor()`	create a cursor, the object that runs statements and holds results
`execute(sql, params)` / `executemany(sql, seq)`	run one statement / run it once per parameter set
`fetchone()`, `fetchmany(n)`, `fetchall()`	read result rows
`conn.commit()` / `conn.rollback()`	make changes permanent / undo them

SQLite is a SQL database in a library: no server, one ordinary file, built into Python (`import sqlite3`). Browsers and phones use it. `":memory:"` gives a temporary database in RAM, ideal for tests.

Placeholders and what comes back

```
import sqlite3

conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE users (name TEXT, plan TEXT)")
conn.executemany("INSERT INTO users VALUES (?, ?)",
                 [("ana", "free"), ("bo", "pro"), ("cy", "pro")])

name = "nobody' OR '1'='1"                 # hostile input

bad = f"SELECT name FROM users WHERE name = '{name}'"
print(bad)
print(conn.execute(bad).fetchall())        # leaks every row

good = "SELECT name FROM users WHERE name = ?"
print(conn.execute(good, (name,)).fetchall())

print(conn.execute("SELECT name FROM users WHERE plan = :p AND name > :n",
                   {"p": "pro", "n": "bo"}).fetchall())
```

```
SELECT name FROM users WHERE name = 'nobody' OR '1'='1'
[('ana',), ('bo',), ('cy',)]
[]
[('cy',)]
```

> ⚠ **TRAP CARD: SQL injection**
>
> Building SQL with f-strings, `%` or `+` lets user input become SQL. The hostile text above turned a one-row lookup into "return everyone". With `?` placeholders the driver sends the values separately and they are always treated as data. **Always pass values as parameters**, including for LLM-generated text.

> ■ **TYPE CHECK**
>
> Parameters must be a *sequence*: `("pro",)` with the trailing comma, not `"pro"`. A bare string is treated as a sequence of characters. Placeholders can stand for *values only*, not table or column names: `"SELECT * FROM ?"` is a syntax error. If a table name must vary, check it against a fixed allow-list. SQLite uses `?` and `:name`; psycopg2 (PostgreSQL) uses `%s`.

```
try:
    conn.execute("SELECT name FROM users WHERE plan = ?",
                 "pro")                        # not a tuple
except sqlite3.ProgrammingError as err:
    print("ProgrammingError:", err)
sql = "SELECT name FROM users WHERE plan = ?"
print(conn.execute(sql, ("pro",)).fetchall())
try:
    conn.execute("SELECT * FROM ?", ("users",))
except sqlite3.OperationalError as err:
    print("OperationalError:", err)
```

```
ProgrammingError: Incorrect number of bindings supplied. The current statement uses 1, and there are 3 supplied.
[('bo',), ('cy',)]
OperationalError: near "?": syntax error
```

Now the row type. `fetch*` returns *tuples*; `fetchall` returns a list of tuples. Column names are in `cursor.description`. Set `conn.row_factory = sqlite3.Row` to also read columns by name.

```
cur = conn.execute("SELECT name, plan FROM users ORDER BY name")
row = cur.fetchone()
print(type(row), row, row[0])
print([d[0] for d in cur.description])
print(cur.fetchmany(1), cur.fetchall(), cur.fetchone())

conn.row_factory = sqlite3.Row
sql = "SELECT name, plan FROM users WHERE name = 'bo'"
row = conn.execute(sql).fetchone()
print(type(row).__name__, row["plan"], row[0], row.keys(), tuple(row))
```

```
<class 'tuple'> ('ana', 'free') ana
['name', 'plan']
[('bo', 'pro')] [('cy', 'pro')] None
Row pro bo ['name', 'plan'] ('bo', 'pro')
```

> ■ **TYPE CHECK**
> A cursor is used up as you read it: `fetchone` returns the next row or `None` when there are no more rows; `fetchall` then returns `[]`. SQL `NULL` becomes `None`. SQLite types are loose: it *coerces* values to the column type when it can and stores the rest as is.

```
conn.row_factory = None
conn.execute("CREATE TABLE t (a INTEGER, b TEXT)")
conn.execute("INSERT INTO t VALUES (1, 'x')")
conn.execute("INSERT INTO t VALUES ('7', 5)")        # coerced, not rejected
print(conn.execute("SELECT a, b, typeof(a), typeof(b) FROM t").fetchall())

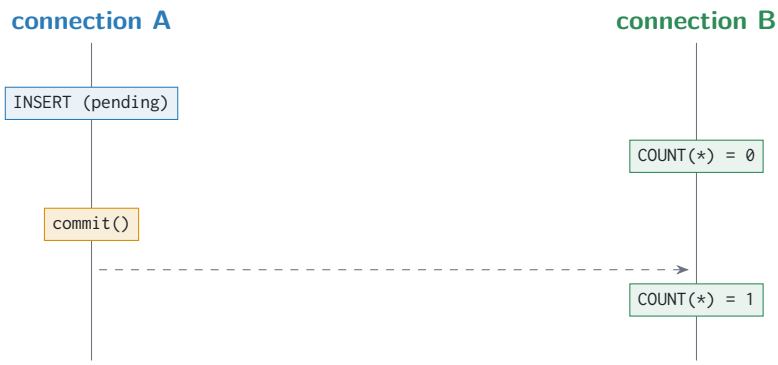
conn.execute("CREATE TABLE k (name TEXT UNIQUE)")
conn.execute("INSERT INTO k VALUES ('a')")
try:
    conn.execute("INSERT INTO k VALUES ('a')")
except sqlite3.IntegrityError as err:
    print("IntegrityError:", err)
```

```
[(1, 'x', 'integer', 'text'), (7, '5', 'integer', 'text')]
IntegrityError: UNIQUE constraint failed: k.name
```

Constraints (`UNIQUE`, `NOT NULL`, `PRIMARY KEY`, foreign keys) make the database reject bad data. That is what the schema is for.

Transactions: commit or lose it

Changes happen inside a *transaction*. Other connections do not see them until you `commit()`. `rollback()` throws them away.




```
import os
import sqlite3
import tempfile

d = tempfile.mkdtemp()
path = os.path.join(d, "app.db")

a = sqlite3.connect(path)
a.execute("CREATE TABLE ev (id INTEGER PRIMARY KEY, msg TEXT)")
a.commit()
a.execute("INSERT INTO ev (msg) VALUES ('one')")
b = sqlite3.connect(path)
print("before commit:", b.execute("SELECT COUNT(*) FROM ev").fetchone())
a.commit()
print("after commit :", b.execute("SELECT COUNT(*) FROM ev").fetchone())

a.execute("INSERT INTO ev (msg) VALUES ('two')")
print("in_transaction:", a.in_transaction)
a.rollback()
print("after rollback:", a.execute("SELECT COUNT(*) FROM ev").fetchone())
```

```
before commit: (0,)
after commit : (1,)
in_transaction: True
after rollback: (1,)
```

The connection is also a context manager: leaving the `with` block commits, or rolls back if an exception escaped. It does *not* close the connection.

```
try:
    with a:
        a.execute("INSERT INTO ev (msg) VALUES ('three')")
        raise RuntimeError("boom")
except RuntimeError:
    pass
print("after failed with:", a.execute("SELECT COUNT(*) FROM ev").fetchone())
with a:
    a.execute("INSERT INTO ev (msg) VALUES ('four')")
print("after good with  :", a.execute("SELECT COUNT(*) FROM ev").fetchone())
a.close()
b.close()

c = sqlite3.connect(path)               # a file keeps its data
print([r[0] for r in c.execute("SELECT msg FROM ev ORDER BY id")])
c.close()
```

```
after failed with: (1,)
after good with  : (2,)
['one', 'four']
```

> ⚠ **TRAP CARD: forgetting `commit()`**
>
> Close a connection without committing and your inserts silently vanish. Use `with conn:` around each unit of work. Also close connections (or use `contextlib.closing`); they hold files and locks.

> ➤ **RULE: one function, parameters, with-block**
>
> Wrap writes in `with conn:`, pass values as parameters, and use `executemany` for bulk inserts: it is much faster than a Python loop of single `execute` calls.

23.7 DuckDB, MySQL and PostgreSQL

Database	Runs as	Python package	Notes
SQLite	library, one file	`sqlite3` (stdlib)	simple apps, tests, local caches
DuckDB	library, in-process	`duckdb`	analytics (OLAP): fast scans and aggregates; queries CSV, JSON, Parquet files directly (Chapter 25)
MySQL	server	`mysqlclient`, `PyMySQL`, `mysql-connector-python`	popular open source server
PostgreSQL	server	`psycopg2` (or `psycopg`)	full-featured, strict, extensible; also does JSON and vectors

Servers are used when many programs on different machines need the same data. The API is the same DB-API; only the connection arguments and the placeholder style change. None of this was run here (no servers, no packages); syntax-checked only.

```
# needs: pip install duckdb psycopg2-binary  (and a running PostgreSQL)
import duckdb
import psycopg2

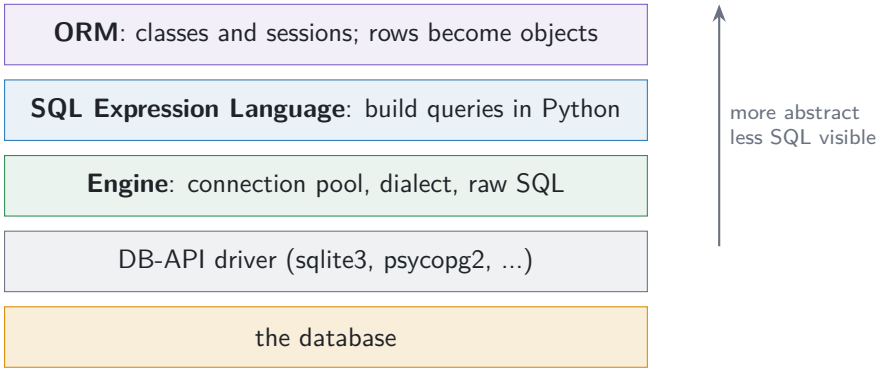
conn = psycopg2.connect(host="localhost", dbname="evals", user="me",
                        password="x")
with conn, conn.cursor() as cur:               # commit on success
    cur.execute("SELECT name FROM models WHERE vendor = %s", ("acme",))
    print(cur.fetchall())                      # %s, not ?, in psycopg2
conn.close()

print(duckdb.sql("SELECT avg(score) FROM 'runs.csv'").fetchall())
```

`psycopg2` needs the PostgreSQL client libraries unless you install the `-binary` variant. Get the password from an environment variable, never from code.

23.8 SQLAlchemy

SQL dialects differ, and DB-API only goes so far. *SQLAlchemy* is the most popular cross-database toolkit. It has three levels; use the lowest that does the job.



The connection string (URL) selects the dialect and driver:

```
dialect+driver://user:password@host:port/dbname
sqlite:///:memory:                    in-memory SQLite
sqlite:///zoo.db                      file in the current folder
postgresql+psycopg2://me:pw@localhost:5432/evals
```

Below is the current (2.x) style for all three levels. It needs `pip install sqlalchemy`; not run here, syntax-checked only. Older tutorials that call `conn.execute("raw string")` do not work on 2.x: wrap raw SQL in `sa.text()`.

```
# needs: pip install sqlalchemy  (2.x)
import sqlalchemy as sa
from sqlalchemy.orm import DeclarativeBase, Mapped, Session, mapped_column

engine = sa.create_engine("sqlite:///:memory:")

# level 1: engine + raw SQL, named parameters
with engine.begin() as conn:                    # begin() commits at the end
    conn.execute(sa.text(
        "CREATE TABLE ev (id INTEGER PRIMARY KEY, msg TEXT)"))
    conn.execute(sa.text("INSERT INTO ev (msg) VALUES (:m)"), {"m": "hi"})
    rows = conn.execute(sa.text("SELECT id, msg FROM ev")).fetchall()
    print(rows)                                 # [(1, 'hi')]
```

Level 2, the expression language, builds the same SQL from Python objects:

```
# level 2: expression language
meta = sa.MetaData()
runs = sa.Table(
    "runs", meta,
    sa.Column("id", sa.Integer, primary_key=True),
    sa.Column("task", sa.String),
    sa.Column("score", sa.Float),
)
meta.create_all(engine)
with engine.begin() as conn:
    conn.execute(sa.insert(runs), [{"task": "qa", "score": 0.9}])
    stmt = sa.select(runs.c.task, runs.c.score).where(runs.c.score > 0.5)
    print(conn.execute(stmt).all())
```

Level 3, the ORM, maps a class to a table and a row to an instance:

```
# level 3: ORM
class Base(DeclarativeBase):
    pass

class Model(Base):
    __tablename__ = "models"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str]
    usd_per_m: Mapped[float]

Base.metadata.create_all(engine)
with Session(engine) as session:
    session.add(Model(name="m-small", usd_per_m=0.10))
    session.commit()
    found = session.scalars(sa.select(Model).where(Model.usd_per_m < 1)).all()
    print(found[0].name)
```

TYPE CHECK

Levels 1 and 2 return `Row` objects that behave like tuples. Level 3 returns *instances of your class* (`Model`); attributes are typed through `Mapped[int]`, `Mapped[str]`, ...

An ORM (object-relational mapper) lets you stay in Python objects. But it is an abstraction, and abstractions leak: when it misbehaves you must understand the SQL it generates. Learn SQL first, then use an ORM for simple, table-shaped apps. FastAPI projects often use SQLAlchemy or SQLModel.

Lighter tools: `dataset` ("databases for lazy people", built on SQLAlchemy) and `records` ("SQL for humans": plain SQL strings, exports via Tablib).

23.9 NoSQL Data Stores

Not all data is a rectangle. NoSQL ("not only SQL") stores trade the strict table model for flexibility or scale. The simplest kind is a *key-value store*: a dictionary on disk or on a server.

Store	Lives	Notes
`dbm`	file, stdlib	dict-like; keys and values are bytes
Memcached	server, RAM	pure cache; may forget data any time
Redis / Valkey	server, RAM + optional disk	keys with rich values (list, hash, set, sorted set); expiry

The dbm family

`dbm` behaves like a dict saved on disk. Modes: `"r"` read, `"w"` write, `"c"` read/write and create. You cannot iterate over the object, but `keys()`, `len()` and `get()` work.

```
import dbm
import os
import tempfile

path = os.path.join(tempfile.mkdtemp(), "defs")
with dbm.open(path, "c") as db:
    db["mustard"] = "yellow"           # str is encoded for you
    db[b"ketchup"] = b"red"
    print(len(db), db["mustard"], type(db["mustard"]))
    print(db.get("none"), db.get("none", b"dflt"))
with dbm.open(path, "r") as db:
    print(db["mustard"].decode("utf-8"))
    print(sorted(db.keys()))
```

```
2 b'yellow' <class 'bytes'>
None b'dflt'
yellow
[b'ketchup', b'mustard']
```

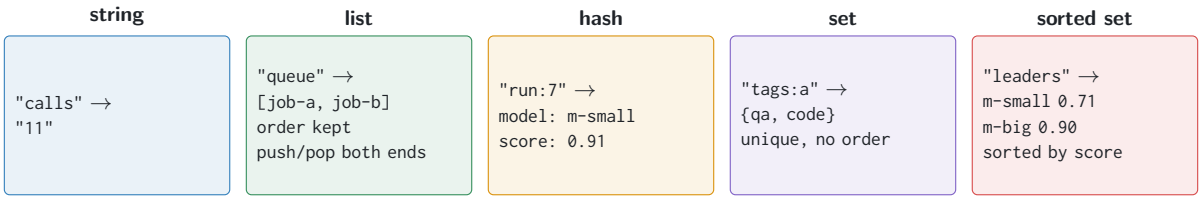
> ■ **TYPE CHECK**
> dbm returns `bytes`, even if you stored a `str`. Decode on the way out.

Memcached

A fast in-memory cache server, often placed in front of a database. Data can disappear at any time (it is a cache). Python drivers: `python-memcached`, `pymemcache`. Operations: set/get, increment/decrement, delete. Use it only for data you can recompute.

Redis

Redis is a *data structure server*: keys map to strings, lists, hashes, sets or sorted sets. Data lives in memory, with optional saving to disk. Redis types map closely to Python types, so it makes a handy shared scratchpad between programs. Install `pip install redis` and run a Redis server on localhost. All code in this section is syntax-checked only; the comments show what you should see.



```python
# needs: pip install redis, and a Redis (or Valkey) server on localhost
import json
import time

import redis

r = redis.Redis(decode_responses=True)          # str in, str out

# strings and counters
r.set("model:default", "m-small")
print(r.get("model:default"))                   # 'm-small'
r.set("calls", 0)
r.incr("calls")
r.incr("calls", 10)
print(r.get("calls"))                           # '11'  (a str!)
print(r.setnx("model:default", "other"))        # False: key exists

# list (a queue)
r.rpush("queue", "job-a", "job-b")
print(r.lpop("queue"), r.llen("queue"))         # job-a 1

# hash (one level of fields, all strings)
r.hset("run:7", mapping={"model": "m-small", "score": "0.91"})
print(r.hget("run:7", "score"), r.hgetall("run:7"))

# set
r.sadd("tags:a", "qa", "code")
r.sadd("tags:b", "qa", "math")
print(r.sinter("tags:a", "tags:b"))             # {'qa'}

# sorted set (leaderboard)
r.zadd("leaders", {"m-small": 0.71, "m-big": 0.90})
print(r.zrange("leaders", 0, -1, withscores=True))
print(r.zrevrange("leaders", 0, 0))             # ['m-big']

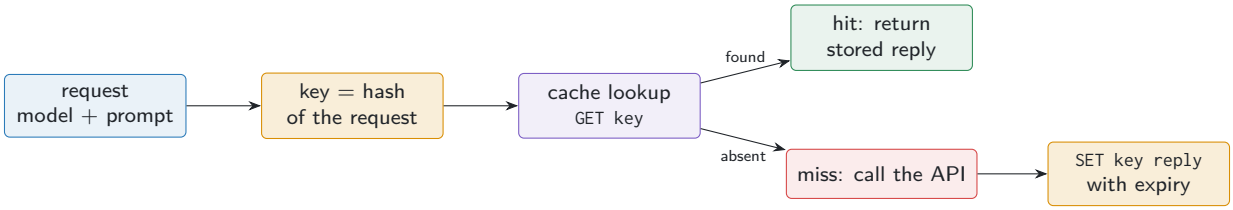
# cache with expiry
r.set("cache:abc123", json.dumps({"text": "hi"}), ex=60)
print(r.ttl("cache:abc123"))                    # about 60
time.sleep(61)
print(r.get("cache:abc123"))                    # None
```

Type	Commands (redis-py method names)
String	`set`, `get`, `setnx` (only if absent), `getset`, `mset`/`mget`, `incr`/`decr`, `incrbyfloat`, `delete`
List	`lpush`/`rpush`, `lpop`/`rpop`, `lrange(k, 0, -1)`, `llen`, `ltrim`, blocking `brpop`
Hash	`hset`, `hget`, `hmget`, `hgetall`, `hkeys`, `hvals`, `hlen`
Set	`sadd`, `smembers`, `srem`, `sinter`, `sunion`, `sdiff`, `scard`
Sorted set	`zadd(k, {member: score})`, `zrange`, `zrank`, `zscore`; use for leaderboards, time series (score = timestamp)
Expiry	`expire(k, seconds)`, `ex=` option of `set`, `ttl`; after that the key is gone

> ■ **TYPE CHECK**
>
> Without `decode_responses=True`, redis-py returns `bytes`: `b'11'`. Numbers come back as text (strings) from `get` even if you stored `11`; `incr` returns an `int`. Hash fields and list items are strings only. For nested data, store `json.dumps(...)` as one string. `smembers` gives a `set`.

Expiry makes Redis a good cache. A common AI use is caching model replies so the same request is not

paid for twice. The flow (*cache-aside*):



The same logic works with a plain dictionary, so it can be tried without a server:

```
import hashlib
import json
import time

cache = {}                                  # key -> (expires_at, value)
calls = 0

def cache_key(model, prompt):
    raw = json.dumps([model, prompt]).encode("utf-8")
    return hashlib.sha256(raw).hexdigest()[:12]

def fake_llm(prompt):                       # stands in for a paid API call
    global calls
    calls += 1
    return prompt.upper()

def ask(model, prompt, ttl=60):
    key = cache_key(model, prompt)
    hit = cache.get(key)
    if hit and hit[0] > time.time():        # found and not expired
        return hit[1]
    value = fake_llm(prompt)
    cache[key] = (time.time() + ttl, value)
    return value

print(ask("m1", "hello"), ask("m1", "hello"), ask("m1", "other"))
print("api calls:", calls, "cache size:", len(cache))
print(cache_key("m1", "hello"))
```

```
HELLO HELLO OTHER
api calls: 2 cache size: 2
f3a65b99fca4
```

> ➤ **RULE: cache keys must contain everything that changes the answer**
>
> Hash the model name, the prompt *and* the sampling settings. Set an expiry so stale answers disappear. Never cache anything that must be fresh or private to one user.

Redis alternative: Valkey

In 2024 Redis moved to a non-open-source license. Valkey is an API-compatible open source fork maintained by Redis core developers. Install `pip install valkey`; the client API mirrors redis-py.

Other NoSQL families

Family	Shape of data / use	Examples (Python package)
Document	JSON-like records with varying fields, nested; the ODM is the ORM of this world	MongoDB (PyMongo), DynamoDB (boto3), CouchDB
Time series	measurements stamped with time (metrics)	InfluxDB, Prometheus client, TimescaleDB (TigerData)
Graph	nodes joined by edges (users who follow users)	Neo4j (`neo4j`), ArangoDB
Wide-column / other	huge data over many machines	Cassandra, HBase
Full-text search	index every word; find the poem about windmills	Elasticsearch, Solr, Whoosh, Lucene
Vector	arrays of numbers (embeddings) for *similarity* search, not exact match	Chroma, pgvector, others
Geospatial	maps and locations	PostGIS, SpatiaLite, GeoPandas

PostgreSQL can imitate several of these: `jsonb` columns store and index JSON documents, arrays store lists in a cell, and the `pgvector` extension adds vector search, all while keeping transactions and foreign keys. For an AI engineer, vector databases are the important one here: they store embeddings and answer "which stored texts are closest in meaning to this one". That is the core of retrieval-augmented generation (RAG).

■ **BOSS FIGHT: choosing storage**

There is no best store; there is the one that matches the shape and the access pattern. Ask: *How big? Who writes and reads, and how many at once? What must survive a crash? Do I query by many fields, or by one key?*

If you need	Start with
a small config, read at startup	TOML or JSON file
a table to exchange or inspect	CSV
queries, joins, one app, local	SQLite
many writers or services	PostgreSQL
a fast cache with expiry	Redis
nearest-neighbour search on embeddings	a vector database or `pgvector`

23.10 Practice

➤ **PRACTICE**

1. Write a CSV file with a column whose values contain commas and quotes. Read it back with `DictReader`. What type is a number column, and how do you convert it?
2. Create a SQLite database file with a table `evals(model, task, score, usd)`. Load rows from a CSV with `executemany`. Commit, reopen the file, and print the three best scores.
3. This query is unsafe: `conn.execute(f"DELETE FROM evals WHERE model = '{m}'")`. Show an input that deletes everything. Rewrite it safely. Why can you not use a placeholder for the table name?
4. Using `models` and `runs` from this chapter, write one query that returns total cost per *vendor*, only for vendors with more than 3000 tokens in total. Which clause filters the buckets?
5. Give an example where `LEFT JOIN` and `JOIN` return different row counts. Which rows are missing in the inner join?
6. (Needs Redis or Valkey.) Cache replies with a 30-second expiry, store run details as a hash, and

show `ttl` counting down.

★ **ACHIEVEMENT UNLOCKED: Persistent Data**

You can now:

- read and write CSV, JSON, TOML and XML and say what types each preserves;
- create SQLite tables and run SELECT, WHERE, ORDER BY, GROUP BY and JOIN queries;
- use parameterized queries, commit and rollback, and explain SQL injection;
- tell the three SQLAlchemy levels apart and know why to learn SQL first;
- pick between a file, SQLite, PostgreSQL, Redis and a vector store for a job.

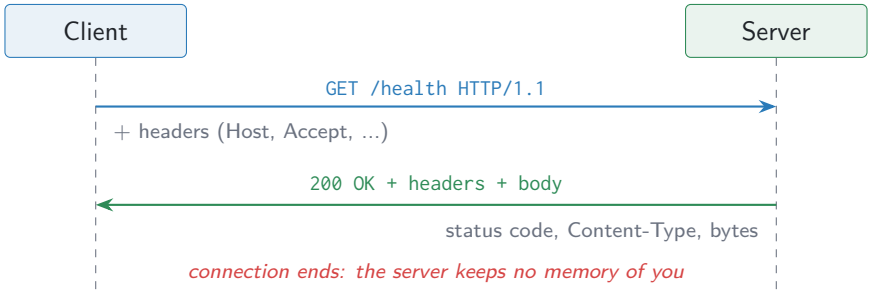
CHAPTER 24

The Web

What this chapter gives you. Almost every AI product you will build talks over HTTP: your code calls a model API, and other code calls your service. This chapter shows how HTTP works, how to write clients and small servers in Python, and how to pull data out of web pages.

24.1 Web Basics

The web is a **client–server** system. A client opens a TCP connection, sends a request in text form (HTTP), and the server sends back a response. TCP/IP only moves bytes; HTTP gives those bytes meaning. The three original ideas were HTTP (the protocol), HTML (a page format) and the URL (an address for a resource).



> ★ **KEY IDEA: HTTP is stateless**
> Every request stands alone. The server does not remember the last one. Caching, shopping carts and logins all need extra tricks, such as **cookies**: a small value the server sends once and the client sends back with each later request.

URL anatomy




```
from urllib.parse import urlparse, parse_qs

u = urlparse("https://api.example.com:8443/v1/answer?q=hi&n=2#top")
print(u.scheme, u.hostname, u.port, u.path)
print(u.query, parse_qs(u.query))
```

```
https api.example.com 8443 /v1/answer
q=hi&n=2 {'q': ['hi'], 'n': ['2']}
```

Note that `parse_qs` gives *lists* of strings, because a key may repeat.

Status codes

Range	Meaning	Examples you will meet
1xx	information	rarely seen by your code
2xx	success	200 OK, 201 Created
3xx	redirect	301/302: the resource lives at another URL
4xx	**your** mistake	400 bad input, 401/403 auth, 404 not found, 429 too many requests
5xx	**their** mistake	500 crash, 502/503 upstream down or overloaded

> ➤ **RULE**
> Treat 429 and 5xx as "try again later" (with a delay) and 4xx as "fix the request". Retrying a 400 forever never helps. Model APIs behave this way too.

Cookies in one example

```
from http.cookies import SimpleCookie

jar = SimpleCookie()
jar["session"] = "abc123"
jar["session"]["path"] = "/"
print(jar.output())
parsed = SimpleCookie("session=abc123; theme=dark")
print({k: m.value for k, m in parsed.items()})
```

```
Set-Cookie: session=abc123; Path=/
{'session': 'abc123', 'theme': 'dark'}
```

24.2 HTTP Testing

HTTP is plain text, so you can poke a server by hand before writing any Python.

Tool	What it is	Use it for
Telnet	old raw TCP terminal; you type the HTTP yourself	seeing the protocol; mostly replaced (and not installed by default on Windows)
curl	the standard command-line web client	quick GET/POST, headers, scripts
HTTPie (`http`)	friendlier curl with JSON by default; `pip install httpie`	readable manual testing
httpbin	a test site that echoes requests back; can run in Docker	safe target for experiments

Here is what Telnet would show you, done with a plain socket against a tiny local service (listed in the next section). The text you send is the whole request:

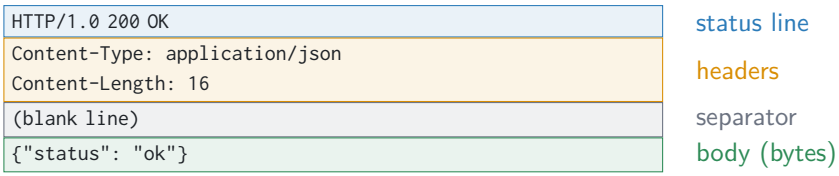
```
import socket

request = (
    "GET /health HTTP/1.1\r\n"
    "Host: localhost\r\n"
    "Connection: close\r\n"
    "\r\n"                      # blank line = end of headers
)
with socket.create_connection(("127.0.0.1", 8765)) as s:
    s.sendall(request.encode("ascii"))
    chunks = []
    while data := s.recv(4096):
        chunks.append(data)
print(b"".join(chunks).decode("ascii"))
```

```
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.12.10
Date: Mon, 02 Nov 2026 17:46:15 GMT
Content-Type: application/json
Content-Length: 16

{"status": "ok"}
```

The first line is the status line. Then come headers, one blank line, then the body. The `Date` line changes on each run.



The same thing with curl, run against that local service (real output):

```
$ curl -s -X POST -H "Content-Type: application/json" \
       -d '{"name":"run-7"}' http://127.0.0.1:8765/runs
{"stored": "run-7"}
$ curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8765/missing
404
```

> ⚠ **TRAP CARD**
>
> JSON needs **double quotes**. A body like `{'a': 1}` is a Python dict literal, not JSON, and a server will reject it. Also, quoting rules differ in PowerShell and cmd; on Windows the simplest fix is to test from Python.

HTTPie sends JSON by default (`http POST url a=1 b=2`) and form data with `-f`. Not run here (needs `pip install httpie`).

24.3 Web Clients

The standard library

The `http` package covers protocol details (`client`, `server`, `cookies`). `urllib` sits on top (`request`, `response`, `parse`, `error`). First, the stub service the next examples call. It plays the role of a tiny model API: it has a health check, an `/answer` endpoint, and a POST route.

```
# web_server.py - a stub service on localhost (stdlib only)
import json
import threading
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from urllib.parse import urlparse, parse_qs


class Handler(BaseHTTPRequestHandler):
    def _send(self, status, payload):
        body = json.dumps(payload).encode("utf-8")
        self.send_response(status)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def do_GET(self):
        url = urlparse(self.path)
        if url.path == "/health":
            self._send(200, {"status": "ok"})
        elif url.path == "/answer":
            q = parse_qs(url.query).get("q", [""])[0]
            self._send(200, {"question": q, "answer": q.upper(),
                             "tokens": len(q.split())})
        else:
            self._send(404, {"error": "no such route"})

    def do_POST(self):
        size = int(self.headers.get("Content-Length", 0))
        data = json.loads(self.rfile.read(size))
        self._send(201, {"stored": data["name"]})

    def log_message(self, *args):    # silence the access log
        pass


server = ThreadingHTTPServer(("127.0.0.1", 8765), Handler)
threading.Thread(target=server.serve_forever, daemon=True).start()
BASE = "http://127.0.0.1:8765"
```

Now a GET, looking at every type along the way:

```
import json
import urllib.request
from web_server import BASE

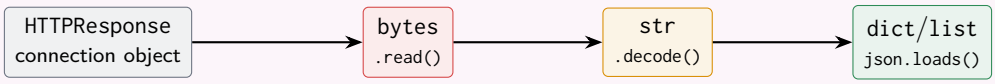
with urllib.request.urlopen(f"{BASE}/health") as resp:
    print(type(resp).__name__, resp.status)
    print(resp.headers["Content-Type"])
    raw = resp.read()

print(type(raw), raw)
text = raw.decode("utf-8")
print(type(text), text)
data = json.loads(text)
print(type(data), data["status"])
```

```
HTTPResponse 200
application/json
<class 'bytes'> b'{"status": "ok"}'
<class 'str'> {"status": "ok"}
<class 'dict'> ok
```

■ TYPE CHECK

The body travels as **bytes**. Turning it into something usable takes three steps, and each has its own type:



`json.load(resp)` (no s) reads and parses a file-like object in one go. Calling `resp.read()` twice returns empty bytes the second time: the stream is used up.

Query strings, POST bodies, errors and timeouts:

```python
import json
import urllib.request
import urllib.error
from urllib.parse import urlencode
from web_server import BASE

query = urlencode({"q": "what is a token?"})
print(query)
with urllib.request.urlopen(f"{BASE}/answer?{query}", timeout=5) as resp:
    reply = json.load(resp)
print(reply)

body = json.dumps({"name": "run-42"}).encode("utf-8")
req = urllib.request.Request(
    f"{BASE}/runs",
    data=body,
    headers={"Content-Type": "application/json"},
    method="POST",
)
with urllib.request.urlopen(req, timeout=5) as resp:
    print(resp.status, json.load(resp))

try:
    urllib.request.urlopen(f"{BASE}/nope", timeout=5)
except urllib.error.HTTPError as err:
    print(err.code, err.reason, json.load(err))
```

```
q=what+is+a+token%3F
{'question': 'what is a token?', 'answer': 'WHAT IS A TOKEN?', 'tokens': 4}
201 {'stored': 'run-42'}
404 Not Found {'error': 'no such route'}
```

⚠ TRAP CARD

`urllib` *raises* `HTTPError` on 4xx/5xx, and the error object is itself a readable response (you can parse its JSON body). Never build query strings by hand with f-strings: use `urlencode`, which escapes spaces and `?`. And always pass `timeout`; without one, a stalled server can hang your program forever.

Requests

`requests` is the popular third-party client. It needs `pip install requests`; the snippets below were syntax-checked only (not run, since no network calls are allowed here).

```
import requests

resp = requests.get(
    "http://127.0.0.1:8765/answer",
    params={"q": "what is a token?"},    # encoded for you
    timeout=5,
)
resp.raise_for_status()      # raises on 4xx/5xx; requests does NOT by default
data = resp.json()           # parsed JSON: a dict here
```

■ TYPE CHECK

Expression	Type	Note
`resp`	`Response`	prints as `<Response [200]>`; truthy if status < 400
`resp.status_code`	`int`	200, 404, ...
`resp.content`	`bytes`	raw body
`resp.text`	`str`	body decoded with the guessed encoding
`resp.json()`	`dict` or `list`	raises if body is not JSON

A `Session` reuses the TCP connection between calls, which is why it is faster when you call the same host many times (the book measured about 3.5x faster than separate calls).

```
import requests

with requests.Session() as s:
    s.headers.update({"Authorization": "Bearer <your-key>"})
    for q in ["a", "b", "c"]:
        r = s.get("http://127.0.0.1:8765/answer", params={"q": q}, timeout=5)
```

Other clients: httpx and aiohttp

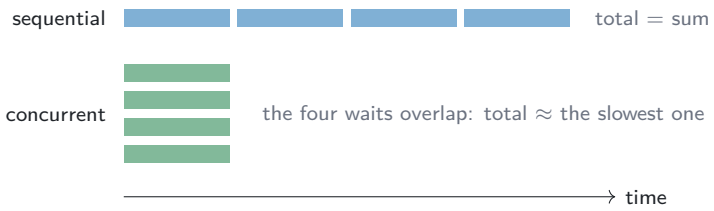
Library	Sync / async	Pick it when
`urllib`	sync, stdlib	no installs allowed; tiny scripts
`requests`	sync only	simple blocking calls
`httpx`	both, `requests`-like API	you want sync now and async later; the Anthropic SDK is built on it
`aiohttp`	async only (also a server)	purely async programs

Async pays off when you wait on many remote hosts: most of the time is waiting for I/O, not computing. Here is the shape of an async fan-out (syntax-checked; needs `pip install httpx`):

```python
import asyncio
import httpx

async def ask_all(questions):
    async with httpx.AsyncClient(timeout=10) as client:
        tasks = [
            client.get("http://127.0.0.1:8765/answer", params={"q": q})
            for q in questions
        ]
        responses = await asyncio.gather(*tasks)
    return [r.json() for r in responses]

answers = asyncio.run(ask_all(["one", "two", "three"]))
```



▲ **LAB RESULT** measured, not guessed

Ten stand-in "API calls" that each wait 0.1 s (a `sleep`), run three ways on this machine: sequential 1.0 s, thread pool 0.1 s, `asyncio.gather` 0.1 s. Waiting overlaps; computing does not (see Chapter 21).

24.4 Web Servers

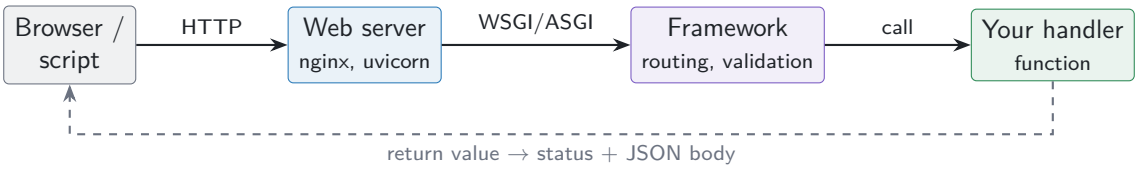
The simplest Python web server

```
$ python -m http.server 9999
Serving HTTP on 0.0.0.0 port 9999 ...
```

It serves files from the current folder and logs each hit as `127.0.0.1 - - [date] "GET / HTTP/1.1" 200 -` (client IP, two unused fields, time, request line, status). Stop with Ctrl+C. `0.0.0.0` means "listen on every network interface"; for tests prefer `–bind 127.0.0.1` so only your machine can reach it. Use it for quick demos only: it is slow and cannot run dynamic code.

From CGI to WSGI to ASGI

Era	How server and Python code connect	Problem / benefit
CGI	start a new program per request	slow: startup cost every time
Embedded (`mod_php`, `mod_python`)	interpreter lives inside the web server	fast, but tied to that server
WSGI	standard sync interface between server and Python app	portable; one request at a time per worker
ASGI	the async counterpart of WSGI	many waiting requests per worker (`async`/`await`)



Apache, nginx and other servers

- **Apache** runs WSGI apps through `mod_wsgi` (embedded or daemon mode). Most of the work is editing Apache's config.
- **nginx** has no embedded Python. It sits in front and forwards requests to a separate WSGI server such as uWSGI or Gunicorn.
- **Other Python servers** fall in two groups: multi-process/multi-thread (CherryPy, Waitress) and event-based (Tornado, gevent, Gunicorn workers). For ASGI apps, `uvicorn` is the usual choice.

24.5 Web Server Frameworks

A web server speaks HTTP; a **framework** is what you write against. The main jobs:

Job	Meaning
Routing	map a URL and verb to a Python function
Parameters	read path, query and body values; validate type/range
Auth	authentication (who are you?) and authorization (what may you do?)
Sessions	keep state across requests
Responses	return data plus the right status code
Static files, templates	serve files; fill HTML with values

Bottle

One file, no dependencies (`pip install bottle`). The decorator binds a URL to a function; `<thing>` captures part of the path.

```
from bottle import route, run

@route("/")
def home():
    return "model service is up"

@route("/greet/<name>")
def greet(name):
    return f"Hello, {name}!"

run(host="localhost", port=9999)
```

Flask

Flask is Bottle's bigger cousin: more extensions, Jinja2 templates. Needs `pip install flask`. Path values and query values arrive differently:

```python
from flask import Flask, request, render_template_string

app = Flask(__name__)

@app.route("/greet/<name>")            # path parameter
def greet(name):
    return f"Hello, {name}!"

@app.route("/search")                  # /search?q=cats&top=3
def search():
    q = request.args.get("q", "")      # query parameters: strings!
    top = int(request.args.get("top", 5))
    page = "<p>{{ q }} (top {{ top }})</p>"      # Jinja2 template
    return render_template_string(page, q=q, top=top)

if __name__ == "__main__":
    app.run(port=9999, debug=True)
```

> ⚠ **TRAP CARD**
>
> `debug=True` shows an interactive debugger page. Never use it on a server reachable from the internet. And query values are always *strings*; convert them yourself (`int(...)`), or use a framework that validates (below).

Django, FastAPI, Litestar

Framework	Style	Best for
Bottle	one file, minimal	learning, tiny tools
Flask	small core + extensions, sync (WSGI)	simple sites and APIs
Django	"batteries included": ORM, admin pages, auth	large database-backed sites; common in job ads
FastAPI	ASGI, type hints, auto docs, Pydantic validation	JSON APIs, **model-serving services**
Litestar	similar to FastAPI, more built-ins	alternative to FastAPI

FastAPI is the one an AI engineer will use most. You describe the request and response with Pydantic models; FastAPI validates input and builds interactive docs at `/docs`. Needs `pip install fastapi uvicorn`; syntax-checked, not run. The model call is a stub.

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class AskIn(BaseModel):
    question: str
    max_tokens: int = 256

class AskOut(BaseModel):
    answer: str

def ask_model(question: str, max_tokens: int) -> str:
    return f"stub answer to: {question}"     # swap in a real SDK call

@app.post("/ask", response_model=AskOut)
def ask(body: AskIn) -> AskOut:
    return AskOut(answer=ask_model(body.question, body.max_tokens))

# run:  uvicorn app:app --reload      then open http://127.0.0.1:8000/docs
```

What the validation does, shown with Pydantic alone (this part really ran):

```python
from pydantic import BaseModel, ValidationError

class AskIn(BaseModel):
    question: str
    max_tokens: int = 256

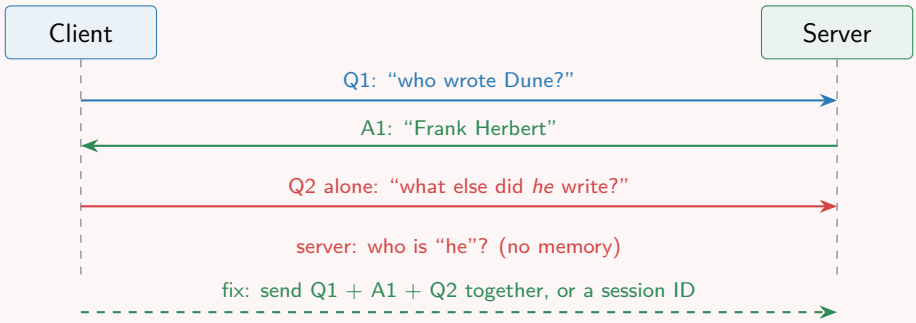
print(AskIn(question="hi"))
print(AskIn(question="hi", max_tokens="300"))     # str "300" -> int 300
try:
    AskIn(question="hi", max_tokens="lots")
except ValidationError as e:
    print(e.error_count(), e.errors()[0]["type"], e.errors()[0]["loc"])
```

```
question='hi' max_tokens=256
question='hi' max_tokens=300
1 int_parsing ('max_tokens',)
```

In FastAPI, bad input like that returns a **422** response automatically; your function never runs. Compare Flask, where you would write those checks by hand.

> ⚠ **TRAP CARD**
>
> Inside `async def` handlers, a blocking call (`requests.get`, `time.sleep`, a synchronous SDK call) freezes the whole event loop and every other request. Either use plain `def` (FastAPI runs it in a thread) or use the async client.

> ■ **BOSS FIGHT: statelessness is why chat needs a history list**
>
> HTTP forgets you between requests. A web service that answers questions is the same: if the second request does not carry the first question, the server has no idea what "it" refers to. The fix is always the same: either the **client sends the context each time** (as model APIs do with the message history), or the server **stores state** and finds it again using an ID the client sends back (a cookie, session ID or conversation ID).



24.6 Database Frameworks

Web and databases go together. Options, from most work to least:

Approach	What you do
Build it yourself	a framework (Bottle/Flask) + a DB package (DB-API, SQLAlchemy) + a driver (PyMySQL)
Web/DB combo tools	Connexion, Datasette, sandman2, Flask-Restless expose a database as an API
Framework with a built-in ORM	Django (also gives automatic admin pages for internal use)
Schemaless store	a NoSQL database (Chapter 23) when row shapes differ a lot

24.7 Web Services and Automation

webbrowser and webview

`webbrowser.open(url)` asks your own browser to show a page; it returns `True` if it could try. (`import antigravity` is a joke that calls it.) `webview` (third party, `pywebview`) instead shows the page in its own native window.

```
import webbrowser
webbrowser.open("http://127.0.0.1:8000/docs")    # opens your default browser
```

Web APIs and REST

Instead of HTML for people, a web API returns data (usually JSON) for programs. A **RESTful** service uses HTTP verbs in set ways, so the *verb* says what you do and the *URL* says to what.

Verb	Meaning	Changes data?	Example
GET	read a resource	no	`GET /runs/42`
HEAD	headers only, no body	no	`HEAD /runs/42`
POST	create a resource	yes	`POST /runs`
PUT	replace (or create)	yes	`PUT /runs/42`
PATCH	change part of it	yes	`PATCH /runs/42`
DELETE	remove it	yes	`DELETE /runs/42`

> ➤ **RULE**
> GET must never change data. Caches, browsers and crawlers assume GET is safe to repeat.

Model APIs are exactly this: a POST with a JSON body (model name, messages) returns a JSON body (the reply, usage counts). The SDK wraps it, but the traffic is plain HTTP.



WebSockets

HTTP is request-then-response, and the server cannot start talking. A **WebSocket** keeps one two-way connection open: good for chat, live notifications, collaboration and games. The book's example uses the third-party `websockets` package (`pip install websockets`); the version below is my own, syntax-checked but not run.

```
import asyncio
from websockets.asyncio.server import serve

async def shout(ws):
    async for message in ws:                # one connection, many messages
        await ws.send(message.upper())

async def main():
    async with serve(shout, "localhost", 8765):
        await asyncio.get_running_loop().create_future()   # run forever

asyncio.run(main())
```

FastAPI can host WebSocket routes beside normal ones. Note that LLM APIs usually stream tokens with **server-sent events** over normal HTTP (one-way, server to client), which is simpler than a WebSocket.

Webhooks

A webhook flips the direction: you give a service a URL, and *it* calls you (as an HTTP client) when something happens, such as a finished batch job. Your service just needs a POST route that returns 2xx quickly. FastAPI documents this well.

24.8 Web Frontends

A frontend faces the user; it is normally HTML, CSS and JavaScript talking to a backend by API. Two Python-friendly ways to avoid writing much JavaScript:

Tool	Idea
htmx	a small JavaScript library driven by HTML attributes: any element can send a request on any event and put the returned HTML into any part of the page
FastHTML	Python functions such as `Div()` and `P()` build the HTML, so the whole app is Python (`pip install python-fasthtml`)

```
from fasthtml.common import FastHTML, serve

app = FastHTML()

@app.get("/")
def home():
    return "<p>Yes, it works.</p>"

serve()          # dev server, default http://localhost:5001
```

Syntax-checked only. For data dashboards, Chapter 25 lists Python-only tools such as Streamlit.

24.9 Crawl and Scrape

A **crawler** (spider) fetches pages automatically; a **scraper** pulls out the data you want. Use an API if one exists; scrape only when data is on the page and nowhere else.



Tool	Role
Scrapy	full crawl-and-scrape framework; powerful but more setup
Beautiful Soup (`beautifulsoup4`)	parses HTML you already have, even broken HTML
requests-html	fetch + parse in one package
`html.parser` (stdlib)	enough for simple jobs

Do not parse HTML with regular expressions; real pages are messy. A runnable stdlib version that collects links:

```
from html.parser import HTMLParser

class LinkFinder(HTMLParser):
    def __init__(self):
        super().__init__()
        self.links = []

    def handle_starttag(self, tag, attrs):
        if tag == "a":
            self.links.append(dict(attrs).get("href"))

page = """<html><body>
<a href="/docs">Docs</a> <p>text <a href="https://x.test/a">A</a>
<a name="anchor-only">no href</a>
</body></html>"""
finder = LinkFinder()
finder.feed(page)
print(finder.links)
```

```
['/docs', 'https://x.test/a', None]
```

Note the `None`: an <a> tag without `href`. Filter it out before using the list. With Beautiful Soup the same job is shorter (needs `pip install beautifulsoup4`; syntax-checked):

```
from bs4 import BeautifulSoup

def get_links(html: str) -> list[str]:
    doc = BeautifulSoup(html, features="html.parser")
    return [a["href"] for a in doc.find_all("a", href=True)]
```

> ⚠ **TRAP CARD**
> Be a polite scraper: read `robots.txt`, slow down, and check the site's terms. The stdlib can parse the rules:

```
from urllib.robotparser import RobotFileParser

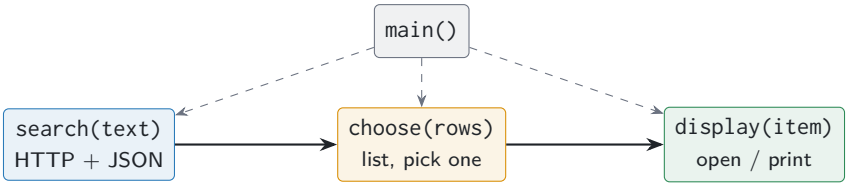
rules = ["User-agent: *", "Disallow: /private/", "Crawl-delay: 5"]
rp = RobotFileParser()
rp.parse(rules)
print(rp.can_fetch("my-bot", "https://site.test/docs/a"))
print(rp.can_fetch("my-bot", "https://site.test/private/x"))
print(rp.crawl_delay("my-bot"))
```

```
True
False
5
```

For AI work, scraped text often becomes the input for chunking and embeddings, so clean HTML to plain text early and keep the source URL with every piece.

24.10 Let's Watch a Movie: a client in four small functions

The book ends with a full program: search an API, list results, let the user pick one, open it in the browser. The lesson is its *shape*: one small function per job. Here is the same shape against our local stub (it ran).




```
import json
import urllib.request
from urllib.parse import urlencode
from web_server import BASE


def search(text):
    url = f"{BASE}/answer?{urlencode({'q': text})}"
    with urllib.request.urlopen(url, timeout=5) as resp:
        return json.load(resp)


def choose(rows):
    for num, row in enumerate(rows):
        print(f"{num}: {row['answer']} ({row['tokens']} words)")
    return rows[0]            # a real program would call input()


def main():
    rows = [search(t) for t in ("hello world", "what is rag")]
    picked = choose(rows)
    print("picked:", picked["question"])


main()
```

```
0: HELLO WORLD (2 words)
1: WHAT IS RAG (3 words)
picked: hello world
```

Most real APIs require a key, sent in a header and never hard-coded. Anonymous APIs are rare because free open resources get abused. Keep keys in environment variables.

24.11 Practice and wrap-up

The web is clients, servers and the services between them. As an AI engineer you will mostly write JSON clients and small FastAPI-style services.

➤ **PRACTICE**

1. Using only `urllib`, write a function that GETs a JSON URL with a query dict and a timeout, and returns a dict. What type does each step return?
2. A request returns status 429, another 404, another 503. Which would you retry, and why?
3. Write a Flask or FastAPI route `/echo` that takes `thing`, `height` and `color` as query parameters and returns them as JSON. Which of them must you convert?
4. Why can a server not answer "and what about the second one?" by itself? Name two fixes.
5. When is `httpx` with `asyncio.gather` faster than a plain loop of `requests`, and when is it not?
6. Extract all `href` values from an HTML string with `html.parser`, skipping tags that have none.

★ **ACHIEVEMENT UNLOCKED: Web client and server**

You can now:

- read a raw HTTP request and response and name each part;
- call a JSON API from Python and handle errors, timeouts and the bytes/str/dict steps;
- choose between WSGI and ASGI frameworks and sketch a validated FastAPI endpoint;
- explain why HTTP is stateless and how chat history works around it;
- pull links out of HTML and respect `robots.txt`.

CHAPTER 25

Data Science

What this chapter gives you. Model work is mostly data work: reading files, cleaning rows, comparing numbers, and turning text into vectors. Here you meet the standard-library tools for numbers, then NumPy (fast arrays) and pandas (tables), which almost every AI library builds on.



each layer is built on the one before it; ML libraries (PyTorch, scikit-learn) speak NumPy

25.1 Standard Python

Sorting with itemgetter and attrgetter

`sorted(items, key=...)` wants a function that picks the sort value. `itemgetter` builds one for indexes and dict keys; `attrgetter` does it for object attributes. Several names break ties in order.

```
from operator import itemgetter, attrgetter
from dataclasses import dataclass

runs = [
    {"model": "haiku", "score": 0.81, "cost": 0.4},
    {"model": "opus", "score": 0.93, "cost": 3.1},
    {"model": "sonnet", "score": 0.90, "cost": 1.2},
]
by_score = sorted(runs, key=itemgetter("score"), reverse=True)
print([r["model"] for r in by_score])
get_two = itemgetter("model", "cost")      # several keys -> a tuple
print(get_two(runs[0]))

@dataclass
class Run:
    model: str
    score: float

objs = [Run("b", 0.5), Run("a", 0.9)]
print(sorted(objs, key=attrgetter("score")))
```

```
['opus', 'sonnet', 'haiku']
('haiku', 0.4)
[Run(model='b', score=0.5), Run(model='a', score=0.9)]
```

Format conversions

Chapters 17, 18 and 23 covered text, binary and database formats. The standard library reads CSV and JSON; pandas (below) handles many more. One trap to remember: CSV values come back as *strings*.

```
import csv, io, json

text = "model,score\nhaiku,0.81\nopus,0.93\n"
rows = list(csv.DictReader(io.StringIO(text)))
print(rows)
print(type(rows[0]["score"]))
for r in rows:
    r["score"] = float(r["score"])       # convert yourself
print(json.dumps(rows))
print(json.loads(json.dumps(rows)) == rows)
```

```
[{'model': 'haiku', 'score': '0.81'}, {'model': 'opus', 'score': '0.93'}]
<class 'str'>
[{"model": "haiku", "score": 0.81}, {"model": "opus", "score": 0.93}]
True
```

Math

`import math` gives constants (`pi`, `e`, `inf`, `nan`) and functions. Behaviours worth knowing:

```
import math
print(math.pi, math.e)
print(math.floor(-2.5), math.ceil(-2.5), round(2.5), round(3.5))
print(math.log(8, 2), math.log2(8), math.log10(1000))
print(2**3, 2.0**3, math.pow(2, 3))
print(math.hypot(3, 4), math.sqrt(16))
try:
    math.sqrt(-1)
except ValueError as e:
    print("ValueError:", e)
print(math.isclose(0.1 + 0.2, 0.3), 0.1 + 0.2 == 0.3, 0.1 + 0.2)
print(math.gcd(24, 16), math.factorial(5))
print(math.inf > 10**100, math.isnan(math.nan), math.nan == math.nan)
```

```
3.141592653589793 2.718281828459045
-3 -2 2 4
3.0 3.0 3.0
8 8.0 8.0
5.0 4.0
ValueError: math domain error
True False 0.30000000000000004
8 120
True True False
```

> ⚠ **TRAP CARD**
> Never compare floats with `==`. Use `math.isclose`. Also: `round(2.5)` is 2 and `round(3.5)` is 4 (round-half-to-even), `nan == nan` is `False`, and `**` with two ints stays an int while `math.pow` always returns a float. The trig functions (`sin`, `cos`, `atan2`, ...) and `radians`/`degrees` are there when you need them.

Complex numbers, Decimal and Fraction

```
print(3 + 2j, (3 + 2j) * 1j, 1j * 1j)
z = 3 + 4j
print(z.real, z.imag, abs(z))

from decimal import Decimal
price, tax = Decimal("19.99"), Decimal("0.06")      # build from STRINGS
total = price + price * tax
print(total, total.quantize(Decimal("0.01")))
print(Decimal(0.1))                  # building from a float keeps its error
print(0.1 + 0.2, Decimal("0.1") + Decimal("0.2"))

from fractions import Fraction
print(Fraction(1, 3) * Fraction(2, 3), Fraction(3, 6), Fraction("0.25"))
```

```
(3+2j) (-2+3j) (-1+0j)
3.0 4.0 5.0
21.1894 21.19
0.1000000000000000055511151231257827021181583404541015625
0.30000000000000004 0.3
2/9 1/2 1/4
```

Type	Good for	Watch out
`float`	almost all science and ML math	tiny binary rounding errors
`Decimal`	money, exact decimal rules	slower; make it from `str`, not float
`Fraction`	exact ratios (`2/9`)	grows big; rarely needed in ML
`complex`	signal processing, `cmath` functions	not used in everyday AI code

> ⚠ **TRAP CARD**
> The book calls `fractions.gcd`. That function was removed in Python 3.9; importing it raises `ImportError`. Use `math.gcd`. Also the book shows `np.float` for NumPy dtypes: that alias was removed from NumPy; use `float` or `np.float64`.

array, statistics and the @ operator

`array.array` packs one numeric type tightly (less memory than a list of Python objects), but it has no math on whole arrays. For real numeric work use NumPy.

```
import sys
from array import array
import statistics as st

nums = list(range(1000))
packed = array("i", nums)
print(sys.getsizeof(nums), sys.getsizeof(packed))   # list = pointers only
print(packed.itemsize, packed.typecode, packed[:3])
try:
    packed.append(1.5)
except TypeError as e:
    print("TypeError:", e)

lat = [120, 135, 128, 410, 131, 125]      # latencies in ms, one outlier
print(st.mean(lat), st.median(lat), round(st.stdev(lat), 1))
print(st.mode(["a", "b", "a"]))
print(st.quantiles(lat, n=4))
```

```
8056 4080
4 i array('i', [0, 1, 2])
TypeError: 'float' object cannot be interpreted as an integer
174.83333333333334 129.5 115.3
a
[123.75, 129.5, 203.75]
```

The outlier (410 ms) drags the mean far above the median. When latency or cost has outliers, report the median or a percentile.

The `@` operator means matrix multiply, but only for types that define it. Plain lists do not; NumPy arrays do.

```
a = [[1, 2], [3, 4]]
try:
    a @ a
except TypeError as e:
    print("TypeError:", e)
```

```
TypeError: unsupported operand type(s) for @: 'list' and 'list'
```

25.2 NumPy

NumPy's `ndarray` is a block of memory holding values of **one type** next to each other. That makes math over whole arrays fast, and it is the data format under embeddings, images and tensors. (Needs `pip install numpy`. NumPy is not installed in this notes environment, so the outputs below are labeled *example output*; I checked the numbers by hand or with plain Python.)

	Python `list`	**NumPy `ndarray`**
Element types	anything, mixed	one `dtype` (float64, int64, ...)
Memory	pointers to separate objects	one contiguous block
`x * 3`	repeats the list 3 times	multiplies every element by 3
`x + y`	joins two lists	adds element by element
2D indexing	`m[1][2]`	`m[1, 2]` (and `m[1][2]` also works)
Math functions	write a loop or comprehension	whole-array operations

Making arrays and reading their shape

Three attributes describe every array: `ndim` (number of dimensions; the book calls it the rank), `shape` (length along each dimension, a tuple) and `size` (total elements).

```
import numpy as np

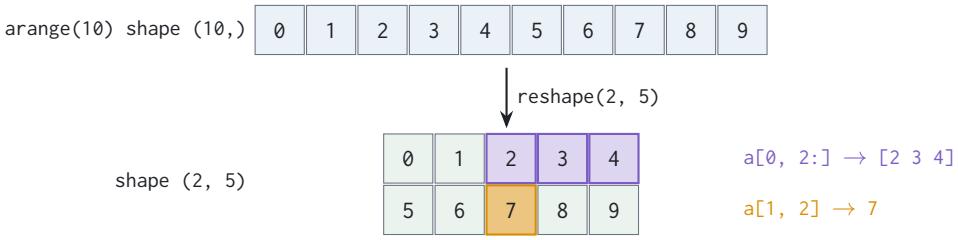
b = np.array([2, 4, 6, 8])          # from a list
print(b, b.ndim, b.shape, b.size)
a = np.arange(7, 11)                # like range(), but an array
f = np.arange(2.0, 3.0, 0.25)       # floats are fine
z = np.zeros((2, 4))                # shape is ONE tuple argument
o = np.ones((3, 5))
rng = np.random.default_rng(0)      # modern random generator
r = rng.random((3, 5))              # values in [0.0, 1.0)
print(z.shape, o.size, r.shape)
```

```
[2 4 6 8] 1 (4,) 4
(2, 4) 15 (3, 5)
```

Example output.

> **⚠ TRAP CARD**
> `np.zeros(2, 4)` is an error: the shape goes in one tuple, `np.zeros((2, 4))`. And `arange` with a float step can produce an unexpected number of items because of rounding; prefer `np.linspace(start, stop, num)` when you care about the count.

Reshape and indexing




```
a = np.arange(10).reshape(2, 5)     # same 10 values, new shape
print(a[1, 2], a[-1, :3], a[0, 2:])
a[:, 2:4] = 1000                    # assign to a slice of every row
print(a)
b = a.reshape(5, 2)                 # sizes must multiply to 10
# a.reshape(3, 4) -> ValueError: cannot reshape array of size 10 ...
```

```
7 [5 6 7] [2 3 4]
[[   0    1 1000 1000    4]
 [   5    6 1000 1000    9]]
```

Example output.

> **⚠ TRAP CARD**
> Slices and `reshape` return **views**, not copies: they share memory with the original. Changing the view changes the source. Call `.copy()` when you need independence. With plain lists, a slice is a new list, so the habit does not carry over.

Array math: no loops

```
arr = np.arange(4)
arr = arr * 3                   # every element at once
print(arr)                      # [0 3 6 9]
print([1, 2] * 3)               # a LIST repeats instead
print(np.zeros((2, 5)) + 17.0)  # one number added to all ten
scores = np.array([0.2, 0.9, 0.4, 0.95])
print(scores[scores > 0.5])     # boolean mask: keep matching items
```

```
[0 3 6 9]
[1, 2, 1, 2, 1, 2]
[[17. 17. 17. 17. 17.]
 [17. 17. 17. 17. 17.]]
[0.9  0.95]
```

Example output (the list line was run in plain Python).

TYPE CHECK

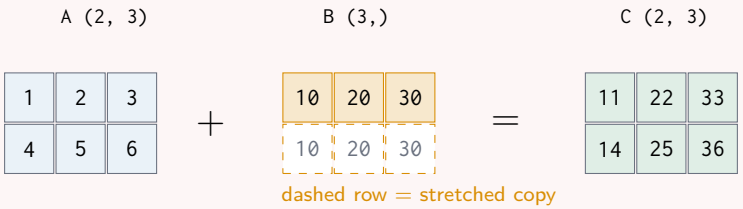
Same-looking values, different behaviour:

- `[1, 2] * 3` is a `list` of six items; `np.array([1, 2]) * 3` is an `ndarray` `[3 6]`.
- `type(x)` tells you which you have. `x.shape` exists only on arrays, not lists.
- `np.array(my_list)` converts a list; `arr.tolist()` converts back (needed for `json.dumps`, which cannot encode an `ndarray`).

Shapes and broadcasting

BOSS FIGHT: broadcasting

When two arrays have different shapes, NumPy *stretches* the smaller one to fit, as long as each dimension matches or is 1 (compared from the right). It does not copy the data; it behaves as if it had. Most shape errors in ML code come from not knowing the shape. Print `.shape` early and often.




```
A = np.array([[1, 2, 3], [4, 5, 6]])    # shape (2, 3)
B = np.array([10, 20, 30])              # shape (3,)
print(A + B)                            # B is applied to every row
# np.ones((2, 3)) + np.ones((2,))  -> ValueError: shapes (2,3) (2,) ...
```

```
[[11 22 33]
 [14 25 36]]
```

Example output.

Linear algebra, and an embedding example

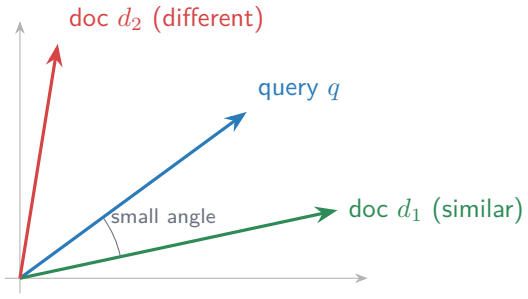
The book solves the system $4x + 5y = 20,\ x + 2y = 13$. Here is the same method on my own system: $2x + y = 5$ and $x - y = 1$ (solution $x = 2,\ y = 1$).

```
coef = np.array([[2, 1], [1, -1]])
rhs = np.array([5, 1])
sol = np.linalg.solve(coef, rhs)
print(sol)
print(np.allclose(coef @ sol, rhs))     # check; @ is matrix multiply
```

```
[2. 1.]
True
```

Example output.

Use `np.allclose` for float comparison, as with `math.isclose`. A far more common job in AI work is comparing two embedding vectors by the angle between them (**cosine similarity**):



$$\cos(q, d) \;=\; \frac{q \cdot d}{\|q\|\,\|d\|}$$

A value near 1 means the vectors point the same way. I checked this calculation in plain Python: for $q = (1, 2, 2)$ and $d = (2, 0, 1)$ the dot product is 4, the norms are 3 and $\sqrt{5}$, and the result is 0.5963.

```
import numpy as np

def cosine(q: np.ndarray, d: np.ndarray) -> float:
    return float(q @ d / (np.linalg.norm(q) * np.linalg.norm(d)))

q = np.array([1.0, 2.0, 2.0])
d = np.array([2.0, 0.0, 1.0])
print(round(cosine(q, d), 4))
```

```
0.5963
```

Example output (matches the plain-Python check).

NumPy also has modules for polynomials, Fourier transforms, statistics and random distributions.

25.3 SciPy

SciPy sits on NumPy and adds ready-made scientific algorithms. You will rarely import it for LLM work, but it is behind many ML tools.

Area	What you get
Optimization	minimize a function; curve fitting
Statistics	distributions, hypothesis tests
Linear regression, interpolation	fit and fill in values
Integration	numerical integrals
Signal and image processing	filters, transforms

Python plus NumPy plus SciPy covers much of what MATLAB or R do. (`pip install scipy`.)

25.4 pandas

pandas handles messy real-world tables: many column types, missing values, dates, and many file formats (CSV, TSV, fixed-width text, Excel, JSON, HTML tables, SQL, HDF5). Its core type is the **DataFrame**, a table with labelled rows (the *index*) and typed columns. One column is a **Series**.

index	model	task	score	latency_ms
0	haiku	summarize	0.81	420
1	haiku	extract	0.88	380
2	sonnet	summarize	0.90	910
3	sonnet	extract	0.94	870

whole grid = **DataFrame** `df` green column = one **Series**: `df["score"]`

Reading a CSV and looking around

I use a small, made-up table of evaluation results. `read_csv` accepts a path, a URL, or any file-like object (here an in-memory string). Install with `pip install pandas`. Not run here; the outputs follow pandas' standard display and I computed the numbers with plain Python.

```
import io
import pandas as pd

csv_text = """model,task,score,latency_ms
haiku,summarize,0.81,420
haiku,extract,0.88,380
sonnet,summarize,0.90,910
sonnet,extract,0.94,870
opus,summarize,0.93,2100
opus,extract,0.97,1980
"""
df = pd.read_csv(io.StringIO(csv_text))     # a DataFrame
print(df)
```

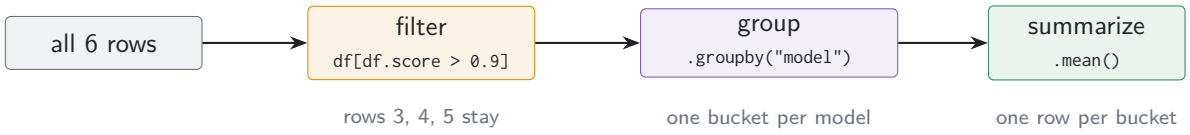
```
    model       task  score  latency_ms
0   haiku  summarize   0.81         420
1   haiku    extract   0.88         380
2  sonnet  summarize   0.90         910
3  sonnet    extract   0.94         870
4    opus  summarize   0.93        2100
5    opus    extract   0.97        1980
```

Example output.

■ TYPE CHECK

Expression	Type	Meaning
`df`	`DataFrame`	the whole table
`df["score"]`	`Series`	one column (1-D, with an index)
`df[["score"]]`	`DataFrame`	double brackets: a one-column table
`df.loc[0]`	`Series`	one row, by index label
`df["score"].to_numpy()`	`ndarray`	the raw values, for NumPy or a model
`df.shape`	`tuple`	`(6, 4)`: rows, columns

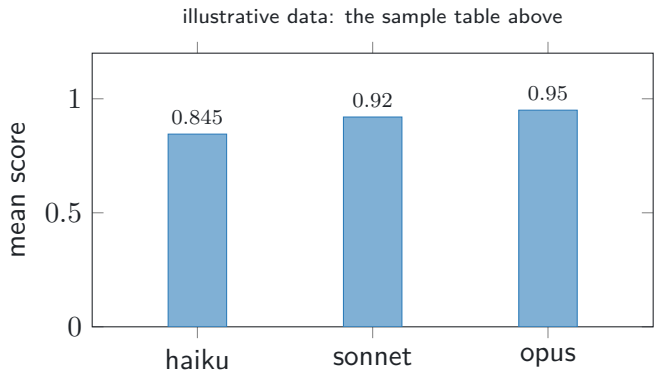
Filter, group, summarize




```
good = df[df["score"] > 0.9]                    # boolean mask on rows
print(list(good.index))                         # [3, 4, 5]
means = df.groupby("model")[["score", "latency_ms"]].mean()
print(means)
print(round(df["score"].mean(), 3))             # Series method -> float
```

```
[3, 4, 5]
        score  latency_ms
model
haiku   0.845       400.0
opus    0.950      2040.0
sonnet  0.920       890.0
0.905
```

Example output.



➤ RULE

Compute on whole columns (`df["score"] > 0.9`, `groupby`, `mean`) instead of looping over rows with `for`. It is shorter and far faster, for the same reason NumPy beats a list loop.

⚠ TRAP CARD

A boolean filter is an expression, not a statement: `df["score"] > 0.9` gives a Series of `True`/`False`; you must pass it back into `df[...]` to get rows. Also, `df["new"] = ...` on a *filtered* frame can trigger the `SettingWithCopyWarning` or silently fail; take `.copy()` first. And a column holding numbers

> read as text (e.g. `"0.81"`) will sort and compare wrongly: check `df.dtypes`.

Dates

`pd.date_range("2025-01-01", periods=3, freq="MS")` returns a `DatetimeIndex` of the first day of three months. It handles calendar details (month ends, business days) that are painful to write by hand. Note that pandas calls frequency codes by letters such as `MS` (month start); check the docs if one is deprecated in your version.

25.5 Polars

Polars is a newer DataFrame library written in Rust. It can use all CPU cores and handle data bigger than memory (lazy mode). Its API differs from pandas: you build *expressions* with `pl.col(...)`. (`pip install polars`; syntax-checked only.)

```
import polars as pl

df = pl.read_csv("results.csv")
best = (
    df.filter(pl.col("score") > 0.9)
      .group_by("model")
      .agg(pl.col("score").max())
)
```

25.6 DuckDB

DuckDB is an in-process SQL database (like SQLite, no server) built for analysis. It can run SQL straight over a pandas DataFrame, CSV or JSON file, and it connects to NumPy and Polars too. (`pip install duckdb`; syntax-checked only.)

```
import duckdb
import pandas as pd

df = pd.DataFrame({"model": ["haiku", "opus"], "score": [0.81, 0.93]})
out = duckdb.sql("SELECT model, score FROM df WHERE score > 0.9").df()
```

DuckDB finds the variable named `df` by name. The result converts back to a pandas DataFrame with `.df()`.

Tool	You write	Reach for it when
pandas	Python methods on DataFrames	the default; small to medium data
Polars	expressions, lazy plans	bigger data, speed, multi-core
DuckDB	SQL	you already think in SQL; query files in place

25.7 Data Visualization

A chart is for insight, not decoration. Pick a tool by how interactive and how shareable the result must be.

Package	Notes	Typical use
Matplotlib	oldest, most control, 2D/3D	static charts, papers, notebooks
seaborn	statistical plots on top of Matplotlib	quick plots straight from DataFrames
Plotly	interactive charts	dashboards, hover details
Bokeh	interactive, browser-based	web plots
Streamlit	turn a script into a web app	share a demo or eval dashboard

```
import matplotlib.pyplot as plt

means = df.groupby("model")["score"].mean()      # a Series
ax = means.plot(kind="bar", ylim=(0, 1), title="Mean score by model")
ax.set_ylabel("score")
plt.tight_layout()
plt.savefig("scores.png")
```

Syntax-checked only. Plotting a Series draws one bar per index label, which is why the `groupby` result above plots with no extra work.

25.8 Practice and wrap-up

Data science is a ladder: plain Python for small jobs, NumPy for numeric speed, pandas for tables, SQL or Polars when the data grows.

➤ **PRACTICE**

1. Sort a list of dicts of eval runs by score (high first), breaking ties by cost (low first), using `itemgetter`. Which argument handles the direction?
2. Show why `0.1 + 0.2 == 0.3` is false, and give two correct ways to compare or compute it.
3. What are the shapes of `np.arange(12).reshape(3, 4)` and its transpose? What happens if you add a shape `(4,)` array to it? A shape `(3,)` array?
4. Change a slice of a NumPy array and then print the original. Explain the result and fix it with `.copy()`.
5. In pandas, what is the difference between `df["a"]` and `df[["a"]]`? When would each be the right one?
6. Write a pandas or SQL query that returns the best score per model from a table of eval runs.

★ **ACHIEVEMENT UNLOCKED: Data wrangler**

You can now:

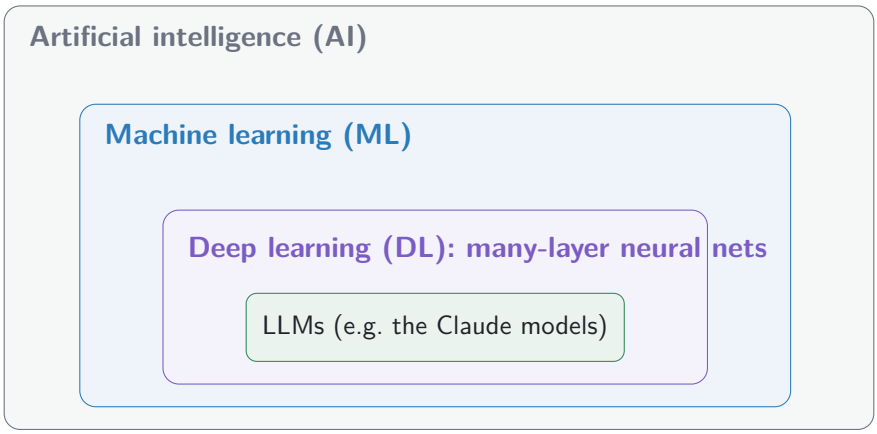
- pick the right number type (`float`, `Decimal`, `Fraction`) and avoid float comparison bugs;
- read an array's `shape`, reshape it, slice it, and predict broadcasting;
- tell a list, an `ndarray`, a `Series` and a `DataFrame` apart by type and behaviour;
- filter, group and summarize a table of results;
- choose between pandas, Polars and DuckDB, and a plotting tool, for a job.

CHAPTER 26

AI

What this chapter gives you. This chapter is the map of the field you are entering: where modern AI came from, which ideas matter (training, loss, attention, retrieval, agents), and how Python fits in. The book's model examples date quickly; **model names change, concepts last**. For hands-on work in this series you use the Anthropic SDK with `claude-haiku-5-5`, `claude-sonnet-5-5` and `claude-opus-5-5`.

26.1 Terms that get mixed up



rule-based expert systems live in the outer ring only (AI without ML)

Term	Meaning
AI	the broad, old goal: machines that do things people do
ML	learn patterns from data instead of being given rules
DL	ML with multi-layer neural networks
LLM	a large deep-learning model trained on text to predict the next token

> ★ **KEY IDEA: programs versus models**
>
> Classic code: *you* write the rules, the computer applies them. Machine learning: you supply examples (inputs and the right outputs), and the computer finds the rules (called **weights**). You cannot read the weights like source code, which is why you must test a model's outputs rather than trust them.

26.2 It Turns Out... (a short history)



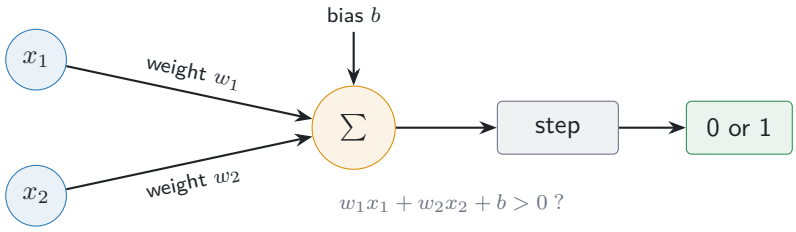
Expert systems

From roughly the 1950s to the 1980s, AI meant symbolic, top-down programs: experts wrote the rules. Chess shows the arc: a custom machine reached master level in 1983, and IBM's Deep Blue beat a

grandmaster in 1997. Later, AlphaGo (2016) beat a Go champion, and a successor learned the game from self-play with no rules taught.

Perceptrons

A **perceptron** is one artificial neuron: multiply each input by a weight, add a bias, and apply a threshold. It worked for simple classification and was proven unable to learn some patterns, which stalled the field for years.



A tiny trainer in pure Python (my own example; it ran). It learns AND, which one neuron can represent, but not XOR, which it cannot:

```
def step(x):
    return 1 if x > 0 else 0

def train(samples, epochs=20, lr=0.1):
    w1 = w2 = b = 0.0
    for _ in range(epochs):
        for (x1, x2), target in samples:
            pred = step(w1 * x1 + w2 * x2 + b)
            err = target - pred
            w1 += lr * err * x1      # nudge weights toward the target
            w2 += lr * err * x2
            b += lr * err
    return w1, w2, b

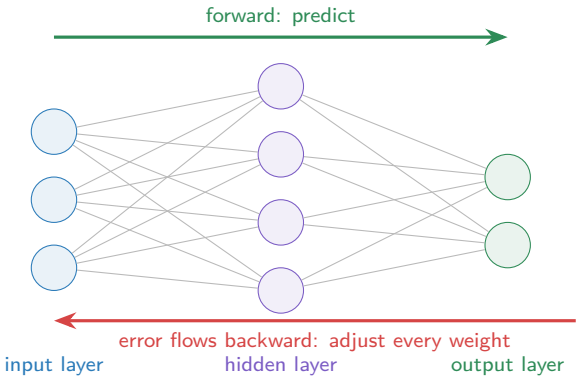
def accuracy(samples, w1, w2, b):
    hits = sum(step(w1 * x1 + w2 * x2 + b) == t for (x1, x2), t in samples)
    return hits / len(samples)

AND = [((0, 0), 0), ((0, 1), 0), ((1, 0), 0), ((1, 1), 1)]
XOR = [((0, 0), 0), ((0, 1), 1), ((1, 0), 1), ((1, 1), 0)]
print("AND", accuracy(AND, *train(AND)))
print("XOR", accuracy(XOR, *train(XOR)))
```

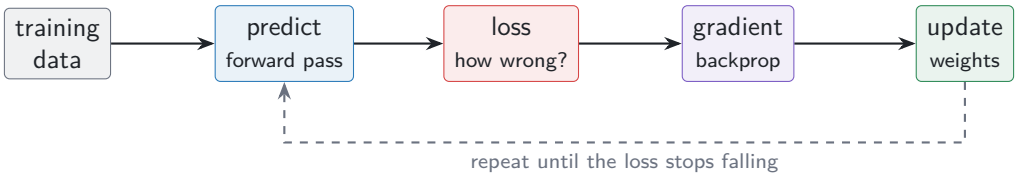
```
AND 1.0
XOR 0.5
```

The breakthrough: hidden layers and backpropagation

In 1986, Rumelhart, Hinton and Williams showed that a **multilayer perceptron** can learn the hard cases, thanks to two additions: **hidden layers** (more weights between input and output) and **backpropagation** (adjusting the weights backward from the error). Hinton and Hopfield shared the 2024 Nobel Prize in Physics for related work.



Training means: start with random weights; predict; measure how wrong you are with a **loss function**; use calculus (the slope, or *gradient*) to learn which way to move each weight; repeat. **Gradient descent** takes bigger steps where the slope is steep. After training, using the model on new data is **inference**.



■ **BOSS FIGHT: learning is minimizing a loss**

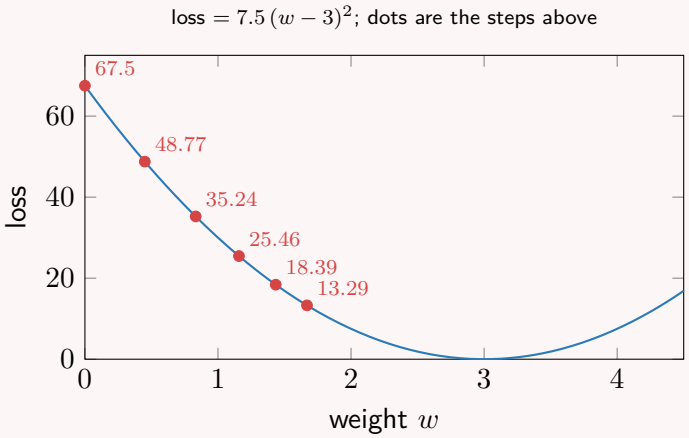
No one writes the model's rules. A loop nudges the weights downhill on the loss curve. The smallest possible example: fit $y = w\,x$ to data that follows $y = 3x$. The loss is the mean squared error. Each step moves w by $-$learning rate $\times$ gradient.

```
xs = [1.0, 2.0, 3.0, 4.0]
ys = [3.0, 6.0, 9.0, 12.0]
w, lr = 0.0, 0.01

def loss(w):
    return sum((w * x - y) ** 2 for x, y in zip(xs, ys)) / len(xs)

for step in range(6):
    grad = sum(2 * (w * x - y) * x for x, y in zip(xs, ys)) / len(xs)
    print(f"step {step}: w={w:.4f} loss={loss(w):.4f} grad={grad:.4f}")
    w -= lr * grad
```

```
step 0: w=0.0000 loss=67.5000 grad=-45.0000
step 1: w=0.4500 loss=48.7687 grad=-38.2500
step 2: w=0.8325 loss=35.2354 grad=-32.5125
step 3: w=1.1576 loss=25.4576 grad=-27.6356
step 4: w=1.4340 loss=18.3931 grad=-23.4903
step 5: w=1.6689 loss=13.2890 grad=-19.9667
```



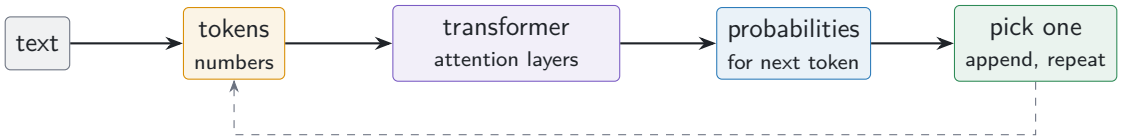
The gradient shrinks as w nears the minimum, so the steps shrink too. Running 200 more steps (my run) ends at $w = 3.0000$. A learning rate that is too big overshoots and the loss grows. Real models do the same thing with billions of weights, and PyTorch computes the gradients for you.

Image recognition: ImageNet and AlexNet

ImageNet is a huge labeled image set. From 2010 a yearly contest asked programs to label images. In 2012 the entry AlexNet won by a wide margin using a **convolutional neural network** (CNN), and interest in deep learning exploded. CNNs scan small patches of an image for patterns.

Large language models

In 2017, the paper "Attention Is All You Need" introduced the **transformer**. Earlier sequence models (RNNs) processed text one step at a time. *Attention* lets the model relate every token to every other token, and it can be computed in parallel on GPUs, so very large models became practical. LLMs look like fancy autocomplete, yet they show surprising abilities at scale.



⚠ TRAP CARD

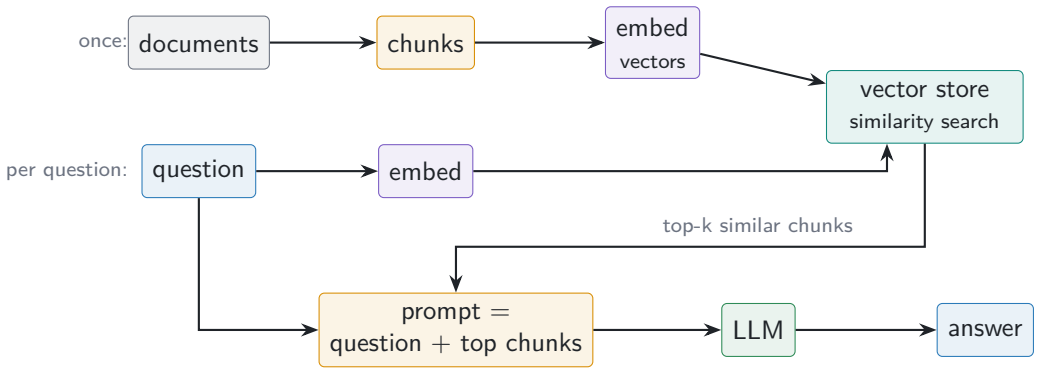
The model sees **tokens**, not letters. A word like *strawberry* may arrive as a couple of chunks (the real split depends on the tokenizer). That is why small models famously miscount letters. When you need an exact answer, do not ask a model to count: write `"strawberry".count("r")` (it returns 3).

The ChatGPT moment

In 2022 OpenAI released ChatGPT. A chat interface turned a research model into something anyone could use: write code, converse, summarize. Since then every major lab has shipped chat models and APIs.

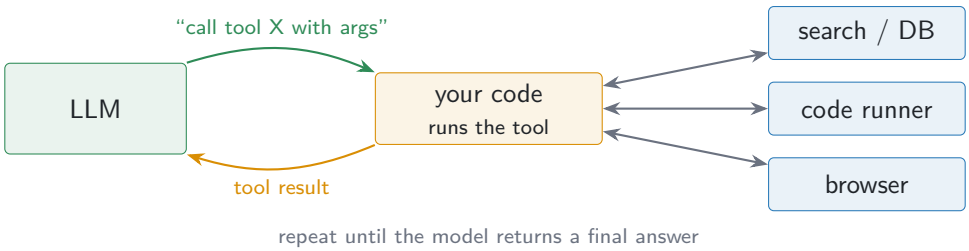
Retrieval augmented generation (RAG)

A model answers from what it learned in training, and it may **hallucinate**: state a wrong answer confidently. **RAG** gives the model fresh, relevant text at question time, and tends to reduce (not remove) hallucinations.



Agents, tools and protocols

Plain models take text in and give text out. Many are **multimodal** too (images, audio, video). Applications now connect a model to outside **tools and data**: a vector store for similarity search, a browser to control, a code runner. In an **agent loop** the model decides which tool to call, your code runs it, and the result goes back to the model, until it can answer.



Name	What it is
LangChain	a Python framework to wire a model to tools and steps
MCP (Model Context Protocol)	Anthropic's open protocol for plugging tools and data sources into models
A2A (Agent2Agent)	Google's protocol for agents to find and talk to each other

Efficiency

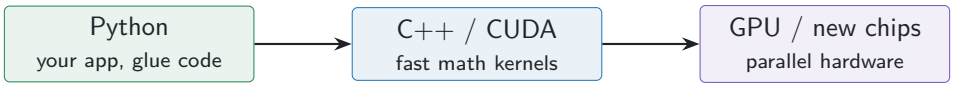
Better algorithms, cheaper and more scalable hardware and the cloud made all this affordable. **GPUs** beat CPUs at the repeated multiply-and-add at the heart of neural networks, because they do many of them in parallel.

26.3 Create Models: Python Frameworks

Python won AI partly because it is free, open and good with data, and because NumPy and SciPy let a slow-looking language drive fast numeric code. The book names the main training frameworks.

Library	What it is	Use it for
scikit-learn	classic ML: classification, clustering, regression	small/tabular problems, baselines
TensorFlow	Google's deep-learning framework (2015), mostly C++ underneath	existing TF codebases
PyTorch	Meta's framework (2016), more "Pythonic"	most current deep-learning research and models
fastai	high-level layer on PyTorch	quick training experiments
Keras	simple API on top of TensorFlow and others	beginner-friendly model building

The book chooses not to teach model building here, and instead shows how to *use* models. That matches a junior AI engineer's day-to-day job: call models, evaluate them, wire them to data.



the usual AI "sandwich": easy on top, fast underneath

26.4 Current Models

The book lists (as of early 2025) models from OpenAI, Anthropic, Google, Perplexity, DeepSeek, xAI and Meta, plus special-purpose models such as AlphaFold (protein structure). That list is already dated; treat it as an example of the *kinds* of providers, not a recommendation.

Kind of model	How you reach it	Trade-off
Hosted frontier model (Claude, GPT, Gemini)	HTTPS API with a key, via an SDK	best quality; costs per token; your data leaves your machine
Open-weights model (Llama and others)	download and run it yourself (Ollama, Hugging Face)	free to run, private; needs RAM/GPU; usually weaker
Special-purpose model	its own API or library	excellent at one job, useless at others

> ➤ **RULE**
> Do not hard-code a model name deep in your program. Put it in one constant or config value, so you can swap `claude-haiku-5-5` for `claude-sonnet-5-5` or `claude-opus-5-5` (cheaper and faster versus stronger) without touching the logic.

Illustrative call with the Anthropic SDK. It needs `pip install anthropic` and an API key in your environment (variable `ANTHROPIC_API_KEY`). Syntax-checked only:

```
import anthropic

MODEL = "claude-haiku-5-5"          # one place to change the model

client = anthropic.Anthropic()      # reads the key from the environment
response = client.messages.create(
    model=MODEL,
    max_tokens=1000,
    messages=[{"role": "user", "content": "Define a limerick in one line."}],
)
for block in response.content:      # content is a LIST of blocks
    if block.type == "text":
        print(block.text)
```

> ■ **TYPE CHECK**
> The SDK returns a **typed object**, not a dict. I built one offline (no API call) to show the types:

```python
from anthropic.types import Message, TextBlock, Usage

msg = Message(
    id="msg_demo", type="message", role="assistant",
    model="claude-haiku-5-5",
    content=[TextBlock(type="text", text="A token is a chunk of text.")],
    stop_reason="end_turn", stop_sequence=None,
    usage=Usage(input_tokens=12, output_tokens=9),
)
print(type(msg).__name__)
print(type(msg.content).__name__, type(msg.content[0]).__name__)
print(msg.usage.input_tokens + msg.usage.output_tokens)
d = msg.model_dump()                    # plain dict, e.g. for json.dumps
print(type(d).__name__, d["content"][0]["text"])
```

```
Message
list TextBlock
21
dict A token is a chunk of text.
```

Expression	Type	Use
`response`	`Message` (object)	attribute access: `.content`, `.usage`
`response.content`	`list` of blocks	loop; do not assume `[0]` is text
`block.text`	`str`	only on blocks whose `type == "text"`
`response.model_dump()`	`dict`	save to JSON, log, compare

26.5 A Million Models: Hugging Face

Hugging Face is a hub with over a million open models and many datasets (the book's count). Its `transformers` library downloads and runs them, on top of PyTorch or TensorFlow (`pip install transformers torch`). The `pipeline` helper is the quickest start. Illustrative sketch (not run; the model id is a placeholder you pick from the hub):

```python
from transformers import pipeline

classify = pipeline("text-classification", model="<model-id-from-the-hub>")
result = classify("The new release fixed my bug.")
# result is a LIST of dicts, e.g. [{"label": "...", "score": 0.97}]
print(result[0]["label"], result[0]["score"])
```

Hub item	Why you care
Models	ready-made weights for text, vision, audio and more; check each model's license
Datasets	data for training and evaluation
Libraries	`transformers` to run models, others to train them

26.6 Working Examples With Ollama

Ollama runs open models on your own computer, free and open source, with a command-line tool and a Python library (`pip install ollama`). The book uses it to try small local models. Typical flow:

```
$ ollama serve                  # start the local server
$ ollama run tinyllama          # downloads the model on first use, then chats
>>> Send a message (/? for help)
```

Illustrative Python use (not run; check the library docs for current response fields):

```python
import ollama

reply = ollama.chat(
    model="tinyllama",
    messages=[{"role": "user", "content": "What is a limerick?"}],
)
print(reply["message"]["content"])
```



everything stays on your machine: no API key, no per-token bill

	Local small model	Hosted frontier model
Cost	free after download; uses your RAM/CPU/GPU	pay per token
Privacy	data stays on your machine	data is sent to the provider
Quality	often weak at counting, poetry, code	much stronger overall
Setup	install, download gigabytes	API key and an SDK

What the book's experiments teach

The author tried two small local models. TinyLlama gave a wrong answer to a letter-counting question, did not write a real villanelle or limerick, did not define a limerick correctly, and wrote Python that did not work. Code Llama (Python version) did better on short functions but wrote test cases that were duplicated and sometimes wrong. The lessons matter more than the models:

- **Small models are uneven.** Match the model to the task; test before you trust.
- **Model output is untrusted input.** Run generated code and check it yourself.
- **Models can be out of date.** One generated snippet used `datetime.utcnow()`, which Python now warns is deprecated.

Checks I ran for these ideas:

```
import warnings
from datetime import datetime, timezone

def utc_now_str():
    return datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M:%S")

s = utc_now_str()
print(len(s), s[4], s[10] == " ")
with warnings.catch_warnings(record=True) as w:
    warnings.simplefilter("always")
    datetime.utcnow()                       # the old, deprecated way
    print(w[0].category.__name__)
print("strawberry".count("r"))
gen = (x for x in range(1, 6))              # generator expression
lst = [x for x in range(1, 6)]              # list comprehension
print(type(gen).__name__, type(lst).__name__, list(gen))
```

```
19 - True
DeprecationWarning
3
generator list [1, 2, 3, 4, 5]
```

Note the last line: "a generator comprehension" (round brackets) is not the same type as a list comprehension (square brackets). A model that returns a list when you asked for a generator has changed the type.

> ➤ **RULE**
>
> Never accept model-written tests as proof. Write your own cases (include edge cases such as an empty string) and run them against the generated function:

```
def count_letter(word, letter):          # pretend a model wrote this
    return word.count(letter)

cases = [("strawberry", "r", 3), ("banana", "a", 3), ("", "x", 0)]
failures = [(w, l, want, count_letter(w, l))
            for w, l, want in cases if count_letter(w, l) != want]
print("failures:", failures)
```

```
failures: []
```

References and next steps

The book points to material on Ollama and its Python library, a general AI tutorial, Chip Huyen's *AI Engineering* and *Designing Machine Learning Systems*, and 3Blue1Brown's visual videos on backpropagation and gradient descent. For a visual learner, the last is the one to watch first.

26.7 Practice and wrap-up

Python became the main language of data science and AI because it glues easy code to fast libraries. The field moved from hand-written rules to learned weights, and from single models to tools and agents.

➤ **PRACTICE**

1. Put these in order of nesting: deep learning, AI, machine learning, LLM. Which one does a rule-based chess program belong to?
2. In your own words: what are the *loss*, the *gradient* and a *learning rate*? What goes wrong with a learning rate that is too large?
3. Why does a language model often fail at "how many r's are in strawberry", and what is the reliable way to get the answer?
4. Draw the steps of a RAG system for a question about your own notes. Which step uses embeddings and which uses the LLM?
5. Ask any model of your choice to write `utc_now_str()`. Run it, check for deprecation warnings, and write three tests of your own.
6. List two reasons to run a small model locally and two reasons to call a hosted one.

★ **ACHIEVEMENT UNLOCKED: AI map**

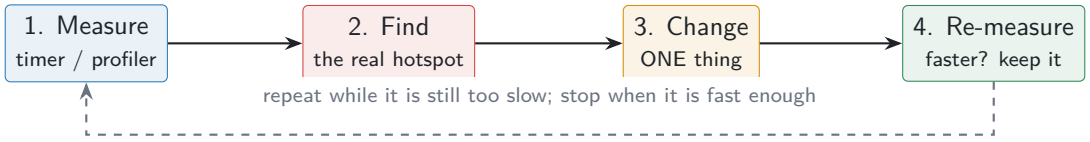
You can now:

- explain AI, ML, DL and LLM, and the key ideas of training: weights, loss, gradient descent;
- describe how transformers, RAG and agents with tools fit together;
- read the types in an SDK response (object, list of blocks, dict) and parse them safely;
- choose between a local and a hosted model, and verify model-written code instead of trusting it;
- write a loop that fits a one-parameter model by gradient descent.

CHAPTER 27

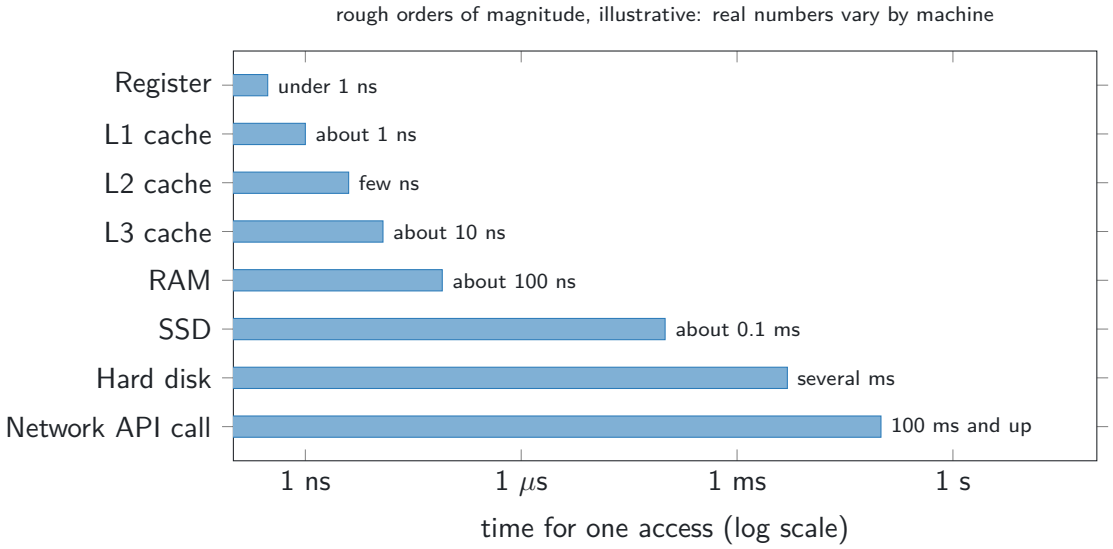
Performance

What this chapter gives you. Python is fast enough until it is not, and people guess wrong about where the time goes. This chapter teaches you to *measure* first (timers and a profiler), then fix the real hotspot with a better algorithm, a cache, concurrency, or a faster tool. In AI work the slow parts are usually waiting on model APIs, big loops over data, and memory use.



27.1 Computer Hardware

A CPU core has registers (hold the numbers in use), an arithmetic-logic unit (does the operations), a control unit (fetches instructions and data) and a clock. Around it sit caches, RAM, an SSD and maybe a hard drive. Going down the list, storage gets bigger, cheaper and **much slower**. The first two kinds lose their data when power is lost; the last two keep it.



The point of the chart: each step down costs 10 to 1000 times more. Waiting on a network call is millions of times slower than adding two numbers. The operating system moves data between rows for you (for example virtual memory pages to disk, which is why a computer that runs out of RAM suddenly crawls). At the software level you can help in three ways:

Lever	Idea	Python example
Data layout	contiguous data fits caches; scattered objects do not	`array`, NumPy arrays instead of lists of floats
Parallelism	the CPU applies one instruction to many values (SIMD), or many cores work at once	NumPy and pandas operations; multiprocessing
Concurrency	do other work while waiting for disk or network	threads, `asyncio` (Chapter 21)

27.2 Measure Timing

Tool	Good for	Note
`time.time()`	wall-clock epoch seconds	can jump if the system clock changes; the book uses it
`time.perf_counter()`	timing a block of code	best clock for intervals
`timeit`	tiny snippets, many repeats	runs the code for you
`cProfile`	which function is slow	per-function table
`tracemalloc`	which code uses memory	peak and current bytes

A stopwatch with perf_counter

```
import time

t0 = time.perf_counter()
time.sleep(0.2)                          # the work being timed
elapsed = time.perf_counter() - t0
print(f"{elapsed:.2f} s")
```

```
0.20 s
```

Run it several times: tiny jobs vary a lot (other programs, caches, power saving). One measurement proves little.

timeit

`timeit.timeit(code, number=n)` runs the code `n` times and returns the *total* seconds for all runs (a `float`). The default `number` is one million, so set it yourself for anything slow. `repeat` gives several totals; take the **minimum**, which has the least noise. You can pass a string, or any callable.

```
from timeit import timeit, repeat

total = timeit("sum(range(100))", number=1000)       # float, seconds
runs = repeat("sum(range(100))", number=1000, repeat=5)
best = min(runs) / 1000                              # seconds per call
print(type(total).__name__, len(runs))
print(f"best per call: {best * 1e6:.1f} us")         # varies per machine

data = list(range(1000))
t = timeit(lambda: sorted(data), number=100)         # pass a callable
print(f"{t:.4f} s for 100 sorts")
```

```
float 5
best per call: 0.4 us
0.0003 s for 100 sorts
```

Your numbers will differ.

A decorator and a context manager

For your own programs, wrap timing in a reusable tool. `functools.wraps` keeps the function's name; `try/finally` prints even if the function raises.

```
import time
from contextlib import contextmanager
from functools import wraps

def timed(func):
    @wraps(func)
    def inner(*args, **kwargs):
        t0 = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            print(f"{func.__name__}: {time.perf_counter() - t0:.3f} s")
    return inner

@contextmanager
def stopwatch(label):
    t0 = time.perf_counter()
    yield
    print(f"{label}: {time.perf_counter() - t0:.3f} s")

@timed
def embed_batch(n):                 # stand-in for an embedding call
    time.sleep(0.05 * n)
    return [0.0] * n

vecs = embed_batch(2)
with stopwatch("parse"):
    time.sleep(0.1)
```

```
embed_batch: 0.100 s
parse: 0.100 s
```

> ⚠ **TRAP CARD**
> Do not time code with `print` in the middle, with the first run only (caches and imports make it slower), or with a stdout redirect that does I/O. And never compare timings from two different machines or from your laptop on battery. Compare two versions on the same machine in the same run.

27.3 Profile

Timers tell you *how long*. A profiler tells you *where*. `cProfile` (written in C, low overhead) counts calls and times every function. Below, a pipeline hides a slow de-duplication step; the profile exposes it. I ran this; file paths are shortened.

```python
import cProfile
import pstats

def tokenize(text):
    return text.split()

def slow_unique(words):
    seen = []
    for w in words:
        if w not in seen:           # linear search in a list
            seen.append(w)
    return seen

def pipeline():
    text = " ".join(f"w{i % 700}" for i in range(20000))
    return slow_unique(tokenize(text))

prof = cProfile.Profile()
prof.runcall(pipeline)
pstats.Stats(prof).sort_stats("cumulative").print_stats(5)
```

```
         20707 function calls in 0.037 seconds

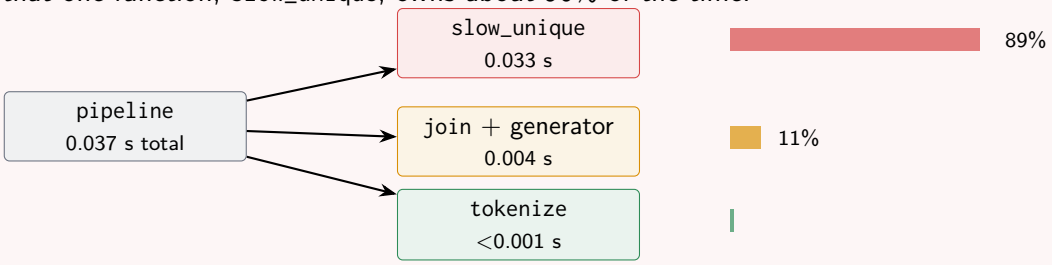
   Ordered by: cumulative time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.037    0.037 pipeline.py:14(pipeline)
        1    0.033    0.033    0.033    0.033 pipeline.py:7(slow_unique)
        1    0.002    0.002    0.004    0.004 {method 'join' of 'str' objects}
    20001    0.003    0.000    0.003    0.000 pipeline.py:15(<genexpr>)
        1    0.000    0.000    0.000    0.000 pipeline.py:4(tokenize)
```

Column	Meaning	When to look
ncalls	number of calls	huge numbers: called too often
tottime	time inside the function itself	the real hotspot
cumtime	time including everything it calls	which top-level step is slow

■ BOSS FIGHT: measure before you optimize

Everyone guesses. In the run above you might blame `tokenize` or the string building. The profiler shows that one function, `slow_unique`, owns about 90% of the time.



The fix is a data-structure change, not clever code: membership in a list scans every item, in a set it is a hash lookup. One line, found only by measuring:

```python
def fast_unique(words):
    return list(dict.fromkeys(words))     # keeps first-seen order, O(n)
```

For memory, `tracemalloc` reports the peak bytes allocated while your code ran (measured here, one million

numbers):

```
import tracemalloc
from array import array

def peak_mb(build):
    tracemalloc.start()
    keep = build()
    _, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return peak / 1_000_000

N = 1_000_000
print("list of ints :", round(peak_mb(lambda: list(range(N))), 1), "MB")
print("array('i')   :", round(peak_mb(lambda: array("i", range(N))), 1), "MB")
gen_mb = peak_mb(lambda: sum(x for x in range(N)))
print("sum(generator):", round(gen_mb, 3), "MB")
```

```
list of ints : 40.0 MB
array('i')   : 4.1 MB
sum(generator): 0.0 MB
```

A generator never holds the whole sequence, which matters when you stream a huge file of documents.

27.4 Algorithms and Data Structures

“Better” can mean faster, shorter, clearer or lighter on memory. Compare with data. All numbers below are measured on this machine (Python 3.12); yours will differ, but the *ratios* are the lesson.

```
from timeit import timeit

def with_append():
    out = []
    for v in range(1000):
        out.append(v * 2)
    return out

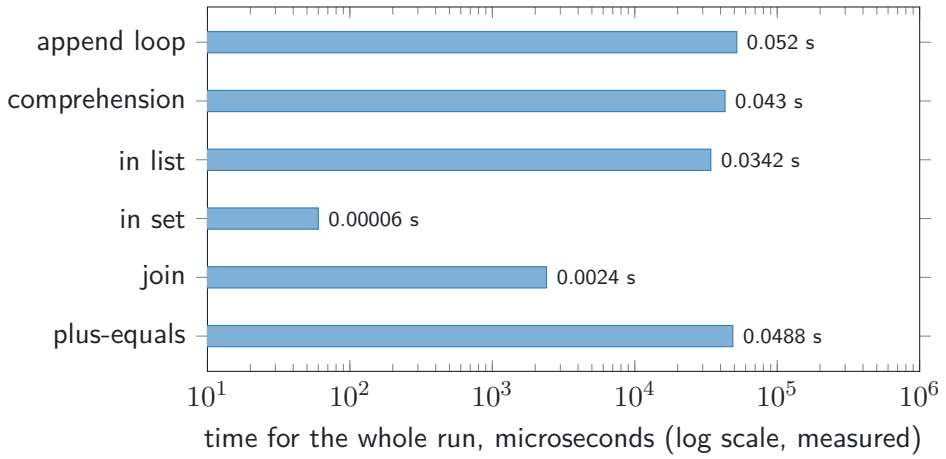
def with_comp():
    return [v * 2 for v in range(1000)]

print("loop :", round(timeit(with_append, number=2000), 3))
print("comp :", round(timeit(with_comp, number=2000), 3))

ids_list = list(range(5000))
ids_set = set(ids_list)
print("list in:", round(timeit(lambda: 4999 in ids_list, number=2000), 4))
print("set in :", round(timeit(lambda: 4999 in ids_set, number=2000), 6))

parts = [str(i) for i in range(2000)]
def plus():
    s = ""
    for p in parts:
        s += p
    return s
print("join :", round(timeit(lambda: "".join(parts), number=500), 4))
print("plus :", round(timeit(plus, number=500), 4))
```

```
loop : 0.052
comp : 0.043
list in: 0.0342
set in : 6e-05
join : 0.0024
plus : 0.0488
```



Choice	Measured gap	Why
comprehension vs append loop	about 1.2× here	fewer bytecode steps; the book saw about 2× on its machine, so gaps change between Python versions
`in` on a set vs a list	about 500×	hash lookup vs scanning every item
`"".join(parts)` vs repeated `+=`	about 20×	join measures once and copies once

> ➤ **RULE**
>
> The big wins come from changing the **algorithm or data structure** (list to set, repeated work to cached work), not from micro-tweaks. Choose the readable form first; optimize only the measured hotspot.

Arrays versus lists and tuples

Contiguous, same-type data is friendlier to the CPU cache. The standard `array` module packs numbers tightly (measured above: 4.1 MB against 40.0 MB for a million ints). NumPy goes further and runs the loop in compiled code (Chapter 25). NumPy is not installed here, so this is a syntax-checked sketch:

```
import numpy as np

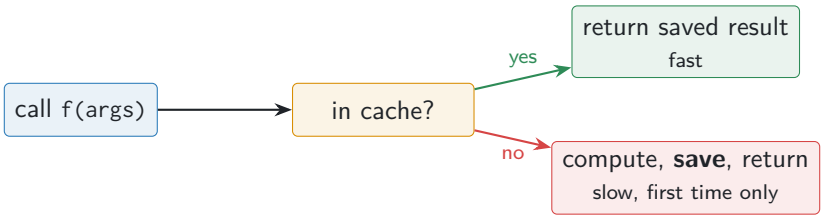
values = list(range(1_000_000))
doubled_slow = [v * 2 for v in values]       # Python loop

arr = np.arange(1_000_000)
doubled_fast = arr * 2                       # compiled loop, no Python per item
```

> ■ **TYPE CHECK**
>
> The speed of NumPy and pandas comes from keeping work *out of Python's per-item loop*. If you call `.apply` or loop over `df.iterrows()`, you lose the benefit. Prefer whole-column operations.

27.5 Cache

If the same call keeps producing the same answer, compute it once. `functools.cache` (unbounded) and `lru_cache(maxsize=N)` (keeps the N most recently used) store results keyed by the arguments. This is ideal for expensive, repeatable work, such as embedding the same text twice.




```
import time
from functools import lru_cache, cache

calls = 0

@cache
def embed(text):                # pretend this calls an embedding API
    global calls
    calls += 1
    time.sleep(0.05)
    return len(text)

t0 = time.perf_counter()
for t in ["a", "bb", "a", "bb", "a"]:
    embed(t)
print(calls, f"{time.perf_counter() - t0:.2f}")
print(embed.cache_info())

@lru_cache(maxsize=2)           # small limit: evicts the oldest
def f(x): return x
for x in (1, 2, 3, 1):
    f(x)
print(f.cache_info())

try:
    @cache
    def g(xs): return len(xs)
    g([1, 2])
except TypeError as e:
    print("TypeError:", e)
```

```
2 0.10
CacheInfo(hits=3, misses=2, maxsize=None, currsize=2)
CacheInfo(hits=0, misses=4, maxsize=2, currsize=2)
TypeError: unhashable type: 'list'
```

Five calls, only two real computations. In the second cache, the last call to `f(1)` missed because `1` had already been evicted.

> ⚠ **TRAP CARD**
>
> Arguments must be **hashable**: pass a tuple or string, not a list. Caching ties up memory (use `lru_cache(maxsize=...)` for unbounded inputs). Never cache a function with side effects or one whose answer changes over time (the current time, a live API result).

27.6 Concurrency for waiting

Chapter 21's lesson applies directly to AI apps: most of your time is spent waiting for model and network calls. Overlap the waits. Measured here with a fake call that sleeps 0.1 s, ten of them:

```
import asyncio
import time
from concurrent.futures import ThreadPoolExecutor

def fake_call(i):
    time.sleep(0.1)                  # stands in for a network wait
    return i

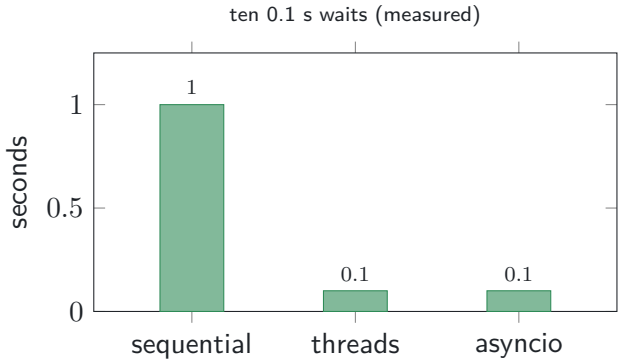
async def fake_acall(i):
    await asyncio.sleep(0.1)
    return i

t0 = time.perf_counter()
seq = [fake_call(i) for i in range(10)]
print(f"sequential: {time.perf_counter() - t0:.1f} s")

t0 = time.perf_counter()
with ThreadPoolExecutor(max_workers=10) as pool:
    res = list(pool.map(fake_call, range(10)))
print(f"threads   : {time.perf_counter() - t0:.1f} s")

async def main():
    return await asyncio.gather(*(fake_acall(i) for i in range(10)))
t0 = time.perf_counter()
res2 = asyncio.run(main())
print(f"asyncio   : {time.perf_counter() - t0:.1f} s", res2 == seq)
```

```
sequential: 1.0 s
threads   : 0.1 s
asyncio   : 0.1 s True
```

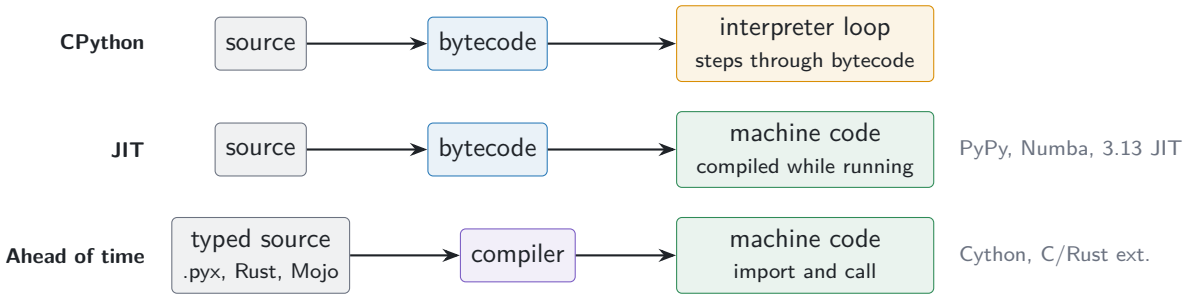


TYPE CHECK

Threads and `asyncio` help with *waiting* (I/O). They do not speed up pure number crunching in CPython, because of the global interpreter lock. For CPU-bound work use NumPy, a compiled tool below, or multiple processes.

27.7 Going faster than CPython

When the algorithm is right and the hotspot is still CPU-bound Python, you can compile it. First, how Python normally runs, and where each tool plugs in:



You can see CPython's bytecode with the `dis` module (output from Python 3.12):

```
import dis
dis.dis("a + b")
```

```
  0           0 RESUME                   0

  1           2 LOAD_NAME                0 (a)
              4 LOAD_NAME                1 (b)
              6 BINARY_OP                0 (+)
             10 RETURN_VALUE
```

Tool	How it works	You change	Good for
Cython	Python with optional C types, compiled to C	add `cdef` types, build step	numeric loops (large speedups when typed)
C or Rust extension	write the hot function in C/Rust, call from Python	write in another language	libraries; Rust is memory-safe. Pydantic's core and `uv` are Rust
Taichi	JIT-compiles Python-like kernels	decorate kernels	numeric and graphics code
PyPy	alternative interpreter with a JIT	run your program with `pypy`	long-running pure-Python code; check C-extension support
Numba	JIT for numeric functions	add `@njit`	loops over numbers and NumPy arrays
CPython 3.13 JIT	experimental built-in JIT	rebuild CPython with the option	experiments only
Mojo	new Python-like language with types, compiled	rewrite hot code in Mojo	ML kernels; still young

Cython

Cython translates Python plus type declarations to C. Not run here (needs a C compiler and a build step); illustrative `.pyx` file:

```
# sumsq.pyx
def sum_sq(int n):
    cdef long total = 0
    cdef int i
    for i in range(n):
        total += i * i
    return total
```

NumPy and SciPy

The cheapest "compiler" is a library that already has compiled loops. NumPy and SciPy do the heavy numeric work in C and Fortran (see Chapter 25), which is a big reason Python is used for data science.

C or Rust extensions

Much of Python itself and its standard library is C wrapped for convenience. You can write your own extension for a hotspot. A recent trend is Rust instead of C because it is fast and safer with memory. Two well-known Python tools built this way: Pydantic's validation core, and the package manager `uv`.

Taichi, PyPy and Numba

CPython is the standard interpreter (written in C; do not confuse it with Cython). It translates your code to bytecode and steps through it. **PyPy** is another interpreter that adds just-in-time compilation and is often much faster on CPU-bound pure Python. **Numba** compiles chosen functions with a decorator. The first call includes compile time, so time later calls:

```
from numba import njit          # pip install numba

@njit                           # = jit(nopython=True)
def sum_sq(n):
    total = 0
    for i in range(n):
        total += i * i
    return total

sum_sq(10)                      # first call compiles; ignore its time
result = sum_sq(10_000_000)     # later calls run machine code
```

Syntax-checked only. A JIT helps with long loops over numbers. It will not help a tiny function called once, or code that is mostly I/O or dictionaries of strings (the book's own tiny `hypot` example showed little gain).

The standard-library JIT

Python 3.13 added an *experimental* JIT, available only if you rebuild CPython with the right flag. That version also has an experimental free-threaded build (no GIL). Neither is something to rely on yet.

Mojo

Mojo is a newer language from Modular (announced in 2023), designed as a faster "Python++" for AI work. Key points from the book:

- **Why:** modern AI stacks are a sandwich: Python on top, C/C++/CUDA in the middle, new hardware underneath. Mojo aims to cover all layers in one language.
- **Who:** Chris Lattner, who earlier created LLVM, Clang, Swift and MLIR.

- **Design:** compiles to machine code; type hints are mandatory (CPython ignores them); `struct` groups data; `fn` replaces `def` for strictly typed functions; it can run existing Python.
- **Claims:** one demo was 35,000× faster than naive Python; another about 60× and twice NumPy. Treat such vendor and demo figures with caution and measure your own workload.
- **Limits:** young; Python compatibility is still incomplete.

> ➤ **RULE**
> Climb the ladder in order and stop when it is fast enough: (1) measure, (2) better algorithm or data structure, (3) cache or avoid repeated work, (4) use built-in or C-backed libraries (`sum`, `sorted`, NumPy, pandas), (5) overlap waiting with threads or `asyncio`, (6) compile the hotspot (Numba, Cython, Rust), (7) different interpreter or language. Each step costs more effort and flexibility than the previous one.

27.8 Practice and wrap-up

Performance work is detective work: measure, find the cause, change one thing, measure again.

> ➤ **PRACTICE**
> 1. Why is `time.perf_counter()` a better stopwatch than `time.time()`? What does `timeit.repeat` return, and which value of it should you report?
> 2. Profile a function of your own. Which column tells you the hotspot, `tottime` or `cumtime`? Why?
> 3. Change a list used for membership tests to a set and measure. Predict the ratio first.
> 4. Why does `@cache` fail for a function that takes a list? How would you fix the call?
> 5. Embedding 200 documents takes 0.1 s of waiting per call. Roughly how long does a sequential loop take, and a pool of 20 threads? What changes if the work were CPU-bound instead?
> 6. Compare three ways to compute the sum of the first n squares: a `for` loop, a generator passed to `sum`, and the closed formula $n(n+1)(2n+1)/6$. Time them for $n = 10^6$ and explain the result.

> ★ **ACHIEVEMENT UNLOCKED: Performance detective**
> You can now:
> - time code correctly with `perf_counter` and `timeit`, and report a stable number;
> - read a `cProfile` table to find the real hotspot, and check memory with `tracemalloc`;
> - replace a slow data structure or algorithm (list to set, `+=` to `join`, loop to comprehension);
> - add caching and concurrency, and know when each one helps;
> - explain where Numba, Cython, PyPy, Rust extensions and Mojo fit, and when to leave them alone.